

DATA ENGINEERING

Distributed Systems, Scalable Processing, and Modern Architectures

FIRST EDITION · 2026

PART I The Data Engineering Landscape

PART II Distributed File Systems

PART III Batch Processing at Scale

PART IV In-Memory Processing

PART V Data Pipelines & Integration

PART VI Insight Delivery

PART VII The Modern Data Stack

Hadoop

HDFS

MapReduce

Apache Spark

Kafka

Delta Lake

HiveQL

MLOps

Data Mesh

Apache Beam

Lakehouse

Diff. Privacy

Rokan Uddin Faruqi

Professor, Department of Computer Science & Engineering
University of Chittagong

First Edition, 2026.

Contents

I	The Data Engineering Landscape	1
1	The Big Data Revolution: Scale, Velocity, and Variety	3
1.1	Why Data Engineering Matters	4
1.2	Defining Big Data: The 3Vs Framework	6
1.2.1	Volume: The Scale Dimension	6
1.2.2	Velocity: The Speed Dimension	7
1.2.3	Variety: The Diversity Dimension	9
1.3	The Extended 5Vs Framework	10
1.4	A Taxonomy of Digital Data	11
1.4.1	Structured Data	12
1.4.2	Semi-Structured Data	13
1.4.3	Unstructured Data	15
1.5	The Engineering Response to Scale	15
1.5.1	Vertical vs. Horizontal Scaling	17
1.5.2	The Commodity Hardware Insight	17
1.5.3	Moving Computation to Data	18
1.6	The CAP Theorem	19
1.6.1	Statement and Definitions	19
1.6.2	The Practical Implication: Choosing C vs. A	20
1.7	Batch vs. Stream Processing	23
1.7.1	Batch Processing	23
1.7.2	Stream Processing	23
1.8	Case Study: Walmart’s Retail Analytics Platform	25
2	Data Integration: Heterogeneity, Quality, and Provenance	39
2.1	The Data Silo Problem	40
2.2	A Formal Framework for Data Integration	41
2.2.1	Schema Mapping	42
2.2.2	Data Exchange	43

2.2.3	Entity Resolution	43
2.3	The Four Dimensions of Data Heterogeneity	44
2.3.1	Structural Heterogeneity	44
2.3.2	Semantic Heterogeneity	45
2.3.3	Syntactic Heterogeneity	46
2.3.4	Scale and Granularity Heterogeneity	47
2.4	Entity Resolution	47
2.4.1	Blocking: Reducing the Candidate Space	48
2.4.2	Similarity Functions	49
2.5	Data Quality: Dimensions and Challenges	49
2.5.1	Missing Values	51
2.5.2	Noise, Outliers, and Inconsistency	51
2.6	Data Provenance and Lineage	52
2.7	ETL vs. ELT: Two Philosophies of Integration	53
2.7.1	ETL: Extract, Transform, Load	54
2.7.2	ELT: Extract, Load, Transform	54
2.8	The Modern Data Integration Stack	56
2.9	Case Study: US Healthcare Interoperability and HL7 FHIR	57

II Distributed Storage 75

3 Distributed File Systems: GFS and HDFS 77

3.1	The Storage Problem at Petabyte Scale	78
3.2	The Google File System	79
3.2.1	Workload Assumptions and Design Decisions	80
3.2.2	GFS Architecture	81
3.3	HDFS Architecture	83
3.3.1	The NameNode	83
3.3.2	DataNodes	84
3.3.3	The Secondary NameNode	85
3.3.4	Block Storage	86
3.4	Replication and Fault Tolerance	86
3.4.1	Rack-Aware Replication	87
3.4.2	Failure Detection and Re-replication	88
3.5	HDFS Read and Write Paths	89

3.5.1	The Read Path	89
3.5.2	The Write Path	91
3.6	HDFS High Availability	92
3.6.1	The HA Architecture	93
3.7	HDFS Design Characteristics and Limitations	94
3.7.1	What HDFS Is Optimised For	94
3.7.2	The Small File Problem	95
3.7.3	No In-Place Random Writes	95
3.7.4	No Low-Latency Random Access	96
3.8	Beyond HDFS: Cloud Object Storage	97
3.9	Case Study: Yahoo!'s Hadoop Cluster (2006–2011)	98
III	Batch Processing at Scale	113
4	The MapReduce Paradigm and the Hadoop Processing Stack	115
4.1	Why a New Computation Model?	117
4.2	The MapReduce Programming Model	118
4.2.1	The Canonical Example: Word Count	119
4.2.2	Additional MapReduce Examples	120
4.3	The Combiner and the Partitioner	120
4.3.1	The Combiner: Local Aggregation	120
4.3.2	The Partitioner: Directing Keys to Reducers	121
4.4	The Full MapReduce Execution Pipeline	122
4.4.1	Input Splits and the InputFormat	122
4.4.2	Task Scheduling and Data Locality	122
4.4.3	The Shuffle and Sort in Detail	123
4.5	Fault Tolerance in MapReduce	123
4.6	Apache Pig: Dataflow Scripting	125
4.7	Apache Hive: SQL-on-Hadoop	127
4.7.1	The Hive Metastore	127
4.7.2	Partitioning and Bucketing	128
4.7.3	HiveQL to MapReduce Compilation	129
4.8	Apache HBase: Column-Family NoSQL on Hadoop	130
4.8.1	The HBase Data Model	131
4.8.2	HBase Architecture	132

4.9	Ecosystem Tool Selection	134
4.10	Case Study: Facebook Data Infrastructure (2008–2012)	135
5	Resource Management, File Formats, and I/O Optimisation	149
5.1	The Hadoop 1.x Scheduling Bottleneck	151
5.2	YARN: Yet Another Resource Negotiator	152
5.2.1	The ResourceManager	152
5.2.2	The NodeManager	152
5.2.3	The ApplicationMaster	153
5.2.4	The Container: YARN’s Resource Unit	154
5.2.5	YARN Job Execution Flow	154
5.3	YARN Schedulers: Sharing the Cluster	156
5.3.1	The FIFO Scheduler	156
5.3.2	The Capacity Scheduler	156
5.3.3	The Fair Scheduler	157
5.3.4	Dominant Resource Fairness	157
5.3.5	YARN Fault Tolerance	158
5.4	Row-Oriented vs. Columnar Storage	159
5.4.1	Row-Oriented Storage	160
5.4.2	Columnar Storage: The I/O Reduction Formula	160
5.5	Big Data File Formats in Depth	161
5.5.1	Apache Avro: Schema-Evolving Row Storage	162
5.5.2	Apache Parquet: Columnar Analytics Storage	162
5.5.3	Apache ORC: Hive-Optimised Columnar Storage	163
5.6	Compression Codecs and Splittability	165
5.6.1	The Splittability Property	165
5.6.2	Compression Codec Comparison	165
5.7	I/O Optimisation Techniques	166
5.7.1	Predicate Pushdown	166
5.7.2	Column Pruning	167
5.7.3	Partition Pruning	168
5.8	Case Study: LinkedIn’s Multi-Framework YARN Cluster (2013–2016)	169

IV	In-Memory Processing	185
6	Apache Spark: In-Memory Distributed Computing	187
6.1	The MapReduce Iteration Problem	189
6.2	Resilient Distributed Datasets	191
6.2.1	Lineage-Based Fault Tolerance	191
6.2.2	Lazy Evaluation and the RDD DAG	192
6.2.3	RDD Persistence	193
6.3	Spark Architecture	193
6.3.1	The Driver Program and SparkSession	194
6.3.2	Executors	195
6.3.3	Cluster Managers	195
6.3.4	DAG Scheduler and Task Scheduler	196
6.4	Transformations and Actions	196
6.4.1	Narrow Transformations	197
6.4.2	Wide Transformations and the Shuffle	198
6.4.3	groupByKey vs. reduceByKey: A Critical Distinction	199
6.4.4	Actions	200
6.5	Spark SQL: DataFrames, Datasets, and the Catalyst Optimizer	201
6.5.1	DataFrames and Datasets	201
6.5.2	The Catalyst Optimizer	202
6.5.3	SparkSQL and Hive Integration	204
6.6	MLlib: Distributed Machine Learning	204
6.6.1	Core Abstractions: Transformer, Estimator, and Pipeline	204
6.6.2	Key MLlib Algorithms	206
6.7	Memory Management in Spark	207
6.7.1	The Unified Memory Model	207
6.7.2	Serialisation and Kryo	208
6.7.3	Data Skew and Salting	209
6.8	Research Spotlights	210
6.9	Case Study: Alibaba's Spark Deployment for Singles' Day (11.11)	213
7	SQL at Scale: HiveQL and the SQL-on-Hadoop Ecosystem	229
7.1	HiveQL: Advanced Language Features	231
7.1.1	Complex Data Types: ARRAY, MAP, and STRUCT	231

7.1.2	LATERAL VIEW and EXPLODE	232
7.1.3	Window Functions	233
7.1.4	User-Defined Functions	235
7.2	HiveServer2 and Concurrent Access	236
7.3	Execution Engine Evolution: MapReduce, Tez, and LLAP . .	237
7.3.1	Hive-on-MapReduce: The Baseline	237
7.3.2	Apache Tez: DAG-Based Execution	237
7.3.3	Hive LLAP: In-Memory Interactive Queries	239
7.4	Query Optimisation in Hive	240
7.4.1	Rule-Based vs. Cost-Based Optimisation	240
7.4.2	Join Execution Strategies	241
7.4.3	Vectorized Execution	242
7.5	Presto and Trino: SQL on Everything	243
7.5.1	Presto Architecture	243
7.5.2	The Connector Model: SQL on Everything	244
7.5.3	Presto vs. Hive: Design Philosophy	245
7.6	The SQL-on-Hadoop Ecosystem	247
7.6.1	Apache Impala	247
7.6.2	Comparative Analysis	247
7.7	Case Study: Airbnb’s Migration to a Unified SQL Platform .	249
V	Data Pipelines and Modern Integration	267
8	Data Pipelines, Data Lakes, and Real-Time Integration	269
8.1	From Batch ETL to Streaming Pipelines	271
8.1.1	The N^2 Integration Problem	271
8.1.2	The Data Integration Tool Landscape	273
8.2	Batch Integration: Apache Sqoop and Apache Flume	273
8.2.1	Apache Sqoop: Bulk RDBMS–HDFS Transfer	274
8.2.2	Apache Flume: Log Ingestion Pipelines	276
8.3	Apache Kafka: The Distributed Append-Only Log	278
8.3.1	The Core Log Abstraction	278
8.3.2	Kafka Architecture: Topics, Partitions, Offsets, and Brokers	280
8.3.3	Producers, Consumers, and Consumer Groups	281

8.3.4	Kafka Throughput: Sequential I/O and Zero-Copy Transfer	283
8.3.5	Retention, Replay, and Exactly-Once Semantics	284
8.4	Lambda Architecture	286
8.4.1	The Three-Layer Design	286
8.4.2	Strengths and the Dual-Code-Path Problem	286
8.5	Kappa Architecture	288
8.5.1	Stream-Only Processing and Log Replay	288
8.5.2	Lambda vs. Kappa: Comparison and Selection	291
8.6	Data Lakes and the Data Swamp Risk	291
8.6.1	Data Lake Architecture	292
8.6.2	The Data Swamp Problem and Governance	292
8.6.3	Table Formats: Delta Lake and Apache Iceberg	294
8.7	Research Spotlight	297
8.8	Case Study: Netflix Keystone Real-Time Data Pipeline	298

VI Insight Delivery 311

9 Data Visualization at Scale 313

9.1	Why Visualization Matters: From Summary to Insight	315
9.1.1	Anscombe's Quartet	315
9.1.2	Visualization in the Big Data Context	316
9.2	Tufte's Principles of Analytical Design	316
9.2.1	The Data-Ink Ratio	317
9.2.2	The Lie Factor	318
9.2.3	Small Multiples and Sparklines	319
9.3	Shneiderman's Visual Information Seeking Mantra	320
9.3.1	The Three-Step Mantra	320
9.3.2	Seven Data Types and Their Visualizations	321
9.4	Big Data Visualization Challenges	322
9.4.1	Challenge 1: Overplotting at Scale	322
9.4.2	Challenge 2: Latency and Pre-Aggregation	325
9.4.3	Challenge 3: Streaming Visualization	327
9.5	The Visualization Tools Landscape	328
9.5.1	Tableau and Power BI: Business Intelligence Platforms	328

9.5.2	D3.js: Data-Driven Documents	330
9.5.3	Grafana and Apache Superset: Open-Source Platforms	332
9.6	Research Spotlight	333
9.7	Case Study: The New York Times Graphics Desk	335
10	Domain Applications: Social Networks, Time Series, and Health-	
	care	349
10.1	Three Domains of Big Data Application	351
10.2	Domain A: Social Media Analytics	351
10.2.1	Graph Structure and Scale	351
10.2.2	Community Detection: The Louvain Algorithm . . .	354
10.2.3	Influence Maximisation	356
10.2.4	Sentiment Analysis: VADER	357
10.2.5	Trend Detection and the Twitter News-Media Finding	360
10.3	Domain B: Time Series Analysis	361
10.3.1	Properties of Time Series Data	361
10.3.2	Classical Forecasting: ARIMA	362
10.3.3	Facebook Prophet: Decomposable Forecasting at Scale	363
10.3.4	Anomaly Detection in Time Series	365
10.3.5	Vectorised Batch Forecasting in Spark	368
10.4	Domain C: Healthcare Big Data	370
10.4.1	Healthcare Data Modalities	370
10.4.2	Clinical Prediction Models	372
10.4.3	Privacy: HIPAA, GDPR, and the CCPA	373
10.4.4	AlphaFold: Big Data and Deep Learning in Science .	374
10.5	Research Spotlight	376
10.6	Case Study: CDC Syndromic Surveillance and the Google Flu Trends Lesson	379
VII	The Modern Data Stack	391
11	The Lakehouse, MLOps, and Emerging Architectures	393
11.1	The Technology Evolution Arc	395
11.2	Lakehouse Architecture	395
11.2.1	The Two-System Problem	397
11.2.2	Delta Lake: ACID on Object Stores	397

11.2.3	Apache Iceberg and Apache Hudi	402
11.3	Data Mesh: Decentralised Domain Ownership	404
11.3.1	The Central Platform Bottleneck	404
11.3.2	The Four Data Mesh Principles	405
11.4	MLOps: Operationalising Machine Learning	406
11.4.1	Hidden Technical Debt in ML Systems	406
11.4.2	The MLOps Pipeline	407
11.4.3	Feature Stores and Training–Serving Skew	410
11.5	Edge Analytics and Cloud-Native Big Data	411
11.5.1	The Three-Tier Edge–Cloud Architecture	411
11.6	Privacy-Preserving Analytics	413
11.6.1	Differential Privacy	413
11.6.2	Federated Learning	415
11.7	The Dataflow Model and Apache Beam	416
11.7.1	The Four Questions	416
11.7.2	Apache Beam: One Pipeline, Multiple Runners	418
11.8	Research Spotlight	420
11.9	Case Study: Enterprise Lakehouse Migration in Production	423

References	435
-------------------	------------

Part I

The Data Engineering Landscape

The Big Data Revolution: Scale, Velocity, and Variety

"The world's most valuable resource is no longer oil, but data."

The Economist, 2017

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why traditional relational database systems cannot handle modern data workloads at scale, and identify the specific limits that are exceeded.
2. Define the three original dimensions of big data (Volume, Velocity, Variety) and the two additional dimensions (Veracity, Value) that form the extended 5Vs framework.
3. Classify any given data source as structured, semi-structured, or unstructured, and identify the appropriate storage and query tool for each category.
4. Distinguish between vertical and horizontal scaling strategies, and explain why distributed, horizontally scaled systems dominate big data engineering.
5. State the CAP theorem, explain the meaning of each property, and

identify how real-world systems (HBase, Cassandra, PostgreSQL) position themselves in the CAP triangle.

6. Compare batch and stream processing models and select the appropriate model for a given engineering requirement.

Every 60 seconds in 2024, users upload 500 hours of video to YouTube, execute more than 5 million Google searches, send over one million WhatsApp messages, and publish approximately 6,000 posts on X (formerly Twitter). These figures are not curiosities; they represent a fundamental transformation in the scale, speed, and diversity of data that computing infrastructure is expected to handle. This chapter establishes the conceptual and technical foundation for the entire book. We begin by examining what makes data “big,” explore how that bigness is formally characterized, and introduce the core theoretical constraints—the CAP theorem in particular—that shape every architectural decision in data engineering.

1.1 Why Data Engineering Matters

In 2018, the International Data Corporation (IDC) projected that the global *datasphere*—the total volume of data generated, captured, copied, and consumed worldwide—would reach **175 zettabytes** by 2025, representing a 23-fold increase over the 7.2 ZB that existed in 2018 (Reinsel, Gantz, and Rydning 2018). To appreciate this scale, consider that one zettabyte equals 10^{21} bytes, or roughly 250 billion standard 4 TB hard drives—more drives than there are stars visible to the naked eye from any single point on Earth.

Traditional *relational database management systems* (RDBMS)—the dominant paradigm since Edgar Codd’s foundational work in 1970—are extraordinary tools for managing well-structured, bounded datasets. A modern relational database running on a well-provisioned server can comfortably manage tens of terabytes and sustain thousands of transactions per second with full ACID guarantees. For a bank’s transaction ledger, a hospital’s patient registry, or an airline’s reservation system, a carefully tuned relational system remains the correct choice. The problem emerges when

Table 1.1: Data volume units and their relationship to everyday storage

Unit	Bytes	Illustrative equivalent
Kilobyte (KB)	10^3	A short email (plain text)
Megabyte (MB)	10^6	A 3-minute MP3 audio track
Gigabyte (GB)	10^9	A feature-length HD film
Terabyte (TB)	10^{12}	~310,000 photos from a 12 MP camera
Petabyte (PB)	10^{15}	All US academic research libraries combined
Exabyte (EB)	10^{18}	All words ever spoken by humans (estimated)
Zettabyte (ZB)	10^{21}	2025 global datasphere
Yottabyte (YB)	10^{24}	Theoretical future scale

data characteristics exceed what any single machine can handle: when datasets are measured in petabytes rather than terabytes; when data arrives not from a single transactional application but from millions of concurrent event streams; and when the data itself is not neatly organized into rows and columns but exists as images, audio recordings, sensor telemetry, and free-form text. In these conditions, the fundamental assumptions of relational database design break down.

Data engineering is the discipline concerned with the design, construction, and maintenance of systems that collect, store, transform, and serve large-scale data for downstream consumers—analysts, machine learning models, business intelligence dashboards, and operational applications. A data engineer builds the infrastructure that makes raw data useful. This field draws on distributed systems, software engineering, database theory, and applied computer science. The tools of modern data engineering—distributed file systems, parallel batch processors, in-memory computation engines, stream processing frameworks, and pipeline orchestrators—are the subjects of this book.

Key Insight | The Data Engineering Mandate

Data engineering is not about *generating* insight—that is the domain of data science and analytics. Data engineering is about making data *available, reliable, and efficiently accessible* at any scale. The data engineer builds the plumbing; analysts and scientists use the water.

1.2 Defining Big Data: The 3Vs Framework

The term “big data” gained formal structure in 2001 when Doug Laney, then a research analyst at the META Group, published a brief but enormously influential research note identifying three dimensions along which data management challenges had outpaced conventional tools (Laney 2001). He labeled these dimensions *Volume*, *Velocity*, and *Variety*—the **3Vs of big data**. Eleven years later, Gartner (which had acquired META Group) codified these dimensions into a formal definition:

Definition | Big Data (Gartner/Beyer & Laney, 2012)

“Big data is high-volume, high-velocity and/or high-variety information assets that demand cost-effective, innovative forms of information processing that enable enhanced insight, decision making, and process automation.”
— Beyer & Laney, Gartner Research, 2012 (Beyer and Laney 2012)

Each of the three dimensions corresponds to a distinct class of engineering challenge. The McKinsey Global Institute, in its landmark 2011 study, reinforced this framing: big data refers to datasets “whose size is beyond the ability of typical database software tools to capture, store, manage and analyze” (Manyika et al. 2011). Let us examine each dimension in turn.

1.2.1 Volume: The Scale Dimension

Volume refers to the sheer quantity of data that is generated, accumulated, and must be stored and processed. Volume is the most intuitively obvious characteristic of big data: a dataset is voluminous when it exceeds the capacity of conventional tools—typically, when a single machine cannot

store and process it within an acceptable elapsed time. The following production examples illustrate current operational scales:

Table 1.2: Volume at production scale (representative 2023–2024 estimates)

Organisation	Volume characteristic
Meta (Facebook)	500 TB of new data ingested per day across all platforms
Google	>100 PB of data processed internally per day (estimated)
Walmart	>2.5 PB of customer transaction data in analytical systems
CERN (LHC)	≈15 PB of particle collision data generated per year
Netflix	>1 PB of user interaction events per day
NYSE	≈1 TB of trading records generated on a peak day

From an engineering perspective, volume creates several concrete challenges. First, no single server—even the highest-specification available in 2024, with up to 64 TB of RAM—can hold a petabyte-scale dataset in memory. Second, sequential reads from a single high-speed SSD array proceed at approximately 7 GB/s; reading one petabyte at that speed requires roughly 41 hours. Third, ingesting 500 TB per day means sustained write throughput of approximately 5.8 GB/s, which no single storage device can sustain reliably. The engineering response is *horizontal distribution*: partition data into blocks and spread those blocks across a cluster of hundreds or thousands of commodity machines—the approach formalized by the Google File System (Ghemawat, Gobioff, and Leung 2003) and, in open-source form, by Hadoop’s HDFS (discussed in Chapter 3).

1.2.2 Velocity: The Speed Dimension

Velocity refers to the speed at which data is generated, arrives at a system, and must be processed or acted upon. High-velocity data is not merely large; it arrives *continuously* and often demands responses on timescales far

shorter than batch processing can provide.

Consider a few representative velocities:

- The Visa payment network processes approximately 24,000 transactions per second at peak load.
- Global IoT deployments generate over one billion device sensor readings per hour across industrial, environmental, and consumer applications.
- A ride-sharing platform (such as Uber or Lyft) receives a GPS ping every three seconds from millions of simultaneously active drivers—at 5 million drivers, that is over 1.6 million location records per second.
- Modern stock exchanges publish quote updates for listed securities at rates exceeding one million messages per second during high-activity sessions.

The engineering challenge of velocity is distinct from that of volume: a system may need to respond to each event within milliseconds (fraud detection, payment authorization) or may tolerate latencies of seconds (real-time dashboards). The key constraint is that high-velocity data *cannot wait* for a nightly batch process—by the time the batch runs, the window for action has closed. This distinction drives the separation between *batch* and *stream* processing architectures, explored in Section 1.7 and in depth in Chapter 8.

Table 1.3: The data processing spectrum by velocity requirement

Processing mode	Latency	Representative tools	Typical use case
Batch	Hours	Hadoop MapReduce, Hive	Nightly ETL, monthly billing
Micro-batch	Seconds	Spark Streaming	Near-real-time dashboards
Stream	Milliseconds	Apache Flink, Kafka Streams	Fraud detection, alerts
Real-time	Sub-ms	In-memory + Kafka	Payment authorization

1.2.3 Variety: The Diversity Dimension

Variety refers to the diversity of data formats, types, schemas, and sources that must be jointly ingested, stored, and analysed. Modern organisations do not operate a single data source with a uniform schema; they accumulate data from dozens of systems, each with its own format and semantics.

Research Spotlight | Laney (2001) — The Origin of the 3Vs

Citation: Laney, D. (2001). *3D Data Management: Controlling Data Volume, Velocity and Variety*. META Group Research Note 949, February 2001. (Laney 2001)

Key insight: Laney’s one-page note, written in the context of e-commerce data challenges, framed data management as a three-dimensional problem—not merely a storage problem. He argued that organisations failing to address all three dimensions would face “increasing management headaches.”

Technical contributions:

- Established the first formal vocabulary (3Vs) for characterising modern data management challenges, unifying what had previously been treated as unrelated operational problems.
- Distinguished *velocity* as a management dimension independent of volume—recognising that *how fast* data arrives is as important as *how much* arrives.
- Introduced *variety* as the hardest dimension to manage, predicting the rise of semi-structured and unstructured data as dominant forms.

Impact today: The 3Vs framework is taught at every major computer science programme and has been cited in thousands of academic papers and industry reports. It remains the canonical definition of big data, even as the framework has been extended to 5Vs and beyond. Gartner officially adopted and extended the definition in 2012.

Variety manifests along several axes. *Format variety* arises when data arrives as CSV files, JSON documents, XML feeds, binary Avro records, columnar Parquet files, DICOM medical images, and MP4 video simultaneously—all of which must be stored and queried together. *Source variety* arises when the same analytical question requires joining data from relational databases,

REST APIs, IoT sensor streams, social media firehoses, and internal event logs. *Semantic variety* arises when a field named date means the transaction date in one system, the record-entry date in another, and the customer birth date in a third.

The foundational distinction among data types—structured, semi-structured, and unstructured—is the key conceptual framework for managing variety, and we explore it in detail in Section 1.4.

1.3 The Extended 5Vs Framework

The 3Vs framework is powerful but incomplete. Two additional dimensions have gained wide acceptance, bringing the framework to five:

Veracity refers to the trustworthiness and quality of data. High-veracity data is accurate, consistent, and free of critical errors; low-veracity data is noisy, inconsistent, or of uncertain provenance. Veracity challenges arise in practical systems for many reasons: GPS sensors drift by tens of metres in dense urban environments; optical character recognition (OCR) on historical documents introduces transcription errors; social media platforms contain significant volumes of bot-generated content; and sensor networks may experience intermittent failures that produce spurious or missing readings. The consequences of low veracity can be severe—a machine learning model trained on inaccurate data will make inaccurate predictions, regardless of how sophisticated the model or how large the training set.

Value refers to the business or societal utility that can be extracted from data. It is the ultimate justification for the considerable engineering investment required to build big data systems. A dataset may score highly on all four other Vs—petabytes of high-velocity, highly varied, high-quality data—and still produce no actionable insight if the right analytical questions are never asked, or if the results are never operationalised. Value is the bridge between technical capability and organisational outcomes.

Research Spotlight | Quantifying the Value Dimension

Citation: Manyika, J. et al. (2011). *Big Data: The Next Frontier for Innovation, Competition, and Productivity*. McKinsey Global Institute, May 2011. (Manyika et al. [2011](#))

Key insight: The report argued that big data has become “a new factor of production,” alongside capital and labour—that is, organisations with superior data capabilities will, over time, systematically outperform those without them, in the same way that access to capital separates competitive enterprises from struggling ones.

Technical and economic contributions:

- Quantified the value potential across five major sectors: healthcare (\$300B+ annually in the US), public sector (\$250B+ in Europe), retail (>60% operating margin improvement for early adopters), manufacturing, and personal location data.
- Identified the acute shortage of data-skilled talent as the primary constraint on value realisation—the report projected a shortfall of 140,000–190,000 professionals with deep analytical expertise in the US alone by 2018.
- Introduced the concept of “data as infrastructure,” arguing that the economic benefits of big data investment are broadly distributed across the economy, much like roads or electricity grids.

Impact today: This report is one of the most cited documents in the history of data-driven business strategy. It directly influenced technology investment priorities at major corporations and governments worldwide, and it validated the creation of dedicated data engineering and data science functions within large organisations.

1.4 A Taxonomy of Digital Data

Managing the Variety dimension of big data requires a clear classification of data types. The field has converged on three primary categories—structured, semi-structured, and unstructured—distinguished principally by the presence, flexibility, and location of their schema.

Table 1.4: The 5Vs framework with engineering implications

Dimension	Definition	Production example	Primary engineering challenge
Volume	Scale of data generated and stored	Meta: 500 TB/day ingested	Distributed storage (HDFS, S3)
Velocity	Speed of generation and processing	Visa: 24,000 tx/s	Stream processing (Kafka, Flink)
Variety	Diversity of formats and sources	CSV + JSON + DICOM + MP4	Schema integration, ETL/ELT
Veracity	Trustworthiness of data quality	GPS drift, OCR errors, bot tweets	Data quality pipelines, validation
Value	Business utility extracted from data	Fraud prevented, revenue gained	Feature engineering, ML pipelines

1.4.1 Structured Data

Definition | Structured Data

Data organised according to a **predefined, rigid schema** in which every record contains the same fields in the same order, with each field having a specified data type. The schema is defined and enforced before data is written (**schema-on-write**). Structured data is typically stored in relational database management systems and queryable directly with standard SQL.

Structured data represents the most mature and well-understood category. Relational database theory, developed by Codd (1970) and refined over five decades, provides powerful tools for storing, querying, and maintaining the integrity of structured data. A bank’s transaction ledger is a

canonical example: every row has an account number, a timestamp, an amount, a currency code, and a transaction type. There are no optional fields, no embedded documents, no ambiguous formats. SQL can answer virtually any analytical question over such data with high efficiency.

The primary limitation of structured data is its rigidity. Adding a new column to a table with hundreds of millions of rows requires a schema migration—a costly, time-consuming, and potentially disruptive operation. More fundamentally, many of the most valuable data types in modern organisations—customer reviews, support transcripts, medical images, sensor streams—cannot be naturally represented in rows and columns without destructive simplification. Despite this, structured data remains the bedrock of transactional systems, and big data engineering tools like Apache Sqoop exist specifically to migrate structured data from RDBMS sources into distributed analytical systems (covered in Chapter 8).

In terms of global prevalence, structured data accounts for only approximately 20% of all digital data—yet it represents the most well-governed, analysable slice of that data.

1.4.2 Semi-Structured Data

Definition	Semi-Structured Data
------------	----------------------

Data with a flexible, self-describing schema embedded within the data itself. Fields are identified by keys or tags that travel with the record, not by position in a fixed row. New fields can be added without modifying a central schema registry. Semi-structured data is typically stored as files or in NoSQL databases and processed with a schema-on-read approach: the schema is imposed at query time, not at write time.
--

Semi-structured data dominates modern web and application architectures. The two most prevalent formats are **JSON** (JavaScript Object Notation) and **XML** (Extensible Markup Language):

```
1 {  
2   "event_id":    "e-928471",  
3   "event_type":  "page_view",  
4   "timestamp":   "2024-03-15T14:22:31Z",
```

```
5  "user": {
6    "id":      "u-441982",
7    "tier":    "premium"
8  },
9  "page":      "/product/laptop-x500",
10 "session_ms": 4820,
11 "metadata": {
12   "user_agent": "Mozilla/5.0 ...",
13   "referrer":   "https://search.example.com"
14 }
15 }
```

Listing 1.1: A JSON event record from a streaming API

Notice that the JSON record carries its own field names (“event_id”, “timestamp”, “user”, etc.) directly within the data. A consumer reading this record does not need to consult an external schema definition to know what each field means. This *self-description* property makes semi-structured data highly flexible: a downstream service that only cares about event_type and user.tier can simply ignore all other fields. When a new field (e.g., ab_test_variant) must be added, it can be appended to future records without invalidating existing ones.

The schema-on-read approach enabled by semi-structured formats is central to the *data lake* architecture: data is ingested in its native format without transformation, and the schema is applied by the analytical tool at query time. This defers the cost of schema design but introduces the risk of schema drift—where the meaning or format of a field changes silently between producers, creating silent data quality failures. Managing this risk (through schema registries, contract testing, and data lineage tracking) is a significant part of production data engineering practice.

Web server access logs, IoT device telemetry published via MQTT, REST API responses, email headers, configuration files in YAML or TOML, and Apache Avro records with embedded schemas all belong to the semi-structured category. This category accounts for roughly 10% of global data by volume.

1.4.3 Unstructured Data

Definition	Unstructured Data
------------	-------------------

Data with no predefined schema and no embedded key-value structure that can be directly queried. The semantic content is encoded in the raw signal—pixels, audio samples, token sequences—and cannot be accessed by a relational or document query engine without substantial preprocessing (feature extraction, natural language processing, or computer vision).
--

Unstructured data is the dominant form of digital data, accounting for an estimated 70–80% of the global datasphere. It encompasses text documents (news articles, legal contracts, clinical notes, social media posts), images (medical scans, satellite photography, product photographs), audio recordings (call-centre conversations, podcasts, speech-enabled device logs), video (surveillance footage, user-generated content, manufacturing inspection recordings), and raw sensor streams (seismic signals, EEG waveforms, raw GPS trajectories).

The defining challenge of unstructured data is that extracting information requires *interpretation*—a step that structured and semi-structured data largely skip. A relational database can compute the average transaction amount in a single SQL aggregation; answering the question “what is the sentiment of customer reviews mentioning our new product?” requires tokenising text, building or loading a language model, running inference, and aggregating the model’s output. This computational chain is far more complex and resource-intensive, which is why the tools introduced in later chapters—Spark MLlib, distributed deep learning frameworks, and specialised feature stores—are essential components of modern data engineering systems.

1.5 The Engineering Response to Scale

Given the 3Vs, the engineering question is: how do we build systems capable of storing petabytes, processing millions of events per second, and integrating dozens of heterogeneous data sources? The answer, which runs

Table 1.5: Comparison of data type characteristics and engineering implications

Dimension	Structured	Semi-structured	Unstructured
Schema	Predefined, rigid	Self-describing, flexible	Absent
Schema timing	On write	On read	N/A
Storage	RDBMS tables	Files, NoSQL, APIs	Object stores, HDFS
Query tool	SQL	JSONPath, Hive, Spark SQL	NLP, CV, Spark MLlib
Global share	~20%	~10%	~70–80%
Schema change cost	High	Low	N/A
Typical ingestion tool	Sqoop	Kafka, Flume	Custom pipelines
Examples	Bank ledger, ERP records	REST API JSON, server logs	MRI scan, audio call

through every chapter of this book, reduces to a small set of foundational principles.

Common Misconception | JSON is Semi-Structured, Not Unstructured

A frequent and consequential error: classifying JSON or XML as “unstructured” because it does not look like a database table. JSON and XML carry an embedded schema in the form of key names and hierarchical nesting. A query engine can navigate this structure directly (using JSONPath, XPath, or Hive’s schema-on-read). An MRI scan, a JPEG image, or an MP3 recording carries *no* such navigable schema—its semantic content is encoded in the binary signal itself, requiring a domain-specific model (computer vision, speech recognition) to interpret. The distinction matters architecturally: semi-structured data can be stored and queried in a data lake with minimal transformation; unstructured data requires a preprocessing pipeline before it becomes

analytically useful.

1.5.1 Vertical vs. Horizontal Scaling

There are two ways to make a computing system more powerful. **Vertical scaling** (scaling up) means adding more resources to a single machine: a faster processor, more RAM, a larger or faster storage array. Vertical scaling is simple—it introduces no distributed-systems complexity—but it has a hard physical ceiling. The most powerful single-node servers available in 2024 top out at approximately 64 TB of RAM and a handful of high-throughput storage controllers. Above this limit, no amount of money can buy a single machine large enough to hold the dataset. Vertical scaling is also exponentially expensive: a server with twice the RAM costs approximately eight times as much.

Horizontal scaling (scaling out) means adding more machines to a cluster and distributing the data and computation across all of them. Each individual machine is a commodity server costing between three and eight thousand dollars. The power of the cluster grows nearly linearly with the number of nodes added: doubling the number of nodes approximately doubles the throughput. Failure of a single node—which is expected, not exceptional, in a large cluster—does not halt the system, because data is replicated across multiple nodes.

1.5.2 The Commodity Hardware Insight

The observation that underpins all of distributed data engineering is deceptively simple: *commodity hardware will fail, and systems must be designed with this expectation from the start*. In a cluster of 1,000 commodity servers, with each server having a disk failure rate of 1% per year, we expect approximately 10 disk failures per year—about one every five weeks. At 10,000 nodes (the scale of Yahoo!’s first production Hadoop cluster in 2008), this becomes roughly one disk failure per day. The solution is not to prevent failures—that is neither economically nor physically feasible—but to design for them. Every piece of data is stored in multiple copies on different nodes; if one node fails, the data remains available from the surviving copies, and the system automatically creates a new replica on a healthy

Table 1.6: Vertical vs. horizontal scaling: a systematic comparison

Attribute	Vertical (Scale Up)	Horizontal (Scale Out)
Scalability ceiling	Hard physical limit (~64 TB RAM)	Near-unlimited (add nodes)
Cost scaling	Superlinear (exponential)	Near-linear per node
Fault model	Single point of failure	Tolerates individual node failures
Complexity	Low (no distributed systems)	High (consistency, coordination)
Latency for small queries	Low	Higher (network overhead)
Typical cost example	Oracle Exadata: >\$500K/yr	1,000 nodes × \$5K = \$5M (one-time)
Used by	Traditional RDBMS, data warehouses	Hadoop, Spark, Cassandra, Kafka

node. This design principle, first articulated for distributed file systems by Google’s engineers (Ghemawat, Gobioff, and Leung 2003), is the cornerstone of HDFS, Kafka, and Cassandra alike.

1.5.3 Moving Computation to Data

The second foundational principle of distributed data processing inverts the traditional computing model. In a conventional system, data is moved to the processor: a query is issued, data is fetched from storage into RAM, and the CPU processes it. This model fails at petabyte scale because the network bandwidth required to move large amounts of data to a central processor becomes the bottleneck—the network simply cannot move data fast enough to keep the processor busy.

The solution, pioneered by Google’s MapReduce system (Dean and Ghe-

mawat 2004) and implemented in every major big data processing framework, is to *move computation to the data*: send the processing logic (a function or query) to the node where the relevant data blocks reside, execute the computation locally, and collect only the (smaller) results. This principle—**data locality**—drastically reduces network traffic and enables the aggregate CPU capacity of an entire cluster to be applied in parallel to a large dataset.

System Design Note | Data Locality in Practice

In HDFS (Chapter 3), a 128 MB data block is stored on three nodes. When a MapReduce or Spark job must process that block, the scheduler first attempts to assign the task to one of the three nodes already holding a replica (task-local scheduling). If those nodes are busy, it next tries a node on the same physical rack (rack-local scheduling). Only if no rack-local node is available does it resort to sending the data over the network. In production clusters, over 90% of tasks execute with data locality, meaning almost all processing is local I/O rather than network transfer.

1.6 The CAP Theorem

Building a distributed system—one that spans multiple machines connected by a network—introduces a fundamental theoretical constraint that shapes every storage and database design decision in data engineering. This constraint is captured by the *CAP theorem*.

1.6.1 Statement and Definitions

The CAP theorem was conjectured by Eric Brewer in his keynote address at the ACM Symposium on Principles of Distributed Computing in 2000 (Brewer 2000) and formally proved by Gilbert and Lynch two years later (Gilbert and Lynch 2002). It states:

Definition | The CAP Theorem (Brewer, 2000; Gilbert & Lynch, 2002)

A distributed data system can simultaneously guarantee **at most two** of the following three properties:

1. **Consistency (C):** Every read operation returns the most recent successfully written value, or an error. All nodes in the system see the same data at the same time.
2. **Availability (A):** Every request to a non-failed node receives a response—not guaranteed to be the most recent write, but a response nonetheless.
3. **Partition Tolerance (P):** The system continues to operate correctly even if an arbitrary number of messages between nodes are lost or delayed (a *network partition*).

Understanding what each property means in operational terms is essential for reasoning about system design. *Consistency*, in the CAP sense, is not the same as the C in ACID transactions (which refers to maintaining database invariants); CAP consistency is *linearisability*—the guarantee that every read sees the most recent write, as if all operations occurred on a single machine. *Availability* means every request gets a response; it does not mean the response contains current data. *Partition tolerance* means the system continues to function when the network splits into disconnected sub-clusters.

1.6.2 The Practical Implication: Choosing C vs. A

The critical practical insight is that network partitions in distributed systems are not theoretical edge cases—they are *inevitable*. Any system operating across multiple machines connected by a real network will, at some point, experience a partition: a switch failure, a misconfigured firewall rule, a miscabling in a data centre, or a brief network congestion event that causes messages to be delayed or dropped. Therefore, in any real distributed system, partition tolerance (P) is not optional. A system that cannot tolerate partitions is effectively a single-machine system.

Given that P is always required, the CAP theorem reduces to a binary choice: when a partition occurs, should the system prioritise **consistency** (refuse requests that cannot be answered with current data, guaranteeing accuracy) or **availability** (answer every request with possibly stale data, guaranteeing responsiveness)? This choice is one of the most important architectural decisions in data engineering.

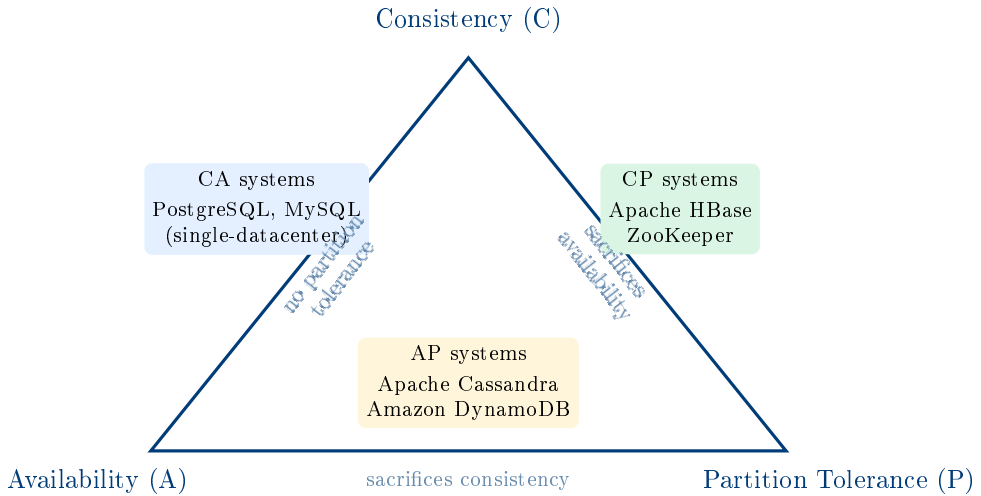


Figure 1.1: The CAP triangle: every distributed system chooses two of the three properties. Because network partitions are unavoidable, real systems choose between CP and AP.

Research Spotlight | Brewer (2000) and Gilbert & Lynch (2002) — The CAP Theorem

Citations:

- Brewer, E.A. (2000). *Towards Robust Distributed Systems*. Keynote, PODC 2000. (Brewer 2000)
- Gilbert, S. & Lynch, N.A. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. *ACM SIGACT News*, 33(2):51–59. (Gilbert and Lynch 2002)

Key insight: Brewer's conjecture, presented as intuition in 2000, was that three desirable properties of distributed systems could not all be simultaneously guaranteed. Gilbert and Lynch provided a formal proof two years later, elevating the observation from engineering folklore to a mathematically rigorous theorem.

Technical contributions:

- Provided the formal definitions of consistency (linearisability), availability, and partition tolerance used in distributed systems theory.
- Proved that no distributed system can achieve all three properties simultaneously using an asynchronous network model.

Table 1.7: CAP positioning of representative data systems

System	CAP	Design rationale
PostgreSQL, MySQL	CA	Designed for single-node or tightly coupled clusters; cannot tolerate true network partitions
Apache HBase	CP	Chooses consistency: if a partition occurs, HBase may become unavailable rather than serve stale data
Apache ZooKeeper	CP	Coordination service where serving stale configuration data would be catastrophic
Apache Cassandra	AP	Chooses availability: always responds, using tunable eventual consistency (read your own writes)
Amazon DynamoDB	AP	Optimised for always-on e-commerce workloads where availability is paramount

- Established the vocabulary (CA, CP, AP) that data architects use to describe and compare distributed systems.

Nuance and evolution: The CAP theorem is often oversimplified. Eric Brewer himself, in a 2012 retrospective, noted that the choice is not always a binary C-vs-A decision: systems like DynamoDB offer tunable consistency levels, allowing different operations to trade consistency for availability at the operation level. The PACELC model (Patterson et al., 2012) extends CAP by noting that even without a partition, systems must trade latency for consistency—a trade-off CAP does not capture. Despite these refinements, the CAP theorem remains the most widely cited result in distributed systems theory.

1.7 Batch vs. Stream Processing

The Velocity dimension of big data—the need to process data that arrives continuously and at high rates—has driven the development of two fundamentally different processing paradigms that data engineers must understand and choose between.

1.7.1 Batch Processing

Batch processing operates on a fixed, bounded dataset accumulated over a period of time. The data is first stored in full (on disk, in a distributed file system, or in a data warehouse), and then a computation—the *batch job*—is run over the entire stored dataset. Classic examples include nightly financial reconciliation, end-of-month billing, weekly machine learning model retraining, and annual census analysis.

Batch processing has several important properties. First, it achieves very high *throughput*: because the entire dataset is available before processing begins, the job can be optimised for sequential I/O, sorted reads, and massive parallelism. Second, batch jobs are inherently *reproducible*: re-running the same job on the same input data will always produce the same output—a property critical for auditing and debugging. Third, batch jobs can be *fault-tolerant* in a simple way: if a task fails, it can be restarted from the last checkpoint without affecting the rest of the job. Fourth, and most importantly for our purposes, batch processing has *latency measured in hours*: a nightly ETL job that begins at 02:00 and finishes at 06:00 delivers results four hours after the data cutoff. For applications that need to respond to data within seconds or milliseconds, this latency is unacceptable.

1.7.2 Stream Processing

Stream processing operates on an unbounded, continuously arriving sequence of data records. Rather than accumulating data and running a batch job, a stream processing engine processes each record (or a small micro-batch of records) as it arrives, maintaining running state across the stream. The key characteristic is *low latency*: a stream processor can emit results within milliseconds to seconds of each event’s arrival.

Stream processing is the appropriate model for applications where the value of a response decays rapidly with time: real-time fraud detection (a stolen card must be blocked before the second transaction clears, not after nightly batch analysis); live stock market monitoring (an algorithmic trading system needs the current quote, not yesterday's close); operational dashboards (a call-centre manager needs to see current queue depth, not yesterday's average); and anomaly detection in industrial systems (a turbine failure signal is valuable in real time, not eight hours later).

Stream processing introduces engineering challenges that batch processing avoids. Records may arrive *out of order* (a mobile device's network connection may be intermittent, causing events to be delayed and arrive after later events). Events may carry an *event time* (when something happened) that differs from *processing time* (when the event arrived at the system). Stateful computations—for example, counting the number of page views per user in the past 60 minutes—must be maintained across an infinite stream, requiring careful memory management and state backends. Fault recovery requires mechanisms like *exactly-once processing* semantics, which add significant implementation complexity.

Key Insight | Lambda and Kappa Architectures

Many production systems need both low latency *and* high accuracy—properties that are difficult to achieve simultaneously. The **Lambda architecture** (Nathan Marz, 2011) addresses this by running both a batch layer (for accurate, comprehensive historical analysis) and a speed layer (for low-latency approximate results) in parallel, merging their outputs in a serving layer. The cost is engineering complexity: every transformation must be implemented twice, in different frameworks. The **Kappa architecture** (Jay Kreps, 2014) simplifies this by replacing the batch layer with a replay of the full event log through the stream processor—using Apache Kafka's message retention to make historical reprocessing possible through the streaming path. Both architectures are examined in depth in Chapter 8.

1.8 Case Study: Walmart's Retail Analytics Platform

Table 1.8: Batch vs. stream processing: a systematic comparison

Attribute	Batch processing	Stream processing
Data model	Bounded, stored dataset	Unbounded, continuously arriving events
Latency	Minutes to hours	Milliseconds to seconds
Throughput	Very high	Moderate (per-event overhead)
Fault recovery	Re-run from checkpoint	Exactly-once semantics required
State management	Simple (no persistent state)	Complex (stateful operators, windows)
Out-of-order data	Not a concern	Central challenge (watermarks)
Debugging	Easy (fixed inputs/outputs)	Hard (live, non-reproducible)
Typical tools	Hadoop MapReduce, Hive, Spark Batch	Kafka, Apache Flink, Spark Streaming
Use cases	ETL, ML training, billing	Fraud detection, monitoring, alerts

Case Study | Walmart — Petabyte-Scale Retail Intelligence

Background. Walmart is the world’s largest retailer by revenue, operating over 10,000 stores across 27 countries and serving approximately 230 million customers per week. The scale of its data generation is correspondingly vast: every store transaction, supply chain event, website interaction, and third-party demand signal contributes to what is one of the largest private-sector data repositories in existence.

Scale. Walmart’s internal data analytics environment holds over **2.5 petabytes** of customer transaction data in its core analytics systems, with an additional multi-petabyte tier of raw, unstructured data in its data lake. The company processes roughly one million customer transactions *per hour*—approximately 280 transactions per second on a typical day, with significant spikes during seasonal events (Thanksgiving weekend in the United States generates transaction volumes three to four times the average daily rate).

Data variety. Walmart’s analytical systems must integrate across all three data categories described in this chapter:

- **Structured:** Point-of-sale transaction records (item, quantity, price, timestamp, store ID, payment method) from 10,000+ stores, all originally stored in Oracle RDBMS systems.
- **Semi-structured:** Web and mobile clickstream events in JSON format (product views, search queries, cart additions, checkout abandonment events) from Walmart.com.
- **Unstructured:** Customer product reviews, call-centre transcript audio, and supplier-submitted product imagery for catalogue enrichment.

Engineering challenges and solutions. The 3Vs framework maps directly onto Walmart’s operational realities:

- **Volume:** Transaction data accumulated over decades cannot fit in any single RDBMS. Walmart migrated its analytical workloads to a Hadoop-based data lake, using HDFS for distributed storage and Apache Hive for SQL-style querying over PB-scale datasets.
- **Velocity:** Supply chain decisions—whether to redistribute inventory across distribution centres—must respond to real-time demand signals, not yesterday’s batch report. Walmart deployed Apache Kafka for real-time event streaming and uses stream processing for demand sensing.
- **Variety:** Integrating transaction records, clickstream events, weather data (a known driver of consumer behaviour), and social media sentiment requires a data integration pipeline that handles schema

mismatches, different temporal granularities, and inconsistent entity representations (the same product may have different identifiers in the RDBMS and the web catalogue).

Business outcomes. Walmart's investment in big data engineering has enabled demand forecasting accuracy that reduced food waste by 15–20% in its grocery divisions (reducing spoilage by anticipating demand more precisely), and dynamic pricing algorithms that run across its full product catalogue multiple times per day. During the COVID-19 pandemic, real-time analytics over social media and purchase patterns allowed Walmart to anticipate demand surges for sanitisers, masks, and canned goods several days before they peaked—enabling pre-positioning of inventory that proved critical when supply chains became constrained.

Lesson. Walmart's evolution from single-RDBMS analytics to a multi-petabyte distributed data platform illustrates the core thesis of this book: the 3Vs are not abstract concepts but concrete engineering constraints that force specific architectural choices. Every tool introduced in subsequent chapters exists because organisations like Walmart encountered these constraints and could not solve them with conventional technology.

Chapter Summary

This chapter established the conceptual and theoretical foundations for the rest of the book. The key points are:

- The global datasphere is projected to reach 175 ZB by 2025, driven by social media, IoT, commerce, and scientific instrumentation. Traditional RDBMS cannot manage data at this scale.
- **Data engineering** is the discipline of building infrastructure that makes large-scale data available, reliable, and efficiently queryable for downstream consumers.
- The **3Vs framework** (Laney, 2001) characterises big data along three

dimensions: *Volume* (scale), *Velocity* (speed), and *Variety* (diversity). The extended **5Vs** adds *Veracity* (data quality) and *Value* (business utility).

- Digital data falls into three categories: **structured** (predefined schema, schema-on-write, ~20% of global data), **semi-structured** (self-describing schema, schema-on-read, ~10%), and **unstructured** (no schema, requires ML/NLP/CV to interpret, ~70–80%).
- **Horizontal scaling** (adding commodity nodes) is the dominant strategy for big data infrastructure because it is cost-effective, fault-tolerant, and has no hard ceiling. Moving **computation to data** (data locality) is the principle that makes distributed computation efficient.
- The **CAP theorem** proves that any distributed system can guarantee at most two of: Consistency, Availability, and Partition Tolerance. Since partitions are unavoidable, the real design choice is CP vs. AP.
- **Batch processing** offers high throughput and simplicity for bounded datasets but incurs high latency. **Stream processing** offers low latency for continuously arriving data but adds complexity. Production systems often combine both paradigms (Lambda or Kappa architectures).

Exercises

Short Questions

SQ1.1. State the 3Vs of big data and give one production-scale numerical example for each dimension.

SQ1.2. What is the CAP theorem? List the three properties and explain what each means in operational terms.

SQ1.3. A single high-end server in 2024 has approximately 64 TB of RAM. Is a 100 PB dataset “big data” by the Volume definition? Justify your answer quantitatively.

SQ1.4. Classify each of the following as structured, semi-structured, or unstructured. Justify each answer in one sentence.

- (a) A table of stock prices in a PostgreSQL database.

- (b) A REST API response in JSON format from a weather service.
- (c) An MP3 recording of a customer service call.
- (d) An XML document containing a purchase order.
- (e) A satellite image of an agricultural field.

SQ1.5. What is “schema-on-read”? How does it differ from “schema-on-write,” and in which type of data is each used?

SQ1.6. Explain the concept of “data locality” in distributed processing. Why does moving computation to data (rather than data to computation) improve performance at petabyte scale?

SQ1.7. An organisation’s fraud detection system must decide whether to approve a payment within 300 milliseconds. Should it use batch or stream processing? Explain why.

SQ1.8. Apache Cassandra is described as an AP system. What does this mean concretely? Give a scenario where choosing Cassandra over a CP system (like HBase) is the correct engineering decision, and a scenario where it is not.

SQ1.9. What is the difference between *event time* and *processing time* in stream processing? Why does this distinction matter for computing aggregations over a stream?

SQ1.10. Identify the primary V (Volume, Velocity, or Variety) that dominates each scenario:

- (a) A genomics research lab accumulating 200 TB of whole-genome sequencing data per month.
- (b) A payment network processing 24,000 card swipes per second.
- (c) An enterprise integrating data from 40 departments, each using a different database vendor and schema.

SQ1.11. What is the Veracity dimension of the 5Vs framework? Give two concrete examples of low-veracity data from different domains.

SQ1.12. Compare the cost-scaling behaviour of vertical and horizontal scaling. Why is vertical scaling said to be “superlinear” in cost?

SQ1.13. What is the Lambda architecture? What problem does it solve, and what is its primary drawback?

SQ1.14. True or false, with a one-sentence justification: “A relational database system can always be made to handle big data workloads by upgrading to a more powerful server.”

Analytical Problems

AP1.1. Data Volume Estimation. A financial services company ingests stock market quote updates at a rate of 1.2 million messages per second during trading hours (6.5 hours per day). Each message is 256 bytes on average, and HDFS stores three replicas of all data. Calculate:

- (a) The raw volume of quote data generated per trading day (in GB).
- (b) The total HDFS storage consumed per trading day (with 3× replication).
- (c) The annual storage requirement (250 trading days per year), in TB.
- (d) If each HDFS DataNode has 48 TB of usable disk capacity, what is the minimum number of DataNodes needed to store one year of data?

AP1.2. CAP Theorem Analysis. For each of the following systems, determine whether its designers should choose CP or AP, and explain the reasoning in terms of what happens when a network partition occurs and the system must choose between C and A.

- (a) An inventory management system for an online retailer. Showing a product as “in stock” when it is actually sold out results in order cancellations and customer dissatisfaction; showing it as “out of stock” when it is available causes lost sales.
- (b) A distributed configuration registry used by microservices to discover the addresses of other services. Serving a stale configuration might cause a service to connect to a decommissioned endpoint.
- (c) A social media “like” counter. The precise count at any given millisecond is not critical; users expect the counter to update within a few seconds.

AP1.3. Batch vs. Stream Latency Analysis. A credit card company runs a fraud detection system. The fraudster’s behaviour pattern (multiple small

test transactions followed by a large transaction) is detectable within a 15-minute time window. The company currently runs a batch fraud detection job that starts at midnight and finishes by 05:00. Show, with a timeline analysis, why this batch job cannot prevent in-session fraud. Estimate the maximum percentage of daily fraud that could be caught by a streaming system with a 10-second detection latency, assuming fraud transactions are uniformly distributed throughout the day.

AP1.4. Scaling Economics. A company must build a system to store and query a 500 TB analytical dataset. Option A is a vertically scaled Oracle Exa-data appliance (licensed at \$1.2M/year, maximum capacity 500 TB). Option B is a horizontally scaled HDFS cluster of commodity servers at \$6,000 per node (12 TB usable per node), with HDFS's 3× replication factor.

- (a) How many DataNodes does Option B require?
- (b) What is the hardware capital cost of Option B?
- (c) If the dataset is expected to grow at 30% per year, how many years before Option A requires a hardware replacement (assume it cannot be expanded beyond 500 TB), and what would Option B's cost be to scale to the equivalent capacity?
- (d) What non-monetary factors also influence this architectural choice?

Design Problems

DP1.1. Architecture Selection for a Global E-commerce Platform. You are the lead data engineer at a mid-sized e-commerce company with operations in 12 countries. The company processes 85 million orders per year and must support the following data workloads:

- Monthly financial reconciliation reports (all orders for the month).
- Real-time fraud scoring for each checkout event (must respond within 500 ms).
- A product recommendation engine that retrains weekly on full order and clickstream history.
- Live operational dashboards showing current order throughput and error rates by country.

For each workload, identify: (a) the dominant V dimension it stresses, (b) whether batch or stream processing is appropriate, and (c) which CAP trade-off the underlying storage must make. Then propose a single integrated architecture (a sketch is sufficient) that serves all four workloads, and identify the single component that is the highest engineering risk.

DP1.2. Data Classification and Integration Strategy. A national weather service wants to build an analytics platform integrating the following data sources:

- Hourly temperature and humidity readings from 12,000 automated weather stations (CSV format, streamed via SFTP).
- Real-time radar imagery in TIFF format, updated every 6 minutes.
- Historical weather observations from 1950 to present, stored in a legacy Oracle database with a proprietary schema.
- Citizen-submitted weather reports via a mobile app (JSON, free-text description field included).
- Satellite cloud-cover imagery (binary NetCDF format).

For each source: (a) classify the data type (structured/semi-structured/unstructured), (b) identify the dominant V dimension, and (c) propose an ingestion mechanism. Then identify the two most difficult integration challenges when combining these sources into a unified query layer, and propose mitigations for each.

Programming Exercises

PE1.1. Data Type Classifier (Python). The following list of data source descriptions must be classified as structured, semi_structured, or unstructured. Implement the `classify_source()` function using keyword-based heuristics. Then extend your solution to print the recommended primary storage technology for each category.

```
1 sources = [  
2     {"name": "Customer transaction records in PostgreSQL",  
3      "format": "relational_table", "schema": "predefined"},  
4     {"name": "HTTP server access log",  
5      "format": "text", "schema": "implicit"},
```

```

6     {"name": "Satellite multispectral imagery (GeoTIFF)",
7       "format": "binary", "schema": "none"},
8     {"name": "Product catalog (XML)",
9       "format": "xml", "schema": "self_describing"},
10    {"name": "Customer service call recording (MP3)",
11      "format": "audio", "schema": "none"},
12    {"name": "E-commerce clickstream events (JSON)",
13      "format": "json", "schema": "self_describing"},
14    {"name": "Annual census data (CSV with fixed columns)",
15      "format": "csv", "schema": "predefined"},
16  ]
17
18  STORAGE_MAP = {
19      "structured": "Relational RDBMS (PostgreSQL, MySQL)
20                    or columnar DW",
21      "semi_structured": "Data lake (HDFS/S3) + Apache Hive /
22                          Spark SQL",
23      "unstructured": "Object store (S3, Azure Blob) + ML
24                      feature pipeline",
25  }
26
27  def classify_source(source):
28      """
29      Returns 'structured', 'semi_structured', or 'unstructured'.
30      Hint: use source['format'] and source['schema'] fields.
31      """
32      # TODO: implement classification logic
33      pass
34
35  for s in sources:
36      label = classify_source(s)
37      print(f"Source : {s['name']}")
38      print(f"Category: {label}")
39      print(f"Storage : {STORAGE_MAP.get(label, 'Unknown')}")
40      print()

```

Listing 1.2: PE1.1: Data type classifier

PE1.2. CAP Trade-off Decision Matrix (Python). Given a set of system requirements, write a function that recommends the appropriate CAP trade-off and a supporting rationale. Then simulate what happens under a network partition for each choice.

```

1 requirements = [

```

```
2     {"system": "Banking ledger (account balances)",
3       "stale_read_acceptable": False,
4       "unavailability_acceptable": True,
5       "description": "Must never show incorrect balance"},
6     {"system": "Social media post counter (likes)",
7       "stale_read_acceptable": True,
8       "unavailability_acceptable": False,
9       "description": "Slight delay in count update is tolerable"
10      },
11     {"system": "Distributed lock manager for microservices",
12       "stale_read_acceptable": False,
13       "unavailability_acceptable": True,
14       "description": "Two services must never acquire the same
15         lock"},
16     {"system": "Product search index",
17       "stale_read_acceptable": True,
18       "unavailability_acceptable": False,
19       "description": "Returning yesterday's index is acceptable"
20      },
21 ]
22
23 def cap_recommendation(req):
24     """
25     Returns a tuple: (cap_choice, rationale_string)
26     where cap_choice is 'CP' or 'AP'.
27
28     Logic:
29     - If stale reads are NOT acceptable -> CP
30       (system must be consistent even if it becomes unavailable)
31     - If stale reads ARE acceptable -> AP
32       (system must stay available even with possibly stale data)
33     """
34     # TODO: implement decision logic and rationale string
35     pass
36
37 def simulate_partition(cap_choice, system_name):
38     """
39     Print what happens to the system during a network partition
40     .
41     """
42     if cap_choice == "CP":
43         print(f" Partition occurs in {system_name}")
```

```

40     print(f" -> System refuses requests on isolated nodes.
41           ")
42     print(f" -> Availability drops; consistency maintained
43           ".")
44     elif cap_choice == "AP":
45         print(f" Partition occurs in {system_name}:")
46         print(f" -> System continues serving requests on
47               isolated nodes.")
48         print(f" -> Data may be stale; eventual consistency
49               applies.")
50
51     for req in requirements:
52         choice, rationale = cap_recommendation(req)
53         print(f"System : {req['system']}")
54         print(f"Choice : {choice} | Rationale: {rationale}")
55         simulate_partition(choice, req['system'])
56         print()

```

Listing 1.3: PE1.2: CAP trade-off recommender

PE1.3. Storage Volume Estimator (Python). Implement a function that estimates the raw and replicated HDFS storage requirements for a streaming data source, and classifies the result as “conventional” (manageable with a single RDBMS), “big data” (requiring distributed storage), or “hyperscale” (requiring cloud object storage tiers in addition to HDFS).

```

1  def estimate_hdfs_storage(records_per_second, avg_record_bytes,
2                           retention_days, replication_factor=3)
3
4      """
5      Parameters
6      -----
7      records_per_second : float -- sustained ingestion rate
8      avg_record_bytes   : int   -- average size of one record
9      retention_days     : int   -- how long to retain data
10     replication_factor  : int   -- HDFS replication (default
11                                3)
12
13     Returns
14     -----
15     dict with keys:
16         raw_tb           : raw data in terabytes
17         replicated_tb    : total HDFS storage in terabytes (with

```

```

        replication)
16     replicated_pb      : total HDFS storage in petabytes
17     classification    : 'conventional' | 'big_data' | '
        hyperscale'
18     """
19     BYTES_PER_TB = 1e12
20     BYTES_PER_PB = 1e15
21
22     # TODO: implement computation
23     # Steps:
24     # 1. Compute bytes per second = records_per_second *
        avg_record_bytes
25     # 2. Compute bytes per day = bytes_per_second * 86400
26     # 3. Compute raw_bytes = bytes_per_day * retention_days
27     # 4. Compute replicated_bytes = raw_bytes *
        replication_factor
28     # 5. Convert to TB and PB
29     # 6. Classify:
30     #    < 50 TB    -> 'conventional'
31     #    50 TB - 10 PB -> 'big_data'
32     #    > 10 PB   -> 'hyperscale'
33     pass
34
35     scenarios = [
36         {"name": "IoT temperature sensors (500K devices, 1 reading/
            min)",
37          "rps": 500_000 / 60, "bytes": 64, "days": 365},
38         {"name": "E-commerce clickstream (peak 200K events/s, 90-
            day retention)",
39          "rps": 200_000,      "bytes": 512, "days": 90},
40         {"name": "Payment network (25K tx/s, 7-year regulatory
            retention)",
41          "rps": 25_000,       "bytes": 256, "days": 2555},
42         {"name": "Video streaming events (50K concurrent sessions,
            1 event/s)",
43          "rps": 50_000,      "bytes": 1024, "days": 180},
44     ]
45
46     for s in scenarios:
47         result = estimate_hdfs_storage(s["rps"], s["bytes"], s["
            days"])
48         if result:
49             print(f"Scenario      : {s['name']}")
50             print(f"Raw storage   : {result['raw_tb']:.1f} TB")

```



```
51     print(f"With 3x rep. : {result['replicated_pb']:.3f} PB  
    ")  
52     print(f"Class          : {result['classification']}")  
53     print()
```

Listing 1.4: PE1.3: HDFS storage volume estimator

Data Integration: Heterogeneity, Quality, and Provenance

“The hardest part of data science isn’t machine learning or statistics. It’s getting data from different places to agree with each other.”

Anonymous data engineer

Learning Objectives

After completing this chapter, you will be able to:

1. Explain the data silo problem and articulate why integration complexity—not storage cost—is the primary barrier to enterprise analytics.
2. Define the three fundamental operations of data integration: schema mapping, data exchange, and entity resolution.
3. Distinguish among the four dimensions of data heterogeneity—structural, semantic, syntactic, and scale—and give a concrete example of each.
4. Describe the six dimensions of data quality and identify appropriate remediation strategies for each.
5. Explain what data provenance is, why it is required by regulations such as GDPR and HIPAA, and how lineage tracking systems cap-

ture it.

6. Compare the ETL and ELT paradigms: their architectures, trade-offs, and appropriate use cases.
7. Identify the right tool (Sqoop, Kafka, Airbyte, dbt) for each stage of a modern integration pipeline.

Chapter 1 established the 3Vs as the defining characteristics of big data: Volume, Velocity, and Variety. Volume and Velocity receive the most attention in the popular press because they are easy to quantify—petabytes of storage, thousands of events per second. But *Variety* is arguably the hardest engineering challenge. Data arrives from dozens of independent sources, each with its own schema, format, quality level, and update cadence. Before a single analytical query can be answered, all of this heterogeneous data must be identified, mapped, cleaned, and unified. This process is called *data integration*, and it is the subject of this chapter.

A 2019 Forrester Research survey found that 73% of enterprise data is never analysed. The primary reason is not a shortage of storage or processing power—it is integration complexity. Data that cannot be found, joined, or trusted cannot be analysed. Data engineering at production scale is, to a greater degree than is often acknowledged, the art of making heterogeneous data coherent.

2.1 The Data Silo Problem

Large organisations accumulate data in *data silos*: independent systems built by separate teams, at different times, for different purposes, using different technology stacks. Each silo does its job well in isolation, but no silo has visibility into the others. The result is an organisation that is simultaneously data-rich and insight-poor.

Consider a representative global retail bank. A customer who has held an account for five years may appear as a distinct record in at least seven separate systems: the core banking RDBMS (account balances, transactions), the mobile application database (session events, in-app behaviour), the loan

origination system (credit applications, repayment history), the customer relationship management platform (support tickets, call transcripts), the anti-money-laundering engine (transaction flags, compliance cases), the marketing automation platform (campaign responses, email open rates), and the regulatory reporting database (filings with supervisory authorities). In each system, the customer may be identified by a different key—an internal account number, a mobile device identifier, a CRM contact ID, a government-issued reference number—and the same attribute (the customer’s full name, for example) may be spelled differently, encoded differently, or stored at different granularities.

The consequence is not merely an engineering inconvenience. It is a fundamental barrier to analytical insight: a data scientist wishing to understand the relationship between a customer’s loan repayment behaviour and their mobile application engagement cannot combine these datasets without first resolving which records in the loan system correspond to which records in the mobile database—a non-trivial operation even for a single bank with thousands of customer records, let alone for a global institution with hundreds of millions.

Key Insight | Integration is the Critical Path

In most data engineering projects, the time spent on data integration—schema mapping, entity resolution, quality cleaning, and pipeline construction—exceeds the time spent on the analytical computation itself. Studies of data science workflows consistently find that engineers spend 60–80% of project time on data preparation rather than on modelling or analysis. Integration quality directly determines analytical quality: a machine learning model trained on incorrectly joined data will encode the integration error into its predictions.

2.2 A Formal Framework for Data Integration

The academic study of data integration has a rich history spanning decades of database research. Halevy, Rajaraman, and Ordille’s landmark 2006 survey—titled “Data Integration: The Teenage Years”—synthesised thirty years of work and established the formal vocabulary that the field still uses

today (Halevy, Rajaraman, and Ordille 2006).

Definition | Data Integration

Given a set of heterogeneous, autonomous data sources $S_1, S_2, \dots, S_n$, each with its own schema Σ_i and content D_i , **data integration** is the problem of providing a unified query interface over a **mediated (global) schema** Σ_G that gives users a single coherent view of all data, regardless of which source it originated from.

This definition makes three important properties explicit. First, data sources are *heterogeneous*: they differ in schema, format, vocabulary, and update cadence. Second, they are *autonomous*: the integration layer has no authority to modify the source systems—it can only read from them. Third, the goal is a *unified query interface*: users should be able to ask a single query against the global schema without knowing which source holds the relevant data or in what format.

Achieving this goal requires three core operations, each of which is a research area in its own right.

2.2.1 Schema Mapping

Schema mapping is the process of defining a formal correspondence between the schema of a source system and the global mediated schema. A schema mapping $M_{i \rightarrow G}$ specifies, for each attribute in the global schema, which attribute in source S_i provides the corresponding data, and what transformation (if any) is required to reconcile differences in naming, data type, units, or encoding.

Schema mappings are expressed as rules. For example:

$$\text{GlobalSchema.customer_id} \leftarrow \text{CRM.contact_id} \quad [\text{identity}] \quad (2.1)$$

$$\text{GlobalSchema.full_name} \leftarrow \text{uppercase}(\text{ATM.CUST_FNAME} + " " + \text{ATM.CUST_LNAME}) \quad (2.2)$$

In practice, deriving accurate schema mappings is one of the most labour-intensive tasks in data engineering. Automated *schema matching* tools—which use string similarity, structural similarity, and machine learning to propose candidate mappings—can reduce this effort significantly,

but they always require human validation before production deployment.

2.2.2 Data Exchange

Data exchange is the process of *materialising* data from source schemas into the global schema. It is the implementation of a schema mapping: given source data D_i and a mapping $\mathcal{M}_{i \rightarrow G}$, produce a dataset D_G that conforms to the global schema Σ_G . Data exchange corresponds to the “Transform” step in an ETL pipeline (discussed in Section 2.7).

2.2.3 Entity Resolution

Entity resolution (also called *record linkage*, *deduplication*, or *identity resolution*) is the process of determining whether two records from different sources (or from the same source) refer to the same real-world entity. It is necessary when different systems use different identifiers for the same object and when direct key-based joins are therefore impossible.

Entity resolution is examined in detail in Section 2.4.

Research Spotlight | Halevy, Rajaraman & Ordille (2006) — Data Integration: The Teenage Years

Citation: Halevy, A., Rajaraman, A., & Ordille, J. (2006). Data Integration: The Teenage Years. *Proceedings of the 32nd VLDB Conference*, pp. 9–16. (Halevy, Rajaraman, and Ordille 2006)

Key insight: The paper’s title is a pointed metaphor: after three decades of research, data integration remained “in its teenage years”—having demonstrated proof of concept and produced fundamental theory, but not yet reaching the robust, deployable maturity that industry needed.

Technical contributions:

- Provided a comprehensive taxonomy of the data integration problem space, distinguishing virtual integration (query rewriting over live sources) from materialised integration (ETL into a data warehouse).
- Formalised the distinction between *local-as-view* (LAV) and *global-as-view* (GAV) approaches to schema mapping— still the canonical framework for integration system design.
- Identified entity resolution as the hardest open problem in data inte-

gration, predicting (correctly) that it would require probabilistic and machine learning approaches at scale.

Impact today: The paper is required reading in database courses at MIT, Stanford, and CMU. Its taxonomy of integration challenges directly influenced the design of modern enterprise data integration platforms, including MuleSoft, Informatica, and Apache Atlas. The LAV/GAV distinction remains the theoretical foundation of modern data virtualisation products.

2.3 The Four Dimensions of Data Heterogeneity

Heterogeneity—the fundamental barrier to integration—manifests along four distinct dimensions. Understanding all four is essential because a pipeline that addresses only one or two will produce subtly incorrect results, often without any obvious indication of failure.

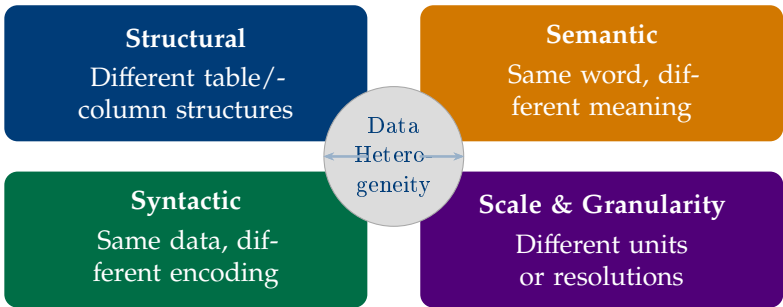


Figure 2.1: The four dimensions of data heterogeneity. A production integration pipeline must address all four simultaneously.

2.3.1 Structural Heterogeneity

Structural heterogeneity arises when two data sources represent the same real-world concept using different table organisations, column names, or normalisation levels. It is the most immediately obvious form of heterogeneity and the first that integration engineers typically address.

Naming conflicts occur when the same attribute has different names in different systems. A customer identifier might be called `cust_id` in the core banking system, `customer_number` in the loan system, and `contact_id` in the CRM. Without a mapping, a JOIN across these tables produces no results.

Structural conflicts occur when the same data is stored at different levels of normalisation. One system may store a customer's address as a single string ("14 Oxford Street, London, W1D 1AU"), while another uses four separate columns (`street_line_1`, `city`, `region`, `postcode`). Converting between these representations requires both parsing and composing—two operations that are error-prone when addresses do not conform to expected patterns.

Table 2.1: Structural heterogeneity: a customer record in three systems

Concept	Core Banking	CRM	ATM Logs
Customer ID	<code>cust_id</code>	<code>contact_id</code>	<code>CARD_HOLDER</code>
First name	<code>first_name</code>	<code>fname</code>	<code>FN</code>
Date of birth	<code>dob (DATE)</code>	<code>birth_date (VARCHAR)</code>	<code>absent</code>
Address	4-column normalised	Single string	<code>absent</code>
Account open date	<code>open_date</code>	<code>create_ts</code>	<code>absent</code>

2.3.2 Semantic Heterogeneity

Semantic heterogeneity—the most dangerous form—arises when two data sources use the same name to mean different things, or different names to mean the same thing. A schema mapping that resolves structural conflicts (renaming columns) will silently produce incorrect results if the underlying semantic conflict is not also resolved.

Classic examples are pervasive:

- The column `price` in the inventory system records the wholesale cost; the same column in the e-commerce system records the retail price. Joining

these tables without disambiguation produces a dataset where “price” has a different meaning for every row.

- The field `date` in an order management system could mean the order date, the shipping date, or the invoice date, depending on which team created the table. All three share the same column name in different tables.
- The concept of a “customer” means a natural person in the retail banking system, but can also mean a legal entity (company) in the corporate banking system. A JOIN on `customer_id` would mix individuals and corporations.

Semantic heterogeneity cannot be detected by examining schemas alone—it requires domain knowledge, documentation review, and often direct consultation with the teams that created each system. This is why data dictionaries and business glossaries are not administrative overhead but engineering necessities.

2.3.3 Syntactic Heterogeneity

Syntactic heterogeneity arises when the same data is encoded in different formats, character encodings, or data types across sources. The value and meaning are equivalent, but the byte-level representation differs. Syntactic heterogeneity must be resolved before any comparison or JOIN can succeed.

Representative examples include:

- Dates stored as “2024-03-15” (ISO 8601, universally sortable) in one system and as “15/03/2024” (UK format) or “03-15-2024” (US format) in another. A lexicographic sort on the date column produces incorrect ordering when formats are mixed.
- Boolean values represented as `true/false` (JSON), `1/0` (integer), “Y”/“N” (legacy COBOL systems), or “yes”/“no” (human-entered data).
- Currency amounts stored as 64-bit floating-point numbers in one system and as decimal strings (e.g., “1,234.56” or “1.234,56” for European locale) in another.
- Character encodings: legacy mainframe systems frequently use EBCDIC encoding; modern web applications use UTF-8; some regional systems use Windows-1252 or ISO-8859 variants.

2.3.4 Scale and Granularity Heterogeneity

Scale and granularity heterogeneity arises when different sources measure the same quantity at different units, resolutions, or temporal granularities. Even when structural, semantic, and syntactic conflicts have been resolved, a mismatch in granularity can invalidate comparisons.

A revenue report that joins monthly aggregates from one system with daily transaction records from another must aggregate the latter before joining; failing to do so produces a systematic distortion where each monthly figure appears 30 times. Similarly, a climate analytics system integrating temperature readings from stations reporting in Celsius and others in Fahrenheit must apply a unit conversion before any cross-station analysis; a system integrating GPS coordinates at metre precision with satellite imagery at 10-metre resolution must account for the resolution mismatch when attributing imagery to ground-truth measurements.

Production Lesson | Integration Failures Are Silent

Of the four heterogeneity dimensions, structural and syntactic failures are the easiest to detect: they typically produce explicit errors (column not found, type mismatch, parse failure). Semantic and granularity failures are far more dangerous because they produce *no error*—the pipeline succeeds, data flows, and the output looks superficially correct. A data warehouse that has silently mixed wholesale prices with retail prices will answer every price-related query without complaint, producing reports that are confidently and systematically wrong. Detecting semantic errors requires domain knowledge and statistical profiling of the integrated output, not just schema validation.

2.4 Entity Resolution

Even after schema mappings have been defined and all four heterogeneity dimensions addressed, a fundamental problem remains: how do we know that record r_1 from source S_1 and record r_2 from source S_2 refer to the same real-world entity? When a common key exists (both systems use a social security number or a product EAN barcode), the answer is trivial: join on

the key. When no common key exists—the common case—the problem is *entity resolution*.

Definition	Entity Resolution
Given two sets of records R_1 and R_2 from sources S_1 and S_2 respectively, entity resolution is the task of identifying all pairs $(r_1, r_2) \in R_1 \times R_2$ that refer to the same real-world entity, using only the attribute values of the records (and no common key).	

The naive approach—comparing every record in R_1 against every record in R_2 —has complexity $O(|R_1| \times |R_2|)$. For datasets with millions of records, this is computationally infeasible. A dataset of 10 million customer records from two sources would require 100 trillion comparisons—at a rate of one billion comparisons per second, that is over 27 hours. At big data scale, the quadratic cost is prohibitive.

2.4.1 Blocking: Reducing the Candidate Space

Blocking is the standard technique for making entity resolution computationally tractable. Rather than comparing all possible pairs, blocking partitions records into **blocks** based on a simple attribute (or combination of attributes) that is likely to match for true duplicates. Only pairs within the same block are compared. A common blocking key is the first three characters of the last name combined with the year of birth; two records can only be a match if they share the same block key.

A well-designed blocking strategy must balance two competing objectives. *Recall* requires that almost all true duplicates fall in the same block—if a blocking key is too aggressive, it splits true duplicates into different blocks and they are never compared. *Efficiency* requires that blocks be small enough to make comparison tractable—if a blocking key is too lenient, blocks are large and the quadratic comparison cost returns. In practice, multiple blocking keys are used in parallel, and a pair is a candidate if it shares at least one blocking key.

2.4.2 Similarity Functions

Within each block, records are compared using *similarity functions* that assign a score $\sigma(r_1, r_2) \in [0, 1]$ to each candidate pair. Two commonly used functions are:

Jaccard similarity operates on sets of tokens. Given two strings s_1 and s_2 tokenised into sets of words or character n-grams T_1 and T_2 :

$$J(T_1, T_2) = \frac{|T_1 \cap T_2|}{|T_1 \cup T_2|} \quad (2.3)$$

Jaccard similarity is robust to word reordering and partial matches, making it well-suited for company names and addresses.

Levenshtein (edit) distance measures the minimum number of single-character insertions, deletions, or substitutions needed to transform one string into another. Normalised edit similarity is:

$$\sigma_{\text{edit}}(s_1, s_2) = 1 - \frac{d_{\text{edit}}(s_1, s_2)}{\max(|s_1|, |s_2|)} \quad (2.4)$$

Edit distance handles typos, transpositions, and abbreviated names (“Corp.” vs “Corporation”).

A *composite similarity score* combines multiple attribute similarities with weights reflecting their discriminative power:

$$\sigma_{\text{composite}}(r_1, r_2) = w_{\text{name}} \cdot \sigma(r_1.\text{name}, r_2.\text{name}) + w_{\text{addr}} \cdot \sigma(r_1.\text{addr}, r_2.\text{addr}) + w_{\text{dob}} \cdot \sigma(r_1.\text{dob}, r_2.\text{dob}) \quad (2.5)$$

where $w_{\text{name}} + w_{\text{addr}} + w_{\text{dob}} = 1$.

A pair (r_1, r_2) is declared a match if $\sigma_{\text{composite}}(r_1, r_2) \geq \theta$ for a threshold θ chosen to balance precision and recall for the specific application.

2.5 Data Quality: Dimensions and Challenges

Data integration can perfectly resolve structural, semantic, syntactic, and granularity heterogeneity, correctly link every entity, and still produce a dataset that is analytically worthless—if the underlying data is of poor quality. The *Veracity* dimension of the 5Vs framework (Chapter 1) captures

this concern. Data quality problems are not exceptions; they are ubiquitous in production systems, and big data engineering requires systematic processes for detecting, quantifying, and remediating them.

The field has converged on six dimensions of data quality, originally formalised by Wang and Strong (1996) in their empirical study of information consumers’ quality requirements.

Table 2.2: The six dimensions of data quality with engineering implications

Dimension	Definition	Detection and remediation
Accuracy	Values correctly represent the real-world entity or event	Statistical profiling; external reference datasets; domain range constraints
Completeness	All required values are present; no critical nulls	NULL ratio checks; conditional completeness rules (if type=“corporate” then tax_id must be non-null)
Consistency	Values are not contradictory within or across sources	Cross-source join validation; referential integrity checks; temporal monotonicity checks
Timeliness	Data is current enough for its intended use	Staleness metrics; data freshness SLAs; arrival-time audits
Validity	Values conform to defined formats, ranges, and vocabularies	Regular expression validation; enumeration checks; numeric range constraints
Uniqueness	Records are not duplicated (within a source or across sources)	Duplicate detection; entity resolution (Section 2.4)

2.5.1 Missing Values

Missing values are among the most common data quality problems in large datasets. They arise from several distinct mechanisms, each with different analytical implications.

Missing Completely at Random (MCAR): the probability that a value is missing is independent of both the observed and missing values. A sensor that fails with a fixed, context-independent probability produces MCAR missingness. Data imputation strategies—such as mean or median imputation—are statistically valid under MCAR.

Missing at Random (MAR): the probability that a value is missing depends on observed values but not on the missing value itself. For example, older customers may be less likely to provide an email address, so the email field is missing more often for older cohorts. This is MAR because age (observed) predicts missingness, but the missing value itself (specific email) does not.

Missing Not at Random (MNAR): the most problematic case. The probability that a value is missing depends on the value itself. Survey data where respondents with very low incomes decline to answer the income question is MNAR—the missingness is informative about the missing value. Imputing MNAR data with mean imputation introduces systematic bias.

In big data engineering, the immediate engineering response to missing values is typically a combination of: (1) quantifying the NULL rate per column in the data quality dashboard, (2) identifying whether the missing values are upstream (the source system never had the value) or midstream (the ETL pipeline dropped them), and (3) applying domain-appropriate imputation, filtering, or flagging. Feeding a machine learning model records with missing feature values without appropriate handling is one of the most common sources of production ML failure.

2.5.2 Noise, Outliers, and Inconsistency

Noise refers to random errors in data values—measurement errors, transmission errors, or OCR mistakes. A temperature sensor that occasionally reads -999.0°C due to a firmware bug introduces noise; a mobile payment amount that loses a decimal point and appears as 10,000 times the correct value is noise. At scale, even a 0.01% noise rate in a 10-billion-row dataset

produces one million corrupted records—enough to systematically distort any aggregate computed over that column.

Outliers may be noise, or they may be genuine rare events. A single transaction of \$50 million in a retail payment network is an outlier; it might represent fraud (noise), or it might represent a legitimate large commercial payment. In data engineering, the distinction between noise and genuine outliers must be resolved using domain knowledge and external validation, not purely statistical criteria.

Inconsistency arises when the same real-world fact is represented differently at different points in time or in different parts of the same system. A customer record that shows the customer’s home city as “New York” in the CRM but “NYC” in the billing system is inconsistent. A product price that changes in the catalogue but is not updated in the recommendation engine creates a temporal inconsistency. Inconsistency is particularly dangerous in slowly changing dimension tables (a concept explored in the data warehousing literature) where historical accuracy must be maintained alongside current values.

2.6 Data Provenance and Lineage

Data provenance refers to the documented record of a data artifact’s origin, transformation history, and movement through a system—the complete answer to the question: “Where did this value come from, and what was done to it?” *Data lineage* is the operationalised form of provenance: the graph of dependencies linking each output dataset to its input datasets and the transformations applied to produce it.

Provenance is not merely a technical nicety; it is increasingly a legal requirement. The European Union’s General Data Protection Regulation (GDPR, 2018) requires organisations to be able to demonstrate the legal basis for processing personal data, trace where personal data flows within and across systems, and respond to subject access requests that require identifying all locations where an individual’s data is held. The United States’ Health Insurance Portability and Accountability Act (HIPAA) requires healthcare organisations to maintain audit trails documenting every access to and movement of protected health information (PHI). Failure to

demonstrate adequate provenance tracking has resulted in regulatory fines exceeding \$10 million in individual GDPR enforcement actions.

Beyond regulatory compliance, provenance serves crucial operational purposes. When an analytical model produces an unexpected result, data lineage allows engineers to trace the result back to the exact source records and transformations that produced it—identifying whether the anomaly originates in a source system, an ETL transformation, or the analytical model itself. In production data pipelines, this capability can reduce the mean time to detect and resolve data quality incidents from days to hours.

Key Insight | Lineage as a First-Class Engineering Artifact

Modern data engineering treats lineage as a first-class artifact, not an afterthought. Tools such as Apache Atlas (metadata management), OpenLineage (open specification for lineage events), and dbt's built-in lineage graph capture transformations automatically as pipelines run. A comprehensive lineage graph answers: which tables feed this model, which source records produced this output row, which transformation version was active when this job ran, and who approved the pipeline change that introduced this transformation. Without this capability, debugging a corrupted KPI dashboard may require days of manual source tracing across dozens of systems.

2.7 ETL vs. ELT: Two Philosophies of Integration

The practical execution of data integration—moving, transforming, and loading data from source systems into analytical targets—is governed by two contrasting paradigms. The choice between them is one of the most consequential architectural decisions in data engineering, because it determines where and when data is transformed, how flexible the system is to evolving requirements, and what trade-offs are made between data freshness, quality, and engineering complexity.

2.7.1 ETL: Extract, Transform, Load

ETL is the traditional paradigm, dominant in data warehousing from the 1980s through the 2010s. It operates in three sequential phases:

Extract: Data is read from source systems—typically RDBMS tables, file exports, or API responses—and written to a staging area. The extraction is designed to be minimally invasive, typically using incremental extraction (change data capture, CDC) rather than full table scans, to avoid overloading source systems.

Transform: In the staging area, the extracted data is subjected to all required cleaning, mapping, enrichment, and aggregation operations before it is written to the target. The target (typically a data warehouse) enforces a strict schema; data that does not conform to the schema is rejected. This means that the transformation step must be comprehensive and correct before any data reaches the warehouse.

Load: The transformed, validated data is loaded into the target data warehouse in bulk. Only data that passes all quality and schema checks is written to the target.

The primary advantage of ETL is data quality at the target: by the time data reaches the warehouse, it has been fully validated and conforms precisely to the intended schema. The primary disadvantage is rigidity: adding a new source or changing a transformation requires modifying the ETL pipeline, which may require schema changes in the warehouse, regression testing of all downstream queries, and potentially a full historical reload.

2.7.2 ELT: Extract, Load, Transform

ELT inverts the order: raw data is extracted from sources and loaded immediately into a data lake (typically cloud object storage such as Amazon S3, Google Cloud Storage, or Azure Data Lake Storage) in its native format, without transformation. Transformation is deferred until analytical consumption time, when a compute engine such as Apache Spark or a transformation tool such as dbt applies the necessary logic on-demand.

ELT has become the dominant paradigm for modern data engineering for several reasons. First, cloud object storage is extraordinarily inexpensive (approximately \$0.023 per GB per month for S3 Standard as of 2024),

making it economically feasible to store raw, untransformed data at any scale. Second, decoupling extraction from transformation enables *schema evolution*: when a source system changes its schema, the raw data in the lake is unaffected, and only the transformation logic needs to be updated. Third, deferred transformation supports multiple downstream use cases with different transformation requirements without requiring the ETL pipeline to anticipate every future analytical need.

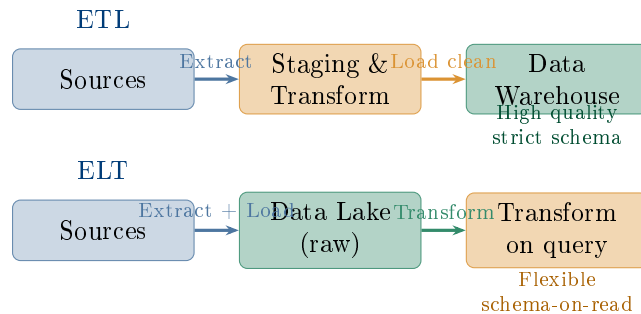


Figure 2.2: ETL vs. ELT: the order of operations determines where and when data quality constraints are enforced.

Common Misconception | ELT Does Not Mean Skip Transformation

A common misunderstanding is that ELT eliminates the need for data transformation. It does not—it defers transformation. The raw data in a data lake is typically not directly queryable or usable for analytics; it requires the same cleaning, normalisation, and enrichment that ETL performs, just applied at query time rather than ingestion time. The *dbt* (data build tool) framework has become the standard for managing this deferred transformation layer, implementing transformation logic as versioned, tested SQL models with lineage tracking. ELT without a robust transformation and testing layer produces a “data swamp”—a lake of raw data that nobody trusts or can efficiently query.

2.8 The Modern Data Integration Stack

Modern data integration pipelines combine tools that address different stages and dimensions of the integration problem. Understanding the ecosystem—which tool solves which problem—is essential for designing production systems.

A typical production ELT stack for a modern data-driven organisation operates as follows. Source systems (RDBMS, SaaS applications, IoT devices) emit data either via batch exports or real-time event streams. Batch sources are ingested using Sqoop (for RDBMS-to-HDFS transfer) or a managed connector platform (Airbyte, Fivetran) that handles incremental extraction and schema change detection. Real-time sources are ingested via Apache Kafka, which acts as a durable, ordered event bus with configurable retention—allowing consumers to replay the full event history at any time. Kafka is examined in depth in Chapter 8.

Once raw data lands in the data lake (HDFS or cloud object storage), transformation is managed by dbt (for SQL-based transformations in a data warehouse engine) or Apache Spark (for large-scale Python/Scala transformations over raw files). The transformation layer produces *data marts*—curated, domain-specific views of the integrated data designed for specific analytical workloads. A metadata and lineage platform such as Apache Atlas automatically captures the transformation history, maintaining the provenance graph required for regulatory compliance.

Schema evolution—the challenge of handling changes in source schemas without breaking downstream consumers—is managed by a *schema registry*. Confluent Schema Registry (for Kafka-based pipelines) maintains versioned schemas for each topic, allowing producers and consumers to negotiate schema compatibility at runtime. When a source system adds a new field, the registry enforces *backward compatibility*: consumers using the old schema can still read new records by treating the new field as null; producers using the new schema can still write records that old consumers can parse.

2.9 Case Study: US Healthcare Interoperability and HL7 FHIR

Case Study | Healthcare Interoperability and HL7 FHIR — Integration at National Scale

Background. The United States healthcare system is one of the world's most data-rich and most data-fragmented. A typical US hospital uses between 20 and 50 different software systems: an electronic health record (EHR) platform, a laboratory information system, a radiology picture-archiving system, a pharmacy management system, a billing and insurance system, a clinical decision support system, and dozens of others. These systems were built by different vendors, at different times, using different data models, and have historically been unable to exchange patient data without significant manual effort.

The integration cost. A 2019 analysis by the Center for Health Information and Decision Systems (University of Maryland) estimated that integration failures cost US hospitals **\$8.3 billion annually** in duplicated tests, delayed care, medication errors attributable to incomplete records, and manual reconciliation labour. A patient transferred between two hospitals using different EHR vendors may have their medication list, allergy history, and recent lab results effectively unavailable to the receiving physician—an information gap that directly affects clinical decisions.

The technical challenge as a heterogeneity problem. Healthcare data integration exhibits all four dimensions of heterogeneity simultaneously. *Structural:* patient identifiers vary by hospital (medical record number, patient account number, national identifier). *Semantic:* the same diagnosis code may mean different things under different coding systems (ICD-9 vs. ICD-10 vs. SNOMED CT). *Syntactic:* dates, dosages, and units of measurement are encoded differently across vendors. *Granularity:* laboratory reference ranges vary by testing equipment and laboratory protocol.

The HL7 FHIR standard. Health Level 7 (HL7) is the standards organisation that developed the Fast Healthcare Interoperability Resources (FHIR) standard to address these challenges. FHIR defines a RESTful API and a set of standardised data resources (Patient, Observation,

Medication, DiagnosticReport, etc.) that any EHR system can implement. By exposing patient data through a FHIR API, a hospital allows any authorised consumer—a mobile app, a referral system, a population health analytics platform—to query standardised patient records without knowing the internal database schema of the EHR.

FHIR resources are represented in JSON or XML. A standardised patient resource looks like:

```

1 {
2   "resourceType": "Patient",
3   "id": "pat-001",
4   "name": [{"family": "Smith", "given": ["John"]}],
5   "birthDate": "1978-04-22",
6   "gender": "male",
7   "identifier": [
8     {"system": "urn:oid:2.16.840.1.113883.4.1",
9      "value": "123-45-6789"}
10  ],
11  "address": [
12    {"line": ["123 Main Street"],
13     "city": "Boston", "state": "MA",
14     "postalCode": "02101"}
15  ]
16 }
```

Listing 2.1: A partial HL7 FHIR Patient resource (JSON)

Integration architecture. A national health information exchange built on FHIR operates as follows. Each participating hospital implements a FHIR server that exposes its EHR data through the standard API. A health information exchange (HIE) platform federates queries across multiple FHIR servers, performing entity resolution (matching the same patient across hospitals using demographic attributes) and aggregating standardised data into a unified longitudinal patient record. The transformation from each hospital’s internal schema to FHIR is handled by a vendor-specific FHIR adapter—essentially an ETL pipeline that runs inside or alongside the EHR system.

Data quality and entity resolution challenges. Even with FHIR standardisation, entity resolution across hospitals remains the hardest tech-

nical challenge. The US has no universal patient identifier (comparable to a national insurance number in many other countries); matching patients across hospitals relies on probabilistic matching on name, date of birth, address, and partial social security number—exactly the similarity-function approach described in Section 2.4. Typical production matching systems achieve 95–98% recall at 98–99% precision, meaning that 1–5% of true patient matches are missed and 1–2% of declared matches are incorrect—a significant error rate in clinical contexts.

Lessons for data engineering. The healthcare interoperability challenge illustrates three principles that apply across all large-scale data integration projects. First, technical standards alone (FHIR) are necessary but not sufficient; data quality, entity resolution, and governance must be addressed alongside standardisation. Second, the cost of integration failure is not abstract—it manifests in clinical outcomes, regulatory penalties, and operational waste. Third, integrating data from autonomous, heterogeneous systems at scale requires exactly the distributed tools—Kafka, Spark, schema registries, and lineage tracking—that constitute modern data engineering practice.

Chapter Summary

- **The data silo problem** is the primary barrier to enterprise analytics: data spread across independent, autonomous systems with incompatible schemas, identifiers, and formats cannot be jointly queried without integration.
- **Data integration** provides a unified query interface over heterogeneous, autonomous sources via three core operations: schema mapping (defining field correspondences), data exchange (materialising transformed data), and entity resolution (linking records representing the same real-world entity).
- **Four dimensions of heterogeneity** must all be addressed: structural (different schemas), semantic (different meanings), syntactic (different encodings), and scale/granularity (different units and resolutions). Struc-

tural and syntactic failures are loud; semantic and granularity failures are silent and dangerous.

- **Entity resolution** identifies matching records across sources with no common key. Blocking reduces the $O(n^2)$ comparison problem; similarity functions (Jaccard, edit distance) score candidate pairs; a composite threshold determines matches.
- **Data quality** has six dimensions: accuracy, completeness, consistency, timeliness, validity, and uniqueness. Missing values may be MCAR, MAR, or MNAR; the appropriate imputation strategy depends on the missingness mechanism.
- **Data provenance** records the origin and transformation history of all data artifacts. It is required by GDPR and HIPAA and is operationally critical for debugging data pipelines. OpenLineage, Apache Atlas, and dbt provide lineage tracking.
- **ETL** transforms data before loading (high quality at the target, rigid schema). **ELT** loads raw data first and transforms on demand (flexible, supports schema evolution, requires a separate transformation layer such as dbt).
- The **modern integration stack** combines Kafka (streaming ingestion), Sqoop/Airbyte (batch ingestion), Spark/dbt (transformation), and Atlas (lineage). Schema registries manage schema evolution across producers and consumers.

Exercises

Short Questions

SQ2.1. Define data integration. What three operations does it require, and what does each operation accomplish?

SQ2.2. Explain the difference between structural and semantic heterogeneity. Give one example of each from the domain of retail e-commerce.

SQ2.3. A source system stores dates as the string "15/03/2024" and a target

system expects dates as DATE values in ISO 8601 format ("2024-03-15"). Which dimension of heterogeneity does this represent? What operation resolves it?

SQ2.4. What is entity resolution? Why is the naive $O(n^2)$ approach infeasible at big data scale, and how does blocking address this?

SQ2.5. Define the six dimensions of data quality and give one concrete example for each from a financial services context.

SQ2.6. What is the difference between MCAR, MAR, and MNAR missingness? Why does the distinction matter when choosing an imputation strategy?

SQ2.7. What is data provenance? Name two regulatory frameworks that require it and explain what each framework demands.

SQ2.8. Compare ETL and ELT: (a) in which order are the transform and load steps applied, (b) where is the data quality enforcement point, and (c) which is better suited for schema evolution?

SQ2.9. True or false, with one-sentence justification: "The ELT paradigm eliminates the need for data transformation."

SQ2.10. What is a schema registry? What problem does it solve in a Kafka-based integration pipeline?

SQ2.11. A retail company finds that 15% of rows in its integrated customer table have NULL in the email column. List three possible reasons for this missingness and explain how you would distinguish between them.

SQ2.12. What is "schema drift"? Give a concrete example of how schema drift can cause a downstream machine learning model to fail silently.

SQ2.13. What is the GAV (global-as-view) approach to schema mapping? How does it differ from LAV (local-as-view), and what is the practical trade-off between them?

SQ2.14. A data engineer says: "Our ETL pipeline runs nightly; by morning we have yesterday's data." Identify the data quality dimension being compromised and explain what architecture change would address it.

Analytical Problems

AP2.1. Entity Resolution Complexity Analysis. A hospital system wants to resolve patient identities across two EHR databases: Hospital A has 2.8 million patient records; Hospital B has 1.4 million patient records. A blocking strategy based on birth-year and first three characters of the last name produces blocks of average size 350 pairs (350 A-records times the B-records in the same block).

- (a) Calculate the number of candidate pairs under naive $O(n^2)$ comparison (all A against all B). At a rate of 5 million comparisons per second, how long would naive comparison take?
- (b) Estimate the total number of candidate pairs after blocking, assuming each blocking key creates blocks of average size 350. (Hint: how many blocks are there if each A-record falls in exactly one block?)
- (c) What is the speedup factor from blocking?
- (d) A composite similarity score requires 12 floating-point operations per pair comparison. Estimate the total computation cost (in FLOPs) for the post-blocking comparisons, and determine whether it is feasible on a single server with 100 GFLOPS of sustained throughput.

AP2.2. ETL vs. ELT Storage and Latency Trade-off. A financial services company ingests 5 TB of raw trading data per day from 40 source systems. In the current ETL architecture, data is transformed and compressed before loading, resulting in a 4:1 compression ratio at the warehouse. The company is evaluating migrating to an ELT architecture with raw data stored in S3 at \$0.023/GB/month, and transformation via Spark on demand.

- (a) Calculate the current warehouse storage cost after 12 months (assume warehouse storage at \$0.20/GB/month, and that the compression ratio applies to all stored data). Compare with the raw data storage cost in S3 over the same period.
- (b) The ETL pipeline requires 4 hours of processing before data is available for queries. The ELT approach makes raw data available immediately but requires 45 minutes of Spark transformation before a specific analytical view is ready. For a use case that needs data refreshed every 6 hours, which approach provides better effective freshness?

- (c) In the ELT system, 12 months of raw data are retained in S3. A new analytical requirement arrives, requiring a transformation that was not anticipated. Estimate the Spark compute cost to retroactively apply this transformation to 12 months of 5 TB/day data at \$0.10 per Spark CPU-hour (assume 10 TB/hour processing throughput per 10 CPUs).

AP2.3. Data Quality Profiling. An e-commerce company’s data engineering team profiles a 50-million-row product catalogue table and finds the following:

- 12% of rows have NULL in product_weight.
- 3.2% of rows have price = 0.
- 0.8% of rows have price > \$10,000 (the 99.9th percentile is \$950).
- 4% of rows appear to be duplicates (same product, same vendor, inserted on different dates).
- 7% of rows have category_id values that do not exist in the categories reference table.

For each issue: (a) identify the data quality dimension it violates, (b) classify it as noise, missing data, inconsistency, or validity violation, and (c) propose a specific remediation and the operational procedure to apply it without breaking downstream pipelines.

AP2.4. Similarity Scoring for Entity Resolution. Two records from different source systems are candidates for entity resolution. Using the composite similarity formula with weights $w_{\text{name}} = 0.5$, $w_{\text{dob}} = 0.3$, $w_{\text{city}} = 0.2$:

Attribute	Source A	Source B
Name	Johnson Michael R.	Michael R Johnson
Date of birth	1985-07-14	07/14/1985
City	Los Angeles	LA

(a) Parse both date representations and confirm they are the same date ($\sigma_{\text{dob}} = 1.0$). (b) Compute the Jaccard similarity on name tokens (split on space and punctuation); let $\sigma_{\text{name}} = J(T_A, T_B)$. (c) Assign $\sigma_{\text{city}} = 0.4$ (“Los Angeles” and “LA” are the same city but share no token; domain knowledge is required). (d) Compute $\sigma_{\text{composite}}$ and determine whether these records are a match at thresholds $\theta = 0.7$ and $\theta = 0.8$. (e) Discuss whether the

threshold choice here is an engineering or a business decision.

Design Problems

DP2.1. Integration Architecture for a Global Logistics Company. A global freight logistics company operates in 60 countries. Each country office uses a locally deployed ERP system (Oracle in North America, SAP in Europe, a custom-built system in Southeast Asia). The company needs a unified analytics platform that answers questions such as: “Which shipment routes had the highest delay rates last quarter?” and “What is the total revenue per customer across all regions?”

The three ERP systems use different customer identifiers, different currency representations (some systems store amounts in local currency only; others in USD), different status code vocabularies for shipment states, and different date formats. Real-time track-and-trace data (GPS position of shipments) arrives from 8,000 vehicles at 30-second intervals.

Design the integration architecture. Your design must specify:

- (a) The integration paradigm (ETL or ELT) and justification.
- (b) How each of the four heterogeneity dimensions is addressed.
- (c) The entity resolution strategy for customers (who may appear in multiple regional ERP systems with different identifiers).
- (d) How the real-time GPS stream is ingested and at what latency it becomes available for analytics.
- (e) How data lineage and provenance are maintained across the pipeline.
- (f) Which specific tools you would use at each stage and why.

DP2.2. Healthcare Data Quality Framework. A national health analytics agency receives patient-level data files from 200 hospitals every month. The files arrive as CSV exports from each hospital’s EHR system. The agency uses this data to compute national statistics on chronic disease prevalence, hospital readmission rates, and treatment outcome metrics.

A preliminary audit reveals: (1) 18 different date formats across the 200 hospitals; (2) ICD-10 codes mixed with ICD-9 codes in the same diagnosis field; (3) patient ages ranging from -3 to 247 (clearly erroneous); (4) 22%

of records missing at least one of: gender, ethnicity, or primary diagnosis; and (5) duplicate patient records at rates between 0.5% and 8% depending on the hospital.

Design a data quality pipeline that the agency can apply to incoming files before they are used in national statistics. Your design must:

- (a) Specify a quality gate for each of the five issues identified, including the rule, the action for records that fail (reject, flag, impute), and the threshold at which an entire hospital's submission is quarantined.
- (b) Describe how MCAR vs. MAR vs. MNAR missingness would be diagnosed for the 22% missing records, and what imputation strategy is appropriate for each case.
- (c) Explain how you would maintain provenance to satisfy HIPAA audit requirements while working with de-identified patient data.
- (d) Propose a data quality SLA (e.g., "we require >98% completeness on primary diagnosis before publishing national statistics") and explain how it would be monitored in production.

Programming Exercises

PE2.1. Entity Resolution with Jaccard Similarity (Python). Implement the Jaccard similarity function for string tokenisation and apply it to resolve company names across two source datasets.

```
1 import re
2
3 source_a = [
4     {"id": "A001", "name": "Google LLC", "city": "Mountain View"},
5     {"id": "A002", "name": "Apple Inc.", "city": "Cupertino"},
6     {"id": "A003", "name": "Microsoft Corp", "city": "Redmond"},
7     {"id": "A004", "name": "Meta Platforms Inc", "city": "Menlo Park"},
8     {"id": "A005", "name": "Amazon.com Inc", "city": "Seattle"},
9 ]
```

```
10
11 source_b = [
12     {"id": "B001", "name": "GOOGLE LLC",          "city": "
13         Mountain View"},
14     {"id": "B002", "name": "Apple Incorporated", "city": "
15         Cupertino"},
16     {"id": "B003", "name": "Microsoft Corporation", "city": "
17         Redmond"},
18     {"id": "B004", "name": "Facebook (Meta)",      "city": "
19         Menlo Park"},
20     {"id": "B005", "name": "Amazon Web Services", "city": "
21         Seattle"}],
22 ]
23
24 def tokenise(text):
25     """Lowercase, remove punctuation, split into tokens."""
26     text = re.sub(r"[.,\-(\)'"]", " ", text.lower())
27     return set(text.split())
28
29 def jaccard(tokens_a, tokens_b):
30     """
31     Return Jaccard similarity between two token sets.
32      $J(A, B) = |A \cap B| / |A \cup B|$ 
33     Return 0.0 if the union is empty.
34     """
35     # TODO: implement
36     pass
37
38 def blocking_key(record):
39     """
40     Return a blocking key: first token of name + first 3 chars
41     of city.
42     Records only compared within the same blocking key.
43     """
44     first_token = tokenise(record["name"])
45     city_prefix = record["city"][:3].lower()
46     # TODO: return key string
47     pass
48
49 def resolve_entities(source_a, source_b, threshold=0.5):
50     """
51     Return list of (a_id, b_id, score) tuples where score >=
52     threshold.
53     Use blocking to reduce comparisons.
54     """
```

```

47     """
48     matches = []
49     # TODO:
50     # 1. Build index of source_b records by blocking key
51     # 2. For each record in source_a, compute its blocking key
52     # 3. Compare only against source_b records with the same
53         key
54     # 4. Compute Jaccard on name tokens; append match if >=
55         threshold
56     return matches
57
58 matches = resolve_entities(source_a, source_b, threshold=0.4)
59 print("Resolved entity pairs:")
60 for a_id, b_id, score in matches:
61     a_rec = next(r for r in source_a if r["id"] == a_id)
62     b_rec = next(r for r in source_b if r["id"] == b_id)
63     print(f"    {a_id} ({a_rec['name']}) <-> {b_id} ({b_rec['name']})
64           "
65           f"score={score:.3f}")

```

Listing 2.2: PE2.1: Company name entity resolution

PE2.2. Data Quality Audit (Python). Implement a data quality profiler that checks a list of records against defined quality rules and produces a summary report.

```

1  import datetime
2
3  records = [
4      {"id": "R001", "name": "Alice Chen",
5       "dob": "1990-05-14", "email": "alice@example.com", "amount": 1250.00},
6      {"id": "R002", "name": "Bob Smith",
7       "dob": "1985-13-01", # invalid month
8       "email": None, "amount": 0.0},
9      {"id": "R003", "name": "", # empty name
10       "dob": "2035-01-01", # future date
11       "email": "carol@example.com", "amount": -50.0},
12      {"id": "R004", "name": "Diana Kumar",
13       "dob": "1978-03-22", "email": "diana@example.com", "amount": 980.50},
14      {"id": "R005", "name": "Eve Johnson",
15       "dob": "1978-03-22", # duplicate dob+name (same as R004
16                           roughly)

```

```
16     "email": None, "amount": 99999.0},
17 ]
18
19 def check_completeness(record, required_fields):
20     """Return list of field names that are None or empty string
21     ."""
22     missing = []
23     for f in required_fields:
24         v = record.get(f)
25         if v is None or v == "":
26             missing.append(f)
27     return missing
28
29 def check_dob_validity(dob_str):
30     """
31     Return True if dob_str is a valid date in ISO format (YYYY-
32     MM-DD)
33     and not in the future.
34     """
35     try:
36         dob = datetime.date.fromisoformat(dob_str)
37         return dob <= datetime.date.today()
38     except (ValueError, TypeError):
39         return False
40
41 def check_amount_validity(amount):
42     """Return True if amount is a positive number."""
43     try:
44         return float(amount) > 0
45     except (ValueError, TypeError):
46         return False
47
48 def profile_records(records):
49     """
50     Run all quality checks and print a summary report.
51     Dimensions checked: Completeness, Validity, Accuracy (range
52     ).
53     """
54     required = ["name", "dob", "email", "amount"]
55     issues = []
56
57     for rec in records:
58         rec_issues = {"id": rec["id"], "problems": []}
```



```
57     # Completeness
58     missing = check_completeness(rec, required)
59     if missing:
60         rec_issues["problems"].append(
61             f"Missing fields: {missing}")
62
63     # Validity: dob
64     if rec.get("dob") and not check_dob_validity(rec["dob"]):
65         rec_issues["problems"].append(
66             f"Invalid or future dob: {rec['dob']}")
67
68     # Validity: amount
69     if rec.get("amount") is not None:
70         if not check_amount_validity(rec["amount"]):
71             rec_issues["problems"].append(
72                 f"Non-positive amount: {rec['amount']}")
73
74     if rec_issues["problems"]:
75         issues.append(rec_issues)
76
77     # Summary
78     total = len(records)
79     clean = total - len(issues)
80     print(f"Quality Report: {clean}/{total} records passed all
81           checks\n")
82     for issue in issues:
83         print(f"    Record {issue['id']}:")
84         for p in issue["problems"]:
85             print(f"        - {p}")
86     return issues
87 profile_records(records)
```

Listing 2.3: PE2.2: Data quality profiler

PE2.3. ETL Pipeline Simulation (Python). Simulate a three-stage ETL pipeline that extracts records from two heterogeneous sources, applies transformations to resolve structural and syntactic heterogeneity, and loads unified records into a target schema.

```
1 import datetime
2
3 # Source A: CRM system (European date format, price in EUR)
```

```
4 source_a = [
5     {"contact_id": "C001", "fname": "Emma", "lname": "Weber",
6      "birth": "22.09.1988", "eur_value": 1400.0},
7     {"contact_id": "C002", "fname": "Luca", "lname": "Rossi",
8      "birth": "05.03.1975", "eur_value": 2200.0},
9 ]
10
11 # Source B: ERP system (US date format, price in USD, full name
12   column)
13 source_b = [
14     {"customer_no": "E045", "full_name": "Chen, Wei",
15      "date_of_birth": "07/18/1992", "usd_value": 3100.0},
16     {"customer_no": "E046", "full_name": "Park, Ji-Young",
17      "date_of_birth": "12/04/1983", "usd_value": 870.0},
18 ]
19 EUR_TO_USD = 1.09 # approximate exchange rate
20
21 # Target schema:
22 # { "id", "first_name", "last_name", "dob" (ISO), "value_usd" }
23
24 def transform_source_a(record):
25     """
26     Map CRM record to target schema.
27     - contact_id -> id
28     - fname/lname -> first_name/last_name
29     - birth (DD.MM.YYYY) -> dob (YYYY-MM-DD)
30     - eur_value * EUR_TO_USD -> value_usd
31     """
32     # TODO: implement transformation
33     pass
34
35 def transform_source_b(record):
36     """
37     Map ERP record to target schema.
38     - customer_no -> id
39     - full_name ("Last, First") -> first_name, last_name
40     - date_of_birth (MM/DD/YYYY) -> dob (YYYY-MM-DD)
41     - usd_value -> value_usd (already USD)
42     """
43     # TODO: implement transformation
44     pass
45
46 def run_etl(source_a, source_b):
```

```
47     """Extract, transform both sources, and load into unified
48         target."""
49
50     # Transform source A
51     for rec in source_a:
52         transformed = transform_source_a(rec)
53         if transformed:
54             target.append(transformed)
55
56     # Transform source B
57     for rec in source_b:
58         transformed = transform_source_b(rec)
59         if transformed:
60             target.append(transformed)
61
62     return target
63
64 unified = run_etl(source_a, source_b)
65 print("Unified target records:")
66 for r in unified:
67     print(f"    {r}")
```

Listing 2.4: PE2.3: ETL pipeline simulation

Table 2.3: ETL vs. ELT: a systematic comparison

Attribute	ETL	ELT
Transformation timing	Before loading (staging area)	After loading (at query time)
Target schema	Predefined, strict (schema-on-write)	Flexible (schema-on-read)
Data quality at target	High (validated before load)	Variable (raw data preserved)
Historical replay	Expensive (re-run ETL jobs)	Easy (re-transform from raw)
Schema evolution	Difficult (pipeline rebuild)	Easy (update transformation only)
Raw data preservation	No (transformed data only)	Yes (raw always available)
Storage cost	Lower (transformed, compressed)	Higher (raw data at full volume)
Latency	Higher (transform-then-load)	Lower (load immediately)
Tooling	Informatica, Talend, SSIS	Spark, dbt, Airbyte, Fivetran
Best suited for	Regulated industries, strict SLAs	Data lakes, ML feature stores, exploratory analytics

Table 2.4: The modern data integration tool ecosystem

Tool	Direction	Pattern	Primary use case
Apache Sqoop	RDBMS ↔ HDFS	Batch	Bulk migration of relational tables into Hadoop
Apache Flume	Log sources → HDFS	Batch/micro-batch	Web server log aggregation
Apache Kafka	Any → Any	Stream	Real-time event bus; decoupled source-to-sink
Airbyte / Fivetran	SaaS APIs → Data lake	Batch/incremental	No-code connector platform for 200+ sources
Apache Spark	Data lake → Any	Batch/stream	Large-scale transformation and enrichment
dbt	Data lake/DW internal	Batch	SQL-based transformation, testing, lineage
Apache Atlas	Any	Metadata	Data governance, lineage graph, cataloging

Part II

Distributed Storage

Distributed File Systems: GFS and HDFS

“We have designed the system for a usage pattern in which most files are mutated by appending new data rather than overwriting existing data. Random writes within a file are practically non-existent.”

Ghemawat, Gobioff & Leung, SOSP 2003

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why conventional file systems (NFS, POSIX, SAN) cannot meet the storage requirements of petabyte-scale data engineering, and identify the specific assumptions they violate.
2. Describe the GFS architecture—master, chunkservers, clients, chunk size, and metadata design—and explain the key design decisions that made it suitable for Google’s workload.
3. Draw and explain the HDFS architecture: NameNode, DataNodes, Secondary NameNode, and their interactions.
4. Trace the complete HDFS read and write paths step by step, including pipeline replication, checksum verification, and lease management.
5. Explain rack-aware replication: the default three-replica placement policy and why it balances fault tolerance against write overhead.

6. Describe the NameNode high-availability (HA) architecture with Active/Standby nodes, ZooKeeper leader election, and shared Journal Nodes.
7. Identify HDFS's design limitations—the small file problem, no random writes, low-latency access—and contrast HDFS with cloud object stores (Amazon S3, Google Cloud Storage).

Chapter 2 established how data moves between systems and how heterogeneous sources are unified. Before any of that integration logic can execute, however, the raw data must live somewhere. In 2003, Google faced a storage problem that no existing file system could solve: billions of web pages, totalling petabytes of data, needed to be stored reliably across thousands of commodity servers, indexed sequentially at enormous throughput, and kept available even as individual servers failed—which, at that scale, happened multiple times per day.

The solution Google published was the *Google File System* (GFS) (Ghemawat, Gobioff, and Leung 2003). Three years later, the Apache Hadoop project reimplemented and open-sourced the same design as the *Hadoop Distributed File System* (HDFS). HDFS became the foundational storage layer for an entire ecosystem of distributed data processing tools—MapReduce, Hive, Pig, HBase, and eventually Spark—and it remains in active production use at organisations ranging from Yahoo! and Facebook to LinkedIn and the New York Times. Understanding HDFS is understanding the bedrock on which the rest of this book's processing stack rests.

3.1 The Storage Problem at Petabyte Scale

A *file system* is the software layer responsible for organising, naming, accessing, and protecting data stored on physical storage devices. The file system abstracts away the physical device—magnetic platters, solid-state cells, or optical discs—and presents the programmer with a uniform interface of named files in a hierarchical directory structure. For four decades, this model served the computing industry well. It continues to serve it well

for most workloads today.

The model breaks at petabyte scale for three fundamental reasons.

Single-machine capacity. A conventional file system is managed by a single host. Even the largest shared file systems— Network File System (NFS) and SAN-attached volumes—ultimately depend on a finite number of storage controllers. By 2003, Google’s web crawl was producing hundreds of terabytes per month, a rate that had already surpassed what any single storage system could absorb. The only solution was to distribute data across a cluster of independent machines.

Throughput requirements. Web indexing requires reading an entire dataset sequentially to reprocess it—there is no practical way to index a subset of the web. A single high-performance SCSI disk in 2003 delivered approximately 50 MB/s of sequential read throughput. Reading one petabyte from a single disk would take over 230 days. Distributing the data across 1,000 disks and reading them in parallel reduces this to approximately five hours—a feasible engineering timeline.

Hardware failure as normal operation. A conventional file system—and the RDBMS systems that sit on top of it—treats hardware failure as an exceptional condition to be recovered from. At Google’s scale in 2003, a cluster of a few thousand commodity PCs experienced approximately three node failures per day as normal operation: disk bearing failures, memory errors, motherboard failures, and network card failures, all occurring on a predictable statistical schedule (Ghemawat, Gobioff, and Leung 2003). Any storage system that treated such failures as exceptional events would be unavailable for recovery most of the time. The failure model must instead treat hardware failure as *the expected case* and design accordingly.

3.2 The Google File System

The Google File System, published by Ghemawat, Gobioff, and Leung at the ACM Symposium on Operating Systems Principles (SOSP) in October 2003, is one of the most consequential systems papers of the past two decades (Ghemawat, Gobioff, and Leung 2003). It describes a purpose-built distributed file system designed not to satisfy general-purpose file system requirements—POSIX compliance, low-latency random access, small file

Table 3.1: Comparison of storage architectures for big data workloads

System	Max capacity	Fault model	Reason it fails at big data scale
POSIX local FS	Single disk / array	Single point of failure	No horizontal scaling; one server limit
NFS	NAS head + array	NAS head is SPOF	Centralized metadata and I/O bottleneck
SAN	Storage array	Expensive RAID	Proprietary hardware; prohibitive cost/PB
RDBMS blob	Server RAM + disk	WAL recovery	Tuned for random access; not sequential scan
GFS / HDFS	Cluster-wide	Expects failure	Designed for this workload

support—but to satisfy the specific requirements of Google’s production workloads.

3.2.1 Workload Assumptions and Design Decisions

GFS was designed around a careful empirical study of Google’s actual workloads. The authors observed that virtually all files in Google’s production environment were large—multiple gigabytes to hundreds of gigabytes—and that data access patterns were dominated by sequential reads and sequential appends, with random writes essentially absent. Files were written once, read many times, and never modified in place after initial creation. Concurrent reads from many clients were the norm; concurrent random writes to the same file were not.

These observations drove six key design decisions that distinguish GFS from conventional file systems:

1. **Large chunk size (64 MB).** Files are divided into fixed-size *chunks* of 64 MB. This is orders of magnitude larger than the 4-KB blocks used by conventional file systems. Large chunks reduce the amount of metadata the master must maintain (one entry per chunk rather than one

per block), reduce client-to-master communication (a client needs the location of fewer chunks to read a large file), and encourage direct TCP connections between clients and chunkservers for sustained sequential I/O.

2. **Centralised, in-memory metadata.** The GFS master holds all file system metadata—the file namespace, chunk-to-file mappings, chunk locations, access control lists—entirely in RAM. This enables sub-millisecond metadata lookups. The master has a single-machine capacity for metadata because large chunk sizes keep the number of chunks manageable: at 64 MB per chunk, one petabyte of data requires only ~16 million chunk entries, which fits comfortably in modern server RAM.
3. **No data caching.** GFS explicitly does not cache file data at either clients or chunkservers. The workload is dominated by streaming reads that iterate through files once; cached data would be evicted before it could be reused. The kernel’s buffer cache handles any incidental temporal locality.
4. **Write-by-append semantics.** GFS optimises for atomic append operations rather than random writes. A client can append data to any file at the current end-of-file position; this corresponds to how log files, web crawl outputs, and search index segments are naturally produced.
5. **Fault tolerance through replication, not redundant hardware.** Each 64 MB chunk is stored on three chunkservers by default. If one fails, the master re-replicates the affected chunks on a surviving server. No RAID arrays or specialised storage hardware are required.
6. **Relaxed consistency.** GFS provides a consistency model weaker than POSIX. Concurrent writes to the same region from different clients may produce an interleaved, “defined but inconsistent” result. Google’s applications were designed to tolerate this—for example, by writing unique chunk identifiers with each record and using deduplication at read time.

3.2.2 GFS Architecture

A GFS cluster consists of three types of components: one *GFS master*, multiple *chunkservers*, and multiple *clients*.

The **GFS master** maintains all file system metadata in RAM. It does not participate in data I/O between clients and chunkservers; once a client

has obtained the chunk location from the master, all data transfer occurs directly between the client and the chunkserver. This separation of the control plane (master) from the data plane (chunkservers and clients) is the central architectural decision that enables GFS to scale: the master handles only lightweight metadata operations, while the bulk data flows on separate direct connections.

The master communicates with chunkservers through two periodic mechanisms. Each chunkserver sends a *heartbeat* to the master every few seconds to confirm it is operational; the absence of heartbeats for a configurable period causes the master to mark the chunkserver as failed. Each chunkserver also sends a periodic *chunk report* listing all the chunk replicas it currently holds, which allows the master to reconstruct its view of chunk locations after a restart.

Chunkservers store chunks as plain files on their local Linux file systems. They do not cache chunks in memory; the kernel's buffer cache provides whatever temporal locality exists. Each chunkserver is responsible for maintaining checksum metadata for every chunk it stores, and it verifies checksums on every read to detect data corruption.

Clients implement the GFS API (which is not POSIX-compatible). A client contacts the master to resolve a file name to a list of chunk handles and their chunkserver locations, then communicates directly with the appropriate chunkservers for data I/O. Clients cache the chunk location information locally for a limited time, reducing master load.

Research Spotlight | Ghemawat, Gobioff & Leung (2003) — The Google File System

Citation: Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). The Google File System. *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 29–43. (Ghemawat, Gobioff, and Leung 2003)

Key insight: The authors' central argument is that *workload-specific design* outperforms general-purpose design at extreme scale. By deliberately abandoning POSIX compliance, random write support, and fine-grained access control, GFS achieved throughput and fault tolerance impossible with any off-the-shelf file system.

Technical contributions:

- Demonstrated that a centralised metadata server (the master) is not

a scalability bottleneck when (a) metadata is kept entirely in RAM, (b) chunk sizes are large enough that the total metadata volume is manageable, and (c) data I/O bypasses the master entirely.

- Introduced *atomic record append*—an operation that allows multiple clients to append to the same file concurrently without prior coordination, enabling producer-consumer pipelines over shared log files.
- Quantified failure rates in commodity clusters and provided the engineering argument that replication (software-based redundancy on commodity hardware) is more cost-effective than hardware RAID at petabyte scale.

Impact today: GFS is cited in over 14,000 academic papers— one of the most-cited systems papers ever published. It directly inspired HDFS (the open-source reimplementation), Amazon S3, Microsoft Azure Blob Storage, and Google’s own successor system, Colossus. The paper is assigned reading at Stanford CS246, MIT 6.830, and CMU 15-721.

3.3 HDFS Architecture

The Hadoop Distributed File System is the open-source Apache implementation of the GFS design. It was initially developed at Yahoo! by Doug Cutting and Mike Cafarella—who had been building a web crawler called Nutch and needed a production-ready open-source replacement for GFS—and contributed to the Apache Software Foundation in 2006. HDFS translates the GFS architecture into Java, with some naming changes (GFS master → NameNode, chunkserver → DataNode, chunk → block) and some incremental design improvements.

3.3.1 The NameNode

The *NameNode* is HDFS’s master process. It is the centralised metadata server responsible for maintaining the file system namespace and the block-to-DataNode mapping. Like the GFS master, it holds all metadata entirely in RAM; the typical NameNode memory footprint is approximately 150 bytes

per file and 150 bytes per block, meaning that one million files average ~300 MB of heap—manageable on a modern server.

The NameNode’s metadata is persisted to disk in two complementary structures. The *FsImage* is a complete, point-in-time snapshot of the file system namespace—a serialised checkpoint of all metadata. The *EditLog* is a write-ahead log of all namespace mutations since the last checkpoint: file creations, deletions, permission changes, and block allocations. On startup, the NameNode replays the EditLog against the last FsImage to reconstruct the current namespace state. Under normal operation, the EditLog grows continuously; it is periodically merged into the FsImage in a process called *checkpointing*.

The NameNode does *not* store the physical locations of blocks in persistent metadata. Instead, block locations are rebuilt from scratch at startup through the DataNode *block report*—each DataNode reports the full list of blocks it currently holds when it connects to the NameNode. This design avoids the risk of stale location data in the FsImage becoming inconsistent with the actual DataNode state after failures or maintenance.

Definition | NameNode Responsibilities

1. **Namespace management:** maintain the directory tree and file-to-block mapping.
2. **Block management:** track which DataNodes hold each block replica; initiate re-replication when a replica is lost.
3. **Access control:** enforce file permissions on all client requests.
4. **Lease management:** grant and revoke write leases that determine which client may currently write to a file.
5. **Load balancing:** coordinate the HDFS Balancer, which moves blocks between DataNodes to equalise disk utilisation.

3.3.2 DataNodes

DataNodes are the worker processes that store actual block data. Each DataNode manages its local storage—one or more hard disks or SSDs—and serves block read and write requests directly from clients. DataNodes are designed to run on commodity hardware; a typical DataNode configuration

in 2024 uses 12–24 spinning hard disks of 8–18 TB each, providing 96–432 TB of raw capacity per node.

DataNodes communicate with the NameNode through two periodic messages. A *heartbeat* (sent every 3 seconds by default) confirms the DataNode is alive and operational. A *block report* (sent every 6 hours by default, or on demand at startup) lists all block replicas the DataNode currently holds. The NameNode uses both to maintain its live view of the cluster's block map. If a heartbeat is absent for more than 10.5 minutes (the default $\text{dfs.heartbeat.interval} \times \text{dfs.namenode.heartbeat.recheck-interval}$), the NameNode marks the DataNode as dead and schedules re-replication of all blocks it held.

DataNodes also perform *block scanning*: each DataNode independently verifies the checksums of all its stored blocks on a configurable schedule (default: every three weeks). A block that fails checksum verification is reported to the NameNode as corrupted, and the NameNode responds by scheduling re-replication from a healthy replica.

3.3.3 The Secondary NameNode

Common Misconception | The Secondary NameNode is Not a Hot Standby

Despite its misleading name, the **Secondary NameNode** is not a standby that takes over if the primary NameNode fails. It performs a single function: it periodically merges the EditLog into the FsImage, producing a new, updated FsImage that it sends back to the primary NameNode. This *checkpointing* service reduces the EditLog size and therefore the NameNode startup time after a crash. If the primary NameNode fails, the Secondary NameNode's checkpoint can be used to recover the namespace, but this requires a manual failover process and results in some minutes of downtime. True hot standby failover requires the High Availability architecture described in Section 3.6.

The checkpointing cycle works as follows. The Secondary NameNode fetches the current FsImage and EditLog from the primary, applies all EditLog transactions to the FsImage in memory, writes the resulting merged FsImage to disk, and sends it back to the primary. The primary then begins writing subsequent namespace mutations to a fresh, empty EditLog.

The net result is that the EditLog never grows indefinitely; checkpointing occurs every hour by default, or when the EditLog exceeds one million transactions.

3.3.4 Block Storage

Blocks are the fundamental unit of storage in HDFS. The default block size is 128 MB (increased from the original 64 MB in Hadoop 2.x). When a client writes a file, HDFS divides it into 128 MB blocks and distributes those blocks across DataNodes; each block is stored three times by default (the *replication factor*).

The large block size—32,768 times larger than a typical ext4 file system block—serves several purposes. It reduces the number of metadata entries the NameNode must maintain. It enables sustained sequential I/O at full disk throughput without per-block overhead. And it reduces the number of round trips between client and NameNode required to locate all blocks of a large file.

A notable edge case: the last block of a file may be smaller than 128 MB. HDFS does not pad it to 128 MB; the block occupies only as much disk space as the actual data. A 1-GB file consists of eight full 128 MB blocks, not nine.

The Complete HDFS Cluster Architecture

Figure 3.1 assembles all three components into a single view of the HDFS cluster architecture. The key point to internalise is the separation of the *control plane* (all metadata operations that involve the NameNode) from the *data plane* (all block transfers that flow directly between clients and DataNodes). The NameNode processes millions of metadata RPC calls per day but never touches a byte of block data; every byte of data is transferred directly between clients and DataNodes on independent TCP connections.

3.4 Replication and Fault Tolerance

HDFS achieves fault tolerance through **block replication**: storing each block on multiple DataNodes such that if any individual DataNode fails, the

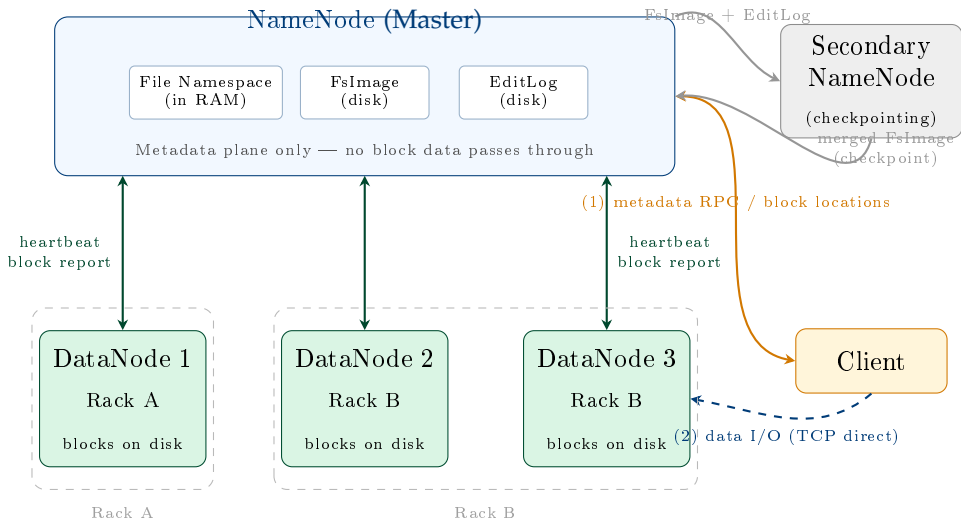


Figure 3.1: Complete HDFS cluster architecture. The NameNode (master) stores all file system metadata in RAM (file namespace, block-to-DataNode map) and persists mutations to the EditLog and FsImage. DataNodes (workers) store blocks on local disks and communicate with the NameNode via periodic heartbeats and block reports. Clients obtain block locations from the NameNode (*step 1 — control plane*), then read/write block data directly to DataNodes (*step 2 — data plane*), bypassing the NameNode entirely. The Secondary NameNode checkpoints the EditLog into the FsImage to bound restart time after a NameNode crash.

remaining replicas ensure data availability. The default replication factor of 3 means that the failure of any single DataNode does not result in any data loss or unavailability.

3.4.1 Rack-Aware Replication

Naive replication—placing all three replicas on randomly chosen DataNodes—would provide poor fault tolerance in a real data centre. Physical servers in a data centre are organised in **racks**: sets of 20–40 servers sharing a top-of-rack switch. A rack-level failure (a switch failure or a power distribution unit trip) takes down every server in the rack simultaneously. If all three replicas of a block happened to reside on nodes within the same rack, a rack failure would destroy all three replicas.

HDFS addresses this with **rack-aware replication**. The default policy for replication factor 3 places:

- **Replica 1:** on the same node as the writing client (or a random node if the client is not running on a DataNode).
- **Replica 2:** on a DataNode in a *different* rack from Replica 1.
- **Replica 3:** on a *different* DataNode in the same rack as Replica 2.

This placement balances three competing objectives. Placing Replica 1 locally (or on a nearby node) minimises write latency for the initial block placement. Placing Replica 2 on a different rack ensures that a single rack failure cannot destroy all replicas. Placing Replica 3 in the same rack as Replica 2 (rather than a third rack) reduces cross-rack network traffic: reads can often be satisfied from within the same rack as the client, reducing the load on the inter-rack network bandwidth.

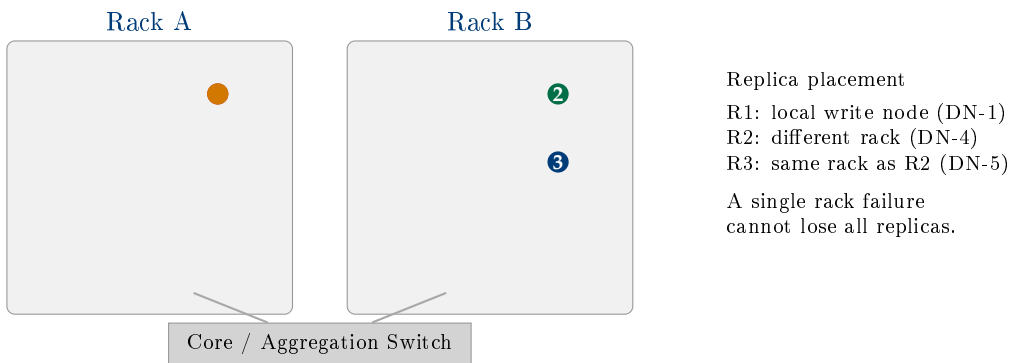


Figure 3.2: HDFS rack-aware replication with replication factor 3. Two replicas in Rack B; one in Rack A. A Rack A failure leaves Replicas 2 and 3 available.

3.4.2 Failure Detection and Re-replication

When the NameNode detects that a DataNode is dead (missed heartbeats for more than the configured threshold), it immediately identifies every block for which that DataNode held a replica. For each such block, the NameNode checks whether the remaining replicas satisfy the replication factor. If a block now has fewer than the required number of replicas, the NameNode schedules *re-replication*: it instructs one of the surviving DataNodes that holds a copy to transfer the block to a new DataNode, restoring the full replication factor.

Re-replication is prioritised by urgency. Blocks that have only one surviving replica (single-replica blocks) are re-replicated first, since they are at

immediate risk of data loss if the last surviving DataNode also fails. Blocks with two surviving replicas are re-replicated next. The re-replication process consumes cluster network bandwidth; in a large cluster where many DataNodes are replaced simultaneously (e.g., during a scheduled maintenance window), the NameNode throttles re-replication to avoid saturating the network.

3.5 HDFS Read and Write Paths

Understanding how data actually flows in and out of HDFS—not merely the architecture of the components but the sequence of operations—is essential for diagnosing performance problems and for understanding why HDFS makes the consistency trade-offs it does.

3.5.1 The Read Path

Reading a file from HDFS involves the following sequence:

1. **Open.** The client calls `FileSystem.open()`. The HDFS client library contacts the NameNode via RPC and requests the block locations for the first few blocks of the target file.
2. **Block location response.** The NameNode returns an ordered list of DataNode addresses for each requested block, sorted by their network proximity to the client. The list prioritises DataNodes on the same node as the client, then the same rack, then other racks—the data locality principle from Chapter 1.
3. **Direct DataNode connection.** The client connects directly to the closest DataNode for the first block and streams the data. The client reads data from the DataNode through a TCP socket; no data passes through the NameNode.
4. **Checksum verification.** As each 512-byte *checksum chunk* is read, the client verifies its CRC-32C checksum against the stored checksum. A mismatch indicates data corruption; the client reports the bad block to the NameNode and retries on an alternative replica.
5. **Block boundary.** When one block is fully read, the client contacts the

- NameNode for the locations of subsequent blocks (if not already cached) and opens a new connection to the appropriate DataNode.
6. **Close.** When all blocks have been read, the input stream is closed.

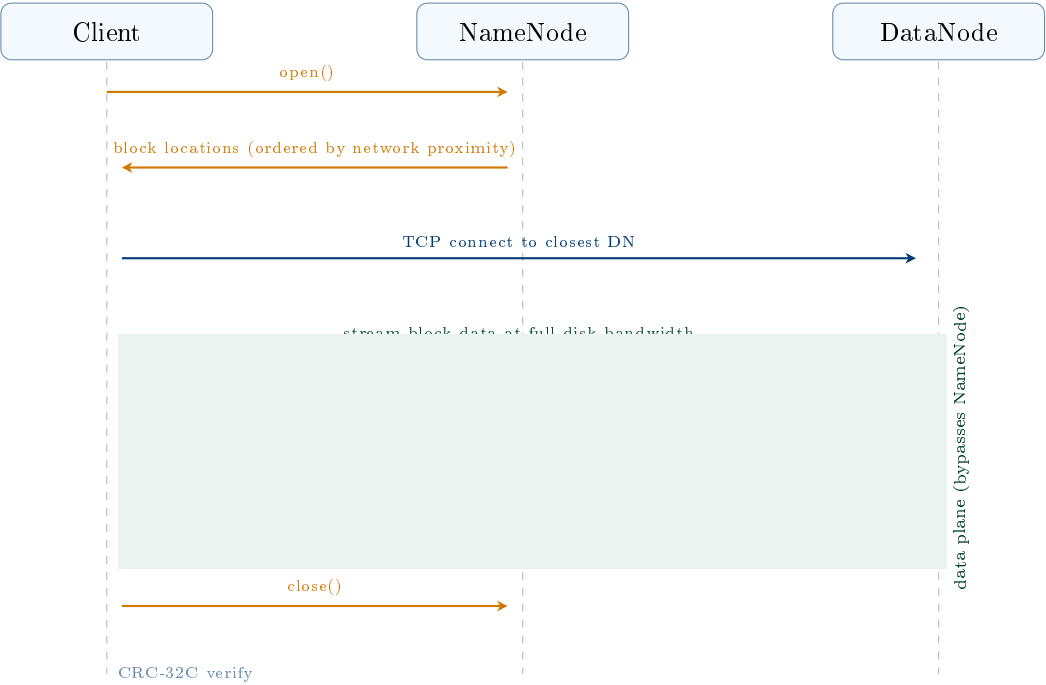


Figure 3.3: HDFS read-path sequence diagram. Steps 1–2 are control-plane operations (RPC to the NameNode); steps 3–6 are data-plane operations (direct TCP to DataNodes). The shaded region highlights that all block data flows between client and DataNode, never through the NameNode. Checksum verification (step 5) runs at the client for every 512-byte chunk; a mismatch triggers a retry on a different replica.

The key performance characteristic of the read path is that data flows directly between client and DataNode, bypassing the NameNode entirely. This means that the NameNode’s throughput is not a bottleneck for read operations: it handles only the metadata lookup (typically measured in microseconds), while data transfer runs at full network speed (typically 1–10 Gbit/s per connection).

3.5.2 The Write Path

Writing to HDFS is more complex than reading, because data must be reliably distributed to multiple DataNodes simultaneously while ensuring consistency in the presence of potential DataNode failures.

1. **Create.** The client calls `FileSystem.create()`. The client library contacts the NameNode to create a new file entry in the namespace. The NameNode verifies that the file does not already exist and that the client has write permission, then creates the file and grants the client a *write lease*.
2. **Block allocation.** As the client accumulates data in a write buffer (default 64 KB per packet), it requests a new block from the NameNode. The NameNode selects DataNodes for the block—applying the rack-aware placement policy—and returns a list of DataNodes ordered by the pipeline sequence.
3. **Pipeline construction.** The client establishes a TCP connection to the first DataNode in the pipeline; that DataNode connects to the second; the second connects to the third. This forms a *replication pipeline*: data flows through the pipeline rather than being sent from the client to each DataNode independently, reducing the client's outbound bandwidth requirement.
4. **Packet streaming.** The client sends data in *packets* of 64 KB down the pipeline. As the first DataNode receives a packet, it writes it to its local disk and simultaneously forwards it to the second DataNode. The second DataNode writes and forwards to the third. Each DataNode sends an *acknowledgement* back up the pipeline once it has durably written the packet.
5. **Block completion.** When all packets for a block have been acknowledged by all three DataNodes, the client notifies the NameNode that the block is complete. The NameNode records the block's DataNode locations in its metadata.
6. **Close.** When the client calls `close()`, all remaining buffered packets are flushed through the pipeline, all blocks are completed, and the NameNode marks the file as complete, releasing the write lease.

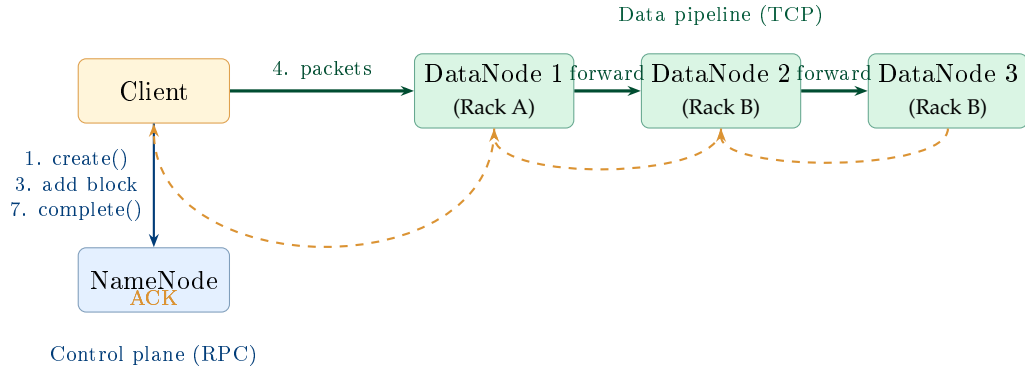


Figure 3.4: HDFS write path. The client communicates with the NameNode only for control messages; all block data flows through the three-DataNode replication pipeline. ACKs propagate back through the pipeline to confirm durable writes.

System Design Note | Pipeline Failure Handling

If a DataNode fails mid-pipeline during a write, HDFS recovers as follows. The pipeline is closed. Packets that were acknowledged by the failed DataNode but not yet confirmed by the client are marked for re-send. A new block replica is allocated on a healthy DataNode, and a new pipeline is constructed excluding the failed DataNode. The NameNode is notified of the incomplete replica on the failed node and schedules re-replication after the write completes. The client experiences a brief pause but the write succeeds as long as at least one DataNode survives. This mechanism allows HDFS to absorb DataNode failures *during* a write without requiring the client to restart the entire write operation.

3.6 HDFS High Availability

In the original HDFS design, the NameNode is a *single point of failure* (SPOF). If the NameNode process crashes or the NameNode machine becomes unavailable, the entire HDFS cluster is inaccessible: no client can resolve file names to block locations, and no DataNode can be instructed

to re-replicate data. At the scale of Yahoo!'s production Hadoop cluster—42,000 nodes in 2011—an unplanned NameNode outage meant that 42,000 DataNodes became inaccessible simultaneously. The resulting loss of analytical capacity during the recovery period (which could take 30–60 minutes for a large namespace) was a significant operational risk.

Hadoop 2.0 (released 2012) introduced *HDFS High Availability* (HA), which eliminates the NameNode SPOF by maintaining two NameNode instances simultaneously: one *Active NameNode* that handles all client and DataNode traffic, and one *Standby NameNode* that continuously mirrors the Active's metadata state and can take over within seconds if the Active fails.

3.6.1 The HA Architecture

The HA architecture requires three key components beyond the standard HDFS setup.

Journal Nodes are a quorum of lightweight processes (a minimum of three, to tolerate one failure) that implement a shared, fault-tolerant edit log. The Active NameNode writes every namespace mutation to the Journal Node quorum as it executes; the Standby NameNode reads from the Journal Node quorum continuously, replaying edits to keep its in-memory namespace state synchronised with the Active. Because the Journal Node quorum is shared, the Standby's namespace state is always at most a few hundred milliseconds behind the Active.

ZooKeeper provides distributed leader election. Each NameNode runs a *ZooKeeper Failover Controller* (ZKFC) process that monitors the NameNode's health and maintains a session with ZooKeeper. If the Active NameNode becomes unhealthy—as detected by the ZKFC—the Active's ZooKeeper session expires, and ZooKeeper triggers a leader election. The Standby's ZKFC wins the election and promotes the Standby to Active.

Fencing prevents the *split-brain* scenario—where both NameNodes believe themselves to be Active and independently accept writes, resulting in divergent metadata states. Before promoting the Standby, the ZKFC executes a fencing operation that either gracefully stops the former Active (via SSH) or, if the former Active is unreachable, forcibly terminates its ability to communicate with DataNodes using SSH-based remote kill commands or

STONITH (“Shoot The Other Node In The Head”) hardware mechanisms.

Table 3.2: NameNode HA component summary

Component	Role
Active NameNode	Handles all client and DataNode traffic; writes edits to JN quorum
Standby NameNode	Continuously replays JN edits; ready to take over within seconds
Journal Nodes (3+)	Shared, quorum-based edit log; tolerates minority failures
ZooKeeper (3+)	Leader election and health monitoring for the two NameNodes
ZKFC	Per-NameNode daemon that manages ZooKeeper session and triggers failover

3.7 HDFS Design Characteristics and Limitations

HDFS is a purpose-built system optimised for specific workloads. Like GFS, its strengths and weaknesses are two sides of the same coin: the design decisions that make it excellent for batch processing of large files make it unsuitable for other access patterns.

3.7.1 What HDFS Is Optimised For

HDFS excels at **high-throughput, sequential access to large files**. A well-tuned HDFS cluster can deliver aggregate read throughput of several GB/s per node: with 100 DataNodes each delivering 200 MB/s from local disk, the cluster sustains 20 GB/s of aggregate read bandwidth. For workloads that need to scan entire datasets—MapReduce jobs, Hive queries that process all records in a table, Spark jobs that re-partition a large dataset—this throughput is the critical metric, and HDFS is optimised for it.

HDFS also provides **fault-tolerant write durability**. A write that returns a success acknowledgement to the client is guaranteed to have been

committed to at least r DataNodes (where r is the replication factor). Hardware failure of any subset of up to $r - 1$ DataNodes simultaneously cannot cause data loss for any successfully written block.

3.7.2 The Small File Problem

HDFS's single most significant operational limitation is its poor handling of large numbers of small files. Each file, regardless of size, consumes approximately 150 bytes of NameNode heap space for its metadata. A single large file split into ten 128 MB blocks consumes approximately 1.65 KB of NameNode memory. Ten million small files—each taking one block—consume 1.5 GB of NameNode heap, regardless of the files' total data volume. At 100 million small files, the NameNode requires 15 GB of heap and metadata operations begin to slow noticeably.

This is a critical practical constraint. Web server log files, sensor telemetry archives, and social media ingestion pipelines naturally produce large numbers of small files if not carefully managed. The standard engineering response is to use *SequenceFiles*—container files that aggregate many small records into a single large HDFS file, providing a sequential index for record-level access—or to pre-aggregate small files before ingestion using tools such as HDFS archive (HAR files) or Apache CombineFileInputFormat in the MapReduce layer.

3.7.3 No In-Place Random Writes

HDFS does not support random writes to existing files. Once a block has been written and acknowledged, its content cannot be modified in place. HDFS supports only sequential writes to new files and atomic appends to the end of existing files (the append semantics introduced in HDFS 0.21 and Hadoop 2.x). This is a deliberate architectural choice that greatly simplifies the consistency model and the replication protocol: if blocks are immutable after writing, the system never needs to coordinate writes across multiple replicas.

For workloads that require in-place updates—a transactional database, an incremental index—HDFS is the wrong tool. Apache HBase (examined in Chapter 4) provides random read/write access on top of HDFS by using

an LSM-tree (Log-Structured Merge-tree) structure that converts random writes into sequential HDFS writes.

3.7.4 No Low-Latency Random Access

HDFS is tuned for throughput, not latency. Opening an HDFS file requires a round trip to the NameNode; reading a specific block requires a TCP connection setup to the appropriate DataNode. Total latency for a random read is typically in the range of 50–200 ms— acceptable for batch processing but wholly unacceptable for interactive queries (which require sub-millisecond storage access) or for OLTP workloads. This is why HDFS is used as the storage substrate for batch and near-batch systems (MapReduce, Spark, Hive), not as a replacement for relational databases or key-value stores.

Table 3.3: HDFS strengths, limitations, and engineering mitigations

Characteristic	Impact	Mitigation / alternative
Optimised for sequential reads	Excellent batch throughput	Use for MapReduce, Spark, Hive
Large block size (128 MB)	Poor for small files	SequenceFiles, HAR archives, Combine-FileInputFormat
No random writes	Cannot update in place	HBase (LSM-tree over HDFS)
High read latency (50–200 ms)	No interactive queries	Presto, Spark SQL with in-memory caching
NameNode SPOF (pre-HA)	Cluster-wide outage	HDFS HA with ZooKeeper
Namespace scalability	100M file ceiling	HDFS Federation (multiple NameNodes)

3.8 Beyond HDFS: Cloud Object Storage

Starting around 2012 and accelerating through 2015–2018, the data engineering industry began a significant migration away from on-premises HDFS clusters toward cloud-based *object storage*: Amazon S3, Google Cloud Storage (GCS), and Azure Data Lake Storage (ADLS). By 2024, cloud object storage has largely supplanted HDFS as the primary storage layer for new data engineering deployments, though HDFS remains in use at organisations that operate their own data centres.

Cloud object storage differs from HDFS in several important ways. Objects in S3 or GCS are identified by a key (essentially a path string) and stored as a flat namespace with no directory hierarchy (though object key prefixes mimic directory structure). Unlike HDFS blocks, objects can be up to 5 TB each. Object storage is practically unlimited in capacity and scales automatically with usage—there is no NameNode to run out of heap, no replication factor to configure manually, and no DataNodes to provision and maintain.

From a pricing and operations standpoint, cloud object storage is dramatically simpler. Amazon S3 charges approximately \$0.023/GB/month for standard storage; there are no servers to operate, no replication factor to configure, and no NameNode HA to maintain. A data engineering team using S3 eliminates the entire operational burden of the HDFS cluster in exchange for a predictable per-GB-per-month cost.

The principal trade-off is **decoupling storage from compute**. In a Hadoop cluster, DataNodes run both storage and compute (MapReduce or Spark tasks), exploiting data locality. Cloud object storage is physically separate from the compute cluster; every read requires a network hop. However, cloud data centre networks now deliver bandwidth of 25–100 Gbit/s between storage and compute instances, which is often higher than the inter-DataNode bandwidth in on-premises Hadoop clusters. Modern frameworks such as Apache Spark have been optimised to run effectively against object storage, and the loss of data locality has proven to be a minor performance penalty in most workloads compared to the operational simplicity of S3.

Key Insight | The Shift to Storage-Compute Separation

The move from HDFS to cloud object storage represents a fundamental architectural shift: from *tightly coupled* storage-compute (where data lives next to the CPU that processes it) to *loosely coupled* (where storage and compute scale independently). In a Hadoop cluster, you cannot add compute capacity without also adding storage, and vice versa. With S3 + cloud compute, you can scale a Spark cluster from 10 to 1,000 nodes and back to 10 without moving a single byte of data. This elasticity has proven to be the dominant engineering advantage, and it is the primary reason cloud object storage has displaced on-premises HDFS for new deployments.

3.9 Case Study: Yahoo!’s Hadoop Cluster (2006–2011)

Case Study | Yahoo! — The First Industrial-Scale Open-Source Distributed File System

Background. Yahoo! was the first organisation outside Google to operate a production Hadoop cluster at true industrial scale. The company had several workloads that required petabyte-scale batch processing: web ranking (recomputing the search index nightly from the full crawl), user behaviour analytics (processing billions of page views per day), and spam detection (scanning email metadata across hundreds of millions of user accounts). After evaluating alternatives, Yahoo! bet its entire search and analytics infrastructure on the open-source Hadoop ecosystem starting in 2006.

Scale and growth. Yahoo!’s Hadoop cluster grew from approximately 1,000 nodes in 2006 to a peak of **42,000 nodes** in 2011—a 42-fold scale-up in five years. At its peak, the cluster stored approximately **100 petabytes** of data in HDFS across two major data centres. The web index computation alone ran tens of thousands of MapReduce jobs per day. Yahoo!’s infrastructure team reported that the cluster experienced approximately **2–3 DataNode failures per day** on average—exactly the failure rate predicted by the GFS paper for commodity hardware at this scale.

Operational challenges. Managing a 42,000-node HDFS cluster required solving engineering problems that the original Hadoop design had not anticipated.

The *NameNode bottleneck* emerged as the cluster grew. By 2009, Yahoo!'s HDFS namespace contained over 50 billion blocks, consuming the full 64 GB of available NameNode heap. Startup time after a planned NameNode maintenance window exceeded 90 minutes—during which the entire cluster was inaccessible. Yahoo!'s engineering team developed several mitigations, including a distributed NameNode extension (*HDFS Federation*) that partitioned the namespace across multiple NameNodes, reducing the per-NameNode block count and memory pressure.

The *small file problem* was acute in production. Yahoo!'s log ingestion pipelines initially wrote one file per server per minute, producing hundreds of millions of small files per day. The engineering team developed automated compaction pipelines that aggregated hourly log files into daily SequenceFiles, reducing the namespace size by a factor of roughly 1,000 while preserving query access.

Contributions to open source. Yahoo! was by far the largest contributor to the Apache Hadoop project during this period. Key features that originated from Yahoo!'s production experience include: HDFS Append (enabling streaming log ingestion), Hadoop Security (Kerberos-based authentication, developed after Yahoo! experienced data governance incidents with an unsecured cluster), HDFS High Availability (eliminating the NameNode SPOF after a NameNode failure caused a 45-minute cluster-wide outage in 2010), and YARN (the resource manager that replaced the original MapReduce-only scheduler, enabling non-MapReduce workloads such as Spark and Tez to share the cluster).

Lessons for data engineering. Yahoo!'s experience establishes several principles that remain relevant today:

- HDFS is exceptionally robust for large-file, sequential-access workloads at any scale—but it requires careful operational management, particularly of the NameNode's memory and the accumulation of small files.

- Production distributed systems require active contributions back to their upstream open-source projects: Yahoo! could not have operated at 42,000 nodes without the HDFS improvements it developed and contributed. This model of “upstream dependency, active contribution” has been adopted by Facebook, LinkedIn, Twitter (X), and most major technology companies.
- Failure planning is not optional. Yahoo!’s decision to invest in HDFS HA before a major outage occurred—rather than after—is the textbook example of proactive operational engineering.

Chapter Summary

- Conventional file systems (NFS, SAN, POSIX) fail at petabyte scale for three reasons: single-machine capacity limits, insufficient sequential throughput, and an inability to tolerate the continuous hardware failures that occur at cluster scale.
- **GFS** (2003) solved these problems by: using large 64 MB chunks, maintaining all metadata in a single master’s RAM, separating control-plane (master) from data-plane (chunkservers), supporting append-only semantics, and relying on replication rather than RAID for fault tolerance.
- **HDFS** is the open-source GFS implementation. Its key components are the **NameNode** (namespace and block mapping in RAM, EditLog + FsImage persistence) and **DataNodes** (block storage, heartbeat, block report, checksum verification).
- **Rack-aware replication** (factor 3, default) places one replica locally, one on a different rack, one on a second node in the same rack as replica 2. A single rack failure cannot cause data loss.
- The **write pipeline** flows client → DN1 → DN2 → DN3 with acknowledgements propagating in reverse. The NameNode handles only control messages; data bypasses it entirely.
- **HDFS HA** uses Active/Standby NameNodes, shared Journal Nodes for the edit log, ZooKeeper for leader election, and fencing to prevent split-brain.

- HDFS limitations include the **small file problem** (NameNode heap exhaustion), **no random writes**, and **high read latency**. Mitigations include SequenceFiles, HBase, and Presto.
- **Cloud object storage** (S3, GCS, ADLS) has largely replaced HDFS for new deployments by offering unlimited capacity, no operational burden, and elastic compute-storage separation—at the cost of data locality.

Exercises

Short Questions

SQ3.1. State three reasons why a conventional NFS-based file system cannot store and process a 500-petabyte dataset.

SQ3.2. What is a GFS chunk? What is the default size and why was this size chosen instead of the 4-KB blocks used by typical file systems?

SQ3.3. Explain the GFS design decision to keep all metadata in the master's RAM. What is the practical ceiling on the number of chunks this imposes, and at what dataset size is that ceiling reached for 64-MB chunks?

SQ3.4. What is the HDFS NameNode? List its five responsibilities and explain what would happen to a running HDFS cluster if the NameNode process were killed.

SQ3.5. What is the difference between the HDFS Secondary NameNode and a hot-standby NameNode? What problem does the Secondary NameNode actually solve?

SQ3.6. Describe the rack-aware replication policy for replication factor 3. If a rack fails, how many replicas of each block remain? Does this satisfy the fault-tolerance goal?

SQ3.7. Trace the HDFS write path step by step from `FileSystem.create()` to the final acknowledgement. At which step is the data actually sent to DataNodes, and how many network hops does it take before the client receives a write acknowledgement?

SQ3.8. What is a DataNode heartbeat and what is a block report? How does the NameNode use each to manage the cluster?

SQ3.9. Explain the HDFS small file problem. If a 100-node cluster holds 200 million files, each occupying one 128-MB block, estimate the minimum NameNode heap required (assume 300 bytes per file–block pair).

SQ3.10. Why does HDFS not support random writes? What are the consistency advantages of the immutable-block design?

SQ3.11. What is HDFS High Availability? List the three components beyond the standard HDFS setup that HA requires and explain the role of each.

SQ3.12. What is a split-brain condition in an HA system? Why is fencing necessary, and what are two mechanisms by which HDFS HA can fence the former Active NameNode?

SQ3.13. Compare HDFS and Amazon S3 on four dimensions: (a) data locality, (b) operational burden, (c) capacity ceiling, (d) cost model.

SQ3.14. A data engineering team ingests one million web server log files per day, each 200 KB in size, directly into HDFS. After one year, the NameNode heap is full. Identify the root cause and propose two engineering solutions.

Analytical Problems

AP3.1. Replication and Storage Overhead Analysis. A genomics research institute stores 50 petabytes of raw sequencing data in HDFS with the default replication factor of 3 and a block size of 128 MB.

- (a) Calculate the total HDFS storage capacity required (including replicas) in petabytes.
- (b) Calculate the number of 128-MB blocks required to store 50 PB of data.
- (c) If each DataNode has 200 TB of usable disk capacity, how many DataNodes are required to store the replicated dataset?
- (d) Estimate the NameNode heap required to hold metadata for all blocks (assume 150 bytes per block). Is this within the 128 GB of RAM on a

modern server?

- (e) If the institute reduces the replication factor to 2, what is the storage saving in petabytes? What is the trade-off?

AP3.2. HDFS Write Throughput Analysis. A streaming ingestion pipeline writes data to HDFS at a sustained rate of 2 GB/s. Each DataNode has a 10 Gbit/s network interface (effective throughput 1.25 GB/s). The replication factor is 3.

- (a) In the write pipeline (client → DN1 → DN2 → DN3), how much network bandwidth does each hop consume at 2 GB/s ingestion rate?
- (b) Is a single DataNode's network interface the bottleneck? If the pipeline is the only network traffic, at what ingestion rate would the first DataNode's network interface saturate?
- (c) How many simultaneous parallel write pipelines (each ingesting a different block) can a 10 Gbit/s interface support at 2 GB/s per pipeline? Why is it beneficial to parallelise writes?
- (d) At 2 GB/s ingestion rate and 128 MB blocks, how many blocks per second does the system create? At what rate does the NameNode receive addBlock RPC calls?

AP3.3. NameNode HA Failover Timing. An HDFS HA cluster has the following configuration:

- ZooKeeper session timeout: 10 seconds
 - ZKFC health check interval: 5 seconds
 - Standby NameNode edit log replay lag: 200 milliseconds maximum
 - SSH fencing timeout: 30 seconds
- (a) What is the minimum time from Active NameNode failure to the Standby becoming Active (assuming fencing via SSH succeeds)?
- (b) What is the maximum time in the worst case (ZooKeeper session timeout has just reset, fencing takes the full 30 seconds)?
- (c) During the failover window, what happens to HDFS clients that are actively reading? What happens to clients that are in the middle of a write?
- (d) Explain why fencing must be completed before the Standby is pro-

moted. Describe the consequences of a successful promotion without fencing, given that the former Active's write lease is still valid.

AP3.4. Cloud vs. On-Premises Storage Economics. A media company stores 2 PB of video content in on-premises HDFS with replication factor 3 (requiring 6 PB raw disk capacity). The data grows at 500 TB/year. DataNode servers cost \$8,000 each with 72 TB usable disk per server; server lifetime is 4 years (amortised \$2,000/server/year). Operations staff costs \$150,000/year to manage the cluster. Amazon S3 charges \$0.023/GB/-month for standard storage.

- (a) How many DataNode servers are needed today? What is the annualised hardware cost (amortised over 4 years)?
- (b) What is the total annual on-premises cost (hardware + operations)?
- (c) What is the annual S3 cost for 2 PB of raw data (no replication factor needed—S3 manages redundancy internally)?
- (d) After 5 years of 500 TB/year growth, compare the cumulative 5-year costs of both approaches (assume S3 pricing remains constant; on-premises requires a hardware refresh at year 4).
- (e) What non-monetary factors should also influence this decision?

Design Problems

DP3.1. HDFS Cluster Design for a Video Streaming Platform. A video streaming company archives 8 PB of source video files for transcoding and long-term retention. The files range from 500 MB to 40 GB each (typical feature film at broadcast quality). The company also processes 2 TB/day of viewer analytics events (one JSON record per play event, approximately 512 bytes each, producing ~4 billion records per day). The system must sustain 10 GB/s sustained write throughput during peak ingestion.

Design the HDFS cluster and ingestion pipeline. Your design must specify:

- (a) The appropriate replication factor for video files vs. analytics events, and the justification for each choice.

- (b) The HDFS block size configuration for video files and for analytics events (can differ per dataset).
- (c) The DataNode hardware specification (disk capacity, network interface speed) and the number of DataNodes required.
- (d) How the 4-billion-record-per-day analytics stream should be ingested without creating the small file problem (propose a specific ingestion architecture using tools from this or previous chapters).
- (e) The HA configuration for the NameNode, including the minimum number of Journal Nodes and ZooKeeper nodes required for quorum.
- (f) The monitoring metrics you would track in production and the alerting thresholds that would trigger an on-call escalation.

DP3.2. HDFS Migration to Cloud Object Storage. A financial services company operates a 500-node on-premises HDFS cluster storing 1.2 PB of trade data, customer records, and regulatory filings. The company is evaluating a migration to Amazon S3. The data has strict regulatory requirements: all data must be encrypted at rest and in transit, all access must be auditable (for SEC and FINRA compliance), and data must be retained for 7 years. Certain datasets must remain on-premises (customer PII, subject to contractual data residency obligations).

Design the migration strategy. Your design must address:

- (a) A data classification scheme that determines which datasets can migrate to S3 and which must remain on-premises.
- (b) The migration mechanism: how to transfer 1.2 PB to S3 with minimal disruption to running production pipelines.
- (c) How encryption at rest and in transit is enforced for S3-stored data.
- (d) How access auditing (“who accessed which object, when”) is implemented in S3.
- (e) The hybrid architecture that allows on-premises Spark jobs to read data from both the remaining HDFS cluster and S3 transparently.
- (f) The rollback plan if the S3 migration reveals unacceptable latency or data access pattern incompatibilities.

Programming Exercises

PE3.1. HDFS Block Layout Simulator (Python). Simulate the block layout for a set of files in HDFS: compute how many blocks each file occupies, how much space is wasted in the last block, and the total NameNode metadata overhead.

```
1 BLOCK_SIZE_MB = 128
2 BYTES_PER_MB = 1024 * 1024
3 METADATA_BYTES_PER_BLOCK = 150 # NameNode heap per block
4
5 files = [
6     {"name": "web_crawl_2024_01.seq",      "size_gb": 312.0},
7     {"name": "user_events_2024_01_01.json", "size_mb": 0.8},
8     {"name": "product_images.tar",        "size_gb":
9         1450.0},
10    {"name": "transaction_log_hourly.orc",  "size_gb": 48.5},
11    {"name": "genome_sample_001.fastq",     "size_gb": 220.0},
12 ]
13
14 def analyse_file(name, size_bytes, block_size=BLOCK_SIZE_MB *
15     BYTES_PER_MB,
16     replication=3):
17     """
18     Return a dict with:
19     num_blocks      : number of HDFS blocks (ceil division)
20     last_block_bytes : size of the final (partial) block
21     wasted_bytes    : padding to fill the last block to
22                       block_size
23                       (HDFS does NOT pad, so wasted_bytes =
24                       0 always;
25                       but report the theoretical waste if
26                       it did)
27     metadata_bytes  : NameNode heap for this file's blocks
28     replicated_bytes : total physical disk used (data *
29                       replication)
30     """
31     import math
32     num_blocks = math.ceil(size_bytes / block_size)
33     last_block_bytes = size_bytes % block_size or block_size
34     # HDFS does not pad the last block
35     wasted_bytes = 0
36     metadata_bytes = num_blocks * METADATA_BYTES_PER_BLOCK
37     replicated_bytes = size_bytes * replication
```

```

32     return {
33         "name": name,
34         "size_bytes": size_bytes,
35         "num_blocks": num_blocks,
36         "last_block_bytes": last_block_bytes,
37         "wasted_bytes": wasted_bytes,
38         "metadata_bytes": metadata_bytes,
39         "replicated_bytes": replicated_bytes,
40     }
41
42 def summarise(files_spec):
43     results = []
44     for f in files_spec:
45         if "size_gb" in f:
46             size = int(f["size_gb"] * 1024 * BYTES_PER_MB)
47         else:
48             size = int(f["size_mb"] * BYTES_PER_MB)
49         results.append(analyse_file(f["name"], size))
50
51     total_blocks = sum(r["num_blocks"] for r in results)
52     total_nn_mb = sum(r["metadata_bytes"] for r in results) /
53         BYTES_PER_MB
54     total_rep_tb = sum(r["replicated_bytes"] for r in results)
55         / (1024**4)
56
57     print(f"{'File':<38} {'Blocks':>7} {'Last block':>12} {'NN
58         heap':>10}")
59     print("-" * 72)
60     for r in results:
61         lb_mb = r["last_block_bytes"] / BYTES_PER_MB
62         nn_kb = r["metadata_bytes"] / 1024
63         print(f"{r['name']:<38} {r['num_blocks']:>7}, "
64             f"{lb_mb:>10.1f} MB {nn_kb:>8.1f} KB")
65     print("-" * 72)
66     print(f"{'TOTAL':<38} {total_blocks:>7}, "
67         f"{'':>12} {total_nn_mb:>8.1f} MB")
68     print(f"\nTotal replicated storage (3x): {total_rep_tb:.2f}
69         TB")
70
71 summarise(files)

```

Listing 3.1: PE3.1: HDFS block layout simulator**PE3.2. Rack-Aware Replica Placement Simulator (Python).** Simulate the

HDFS rack-aware replica placement policy for a set of blocks being written to a cluster of DataNodes distributed across racks.

```
1 import random
2
3 # Cluster topology: rack -> list of DataNode IDs
4 cluster = {
5     "rack-A": ["dn01", "dn02", "dn03", "dn04", "dn05"],
6     "rack-B": ["dn06", "dn07", "dn08", "dn09", "dn10"],
7     "rack-C": ["dn11", "dn12", "dn13", "dn14", "dn15"],
8 }
9
10 def get_rack(dn_id, cluster):
11     for rack, nodes in cluster.items():
12         if dn_id in nodes:
13             return rack
14     return None
15
16 def place_replicas(writer_dn, cluster, replication=3):
17     """
18     Apply HDFS rack-aware placement for replication factor 3:
19     R1: same node as writer (or writer's rack if writer not
20         in cluster)
21     R2: a node on a DIFFERENT rack from R1
22     R3: a DIFFERENT node on the SAME rack as R2
23
24     Returns list of (dn_id, rack) tuples, one per replica.
25     """
26     writer_rack = get_rack(writer_dn, cluster)
27     if writer_rack is None:
28         # Writer is not a DataNode; pick any node for R1
29         writer_rack = random.choice(list(cluster.keys()))
30         r1 = random.choice(cluster[writer_rack])
31     else:
32         r1 = writer_dn
33
34     # R2: different rack
35     other_racks = [r for r in cluster if r != writer_rack]
36     rack2 = random.choice(other_racks)
37     r2 = random.choice(cluster[rack2])
38
39     # R3: different node on same rack as R2
40     rack2_others = [n for n in cluster[rack2] if n != r2]
41     r3 = random.choice(rack2_others)
```



```

41
42     return [(r1, writer_rack), (r2, rack2), (r3, rack2)]
43
44 def simulate_writes(num_blocks, writer_dn, cluster):
45     placements = []
46     for i in range(num_blocks):
47         replicas = place_replicas(writer_dn, cluster)
48         placements.append(replicas)
49
50     # Count blocks per DataNode
51     from collections import Counter
52     node_counts = Counter()
53     for rep_list in placements:
54         for dn, rack in rep_list:
55             node_counts[dn] += 1
56
57     print(f"Block placement for {num_blocks} blocks "
58           f"(writer: {writer_dn}, rack: {get_rack(writer_dn,
59           cluster)})\n")
60     print(f"{'Block':>6} R1           R2           R3")
61     print("-" * 55)
62     for i, rep in enumerate(placements[:10]):
63         cols = [f"{dn} ({rack})" for dn, rack in rep]
64         print(f"{i+1:>6} {cols[0]:<16}{cols[1]:<16}{cols[2]:<16}")
65     if num_blocks > 10:
66         print(f"... ({num_blocks - 10} more blocks)")
67
68     print(f"\nDataNode block distribution (top 5):")
69     for dn, count in node_counts.most_common(5):
70         rack = get_rack(dn, cluster)
71         print(f"  {dn} ({rack}): {count} blocks")
72
73 simulate_writes(num_blocks=50, writer_dn="dn03", cluster=
74                 cluster)

```

Listing 3.2: PE3.2: Rack-aware placement simulator

PE3.3. Failure Simulation and Re-replication Scheduler (Python). Simulate the NameNode's re-replication logic when DataNodes fail: identify under-replicated blocks and schedule re-replication from surviving replicas.

```

1  # Initial block map: block_id -> list of DataNode IDs holding a
   replica

```

```

2 block_map = {
3     "blk_001": ["dn01", "dn04", "dn07"],
4     "blk_002": ["dn02", "dn05", "dn09"],
5     "blk_003": ["dn01", "dn06", "dn11"],
6     "blk_004": ["dn03", "dn07", "dn12"],
7     "blk_005": ["dn01", "dn02", "dn13"],
8     "blk_006": ["dn04", "dn08", "dn14"],
9     "blk_007": ["dn02", "dn06", "dn10"],
10 }
11
12 REPLICATION_FACTOR = 3
13 HEALTHY_NODES = {"dn01", "dn02", "dn03", "dn04",
14                  "dn05", "dn06", "dn07", "dn08",
15                  "dn09", "dn10", "dn11", "dn12", "dn13", "dn14"}
16
17 def simulate_failure(failed_nodes, block_map, healthy_nodes,
18                     replication=REPLICATION_FACTOR):
19     """
20     Remove failed_nodes from block_map and healthy_nodes.
21     Return a list of re-replication tasks:
22         (block_id, source_dn, priority)
23     where priority = 'CRITICAL' (1 surviving replica),
24                    'HIGH'      (2 surviving replicas),
25                    'NORMAL'   (0 deficit, should not appear)
26     Tasks are sorted highest priority first.
27     """
28     live_nodes = healthy_nodes - set(failed_nodes)
29     tasks = []
30
31     for blk_id, replicas in block_map.items():
32         surviving = [r for r in replicas if r in live_nodes]
33         deficit = replication - len(surviving)
34         if deficit <= 0:
35             continue # no re-replication needed
36         priority = "CRITICAL" if len(surviving) == 1 else "HIGH"
37
38         # Source: first surviving replica
39         source = surviving[0] if surviving else None
40         tasks.append((blk_id, source, priority, deficit,
41                      surviving))
42
43     # Sort: CRITICAL first, then HIGH
44     tasks.sort(key=lambda t: 0 if t[2] == "CRITICAL" else 1)

```

```
43     return tasks, live_nodes
44
45     failed = {"dn01", "dn07"}
46     tasks, live = simulate_failure(failed, block_map, HEALTHY_NODES
47                                   )
48
49     print(f"Failed nodes: {failed}")
50     print(f"Surviving nodes ({len(live)}): {sorted(live)}\n")
51     print(f"'Block':<10} {'Source':<8} {'Priority':<10} "
52           f"'Deficit':>8} {'Surviving replicas'}")
53     print("-" * 60)
54     for blk, src, pri, deficit, surviving in tasks:
55         print(f"{blk:<10} {str(src):<8} {pri:<10} "
56               f"{deficit:>8} {surviving}")
57     print(f"\n{len(tasks)} blocks require re-replication.")
```

Listing 3.3: PE3.3: Re-replication scheduler

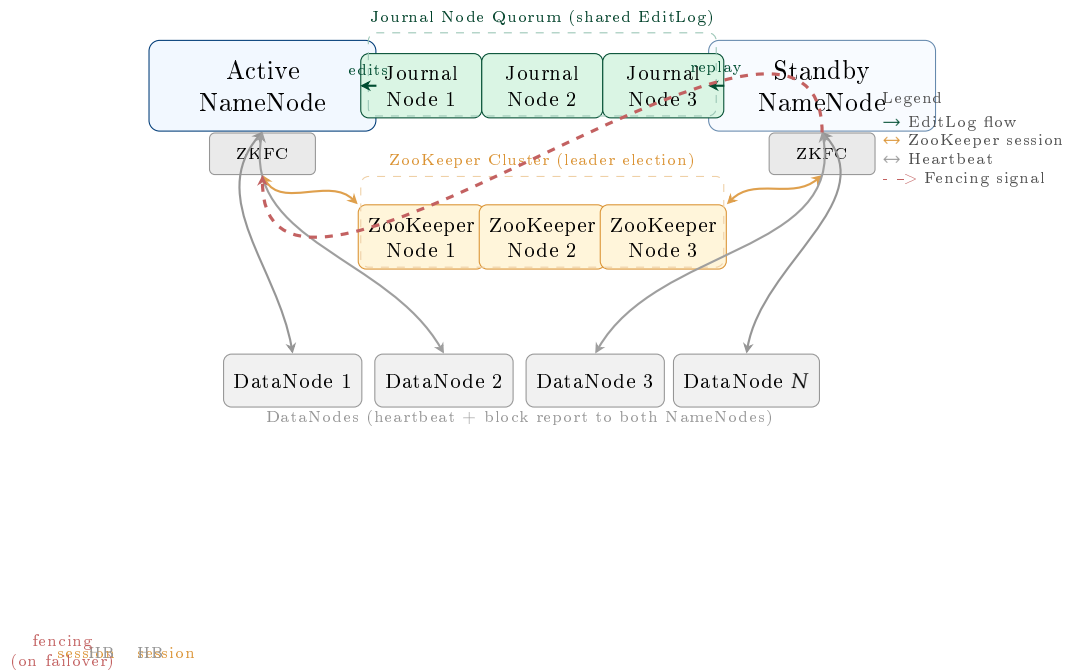


Figure 3.5: HDFS High Availability architecture. The Active NameNode writes every namespace mutation to the Journal Node quorum (shared EditLog); the Standby NameNode continuously replays those edits to maintain a synchronised copy of the namespace. Both NameNodes run a ZooKeeper Failover Controller (ZKFC) that monitors health and maintains a ZooKeeper session. On Active failure, the ZKFC fences the former Active (preventing split-brain), then promotes the Standby to Active in seconds. DataNodes send heartbeats and block reports to *both* NameNodes, so the Standby has a live block map ready for immediate service.

Part III

Batch Processing at Scale

The MapReduce Paradigm and the Hadoop Processing Stack

“Our abstraction is inspired by the map and reduce primitives present in Lisp and many other functional languages. We realized that most of our computations involved applying a map operation to each logical record in our input in order to compute a set of intermediate key/value pairs, and then applying a reduce operation to all the values that shared the same key.”

Dean & Ghemawat, OSDI 2004

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why a new computation model was required for petabyte-scale data on distributed file systems, and identify the specific limitations of SQL and MPI that MapReduce addresses.
2. Write a MapReduce program by defining the Map and Reduce functions, and trace the complete execution through Map → Shuffle/-Sort → Reduce using a concrete example.
3. Explain the roles of the Combiner (local optimisation) and the Partitioner (reducer assignment) and quantify the network savings a Combiner provides.

4. Describe the full MapReduce execution pipeline: input splits, task scheduling with data locality, speculative execution, and output materialisation.
5. Explain how MapReduce achieves fault tolerance through deterministic task re-execution.
6. Write Pig Latin scripts using LOAD, FILTER, FOREACH, GROUP, JOIN, and STORE, and explain how Pig Latin is compiled to MapReduce.
7. Write HiveQL queries using CREATE TABLE, partitioning, bucketing, and SELECT with GROUP BY, and explain the role of the Hive Metastore.
8. Describe the HBase data model (row key, column families, column qualifiers, timestamps) and the HBase architecture (HMaster, RegionServers, WAL, MemStore, HFile).
9. Select the appropriate tool from the Hadoop stack—raw MapReduce, Pig, Hive, or HBase—for a given data engineering problem.

Chapter 3 established how petabytes of data are stored reliably across a cluster of commodity servers. Storage, however, is only half the challenge. The data must be processed: indexed, aggregated, joined, filtered, and analysed. In 2003, Google’s engineers faced precisely this problem. Their GFS cluster held hundreds of terabytes of web crawl data; building the search index required processing every document, aggregating term frequencies, and computing PageRank—computations that would take thousands of hours on a single machine.

The solution they published in 2004 is the subject of this chapter: *MapReduce*, a programming model that allows a programmer to express a large-scale computation as two simple functions—*map* and *reduce*—and have the framework automatically parallelise the execution across hundreds or thousands of machines, handling data partitioning, scheduling, fault tolerance, and inter-machine communication transparently (Dean and Ghemawat 2004). Over the following decade, MapReduce became the computational backbone of an entire ecosystem of data processing tools: Apache Pig, Apache Hive, and Apache HBase, all of which this chapter also covers.

4.1 Why a New Computation Model?

The data processing challenges of petabyte-scale distributed storage do not yield to the tools that work well for smaller datasets. Two dominant paradigms—relational SQL and MPI-style parallel programming—both fail in this environment for distinct reasons.

SQL on a single server requires data to be loaded into a relational schema, supports random access via indexes, and executes on a single instance (or a tightly coupled SMP cluster). None of these properties hold for unstructured petabyte data stored on thousands of commodity DataNodes. Sequential full-table scans of a petabyte dataset at 200 MB/s on a single server would take over 57 days. SQL's assumption of a schema-on-write data model is incompatible with semi-structured and unstructured data (web pages, log files, binary media). SQL-on-Hadoop tools such as Hive (discussed in Section 4.7) resolve these issues by compiling SQL into MapReduce jobs.

MPI (Message Passing Interface) is the standard for scientific high-performance computing. MPI programmes explicitly manage inter-process communication, data partitioning, and synchronisation. This gives the programmer maximum flexibility but at enormous cost: the programmer must manually handle every aspect of distribution. More critically, MPI does not handle fault tolerance: if any MPI process fails, the entire job must typically be restarted from scratch. In a cluster of thousands of commodity machines where failures occur daily, a long-running MPI job that restarts on every node failure is effectively unusable.

MapReduce resolves both limitations. It enforces a structured programming model (Map and Reduce functions) that is restrictive enough for the framework to automate parallelisation, scheduling, and fault recovery—but expressive enough to implement a broad class of large-scale data transformations. The framework, not the programmer, decides how many tasks to spawn, which machine runs each task, how to partition intermediate data, and what to do when a machine fails.

Key Insight | The Expressiveness of Map + Reduce

The surprising result of Dean and Ghemawat's paper is that an enormous variety of large-scale computations—word frequency, inverted index construction, PageRank, distributed sort, join operations, machine learning feature extraction—can be expressed as a single Map phase followed by a single Reduce phase (or a sequence of MapReduce stages). The restrictive two-function interface is a *feature*, not a limitation: its uniformity is exactly what allows the framework to manage all distribution and fault tolerance automatically.

4.2 The MapReduce Programming Model

MapReduce adopts the functional programming concepts of *map* and *reduce* from Lisp and ML, applied to distributed key-value pairs at petabyte scale. The programmer provides two pure functions—with no shared mutable state and no side effects—and the framework handles everything else.

Definition | The MapReduce Functions

Given input data represented as a set of key-value pairs (k_1, v_1) :

Map:

$$\text{map} : (k_1, v_1) \longrightarrow \text{list}((k_2, v_2))$$

The Map function processes each input record independently and emits zero or more intermediate key-value pairs.

Reduce:

$$\text{reduce} : (k_2, \text{list}(v_2)) \longrightarrow \text{list}((k_3, v_3))$$

The Reduce function receives all intermediate values associated with a single key k_2 and aggregates them into the final output.

Between the Map and Reduce phases, the framework executes the *shuffle and sort*: it collects all intermediate (k_2, v_2) pairs emitted by all Map tasks, groups them by key, and sorts each group, delivering to each Reduce task an iterator over all values $\text{list}(v_2)$ for a single key k_2 . The programmer

never writes shuffle-and-sort logic; it is the framework's responsibility.

4.2.1 The Canonical Example: Word Count

The canonical MapReduce example is word count: given a collection of text documents, compute the frequency of every distinct word across the entire collection. This example is simple enough to follow in detail, yet it illustrates every essential phase of MapReduce execution.

```

1  # ----- Map function -----
2  # Input:  (docid, document_text)
3  # Output: (word, 1) for every word in the document
4
5  def map(docid, text):
6      for word in text.split():
7          emit(word.lower(), 1)
8
9  # Example: map("doc1", "To be or not to be") emits:
10 #   ("to", 1), ("be", 1), ("or", 1), ("not", 1), ("to", 1), ("
    be", 1)
11
12 # ----- Shuffle and Sort (framework-managed) -----
13 # Groups all emitted pairs by key, sorts alphabetically:
14 #   ("be", [1, 1, 1, ...])    <- all 1s from every document
15 #   ("not", [1, 1, ...])
16 #   ("or", [1, ...])
17 #   ("to", [1, 1, 1, ...])
18
19 # ----- Reduce function -----
20 # Input:  (word, iterator_of_counts)
21 # Output: (word, total_count)
22
23 def reduce(word, counts):
24     total = sum(counts)
25     emit(word, total)
26
27 # Example: reduce("be", [1,1,1,1,1]) emits: ("be", 5)

```

Listing 4.1: MapReduce word count: Map and Reduce functions

Each document in the collection is assigned to an independent Map task; if there are 10 million documents, 10 million Map tasks run in parallel (subject to cluster capacity). Each Map task processes only its own docu-

ments and emits word counts for that subset. The framework then collects all $(word, 1)$ pairs from all Map tasks, groups them by word, and routes each group to a single Reduce task, which sums the counts. The output is the complete word frequency table for the entire collection.

4.2.2 Additional MapReduce Examples

Word count is illustrative, but the MapReduce model extends to a much wider class of computations. Two further examples demonstrate its generality.

Inverted index construction. A search engine’s inverted index maps each term to the list of documents containing it. The Map function emits $(term, docid)$ for each term in each document. After shuffle and sort, each Reduce task receives $(term, [docid_1, docid_2, \dots])$ and concatenates the list into the posting list for that term.

Distributed sort. To sort a petabyte dataset, the Map function emits each record as $(sort_key, record)$. The shuffle and sort phase sorts all intermediate keys globally (using a Partitioner that distributes key ranges uniformly across Reducers). Each Reducer receives records in sorted order for its key range; the concatenation of all Reducer outputs is globally sorted. This pattern—TeraSort—is the standard benchmark for distributed sorting, and Hadoop clusters have set world records for sorting 1 TB in minutes.

4.3 The Combiner and the Partitioner

Two optional but important components sit between the Map and Reduce phases: the Combiner and the Partitioner. Both are performance optimisations that the programmer can provide without changing the correctness of the computation.

4.3.1 The Combiner: Local Aggregation

In the word count example, each Map task might process a 128 MB input split containing hundreds of thousands of words. If the word “the” appears 50,000 times in that split, the Map task emits 50,000 $(“the”, 1)$ pairs, all of which must be transferred over the network to the Reducer responsible for

“the”. This is wasteful: the network cost is proportional to the number of word occurrences, not the number of distinct words.

The **Combiner** is a locally executed mini-Reducer that runs immediately after the Map phase *on the same node*. It receives the Map task’s output and aggregates values for each key before they are shuffled across the network. For word count, the Combiner would reduce the 50,000 (“the”, 1) pairs to a single (“the”, 50000), reducing the shuffle volume for this key by a factor of 50,000.

Combiners can be applied whenever the Reduce function is *commutative* and *associative*: if $f(a, f(b, c)) = f(f(a, b), c)$ and $f(a, b) = f(b, a)$, then partial application of f is always safe. Sum, count, min, and max all satisfy this property. Average does not—the Combiner cannot compute a local average because it does not know how many records the Reducer will receive from other Map tasks.

In practice, enabling a Combiner can reduce shuffle network traffic by 80–95% for aggregation workloads, dramatically reducing job completion time on bandwidth-constrained clusters.

4.3.2 The Partitioner: Directing Keys to Reducers

The **Partitioner** determines which Reducer receives each intermediate key. Given a key k and the number of Reduce tasks R , the default HashPartitioner computes:

$$\text{reducer_id}(k) = h(k) \bmod R \quad (4.1)$$

where $h(k)$ is the hash of key k . This distributes keys uniformly across all Reduce tasks in expectation—the fundamental requirement for preventing *reducer skew* (one reducer receiving far more data than others and becoming the job’s bottleneck).

For workloads where the hash distribution is insufficient—for example, when sorting requires that all keys in a range go to a specific Reducer—the programmer provides a custom Partitioner. The TeraSort example uses a **TotalOrderPartitioner** that divides the keyspace into R equal-sized ranges based on a sample of the input, ensuring that each Reducer receives a balanced partition of the sorted key space.

4.4 The Full MapReduce Execution Pipeline

Having defined the programming model, we can now trace the complete execution of a MapReduce job from input files on HDFS to output files on HDFS, through the framework's scheduling, data-locality optimisation, and fault recovery mechanisms.

4.4.1 Input Splits and the InputFormat

A MapReduce job begins with the client submitting a job to the cluster. The framework reads the input files from HDFS and divides them into *input splits*—logical chunks of input data, typically aligned to HDFS block boundaries (128 MB by default). One Map task is instantiated per input split; the number of Map tasks is therefore approximately $\lceil \text{total input size} / \text{block size} \rceil$, matching the number of HDFS blocks that contain the input.

The *InputFormat* is the abstraction that translates raw HDFS bytes into (k_1, v_1) pairs that the Map function consumes. The standard `TextInputFormat` produces `(byte_offset, line)` pairs—one per line of text. `SequenceFileInputFormat` reads Hadoop SequenceFile format. `CombineFileInputFormat` merges many small files into a single split, mitigating the small file problem in the context of MapReduce job scheduling.

4.4.2 Task Scheduling and Data Locality

The cluster *JobTracker* (in Hadoop 1.x) or YARN *ResourceManager* (Hadoop 2.x and later, covered in Chapter ??) accepts the submitted job and begins scheduling Map tasks. For each input split, the framework attempts to schedule the Map task on one of the three `DataNodes` that hold a replica of the corresponding HDFS block—applying the data locality principle introduced in Chapter 1. If a data-local slot is unavailable, the framework falls back to rack-local, then to off-rack scheduling.

Data locality is critical for MapReduce performance. A Map task reading its input from a local disk can sustain 100–300 MB/s of throughput; the same task reading from a remote rack over a 1 Gbit/s network connection is limited to approximately 125 MB/s, and in a congested cluster may receive less. For a job with thousands of Map tasks all reading simultaneously, the

difference between local and off-rack I/O can mean the difference between a one-hour job and a three-hour job.

Reduce tasks have no concept of data locality with respect to the original input—their input comes from the network (the shuffle phase). Reduce tasks are scheduled on any available node with sufficient memory and CPU capacity.

4.4.3 The Shuffle and Sort in Detail

The shuffle is the most network-intensive phase of MapReduce. As each Map task produces output, it does not write it directly to HDFS (which would incur expensive replication); instead, it writes intermediate data to a local *spill buffer* in RAM (default 100 MB). When the buffer is 80% full, a background thread *spills* the buffer content to disk after sorting by key within the spill and applying the Combiner (if present). Multiple spills from a single Map task are merged into a single sorted file before the task completes.

Each Reduce task then actively *fetches* its portion of the intermediate data from each Map task's output file via HTTP (a process called the *copy phase*). As data arrives from different Map tasks, the Reduce side merges the multiple sorted streams (using a merge-sort) to produce a single sorted sequence of (k_2, v_2) pairs for each key. This merge-sort is the “sort” in shuffle-and-sort, and it is what guarantees that the Reduce function sees values for a single key in a well-defined, contiguous iterator.

4.5 Fault Tolerance in MapReduce

MapReduce's fault tolerance model is both simple and elegant: all Map and Reduce tasks are *deterministic* and *idempotent*. A Map task that processes the same input split will always produce the same output; a Reduce task that processes the same sorted key-value sequence will always produce the same output. This determinism is the foundation of fault recovery.

The *JobTracker* (or YARN ResourceManager) monitors the progress of every running task through periodic *heartbeats*. If a task has not reported progress within a configurable timeout (default 10 minutes), it is considered

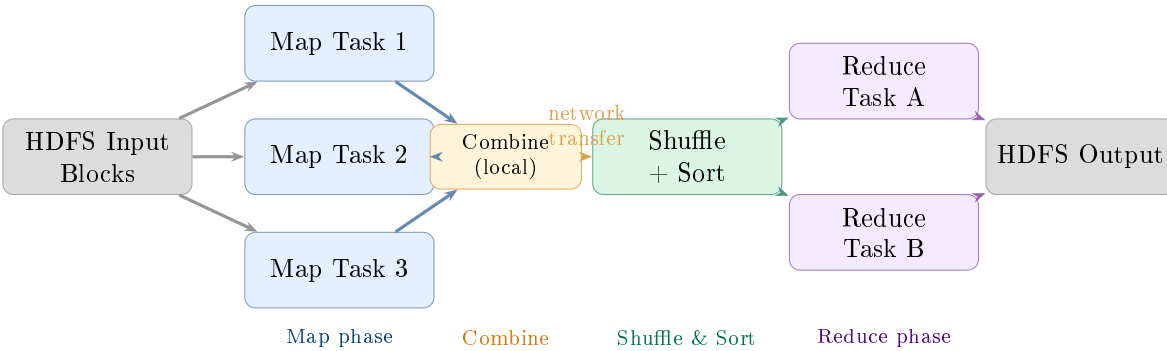


Figure 4.1: The full MapReduce execution pipeline. Map tasks run in parallel, each processing one input split. The optional Combiner reduces shuffle traffic. The framework-managed shuffle groups and sorts all intermediate data by key before delivering it to Reduce tasks.

failed. The framework re-schedules the failed task on a different node, which re-reads the input split from HDFS and re-executes the Map function from scratch. Because HDFS provides three replicas of every input block, the task can always find its input on a surviving DataNode.

For Reduce tasks, re-execution is more expensive because the Reduce task may have already fetched a significant portion of its shuffle data. When a Reduce task fails, the framework restarts it from the beginning of the copy phase; all already-fetched intermediate data is discarded and re-fetched from Map task outputs.

Speculative execution addresses a different failure mode: the slow task (*straggler*). In a large cluster, statistical variation in hardware performance means that some tasks will always run significantly slower than average, even without a hard failure. A single slow Map task on a degraded disk or a hot machine can delay the entire job, because the Reduce phase cannot begin until all Map tasks complete. MapReduce mitigates this by detecting tasks that are running slower than expected and launching *backup tasks*—duplicate executions of the same task on different nodes. Whichever task—original or backup—completes first writes its output; the other is killed. This has no correctness impact because both produce identical outputs (determinism), but can reduce job completion time by 40% in practice.

4.6 Apache Pig: Dataflow Scripting

Table 4.1: MapReduce fault tolerance mechanisms

Failure type	Detection	Recovery mechanism
Map task failure	Heartbeat timeout	Re-schedule on another node; re-read HDFS block
Reduce task failure	Heartbeat timeout	Re-schedule; re-fetch all shuffle data
TaskTracker failure	Heartbeat timeout	Re-schedule all tasks that were running on that node
Slow task (straggler)	Progress rate below mean	Launch speculative backup task; keep first to complete
JobTracker failure	Not auto-recovered (Hadoop 1.x)	Manual restart; job history lost unless YARN HA used

Raw MapReduce is powerful but verbose. A simple join of two datasets requires writing multiple Java classes: an InputFormat for each source, a custom Mapper that tags records with their source, a custom Partitioner, and a Reducer that handles the merge logic—hundreds of lines of boilerplate code. Apache Pig addresses this by providing a high-level dataflow scripting language called *Pig Latin*, which compiles to one or more MapReduce jobs automatically.

Definition | Apache Pig

Apache Pig is a dataflow platform for analysing large datasets on Hadoop. Pig Latin programs describe a **directed acyclic graph (DAG) of data transformations**: each statement loads, filters, transforms, groups, joins, or stores a relation. The Pig compiler translates the DAG into a sequence of MapReduce jobs, optimising the plan by combining compatible operations into single jobs where possible.

A Pig Latin script operates on *relations*—bag-like collections of tuples, similar to relational tables but without requiring a predefined schema. The key operators are:

- LOAD: read data from HDFS into a named relation.
- FILTER: select tuples matching a predicate.
- FOREACH ... GENERATE: project or transform fields (analogous to SQL's SELECT).
- GROUP: group tuples by a key (generates a relation of (key, bag) pairs).
- JOIN: equi-join two relations on a key.
- ORDER BY: sort a relation by a key.
- STORE: write a relation to HDFS.

The following Pig Latin script analyses a web server access log to find the most-requested URLs that returned HTTP 404 errors:

```
1  -- Load the access log (tab-delimited)
2  logs = LOAD 'hdfs:///data/access_log'
3         USING PigStorage('\t')
4         AS (host:chararray, ts:chararray, method:chararray,
5            url:chararray, status:int, bytes:long);
6
7  -- Keep only 404 responses
8  errors = FILTER logs BY status == 404;
9
10 -- Group by URL
11 grouped = GROUP errors BY url;
12
13 -- Count 404s per URL
14 counts = FOREACH grouped GENERATE
15           group          AS url,
16           COUNT(errors) AS num_errors;
17
18 -- Sort descending
19 ranked = ORDER counts BY num_errors DESC;
20
21 -- Keep top 20
22 top20 = LIMIT ranked 20;
23
24 -- Write output to HDFS
25 STORE top20 INTO 'hdfs:///output/top404' USING PigStorage(',');
```

Listing 4.2: Pig Latin: 404 error frequency analysis

The Pig compiler translates this seven-statement script into typically two MapReduce jobs: one for the FILTER + GROUP + FOREACH (a shuffle-intensive aggregate), and one for the ORDER BY + LIMIT (a second sort phase). The programmer writes dataflow logic; the framework decides how many MapReduce stages are needed and how to parallelise each one.

Pig is particularly well-suited for ad hoc exploratory analysis on HDFS data, for ETL pipelines that require complex multi-step transformations, and for data scientists who are comfortable with a scripting language but do not want to write Java MapReduce code. Its primary limitation—shared with raw MapReduce—is high latency: even simple queries require at least one MapReduce job, with startup overhead of 30–60 seconds before computation begins. For interactive analytical queries, Hive with a modern execution engine (Tez or Spark) or Presto (Chapter 7) is more appropriate.

4.7 Apache Hive: SQL-on-Hadoop

Apache Hive, developed at Facebook and open-sourced in 2008, takes a different approach to Hadoop usability: instead of a new scripting language, it provides a SQL-like query language called *HiveQL* (or simply “Hive SQL”) that compiles to MapReduce (and, since Hive 0.13, to Apache Tez or Apache Spark). Hive allows data analysts comfortable with SQL to query petabyte-scale datasets on HDFS without learning MapReduce or Pig Latin.

4.7.1 The Hive Metastore

For SQL queries to work over HDFS files, Hive needs a way to map SQL table names and column names to HDFS paths and file formats. This mapping is managed by the *Hive Metastore*: a relational database (typically MySQL or PostgreSQL) that stores table definitions, schema information, partition metadata, and storage descriptors.

When a user creates a Hive table:

```
1 CREATE TABLE web_events (
```

```

2      user_id      BIGINT ,
3      event_type   STRING ,
4      url          STRING ,
5      session_id   STRING ,
6      duration_ms  INT
7  )
8  PARTITIONED BY (dt STRING)
9  STORED AS ORC
10 LOCATION 'hdfs:///data/web_events';

```

Listing 4.3: HiveQL: creating a partitioned ORC table

Hive records in the Metastore that the table `web_events` maps to the HDFS directory `/data/web_events`, is stored in ORC format, and is partitioned by the column `dt` (date string). Data for 2024-01-15 lives in the subdirectory `/data/web_events/dt=2024-01-15/`. The Metastore is the source of truth that the HiveQL compiler consults to generate the correct HDFS path for each query.

4.7.2 Partitioning and Bucketing

Partitioning is one of the most important performance techniques in Hive. A partitioned table stores its data in separate HDFS subdirectories, one per partition value. A query that filters on the partition column can skip reading all irrelevant partitions entirely—a technique called *partition pruning*. For a table with 365 date partitions and 1 TB per partition (365 TB total), a query filtering on a single day reads only 1 TB rather than 365 TB:

```

1  -- Only reads /data/web_events/dt=2024-01-15/
2  -- Hive's query planner prunes all other 364 partitions
3  SELECT  event_type,
4          COUNT(*)      AS event_count,
5          AVG(duration_ms) AS avg_duration_ms
6  FROM    web_events
7  WHERE   dt = '2024-01-15'      -- partition filter
8  AND     event_type IN ('purchase', 'add_to_cart')
9  GROUP BY event_type
10 ORDER BY event_count DESC;

```

Listing 4.4: HiveQL: partition pruning in a query

Bucketing provides a second level of data organisation within a partition. Rows are assigned to a fixed number of *buckets* (files) based on the

hash of a designated column. Bucketing improves the efficiency of equi-joins (two tables bucketed on the join key and with the same number of buckets can be joined without a full shuffle) and enables efficient sampling (read only one bucket for a $1/N$ sample).

4.7.3 HiveQL to MapReduce Compilation

When a user submits a HiveQL query, the Hive driver executes the following steps:

1. **Parse:** convert the SQL string into an abstract syntax tree (AST).
2. **Semantic analysis:** resolve table and column names against the Metastore; validate types and permissions.
3. **Logical plan:** convert the AST into a logical query plan (a DAG of relational operators: scan, filter, project, aggregate, join).
4. **Optimisation:** apply rule-based optimisations (predicate push-down, column pruning, partition pruning) to reduce data read volume.
5. **Physical plan:** translate the logical plan into a sequence of MapReduce (or Tez, or Spark) jobs.
6. **Execution:** submit the jobs to the cluster and monitor their progress.

A `SELECT ... GROUP BY` query maps straightforwardly to a single MapReduce job: the Map function applies the predicate and projects the required columns; the Reduce function aggregates by the `GROUP BY` key. A multi-table `JOIN` may generate two or more MapReduce jobs depending on the join strategy (common-join, map-join, or bucket-map-join).

Research Spotlight | Thusoo et al. (2009) — Hive: A Warehousing Solution over a Map-R

Citation: Thusoo, A., et al. (2009). Hive – A Warehousing Solution over a Map-Reduce Framework. *Proceedings of the VLDB Endowment*, 2(2):1626–1629. (Thusoo et al. 2009)

Key insight: The paper’s central argument is that SQL is the right interface for large-scale data analysis—not because SQL is technically superior to MapReduce, but because SQL is what hundreds of millions of analysts already know. Hive’s contribution is a translation layer, not a new programming model.

Technical contributions:

- Defined the Hive data model: tables, partitions, buckets, and complex data types (arrays, maps, structs) as SQL extensions compatible with semi-structured data.
- Introduced the *Metastore* architecture for mapping SQL schemas to HDFS directory structures, enabling schema-on-read while providing schema-on-write semantics for the SQL layer.
- Demonstrated that a HiveQL-to-MapReduce compiler could express all SQL-92 analytics workloads (aggregation, joins, subqueries, user-defined functions) on petabyte datasets.

Impact today: Hive is the most-used large-scale SQL engine in the history of open-source data engineering. Its Metastore is now an independent component used by Spark SQL, Presto/Trino, and Amazon Athena as their shared table catalog. The HiveQL dialect has become the *de facto* standard for analytical SQL over big data platforms, influencing the syntax of SparkSQL, BigQuery, and Snowflake.

4.8 Apache HBase: Column-Family NoSQL on Hadoop

MapReduce, Pig, and Hive are all batch processing tools: they read from HDFS, process data, and write back to HDFS, with latencies measured in minutes to hours. There is a class of workload for which this latency is unacceptable: applications that need to read or write individual records by key in milliseconds—user profile lookups, messaging systems, real-time recommendation updates, and fraud scoring.

Conventional HDFS provides no such capability: its read path requires a NameNode round trip and a DataNode TCP connection per block, delivering 50–200 ms latency per operation. Apache HBase, modelled directly on Google’s Bigtable (Chang et al. 2006), provides *random read/write access* at sub-10 ms latency on data stored on HDFS—bridging the gap between the Hadoop ecosystem and the performance requirements of operational applications.

4.8.1 The HBase Data Model

HBase stores data in a model that differs fundamentally from both the relational model and the key-value model.

Definition | The HBase Data Model

An HBase table is a sorted map of:

$(\text{row_key}, \text{column_family} : \text{qualifier}, \text{timestamp}) \longrightarrow \text{value}$

- **Row key:** an arbitrary byte array, lexicographically sorted. Row key design is the central HBase schema design decision; the entire table is physically sorted by row key, enabling efficient range scans.
- **Column family:** a named group of columns, defined at table creation time. All columns in a family are stored together on disk.
- **Column qualifier:** a dynamic column name within a family, created on the fly without altering the table schema. A single row can have zero or thousands of qualifiers within a family.
- **Timestamp:** each cell stores multiple versions, indexed by timestamp. The most recent version is returned by default; older versions are accessible by specifying a timestamp.

Consider a social network's user activity table:

```

1 # Row structure: (row_key, column_family:qualifier, timestamp)
   -> value
2
3 # User u-10042's profile
4 ("u-10042", "profile:name",      T3) -> "Ada Lovelace"
5 ("u-10042", "profile:country",   T3) -> "GB"
6 ("u-10042", "profile:join_date", T1) -> "2019-03-14"
7
8 # User u-10042's activity (sparse: not every qualifier exists
   for every user)
9 ("u-10042", "activity:last_login", T5) -> "2024-01-15T14:22:31
   Z"
10 ("u-10042", "activity:post_count", T4) -> "847"
11 ("u-10042", "activity:follower_ct", T4) -> "12941"
12
13 # Multiple versions: profile:name was updated

```

```

14 ("u-10042", "profile:name", T1) -> "Ada Byron"
15 ("u-10042", "profile:name", T3) -> "Ada Lovelace"  <- default (
    latest)

```

Listing 4.5: HBase data model: user activity table

The row key design `u-10042` is intentional: all rows for a user’s profile and activity are co-located in the same row, enabling an atomic, single-row fetch of the complete user record without a join. This *denormalisation by design* is characteristic of column-family databases and reflects the NoSQL principle that schema design should optimise for the read access pattern, not for normalisation.

4.8.2 HBase Architecture

HBase’s architecture closely mirrors the GFS/HDFS master-worker pattern, adapted for random access.

The *HMaster* is the cluster coordinator. It handles table creation, deletion, and schema changes; assigns *regions* (contiguous ranges of row keys) to *RegionServers*; and monitors RegionServer health via ZooKeeper. The HMaster is not in the read/write data path—all client I/O goes directly to RegionServers after the initial routing lookup.

RegionServers are the worker processes that handle all read and write operations. Each RegionServer manages between 10 and 1,000 regions. When a client writes a record, the RegionServer:

1. Appends the write to the *Write-Ahead Log* (WAL)—an append-only HDFS file that ensures durability even if the RegionServer crashes before flushing to disk.
2. Writes the record into the *MemStore*: an in-memory sorted map (implemented as a Red-Black tree). All reads first check the MemStore.
3. Returns success to the client.

When the MemStore grows beyond a threshold (default 128 MB), it is *flushed* to HDFS as an immutable *HFile* (a sorted, block-compressed file in the HDFS namespace). Reads are served from a combination of the MemStore (for recent writes) and HFiles (for older data), with a Bloom filter on each HFile to avoid unnecessary disk reads for absent keys.

Over time, many small HFiles accumulate for a region. *Compaction* pe-

periodically merges HFiles into larger, consolidated files, eliminating deleted records and expired versions. This is the LSM-tree (Log-Structured Merge-tree) storage model: by converting all writes into sequential appends (WAL and HFile flushes), HBase achieves write throughput competitive with hash-based databases while maintaining sorted order for efficient range scans.

Research Spotlight | Chang et al. (2006) — Bigtable: A Distributed Storage System for S

Citation: Chang, F., et al. (2006). Bigtable: A Distributed Storage System for Structured Data. *Proceedings of the 7th USENIX OSDI Conference*, pp. 205–218. (Chang et al. 2006)

Key insight: Bigtable demonstrates that a single, flexible data model—the sorted, multi-dimensional map—can efficiently support workloads as diverse as web indexing (sparse, write-heavy), Google Earth (large values, read-heavy), and Google Finance (time-series with many versions). The same model that serves all of Google’s internal applications is now accessible to any developer through HBase and Google Cloud Bigtable.

Technical contributions:

- Introduced the *column family* abstraction—grouping related columns for co-location on disk—which enables efficient access to subsets of columns without reading entire rows.
- Described the *Tablet* architecture (equivalent to HBase’s Region): range-partitioned, sorted data units managed by a Tablet Server, with the master handling only assignment and load balancing.
- Demonstrated the LSM-tree storage model in a production distributed system, establishing the architectural pattern that HBase, Cassandra, RocksDB, and LevelDB all follow.

Impact today: Bigtable is cited in over 13,000 papers. HBase (the open-source implementation) serves as the storage backend for Apache Phoenix (SQL on HBase), Facebook’s internal messaging infrastructure (1 billion messages per day at peak), and numerous real-time recommendation systems. Google Cloud Bigtable (the managed service version) is used by companies including Spotify, Dow Jones, and T-Mobile.

4.9 Ecosystem Tool Selection

With four tools in the Hadoop processing stack—raw MapReduce, Pig, Hive, and HBase—the practical question for a data engineer is which tool to use for a given problem. The selection matrix below provides a decision framework based on access pattern, latency requirement, user skill set, and data structure.

Table 4.2: Hadoop processing stack tool selection matrix

Tool	Best use case	Latency	Interface	Not suited for
Raw MapReduce	Custom iterative algorithms; tight performance control	Hours	Java	Interactive queries; rapid development
Apache Pig	Multi-step ETL; ad hoc data exploration; complex transforms	Hours	Pig Latin (script)	Interactive queries; SQL-familiar analysts
Apache Hive	Reporting & BI; data warehouse queries; SQL analysts	Minutes (batch)	HiveQL (SQL)	Sub-second queries; random reads
Apache HBase	Random read/write; operational data; real-time lookups	<10 ms	Java API / REST	Full table scans; complex analytics

A practical guideline: use **Hive** as the default for SQL-capable analysts running reporting and BI workloads over HDFS data; use **Pig** when

a pipeline requires complex multi-step ETL that would be awkward to express in SQL; use **raw MapReduce** when performance tuning or an algorithm that does not map cleanly to Map/Reduce (such as iterative graph algorithms) is required; use **HBase** when the application requires sub-millisecond random access to individual records by key. In practice, Hive over Apache Tez (which avoids materialising intermediate results to HDFS) or Hive over Spark (Chapter 6) replaces much of the original Hive-on-MapReduce workload, delivering 10–100× lower latency.

4.10 Case Study: Facebook Data Infrastructure (2008–2012)

Case Study | Facebook — Hive and HBase at the Petabyte Frontier

Background. By 2008, Facebook had accumulated more than 2.5 petabytes of user data and was generating approximately 15 terabytes of new compressed data per day. The company needed to provide its data analytics team—several hundred analysts—with the ability to query this data interactively in SQL, without requiring them to write MapReduce jobs. Simultaneously, Facebook Messenger was processing over one billion messages per day and required sub-millisecond random access to individual message threads for retrieval.

The Hive deployment. Facebook’s data infrastructure team developed Hive internally and open-sourced it to Apache in 2008. By 2010, Facebook’s Hadoop cluster had grown to over 2,400 nodes with 30 PB of storage, running over 7,500 Hive queries per day submitted by analysts across the company. The Hive Metastore tracked more than 5,000 registered tables, the largest of which—Facebook’s “events” table—exceeded 1 PB of compressed ORC data.

A key engineering challenge was query latency. Early Hive-on-MapReduce queries over large tables took 20–60 minutes due to MapReduce job startup overhead and the requirement to materialise all intermediate results to HDFS. Facebook’s team introduced several optimisations that were later contributed to open source: predicate push-down (filter early), partition pruning (read only relevant data partitions), and

index-based skip-scan. These reduced the median analyst query time from 35 minutes to under 5 minutes.

The HBase deployment. Facebook used HBase as the storage backend for Facebook Messenger and Inbox. Each user’s message thread was stored as an HBase row, with individual messages as column qualifier-value pairs. The row key design was critical: Facebook chose a reversed-timestamp prefix to ensure that the most recent messages for a thread were lexicographically adjacent on disk, enabling efficient range scans for loading a conversation inbox.

At peak, Facebook’s HBase cluster served over **one billion messages per day** across a cluster of several hundred RegionServers. The P99 read latency for fetching a user’s 20 most recent messages was approximately 8 milliseconds—competitive with purpose-built NoSQL databases and far below the latency achievable with HDFS-native batch access.

The architectural lesson. Facebook’s use of both Hive and HBase simultaneously—on the same underlying HDFS cluster—illustrates a key principle of data engineering: different workloads require different tools, but those tools can share the same storage layer. Hive queries and HBase operations both read from and write to HDFS DataNodes; the NameNode manages the unified namespace. This shared storage model avoids data duplication and simplifies the operational burden of managing two separate storage systems.

By 2012, Facebook’s Hadoop cluster had grown to over 100 petabytes, making it the largest known Hadoop deployment in the world at that time. The infrastructure team published a series of papers and blog posts documenting their engineering decisions, which directly influenced the design of subsequent generations of distributed analytics tools—particularly the development of Apache Spark, which addressed the primary limitation of the Hive-on-MapReduce stack: latency.

Chapter Summary

- **MapReduce** resolves two limitations of prior parallel computing paradigms: unlike SQL, it requires no predefined schema and handles unstructured data on HDFS; unlike MPI, it handles fault tolerance automatically through deterministic task re-execution.
- The MapReduce model requires the programmer to provide two functions: **map** $(k_1, v_1) \rightarrow \text{list}(k_2, v_2)$ and **reduce** $(k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$. The framework handles parallelisation, shuffle-and-sort, and fault recovery.
- The **Combiner** (local mini-reducer) reduces shuffle network traffic for commutative-associative operations. The **Partitioner** controls key-to-reducer assignment via hash or custom range logic.
- **Fault tolerance** is achieved through deterministic task re-execution on failure; **speculative execution** launches backup tasks for slow stragglers.
- **Apache Pig** compiles a dataflow scripting language (Pig Latin) to MapReduce, enabling multi-step ETL without Java programming.
- **Apache Hive** compiles HiveQL (SQL dialect) to MapReduce. The **Meta-store** maps SQL tables to HDFS paths. **Partitioning** enables partition pruning; **bucketing** accelerates equi-joins and sampling.
- **Apache HBase** provides random read/write access at <10 ms latency on HDFS, using the LSM-tree model: WAL for durability, MemStore for recent writes, HFiles for historical data, compaction to merge HFiles.
- Tool selection: Hive for SQL analytics; Pig for scripted ETL; raw MapReduce for custom algorithms; HBase for random-access operational data.

Exercises

Short Questions

SQ4.1. Why does SQL on a single server fail for petabyte-scale data on HDFS? Name three specific assumptions SQL makes that do not hold in the

HDFS environment.

SQ4.2. Write the type signatures of the Map and Reduce functions in the MapReduce model. What is the framework's responsibility between the Map and Reduce phases?

SQ4.3. Trace the word count MapReduce computation for the input string "the cat sat on the mat the cat". List every (k, v) pair emitted by the Map function, the grouped shuffle output, and the final (k, v) output of the Reduce function.

SQ4.4. What is the Combiner? For what class of Reduce functions can a Combiner be applied safely? Give an example of a Reduce function for which a Combiner would give incorrect results.

SQ4.5. What is speculative execution in MapReduce? Under what circumstances does the framework launch a backup task, and why does it not affect the correctness of the computation?

SQ4.6. A MapReduce job has 1,000 Map tasks and 50 Reduce tasks. Describe what happens when one Map task fails partway through execution. Describe what happens when one Reduce task fails after it has already fetched 80% of its shuffle data.

SQ4.7. What is the Hive Metastore? What information does it store, and what would happen to a HiveQL query if the Metastore database were unavailable?

SQ4.8. Explain partition pruning in Hive. A table has 365 daily partitions each containing 500 GB of ORC data (total 182 TB). A query filters on a single day. How much data does Hive read, and what is the speedup factor compared to a full-table scan?

SQ4.9. What is the HBase data model? Describe the four coordinates that uniquely identify a cell in an HBase table.

SQ4.10. Explain the LSM-tree write path in HBase: what happens to a client write at each of the WAL, MemStore, and HFile stages?

SQ4.11. What is HBase compaction? Explain the difference between minor and major compaction, and why compaction is necessary for read performance.

SQ4.12. Compare Apache Pig and Apache Hive on three dimensions: tar-

get user, language paradigm, and primary use case.

SQ4.13. A data engineer needs to find the top 10 most active users in a 1-petabyte social media activity dataset (schema: `user_id`, `event_type`, `ts`). Which Hadoop tool would you recommend, and why?

SQ4.14. True or false, with justification: “HBase stores data in HDFS, so its read latency is determined by HDFS block access latency (50–200 ms).” (*Hint: consider the MemStore and the block cache.*)

Analytical Problems

AP4.1. MapReduce Shuffle Volume Analysis. A MapReduce word count job processes a 1 TB corpus of English text. The average word length is 5 characters, and the average word frequency is 100 occurrences per 128-MB input split. There are 100,000 distinct words in the corpus. The cluster has 500 Map tasks.

- (a) Estimate the total number of (*word*, 1) pairs emitted by all Map tasks (assume 128 MB per split, and that the text has an average word density of 150 words per KB).
- (b) Estimate the total shuffle data volume in GB without a Combiner (each pair requires approximately $|word| + 8$ bytes for the key-value pair overhead).
- (c) With a Combiner enabled, each Map task emits at most one pair per distinct word. Estimate the new shuffle volume and the reduction factor.
- (d) If the cluster’s inter-node network bandwidth is 10 Gbit/s per node, and the shuffle phase transfers all data through the network simultaneously, estimate the minimum time for the shuffle to complete (a) without and (b) with the Combiner.

AP4.2. Hive Partitioning Strategy. An e-commerce company has a transactions table (500 TB total, 5 years of data) with columns: `transaction_id`, `customer_id`, `product_id`, `amount`, `country`, `ts` (timestamp).

- (a) The company’s most frequent query pattern is: “What was the total

revenue per product in Germany in Q1 2024?” Propose a partitioning scheme. Which column(s) should be the partition key(s), and how much data does the optimised query read?

- (b) A second query pattern is: “For a given customer, list all their transactions across all time.” Evaluate whether your partitioning scheme from (a) also optimises this query. If not, propose how to handle both query patterns (hint: consider secondary indexes or a separate HBase table).
- (c) If the data is additionally bucketed on customer_id into 256 buckets, and a join query joins this table with a 30-GB customers dimension table on customer_id, explain what bucket-map-join does and estimate the speedup over a common join.

AP4.3. HBase Row Key Design Analysis. A healthcare analytics system stores patient vital signs readings in HBase. Each reading has: patient_id (8-char string), metric (heart_rate, SpO2, temperature, blood_pressure), and timestamp (epoch milliseconds).

The following three row key designs are proposed:

- **Design A:** patient_id + timestamp (e.g., P-000042|1705315200000)
- **Design B:** timestamp + patient_id (e.g., 1705315200000|P-000042)
- **Design C:** patient_id + reverse_timestamp (e.g., P-000042|9994684799999)

- (a) For Design B, describe the write hotspot problem: if all writes have approximately the same timestamp, what happens to the distribution of writes across RegionServers?
- (b) For Design A, explain how a range scan for “all vitals for patient P-000042 in the last 24 hours” would be executed.
- (c) For Design C (reverse timestamp), explain what a scan for “most recent 10 readings for patient P-000042” looks like. Why does this design outperform Design A for this access pattern?
- (d) Propose a salting strategy for Design A that eliminates the write hotspot for high-frequency patients.

AP4.4. MapReduce vs. Pig vs. Hive Performance Comparison. A financial services firm must compute the total trading volume per stock symbol

per day for one year of trade records (1.5 TB 150 DataNodes, 10 Gbit/s network). Estimate completion time for each tool:

- (a) **Raw MapReduce** (Java): Map emits (symbol+date, amount); Reduce sums. Assume 80% data locality, 200 MB/s local read. 50 Reduce tasks. Estimate total Map I/O time, shuffle time (assume 20% of input volume is shuffled), and Reduce time.
- (b) **Apache Pig**: same logical plan as MapReduce, compiled to one MapReduce job with 30 seconds additional startup overhead per job. Estimate total elapsed time.
- (c) **Apache Hive on MapReduce**: same as Pig with 60 seconds additional startup overhead (Metastore lookup + plan compilation). But if the table is partitioned by date, and the query selects a 30-day period, how much data is read, and what is the new estimated time?
- (d) Which tool minimises calendar time? Which minimises engineering effort? Which minimises query latency for an analyst who runs this query daily?

Design Problems

DP4.1. MapReduce Pipeline for a Search Engine Inverted Index. Design a MapReduce pipeline that builds a compressed inverted index from a 500-terabyte web crawl stored in HDFS. Each crawl record is a JSON document containing the URL, crawl timestamp, and raw HTML. The inverted index maps each term to a posting list of (url, tf-idf_score) pairs, sorted by score descending.

Your design must specify:

- (a) The number of MapReduce stages required and what each stage computes (hint: tf-idf requires knowing the total document frequency of each term across the entire corpus, which is not available in a single pass).
- (b) The Map and Reduce function type signatures and logic for each stage.
- (c) The Partitioner strategy to ensure all records for the same term go to the same Reducer in the appropriate stage.

- (d) How the Combiner can be applied in Stage 1 and what the network savings are for a typical term with 10,000 occurrences per 128-MB input split.
- (e) How the pipeline handles the case where a single extremely common term (e.g., “the”) causes a Reducer skew: the single Reducer responsible for “the” processes 100× more data than the average Reducer.

DP4.2. HBase Schema Design for a Real-Time Recommendation System.

An online streaming music service needs to store and serve user listening history and song metadata for a real-time recommendation engine. Requirements:

- 50 million users; each user has an average of 2,000 play events, growing at 10 events/day.
- Each play event: `user_id`, `song_id`, `play_ts`, `duration_played_sec`, `skipped` (boolean).
- The recommendation engine reads the 200 most recent plays for a user in <5 ms.
- A nightly batch job (Hive or Spark) must scan all plays in the last 7 days across all users to recompute collaborative filtering features.

Design the HBase schema. Your design must specify:

- (a) The table name(s), column families, and column qualifiers.
- (b) The row key format, with explicit justification for how it supports the 5-ms recommendation read requirement.
- (c) How the “most recent 200 plays” query is executed (scan direction, stop condition, use of reverse timestamp).
- (d) How to configure cell versioning to automatically expire play events older than 90 days without a separate compaction job.
- (e) How the nightly batch Hive/Spark job reads from HBase (what API it uses and what the scan looks like).
- (f) The estimated RegionServer count if each region is limited to 10 GB, and the strategy for pre-splitting regions at table creation to avoid write hotspots.

[Programming Exercises](#)

PE4.1. MapReduce Word Count in Python (MRJob-style). Implement a word count MapReduce job in pure Python without any framework, simulating the Map, Shuffle/Sort, and Reduce phases. Extend it with a Combiner and measure the shuffle volume reduction.

```

1  import re
2  from collections import defaultdict
3
4  documents = [
5      ("doc1", "To be or not to be that is the question"),
6      ("doc2", "All the world is a stage and all the men and
7          women"),
8      ("doc3", "To be is to do to do is to be to be is to do"),
9      ("doc4", "Be not afraid of greatness some are born great"),
10 ]
11
12 # ----- MAP PHASE -----
13 def mapper(doc_id, text):
14     """Emit (word, 1) for every word in text."""
15     for word in re.findall(r'[a-z]+', text.lower()):
16         yield (word, 1)
17
18 # ----- COMBINER (optional local aggregation) -----
19 def combiner(map_output):
20     """
21     Locally aggregate (word, 1) pairs from a single mapper.
22     Input: iterable of (word, 1) tuples
23     Output: iterable of (word, local_count) tuples
24     """
25     local_counts = defaultdict(int)
26     for word, count in map_output:
27         local_counts[word] += count
28     return list(local_counts.items())
29
30 # ----- SHUFFLE AND SORT -----
31 def shuffle_and_sort(all_map_output):
32     """
33     Group all (word, value) pairs by word.
34     Returns dict: word -> list of values
35     """
36     grouped = defaultdict(list)
37     for word, val in all_map_output:

```

```
37         grouped[word].append(val)
38     return dict(sorted(grouped.items())) # sorted by key
39
40 # ----- REDUCE PHASE -----
41 def reducer(word, values):
42     """Sum all counts for a word."""
43     return (word, sum(values))
44
45 # ----- JOB RUNNER -----
46 def run_mapreduce(documents, use_combiner=True):
47     all_map_output = []
48     per_mapper_counts = []
49
50     for doc_id, text in documents:
51         map_out = list mapper(doc_id, text))
52         per_mapper_counts.append(len(map_out))
53
54         if use_combiner:
55             map_out = combiner(map_out)
56
57         all_map_output.extend(map_out)
58
59     shuffle_out = shuffle_and_sort(all_map_output)
60
61     results = {}
62     for word, values in shuffle_out.items():
63         _, count = reducer(word, values)
64         results[word] = count
65
66     shuffle_volume = sum(len(str(w)) + len(str(v))
67                          for w, v in all_map_output)
68     return results, shuffle_volume, per_mapper_counts
69
70 # Run without and with Combiner
71 res_no_comb, vol_no, cnts_no = run_mapreduce(documents,
72                                              use_combiner=False)
73
74 res_with_comb, vol_yes, cnts_yes = run_mapreduce(documents,
75                                                  use_combiner=True)
76
77 print("Top 10 words:")
78 for w, c in sorted(res_with_comb.items(), key=lambda x: -x[1])
79 [:10]:
80     print(f" {w:<15} {c}")
```

```

78 print(f"\nShuffle volume WITHOUT combiner: {vol_no:,} bytes")
79 print(f"Shuffle volume WITH combiner: {vol_yes:,} bytes")
80 print(f"Reduction factor: {vol_no / vol_yes:.1f}x")

```

Listing 4.6: PE4.1: MapReduce word count with Combiner

PE4.2. HiveQL Query Writing and Partitioning Analysis (Python simulation). Simulate the effect of Hive partitioning on query I/O by implementing a partition-pruning model that shows how much data is skipped for various query predicates.

```

1  import random
2  from datetime import date, timedelta
3
4  # Simulate a partitioned Hive table: (date, country) -> size_gb
5  random.seed(42)
6  countries = ["US", "GB", "DE", "FR", "JP", "IN", "BR", "AU"]
7  start = date(2023, 1, 1)
8
9  # Build partition metadata (what the Hive Metastore stores)
10 partitions = {}
11 for i in range(365):
12     dt = (start + timedelta(days=i)).isoformat()
13     for country in countries:
14         size_gb = random.uniform(0.5, 2.5)
15         partitions[(dt, country)] = round(size_gb, 3)
16
17 total_tb = sum(partitions.values()) / 1024
18 print(f"Total table size: {total_tb:.2f} TB across "
19       f"{len(partitions):,} partitions\n")
20
21 def query_io(predicate_dt=None, predicate_country=None):
22     """
23     Simulate partition pruning for a query with optional
24     filters.
25     predicate_dt      : exact date string or None (full scan)
26     predicate_country : country code or None (full scan)
27     Returns: (partitions_read, gb_read)
28     """
29     read = {k: v for k, v in partitions.items()
30             if (predicate_dt is None or k[0] ==
31                 predicate_dt)
32             and (predicate_country is None or k[1] ==
33                 predicate_country)}

```

```

31     return len(read), sum(read.values())
32
33 queries = [
34     ("Full table scan",
35      dict(predicate_dt=None, predicate_country=None)),
36     ("Single date (2024-01-15)",
37      dict(predicate_dt="2024-01-15", predicate_country=None)),
38     ("Single country (US), all dates",
39      dict(predicate_dt=None, predicate_country="US")),
40     ("Single date + country (2024-01-15, DE)",
41      dict(predicate_dt="2024-01-15", predicate_country="DE")),
42 ]
43
44 total_parts = len(partitions)
45 total_gb    = sum(partitions.values())
46
47 print(f"{'Query':<45} {'Parts read':>12} {'GB read':>10} {''
48       Pruned':>8}")
49 print("-" * 78)
50 for desc, kwargs in queries:
51     n_parts, gb = query_io(**kwargs)
52     pruned_pct = (1 - n_parts / total_parts) * 100
53     print(f"{'desc':<45} {'n_parts':>12,} {'gb':>10.1f} {'pruned_pct'
54           ':>7.1f}%")

```

Listing 4.7: PE4.2: Hive partition pruning simulator

PE4.3. HBase MemStore and Compaction Simulator (Python). Simulate HBase’s write path: WAL append, MemStore insertion, flush to HFile, and minor compaction.

```

1  import time
2
3  MEMSTORE_THRESHOLD = 10 # flush after 10 entries (demo scale)
4  COMPACTION_THRESHOLD = 3 # compact after 3 HFiles
5
6  class HBaseRegionSimulator:
7      def __init__(self, region_name):
8          self.region_name = region_name
9          self.wal          = []          # Write-Ahead Log (
10             list of ops)
11          self.memstore     = {}          # row_key -> {col ->
12             value}
13          self.hfiles       = []          # list of sorted dicts

```

```

12     self.write_count    = 0
13
14     def put(self, row_key, column, value):
15         ts = int(time.time() * 1000)
16         # 1. Append to WAL
17         self.wal.append({"op": "PUT", "row": row_key,
18                         "col": column, "val": value, "ts": ts
19                         })
20         # 2. Write to MemStore
21         if row_key not in self.memstore:
22             self.memstore[row_key] = {}
23         self.memstore[row_key][column] = (value, ts)
24         self.write_count += 1
25
26         # 3. Flush if threshold reached
27         if len(self.memstore) >= MEMSTORE_THRESHOLD:
28             self._flush()
29
30     def _flush(self):
31         if not self.memstore:
32             return
33         # Sort by row key (lexicographic) and materialise as
34         # HFile
35         hfile = dict(sorted(self.memstore.items()))
36         self.hfiles.append(hfile)
37         print(f"    [FLUSH] MemStore -> HFile #{len(self.hfiles)}
38               "
39               f"({len(hfile)} rows)")
40         self.memstore.clear()
41
42         # Compact if too many HFiles
43         if len(self.hfiles) >= COMPACTION_THRESHOLD:
44             self._minor_compact()
45
46     def _minor_compact(self):
47         print(f"    [COMPACT] Merging {len(self.hfiles)} HFiles
48               ...")
49         merged = {}
50         for hfile in self.hfiles:
51             for row, cols in hfile.items():
52                 if row not in merged:
53                     merged[row] = {}
54                 merged[row].update(cols)
55         self.hfiles = [dict(sorted(merged.items()))]

```

```

52     print(f" [COMPACT] Done. 1 HFile with {len(merged)}
      rows.")
53
54     def get(self, row_key):
55         # Check MemStore first
56         if row_key in self.memstore:
57             return {"source": "memstore", "data": self.memstore
                    [row_key]}
58         # Scan HFiles (most recent first)
59         for hfile in reversed(self.hfiles):
60             if row_key in hfile:
61                 return {"source": "hfile", "data": hfile[
                    row_key]}
62         return None
63
64     def status(self):
65         print(f"\nRegion '{self.region_name}' status:")
66         print(f"  WAL entries : {len(self.wal)}")
67         print(f"  MemStore      : {len(self.memstore)} rows")
68         print(f"  HFiles         : {len(self.hfiles)}")
69
70 # Simulate 25 writes to a user-activity HBase region
71 region = HBaseRegionSimulator("user-activity-0001")
72
73 users = [f"u-{i:05d}" for i in range(1, 8)]
74 import random; random.seed(0)
75 for _ in range(25):
76     uid = random.choice(users)
77     col = random.choice(["activity:logins", "activity:purchases
78         ",
79                         "profile:name", "profile:country"])
80     val = str(random.randint(1, 1000))
81     region.put(uid, col, val)
82
83 region._flush() # flush remaining MemStore
84 region.status()
85
86 # Test a GET
87 print(f"\nGET u-00001:", region.get("u-00001"))

```

Listing 4.8: PE4.3: HBase write path and compaction simulator

Resource Management, File Formats, and I/O Optimisation

“Storing data in HDFS is cheap. Storing it in the wrong format is expensive — you pay the cost every single time you query it.”

Julien Le Dem, Apache Parquet co-creator, 2014

Learning Objectives

After completing this chapter, you will be able to:

1. Identify the specific bottlenecks of the Hadoop 1.x JobTracker architecture and explain why they imposed an approximate 4,000-node scalability ceiling.
2. Describe YARN’s three-component architecture—Resource-Manager, NodeManager, and ApplicationMaster—and trace the complete job execution flow from client submission to container completion.
3. Explain what a YARN Container is, why it replaced the fixed MapReduce *slot* model, and how it enables framework-agnostic resource sharing.
4. Compare YARN’s three pluggable schedulers—FIFO, Capacity, and Fair—and select the appropriate scheduler for a given multi-tenant

production scenario.

5. Describe YARN's three-tier fault tolerance model (NodeManager, ApplicationMaster, and ResourceManager failures) and identify which tier each failure type is handled at.
6. Explain the fundamental difference between row-oriented and columnar storage, derive the I/O reduction formula for a k -column query on a c -column table, and apply it to a concrete performance calculation.
7. Describe the internal structure of a Parquet file—row groups, column chunks, and pages—and explain how the footer enables predicate pushdown to skip row groups without reading them.
8. Compare Avro, Parquet, and ORC on orientation, schema model, splittability, compression, and primary use case; and select the appropriate format for a given data engineering workload.
9. Explain the splittability property for compression codecs and explain why Gzip-compressed CSV is an anti-pattern in HDFS.
10. Describe Dominant Resource Fairness (DRF) and explain why single-resource fairness metrics fail for heterogeneous workloads.

Chapters 3 and 4 established the two pillars of the original Hadoop stack: HDFS stores data, and MapReduce processes it. By 2012, however, two engineering problems were threatening the viability of that stack at production scale.

The first problem was *scheduling*. A Hadoop 1.x cluster could only run MapReduce jobs; when Apache Spark (2010) and Apache Tez (2013) emerged as significantly faster alternatives, they had no path to cluster resources because the MapReduce JobTracker was hardcoded to manage only MapReduce tasks. Organisations that wanted to use Spark had to maintain a completely separate cluster, with all the associated duplication of hardware, operations effort, and data movement overhead.

The second problem was *storage format*. As analytical workloads evolved from full-table scans to targeted column selections—driven by the growth of SQL analytics (Hive) and machine learning feature engineering—the row-oriented file formats that Hadoop had inherited (CSV, JSON, sequence

files) became serious performance bottlenecks. Selecting 3 columns from a 100-column CSV table on HDFS required reading and discarding 97% of the data on every query.

YARN (Vavilapalli et al. 2013) and columnar file formats (Parquet, ORC) were the industry's responses to these two problems. This chapter covers both in depth, then examines the compression and I/O optimisation techniques that complete the performance picture.

5.1 The Hadoop 1.x Scheduling Bottleneck

In Hadoop 1.x, cluster resource management and MapReduce job execution were fused into a single process: the *JobTracker*. The JobTracker performed four distinct functions simultaneously:

1. Accepted job submissions from clients.
2. Allocated fixed *slots*—Map slots and Reduce slots—to tasks running on TaskTrackers.
3. Monitored every running task and detected failures.
4. Maintained the job history for completed jobs.

This fusion created three interlocking problems that collectively prevented Hadoop 1.x from scaling beyond approximately 4,000 nodes.

Framework lock-in. The slot allocation model was hardcoded for MapReduce. A Map slot could run exactly one Map task; a Reduce slot could run exactly one Reduce task. There was no abstraction for the resource requirements of a non-MapReduce task—no way for a Spark executor to request a container of 8 GB RAM and 4 CPU cores, for example. Any framework that needed to run on the cluster had to pretend to be MapReduce.

Rigid slot granularity. Map slots and Reduce slots were configured with fixed memory and CPU limits cluster-wide. A job with small Map tasks and large Reduce tasks could not allocate more memory to Reducers at the expense of Mappers; both were constrained by the same slot size. A node with 10 Map slots and 0 available Reduce slots could not repurpose its unused Map slots to serve waiting Reduce tasks, even if it had spare RAM and CPU capacity.

Single-point scalability. The JobTracker’s memory footprint grew linearly with the number of running tasks: each task required a metadata entry in the JobTracker’s heap. At 4,000 nodes with 10 tasks per node, the JobTracker managed 40,000 task records simultaneously. Beyond this scale, JVM garbage collection pressure made the JobTracker unresponsive (Vavilapalli et al. 2013).

5.2 YARN: Yet Another Resource Negotiator

Apache Hadoop YARN, published by Vavilapalli et al. in 2013, resolved all three bottlenecks through a single architectural decision: separate cluster resource management from application-specific execution logic (Vavilapalli et al. 2013). YARN replaces the monolithic JobTracker with three distinct components, each with a clearly bounded responsibility.

5.2.1 The `ResourceManager`

The *ResourceManager* (RM) is the cluster-level resource scheduler. It runs on a dedicated master node and contains two sub-components.

The *Scheduler* is responsible for allocating *containers*— the YARN resource abstraction—to running applications. The Scheduler is *pluggable*: operators can choose from three built-in policies (described in Section 5.3) or implement a custom policy. Critically, the Scheduler has no knowledge of the application logic; it only knows the container requirements (`{vcores: N, memory: M MB}`) and the cluster’s current capacity. This separation means the same Scheduler can manage MapReduce tasks, Spark executors, Samza stream processors, and any other future framework without modification.

The *ApplicationsManager* accepts job submissions from clients and coordinates the launch of each application’s first container, which will host the `ApplicationMaster`.

5.2.2 The `NodeManager`

A *NodeManager* (NM) runs on every worker node in the cluster. Its responsibilities are narrowly scoped to the local machine:

- Launch containers requested by ApplicationMasters, using Linux cgroups for CPU isolation and enforced memory limits.
- Monitor the resource usage (CPU, RAM, disk I/O, network) of every container on the local node and report to the ResourceManager via periodic heartbeats.
- Kill any container that exceeds its allocated memory limit to protect other containers on the same node.
- Aggregate container logs and upload them to HDFS for post-job debugging.

The NodeManager has no global view of the cluster; it knows only about its own node. This is a deliberate design choice: the scalability of the YARN control plane depends on the RM and each NM having well-bounded state, with the RM collecting aggregate information rather than processing per-task details.

5.2.3 The ApplicationMaster

The *ApplicationMaster* (AM) is the component that makes YARN framework-agnostic. One ApplicationMaster runs per submitted job, inside its own container allocated by the ResourceManager. The AM contains all application-specific logic that the old JobTracker had embedded:

- **Resource negotiation:** the AM communicates with the RM to request containers for its tasks. A Spark AM requests containers with specific CPU and memory configurations for Spark Executors; a MapReduce AM requests separate Map-phase and Reduce-phase containers.
- **Task placement:** the AM instructs NodeManagers to launch specific tasks inside allocated containers. The AM can apply its own data-locality preferences when choosing which nodes to request containers from.
- **Progress monitoring:** the AM tracks the progress of all running tasks and re-requests containers for failed tasks.
- **Framework-specific fault tolerance:** the MapReduce AM implements speculative execution; the Spark AM manages executor loss and task re-submission.

The AM model means YARN is, in the authors' words, "a cluster oper-

ating system” (Vavilapalli et al. 2013): the RM and NMs are the kernel that manages hardware resources; the AM is a user-space process that implements the policy of a specific framework. Just as Linux supports arbitrary user-space processes, YARN supports arbitrary AMs.

5.2.4 The Container: YARN’s Resource Unit

Definition	YARN Container
A YARN Container is an allocation of a specific quantity of cluster resources on a specific node: $\text{Container} = \{ \text{node} : m, \text{ vCores} : n, \text{ memory} : M \text{ MB} \}$ The Container replaces Hadoop 1.x’s fixed Map/Reduce slot with a flexible, application-specified resource envelope. The NodeManager on node m enforces the allocated limits; any container exceeding its memory allocation is immediately killed by the NM.	

The Container abstraction is the key enabler of framework diversity. Because a Container is specified in terms of generic resources (vCores and RAM)—not in terms of task types—the same ResourceManager can grant containers to a MapReduce AM requesting small (2 vCPU, 4 GB) Map-task containers, a Spark AM requesting large (8 vCPU, 32 GB) executor containers, and a Samza AM requesting tiny (1 vCPU, 2 GB) stream-task containers, all simultaneously on the same cluster.

5.2.5 YARN Job Execution Flow

The YARN job execution follows six steps from client submission to completion:

1. **Submit.** The client submits the job to the ResourceManager’s ApplicationsManager, providing the application JAR (or binary), configuration, and resource requirements.
2. **Launch AM container.** The RM allocates one container on an available NodeManager for the ApplicationMaster. The NM launches the AM process inside that container.

3. **AM registers and requests resources.** The AM registers itself with the RM and submits resource requests: a list of containers with their required vCores, RAM, and preferred nodes (for data locality).
4. **RM grants containers.** The Scheduler allocates containers based on the scheduling policy and current cluster availability. Granted containers are communicated to the AM.
5. **AM launches tasks.** For each granted container, the AM contacts the appropriate NodeManager via RPC and instructs it to launch a specific task process inside the container.
6. **Task execution, monitoring, and completion.** Tasks run inside containers, report progress to the AM. On task completion, the AM releases the container back to the RM. When all tasks complete, the AM unregisters from the RM, and the RM releases the AM's own container.

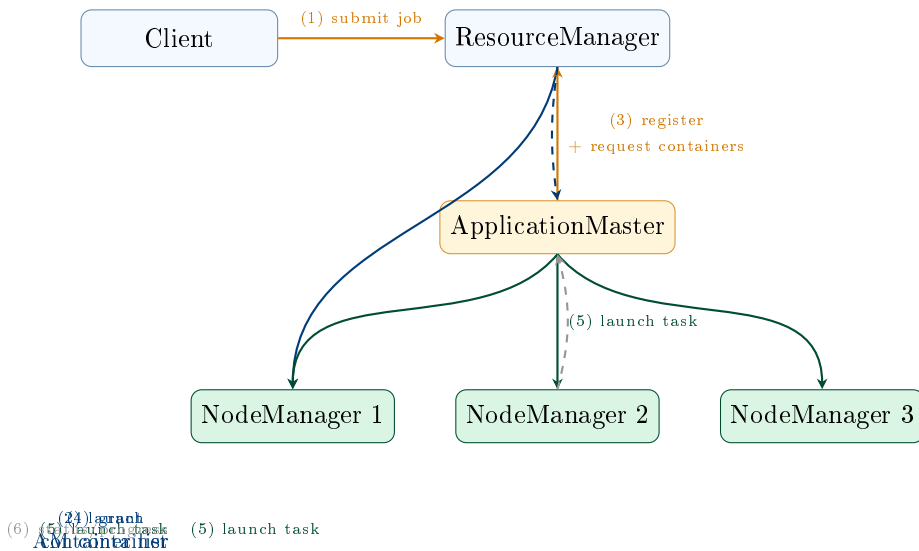


Figure 5.1: YARN job execution flow. The ResourceManger (RM) manages global resources and launches the ApplicationMaster (AM) in its own container. The AM then negotiates additional containers with the RM and instructs NodeManagers to launch application tasks, monitoring their progress independently of the RM.

5.3 YARN Schedulers: Sharing the Cluster

With multiple teams submitting jobs simultaneously to a shared cluster, the Scheduler’s policy determines which jobs get resources first and how much capacity each team is guaranteed. YARN provides three built-in scheduler implementations.

5.3.1 The FIFO Scheduler

The *FIFO (First In, First Out) Scheduler* maintains a single job queue and assigns resources in submission order. The first job submitted receives all available cluster capacity until it completes; subsequent jobs wait. FIFO is simple to understand and configure, but it is unsuitable for multi-tenant clusters. A single long-running batch job (e.g., a multi-hour ETL pipeline) can starve time-sensitive interactive queries for hours. FIFO is appropriate only for single-user development clusters or automated pipeline clusters that run one job at a time.

5.3.2 The Capacity Scheduler

The *Capacity Scheduler*, the default in Hadoop distributions from Hortonworks and Cloudera, divides the cluster’s total capacity into named *queues*. Each queue has a guaranteed minimum capacity and a maximum capacity:

- The **minimum capacity** ensures that jobs in this queue can always obtain at least X% of cluster resources, even when other queues are busy.
- The **maximum capacity** prevents a single queue from consuming the entire cluster, even when the cluster is otherwise idle.
- **Elastic borrowing**: when a queue has unused capacity, other queues can borrow it. The capacity is reclaimed—with a configurable grace period for preemption—when the owning queue submits new jobs.

A typical multi-team Hadoop cluster might configure:

Queue	Min capacity	Max capacity
production	60%	80%
analytics	30%	60%
adhoc	10%	40%

The production queue always has at least 60% of cluster capacity available for revenue-generating pipelines, even if the analytics and adhoc queues are idle.

5.3.3 The Fair Scheduler

The *Fair Scheduler*, the default in Apache Hadoop distributions and historically used by Facebook and Twitter, allocates resources so that all running jobs receive an equal share of available capacity over time. When only one job is running, it receives 100% of the cluster. When a second job is submitted, it begins receiving resources immediately; over a short window, both jobs converge on 50% each. When the second job completes, the first job again receives 100%.

The key property of the Fair Scheduler is its treatment of newly submitted short jobs. Under the Capacity Scheduler, a short interactive query submitted to a busy production queue must wait behind already-running batch jobs. Under the Fair Scheduler, the new query preempts some resources from running jobs immediately, completing quickly before returning those resources to the batch jobs. This makes the Fair Scheduler particularly effective for clusters that serve a mix of batch pipelines and interactive analytical queries.

5.3.4 Dominant Resource Fairness

Both the Capacity and Fair Schedulers must decide how to measure “fairness” in a cluster with multiple resource dimensions (vCores and RAM). A job requiring many CPUs but little memory and a job requiring few CPUs but much memory have different resource profiles; allocating equal CPU to both may leave one job RAM-starved while the other has excess RAM.

Dominant Resource Fairness (DRF), introduced by Ghodsi et al. at NSDI 2011 (Ghodsi et al. 2011), resolves this by defining each job’s fairness share in terms of its *dominant resource*—the resource type for which the job’s relative allocation is highest. If a job consumes 20% of the cluster’s CPU but only 5% of its RAM, its dominant resource is CPU and its dominant share is 20%. The DRF scheduler equalises dominant shares across all running jobs, ensuring that no job dominates the cluster in the resource dimension that

matters most to it. YARN's Fair Scheduler implements DRF as its fairness metric for multi-resource scheduling.

5.3.5 YARN Fault Tolerance

YARN's fault tolerance is layered across three tiers, matching the three components of its architecture.

NodeManager failure. If a NodeManager stops sending heartbeats to the ResourceManager (default timeout: 10 minutes), the RM marks it as dead. All containers on that NodeManager are immediately marked as failed, and the RM notifies the relevant ApplicationMasters. Each AM decides how to respond according to its framework's fault tolerance policy: a MapReduce AM re-requests replacement containers for failed Map or Reduce tasks; a Spark AM marks the executor as lost and reschedules its tasks on surviving executors.

ApplicationMaster failure. If the AM process crashes, the RM detects the failure via a missed heartbeat and restarts the AM in a new container (up to a configurable maximum number of attempts, default 2). The restarted AM can recover job state from HDFS checkpoints if the framework supports it. MapReduce AMs checkpoint completed task outputs to HDFS, allowing a restarted AM to resume from the last completed stage. Early versions of Spark did not checkpoint AM state, so AM failure required restarting the entire job.

ResourceManager failure. In Hadoop 1.x, RM failure was catastrophic: the entire cluster became inaccessible. YARN 2.4 introduced *ResourceManager High Availability*, mirroring the HDFS HA design: an Active RM and a Standby RM, with ZooKeeper providing leader election and automatic failover in under 60 seconds. The Active RM's state (running applications, container allocations) is periodically checkpointed to ZooKeeper so the Standby can reconstruct it on promotion.

Research Spotlight | Vavilapalli et al. (2013) — Apache Hadoop YARN: Yet Another Resource

Citation: Vavilapalli, V. K., et al. (2013). Apache Hadoop YARN: Yet Another Resource Negotiator. *Proceedings of the 4th ACM Symposium on Cloud Computing (SoCC 2013)*, pp. 5:1–5:16. (Vavilapalli et al. 2013)

Key insight: The paper's central thesis is that the fusion of resource

management and MapReduce execution logic in Hadoop 1.x's JobTracker was the primary scalability and extensibility bottleneck. Separating these concerns—through the RM/NM/AM decomposition—makes the cluster a general-purpose computing platform rather than a MapReduce appliance.

Technical contributions:

- Defined the Container abstraction (`{node, vCores, memory}`) as the universal resource unit, replacing the rigid Map/Reduce slot model and enabling flexible, framework-specific resource allocation.
- Introduced the ApplicationMaster pattern: a per-application process that encapsulates framework-specific scheduling, fault tolerance, and monitoring logic, making YARN framework-agnostic by design.
- Demonstrated scalability to 6,000 nodes with 100,000 simultaneous tasks, exceeding Hadoop 1.x's 4,000-node limit by 50%, with ResourceManager scheduling latency below 1 second at 1,000 heartbeats/second.

Impact today: YARN is cited over 3,000 times and became the default cluster OS for the entire Hadoop ecosystem. Its three-tier architecture directly influenced Kubernetes's Pod/Node/Master model; the Container abstraction is the direct ancestor of the Kubernetes Pod. Apache Spark, Apache Tez, Apache Samza, Apache Flink, and TensorFlow on Hadoop all ran as YARN applications. The ApplicationMaster pattern established a design precedent that every subsequent cluster computing framework has replicated.

5.4 Row-Oriented vs. Columnar Storage

The second major performance dimension this chapter addresses is the physical organisation of data within files. The choice between row-oriented and columnar (column-oriented) storage can change the I/O cost of an analytical query by one to two orders of magnitude, making it one of the highest-leverage decisions in a data engineering system.

5.4.1 Row-Oriented Storage

In a *row-oriented* (or *N-ary storage model*, NSM) file format, all fields of a single record are stored contiguously on disk:

user_id=1, name="Alice", age=24, score=91, country="US", ...
user_id=2, name="Bob", age=31, score=78, country="DE", ...
user_id=3, name="Carol", age=27, score=88, country="JP", ...

Row-oriented storage is optimal for workloads that need to read or write complete records: transactional databases (OLTP), log ingestion pipelines, and streaming append operations. Writing a new record requires a single sequential write. Reading a specific record by key requires reading one contiguous block of bytes.

Row-oriented storage is *suboptimal* for analytical workloads that select a small number of columns from a wide table. A query `SELECT score FROM users` must read every field of every row— `user_id`, `name`, `age`, `country`, and all other columns—even though only the `score` field is needed. For a table with c columns, a query selecting k columns reads c/k times more data than necessary.

5.4.2 Columnar Storage: The I/O Reduction Formula

In a *columnar* (or *decomposition storage model*, DSM) file format, all values of a single column are stored contiguously on disk:

user_id: [1, 2, 3, ...]	name: [Alice, Bob, Carol, ...]
score: [91, 78, 88, ...]	

For the same `SELECT score FROM users` query, a columnar store reads *only the score column chunk*—a contiguous block containing `[91, 78, 88, ...]`, and nothing else. The `user_id`, `name`, `age`, and other column chunks are not read at all.

Definition	Columnar I/O Reduction
Let a table have c columns, n rows, and $\bar{s}$ bytes per cell on average. A	

query selecting k columns:

$$\text{Row store I/O} = n \cdot c \cdot \bar{s} \quad (5.1)$$

$$\text{Column store I/O} = n \cdot k \cdot \bar{s} \quad (5.2)$$

$$\text{I/O reduction} = \frac{k}{c} \quad (5.3)$$

For $k = 3$, $c = 100$: the columnar store reads only 3% of the data that a row store would read—a 33× reduction.

Additional compression efficiency: because all values in a column chunk have the same data type, they compress far better than a mixed row. A column of country codes (“US”, “DE”, “JP”, ...) with high repetition compresses 10–50× better than an entire row containing a mix of integers, strings, and floats.

Row Store (CSV, JSON)			
user_id	name	age	score
1	Alice	24	91
2	Bob	31	78
3	Carol	27	88
4	Dave	29	95

Query reads ALL 4 columns

Column Store (Parquet, ORC)			
user_id	name	age	score
1	Alice	24	91
2	Bob	31	78
3	Carol	27	88
4	Dave	29	95

Query reads ONLY score column

Figure 5.2: Row-oriented vs. columnar storage for a `SELECT score FROM users` query. The row store reads all four columns for every row (red box). The column store reads only the score column chunk (green box), skipping `user_id`, `name`, and `age` entirely. For 100 columns with a 3-column query, the columnar store provides a 33× I/O reduction.

5.5 Big Data File Formats in Depth

Three file formats dominate the modern Hadoop and cloud data lake ecosystem: Apache Avro, Apache Parquet, and Apache ORC. Each represents a different design point in the space of storage orientation, schema management, compression efficiency, and processing engine compatibility.

5.5.1 Apache Avro: Schema-Evolving Row Storage

Apache Avro is a row-oriented serialisation framework developed at Yahoo! in 2009 and now an Apache top-level project. Avro stores each record as a single contiguous binary blob, with the schema described as a JSON document embedded in the file header. It is the preferred format for data ingestion pipelines—particularly Kafka-to-HDFS streams—for two reasons.

Schema evolution. As event schemas change over time (new fields added, field names changed, types widened), Avro allows *reader schemas* and *writer schemas* to differ according to well-defined compatibility rules. A consumer reading data written with an older schema can specify a new reader schema; Avro’s binary codec automatically:

- Fills missing fields with their declared default values.
- Ignores fields present in the data but absent from the reader schema.
- Promotes numeric types (e.g., int to long).

This forward and backward compatibility makes Avro the standard for streaming ingestion (Kafka) to long-term storage (HDFS), where the schema evolves continuously as the application changes.

Splittability. An Avro file is divided into *blocks* (not to be confused with HDFS blocks) separated by sync markers. A Hadoop InputFormat can seek to a sync marker and start reading from any block without decompressing from the beginning, making Avro files natively splittable.

5.5.2 Apache Parquet: Columnar Analytics Storage

Apache Parquet, developed jointly by Twitter and Cloudera in 2013, is the de facto standard for analytical data storage in the Hadoop ecosystem and on cloud data lakes. Its internal organisation is designed to minimise I/O for column-selective analytical queries.

File structure. A Parquet file is divided into *row groups* (default ~128 MB, aligning with HDFS block size). Each row group is subdivided into *column chunks*—one per column. Each column chunk is further divided into *pages* of approximately 1 MB each. Data within a page is optionally dictionary-encoded (replacing repeated values with short integer codes) and then compressed with a configurable codec (Snappy by default).

Predicate pushdown. The Parquet *file footer* stores column-level statistics—

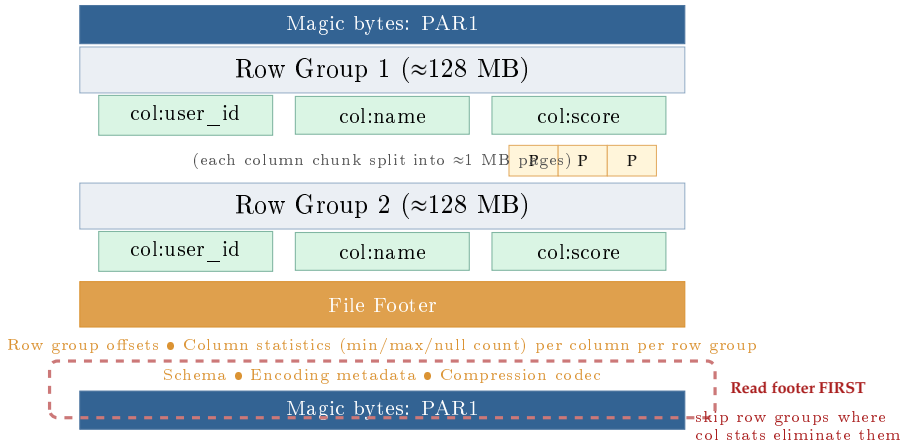


Figure 5.3: Parquet file internal structure. The footer contains per-column, per-row-group statistics (min, max, null count) that enable predicate pushdown: a query with `WHERE score > 90` reads the footer first, then skips all row groups where `max(score) ≤ 90`, reading zero bytes from those row groups.

minimum value, maximum value, and null count—for every column chunk in every row group. A query engine (Spark SQL, Hive on Tez, Presto) reads the footer first. If the footer shows that the score column in Row Group 1 has `min = 60`, `max = 85`, and the query filters `WHERE score > 90`, the engine can skip the entire Row Group 1 without reading a single byte of its data. This *predicate pushdown* to the storage layer can reduce the physical I/O of a selective query from terabytes to gigabytes.

Dictionary encoding. Before compression, Parquet applies *dictionary encoding* to columns with low cardinality. A column containing country codes with 200 distinct values out of 100 million rows stores the 200 distinct values in a compact dictionary, then stores each row’s value as a small integer index into the dictionary. This can reduce the size of a low-cardinality column by 80–95% before compression is even applied.

5.5.3 Apache ORC: Hive-Optimised Columnar Storage

Apache ORC (Optimised Row Columnar) was developed at Hortonworks in 2013 specifically for Apache Hive and is the recommended format for the Hive ecosystem. ORC and Parquet are architecturally similar—both are columnar, self-describing, and support predicate pushdown—but differ in several engineering choices.

ORC uses *stripes* (analogous to Parquet’s row groups, default 250 MB) and stores a *file-level index* in the footer plus a *stripe-level index* within each stripe. The stripe-level index provides statistics at 10,000-row granularity within the stripe, enabling finer-grained skipping than Parquet’s row-group granularity.

ORC’s default compression codec is Zlib (gzip), which achieves higher compression ratios than Parquet’s default Snappy at the cost of slower decompression speed. For Hive workloads where query latency is measured in minutes rather than seconds, the storage savings from Zlib typically outweigh the decompression overhead. ORC also supports Hive’s ACID (atomicity, consistency, isolation, durability) transactions—a feature Parquet does not natively support—enabling UPDATE and DELETE operations on Hive tables.

Table 5.1: Comparison of the five major Hadoop file formats

Format	Orient.	Schema	Split.	Compress.	Primary use case
Text/CSV	Row	External	With codec	None (default)	Small data; portability
JSON	Row	Self (verbose)	No (raw)	External	APIs; log ingestion
Avro	Row	Self (JSON)	Yes	Snappy/Deflate	Kafka ingest; schema evolution
Parquet	Columnar	Self	Yes	Snappy/Gzip	Spark, Presto, ML; analytics
ORC	Columnar	Self	Yes	Zlib/Snappy	Hive warehouse; ACID; archival

5.6 Compression Codecs and Splittability

Compression reduces the physical size of data on disk and in transit, lowering both storage costs and I/O time. In the Hadoop ecosystem, choosing the right compression codec requires balancing three dimensions: compression ratio, decompression speed, and—most critically for distributed processing—*splittability*.

5.6.1 The Splittability Property

Definition	Splittability
A file format is splittable if a Hadoop InputFormat can start reading from any point in the file without decompressing from the beginning. Splittability determines whether HDFS can assign multiple independent Map tasks to different portions of the same file, enabling full parallelism.	

The practical consequence is dramatic. A 10 GB unsplittable Gzip CSV file on HDFS is stored in 80 consecutive 128 MB blocks. HDFS distributes these blocks across the cluster normally, but the MapReduce or Spark job can assign only a *single* task to this file: the task must read all 80 blocks sequentially from start to finish, since decompression must begin at byte 0 and proceed in order. No parallelism is possible.

A 10 GB Parquet+Snappy file with 128 MB row groups is similarly stored in 80 blocks. But the Parquet InputFormat can start reading at any row group boundary independently. The job assigns 80 parallel tasks, each reading one row group from a local DataNode—achieving 80× parallelism and completing the job in 1/80th the wall-clock time of the single-task case.

5.6.2 Compression Codec Comparison

Common Misconception	The Gzip Anti-Pattern
The most common performance anti-pattern in Hadoop data pipelines is storing large datasets as Gzip-compressed CSV or JSON files. Gzip's block format does not contain synchronisation markers that allow random seeks; decompression must begin at byte 0. A 1 TB Gzip CSV	

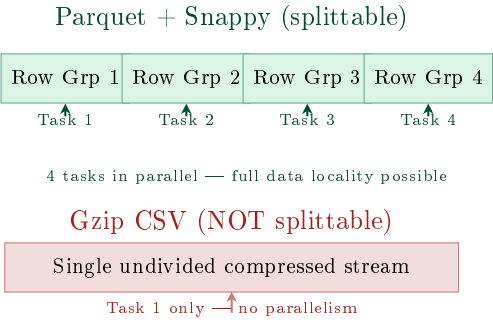


Figure 5.4: Splittability comparison. Parquet’s row-group boundaries allow N tasks to process N row groups in parallel. A Gzip-compressed CSV file has no internal boundaries the InputFormat can seek to, so the entire file is processed by a single task.

file processed by a 1,000-node Spark cluster is forced to run on *a single task*, using one core on one node, while the other 999 nodes sit idle. The correct alternative is **Parquet + Snappy**: the Parquet format provides splittable boundaries (row groups) even though Snappy itself is not splittable, and Snappy’s very fast decompression minimises CPU overhead.

5.7 I/O Optimisation Techniques

Beyond format and codec selection, three additional query-time optimisation techniques reduce the I/O cost of analytical workloads on columnar storage systems.

5.7.1 Predicate Pushdown

Predicate pushdown moves filter conditions from the query engine into the storage layer. In Parquet, the file footer stores column-level statistics (min, max, null count) per row group. When a query contains a filter predicate (e.g., `WHERE revenue > 1000000`), the query engine evaluates the predicate against the footer statistics for each row group *before reading the row group’s data*. Any row group for which the predicate is definitely false

Table 5.2: Hadoop compression codecs: performance and splittability

Codec	Speed	Ratio	Splittable	Recommended use
Snappy	Fastest	Medium	No (but Parquet/Avro row groups are)	Parquet, Avro in Spark/Hive (default)
Gzip	Slow	High	No	Avoid for large HDFS files; use Parquet+Snappy instead
LZO	Fast	Medium	Yes (with index)	Legacy Hadoop; requires external index file
Bzip2	Slowest	Highest	Yes (natively)	Large plain-text files where I/O cost dominates
Zstd	Fast	High	No (block-level)	ORC archival; Parquet cold storage

(i.e., $\max(\text{revenue}) \leq 1,000,000$) is skipped entirely. Only row groups where the predicate *might* be true are read.

In practice, for a time-series table partitioned by date and sorted by user ID within each partition, predicate pushdown on a user-ID filter can skip 99% of the data in large Parquet files, reducing a 1-hour query to under 2 minutes.

5.7.2 Column Pruning

Column pruning (also called *projection pushdown*) ensures that only the columns required by the query are read from disk. In a row-oriented format, this is impossible: all columns of a row are stored together, so reading one requires reading all. In a columnar format, each column is stored

independently; the query engine passes the list of required columns to the file reader, which opens and reads only those column chunks. A query selecting 5 of 200 columns from a Parquet file reads exactly $5/200 = 2.5\%$ of the storage bytes.

Apache Spark applies column pruning automatically for Parquet files through the Catalyst query optimiser: the optimiser analyses the query's column references before execution and passes only the referenced columns to the Parquet reader. No manual configuration is required.

5.7.3 Partition Pruning

Partition pruning, introduced in the context of Hive in Chapter 4, operates at the file system (HDFS directory) level rather than within a file. A table partitioned by *dt* (date) stores each day's data in a separate HDFS subdirectory (`/data/table/dt=2024-01-15/`). A query filtering on a specific date reads only that subdirectory's files, skipping all other partitions entirely. Partition pruning, predicate pushdown, and column pruning are complementary and can be applied simultaneously:

1. **Partition pruning** eliminates entire HDFS directories based on the partition key filter.
2. **Predicate pushdown** eliminates row groups within the remaining Parquet files based on column statistics in the footer.
3. **Column pruning** eliminates irrelevant column chunks within the remaining row groups.

The combined effect can reduce the I/O of a targeted analytical query from terabytes to megabytes—multiple orders of magnitude—purely through optimisations at the storage layer, without any changes to the query itself.

5.8 Case Study: LinkedIn's Multi-Framework YARN Cluster (2013–2016)

Case Study | LinkedIn — Consolidating Five Frameworks on One 5,000-Node YARN Cluster

Background. By 2013, LinkedIn was processing approximately 500 TB of data per day for its professional networking platform—analysing member profiles, connections, job postings, messages, and advertising signals to power features used by over 300 million members. The company ran five distinct data processing frameworks: MapReduce (for Hive ETL), Apache Spark (for the People You May Know recommendation algorithm, PYMK), Apache Samza (for real-time Kafka stream processing), Apache Pig (for ad targeting feature engineering), and Apache Tez (for interactive Hive-on-Tez queries). Before YARN, these frameworks ran on *separate siloed clusters*—three distinct Hadoop clusters, each configured and operated independently.

The silo problem. The siloed cluster architecture created two systemic inefficiencies. First, each cluster was under-utilised: the MapReduce cluster averaged 45% utilisation (heavily used during nightly batch runs, idle during the day), while the Spark cluster was heavily utilised for the PYMK job's 4-hour compute window and nearly idle the rest of the time. Combining both clusters would have provided the same total capacity at significantly higher average utilisation. Second, cross-cluster data movement added 30–90 minutes of latency to pipelines that needed data from multiple clusters: an ETL job on the MapReduce cluster had to export results to HDFS and wait for the Spark cluster to ingest them.

The unified YARN architecture. In 2013, LinkedIn migrated to a unified 5,000-node YARN cluster using the Capacity Scheduler with six queues:

Queue	Capacity	Primary workload
production	40%	Hive ETL; Pig ad features
spark	25%	PYMK; Spark ML models
samza	15%	Kafka stream processing
adhoc	10%	Analyst Hive queries
tez	7%	Interactive Hive-on-Tez
reserve	3%	Overflow / burst capacity

The PYMK pipeline. LinkedIn’s People You May Know (“Friend recommendations”) feature illustrates how multiple frameworks share the YARN cluster in a single end-to-end pipeline:

1. **Ingest** (Samza, samza queue): a Kafka consumer reads member-profile-view events at approximately 2 million events per second, materialising them to HDFS in Avro format.
2. **Feature store** (Hive, production queue): a nightly Hive ETL job joins first-, second-, and third-degree connection data for all 300 million members into a wide feature table stored in ORC+Zlib format (partitioned by member-ID bucket).
3. **Graph model** (Spark GraphX, spark queue): a Spark job runs label propagation over the social graph to score approximately 10 billion candidate recommendation pairs per day. The PYMK Spark job used 600 executors each with 8 GB RAM and 4 vCPUs—4,800 GB of RAM and 2,400 vCPUs drawn from the spark queue’s 25% allocation of the 5,000-node cluster.
4. **Serve**: top-*k* recommendations per member are written to Voldemort (LinkedIn’s key-value store) for real-time serving at sub-10-millisecond latency.

Quantified outcomes. The migration from three siloed clusters to a unified YARN cluster produced measurable improvements across every operational dimension:

- **Utilisation:** average cluster utilisation rose from ~45% (across three separate clusters) to ~78% (on the unified cluster)—a 73% increase in effective compute capacity from the same hardware.

- **PYMK latency:** the PYMK Spark job recompute time dropped from 18 hours (on a separate, undersized Hadoop 1.x MapReduce cluster) to 4 hours (on YARN with Spark GraphX), enabling daily rather than every-other-day recommendation refreshes.
- **Operational cost:** consolidation eliminated one full team's worth of cluster operations overhead and reduced inter-cluster data synchronisation latency by 90%.
- **Peak containers:** at peak, the unified cluster ran over 400,000 simultaneous containers across all six queues.

The architectural lesson. LinkedIn's deployment established a principle that has since become an industry standard: *the cluster is the computer, and the resource manager is its operating system*. Just as an operating system allows multiple processes with different CPU and memory profiles to share hardware fairly, YARN allows multiple data processing frameworks with different resource profiles to share a cluster fairly. The Capacity Scheduler's queue hierarchy provides the "process priority" mechanism; the Container is the "process memory allocation"; the NodeManager's cgroup-based isolation is the "memory protection". This analogy— cluster OS / cluster kernel—was explicitly used by the YARN paper's authors and has guided the design of Kubernetes, Mesos, and every subsequent cluster resource manager.

Chapter Summary

- Hadoop 1.x's **JobTracker** bottleneck had three causes: framework lock-in (MapReduce only), rigid fixed-size slots, and single-process scalability limits ($\approx 4,000$ nodes).
- YARN separates resource management into three components: **ResourceManager** (global scheduler), **NodeManager** (per-node agent), and **ApplicationMaster** (per-job framework logic). A **Container** ({node, vCores, memory}) replaces the fixed slot.
- YARN's three schedulers: **FIFO** (single queue), **Capacity** (guaranteed

multi-queue allocation), and **Fair** (equal sharing over time). **DRF** extends fairness to multiple resource dimensions.

- **Row-oriented** formats (CSV, JSON, Avro) store all fields of a record contiguously. **Columnar** formats (Parquet, ORC) store all values of a column contiguously. For a k -column query on a c -column table, columnar I/O is reduced by factor k/c .
- **Avro**: row-oriented, schema evolution (reader/writer schemas), splittable, Kafka-native. **Parquet**: columnar, row groups, footer statistics for predicate pushdown, Spark/Presto native. **ORC**: columnar, stripe-level index, Zlib compression, Hive ACID support.
- A file is **splittable** if multiple tasks can read different ranges independently. Gzip CSV is not splittable (one task only). Parquet/Avro blocks are splittable even if the codec (Snappy) is not.
- The three I/O optimisation layers apply in order: **partition pruning** (skip directories), **predicate pushdown** (skip row groups via footer statistics), **column pruning** (read only required column chunks).

Exercises

Short Questions

SQ5.1. What is the YARN ResourceManager's Scheduler component? What information does the Scheduler use to make allocation decisions, and what information does it deliberately *not* know?

SQ5.2. Explain the role of the ApplicationMaster (AM) in YARN. Why does one AM per job—rather than a centralised AM for all jobs—improve the scalability of the ResourceManager?

SQ5.3. What is a YARN Container? Write the resource specification for a container that would host a Spark executor requiring 8 CPU cores and 24 GB of RAM on node `dn-07.cluster.internal`.

SQ5.4. Explain the difference between YARN's Capacity Scheduler and Fair Scheduler. Give a production scenario where the Capacity Scheduler

is the better choice, and a scenario where the Fair Scheduler is better.

SQ5.5. What is Dominant Resource Fairness (DRF)? Give an example where allocating equal CPU across two jobs would result in an unfair outcome, and explain how DRF resolves it.

SQ5.6. A YARN NodeManager stops sending heartbeats to the ResourceManager. Trace the sequence of events that occurs: which component detects the failure, what happens to the containers on that NodeManager, and which component decides what to do next?

SQ5.7. What is the difference between the *write schema* and the *read schema* in Apache Avro? Give a concrete example where they differ and explain how Avro handles the mismatch.

SQ5.8. Explain predicate pushdown in Apache Parquet. What information is stored in the Parquet file footer, and how does a query engine use it to avoid reading data?

SQ5.9. What is the splittability property for a Hadoop file format? A 10 GB file on a 100-node cluster is stored as Gzip-compressed CSV. How many Map tasks does MapReduce assign to this file, and why?

SQ5.10. Compare Parquet and ORC on three dimensions: primary processing engine, default compression codec, and ACID transaction support.

SQ5.11. A data engineer proposes storing a 500 TB analytics warehouse as raw JSON files (one JSON object per line) in HDFS. Identify three specific performance problems with this proposal and propose a better alternative.

SQ5.12. Explain column pruning (projection pushdown). For a 200-column Parquet table, a query references 6 columns. What fraction of the physical storage does the Parquet reader access?

SQ5.13. What does it mean for a columnar format to use *dictionary encoding*? For a column containing the values “US”, “GB”, “DE”, “FR” repeated across 500 million rows, explain how dictionary encoding reduces the column chunk size.

SQ5.14. YARN’s ResourceManager HA uses ZooKeeper for failover. What role does ZooKeeper play, and what happens to running applications during the 30–60 second RM failover window?

Analytical Problems

AP5.1. YARN Capacity Planning. An e-commerce company operates a 2,000-node YARN cluster. Each node has 64 GB RAM and 16 vCPUs. The Capacity Scheduler is configured with three queues: batch (50%), streaming (30%), adhoc (20%). Container sizes: batch MapReduce mapper: {4 GB, 2 vCPU}; streaming Samza task: {2 GB, 1 vCPU}; adhoc Spark executor: {8 GB, 4 vCPU}.

- (a) Calculate the total cluster RAM and vCPU capacity.
- (b) For each queue, calculate the maximum number of concurrent containers, and identify whether the bottleneck resource is RAM or vCPU.
- (c) A batch job needs 5,000 mapper containers simultaneously. How many batch nodes can serve this simultaneously? If demand exceeds the batch queue's allocation, describe what happens under (i) Capacity Scheduler with preemption and (ii) Capacity Scheduler without preemption.
- (d) The streaming team submits a Samza job with 3,000 tasks, each requiring {2 GB, 1 vCPU}. Calculate the dominant resource for the streaming queue using DRF.

AP5.2. File Format I/O Comparison. A retail company stores 5 years of transaction records in HDFS. The table has 80 columns, 10 billion rows, and an average cell size of 12 bytes. A weekly analytics query selects 4 columns, filters on a date range covering 7 days out of 365, and has a predicate on one column that eliminates 90% of row groups.

- (a) Calculate the total uncompressed table size in terabytes.
- (b) If stored as CSV, estimate the I/O for the weekly query (no partition pruning, no predicate pushdown).
- (c) If stored as Parquet + Snappy, partitioned by date, with predicate pushdown: apply partition pruning (7/365 days), then predicate pushdown (90% of remaining row groups skipped), then column pruning (4/80 columns). Calculate the remaining I/O.
- (d) Express the I/O reduction as a ratio. If each I/O unit (TB read) costs

the cluster 5 minutes of compute time on a 100-node cluster, estimate the query time saving.

AP5.3. Splittability and Parallelism Analysis. A genomics pipeline ingests 2 TB of DNA sequencing data per day. Three format options are proposed:

- **Option A:** single 2 TB Gzip-compressed FASTQ file
 - **Option B:** 16,000 Gzip-compressed FASTQ files of 128 MB each
 - **Option C:** Parquet files with 128 MB row groups (2 TB total across $\approx 16,000$ row groups)
- (a) For each option, calculate the number of Map tasks HDFS assigns, and explain whether data-local execution is achievable.
 - (b) Option B produces 16,000 files. What is the NameNode memory overhead (assume 300 bytes per file + block pair, and each file fits in one 128 MB block)?
 - (c) Option C uses a single logical Parquet file of 2 TB divided into 16,000 row groups. Assuming perfect data locality, how many Map tasks run in parallel if the cluster has 500 nodes with 40 task slots each?
 - (d) Rank the three options on: (i) MapReduce parallelism, (ii) NameNode memory pressure, (iii) engineering simplicity. Recommend one option for the production pipeline.

AP5.4. YARN Failure Cascade Analysis. A 500-node YARN cluster is running a Spark job with 400 executor containers distributed across 400 nodes. Each executor holds 50 GB of cached RDD data (4 partitions of 12.5 GB each). A power distribution unit trip simultaneously takes down 50 nodes.

- (a) How many executor containers are immediately lost?
- (b) Which YARN components detect the node failure, and in what order?
- (c) The Spark ApplicationMaster requests 50 replacement executors. If the remaining 450 nodes each have 4 available executor slots, how quickly can the RM satisfy the request (assume 500 ms scheduling cycle)?
- (d) The replacement executors must recompute the 50 lost executors' cached RDD partitions (200 partitions, 12.5 GB each, at 100 MB/s recompute

throughput). Estimate the total recompute time.

- (e) What data engineering design change would eliminate the need for full RDD recomputation after executor failure?

Design Problems

DP5.1. Data Lake Format Strategy for a Financial Services Platform. A global investment bank ingests 15 sources of market data into a central data lake: real-time tick data (1 billion trades/day, each trade: {symbol, price, volume, timestamp, exchange, side}), reference data (static security metadata: 2 million instruments, 120 columns), and regulatory reporting data (daily settlement reports, 80 columns, 7-year retention requirement).

Design the format and compression strategy for each data tier. Your design must specify:

- (a) The HDFS file format and compression codec for each of the three source types, with explicit justification for each choice.
- (b) The partitioning scheme for tick data (which columns to partition on, at what granularity, and why).
- (c) The YARN queue configuration for three consumer teams: algorithmic trading analytics (latency-sensitive, needs <5-minute query response), risk analytics (batch, 2-hour window nightly), and compliance reporting (weekly, long-running, low priority).
- (d) How you would handle schema evolution if the tick data schema adds three new fields for a new exchange that emits extended market data.
- (e) The estimated storage reduction from converting raw CSV tick data (at 35 bytes per event on average) to Parquet+Snappy (assuming 6:1 compression ratio on this workload) for one year of data.

DP5.2. YARN Migration from Three Siloed Clusters. A media company runs three separate Hadoop clusters: Cluster A (1,000 nodes, MapReduce for nightly content ETL), Cluster B (500 nodes, Spark for recommendation models), Cluster C (300 nodes, Kafka/Samza for real-time user event processing). Average utilisation: Cluster A 40%, Cluster B 35%, Cluster C 55%. The company plans to consolidate into a single YARN cluster.

Design the consolidation. Your design must address:

- (a) Total node count for the unified cluster, given a target average utilisation of 70% (justify your sizing).
- (b) Capacity Scheduler queue configuration: queue names, capacity percentages, and maximum capacities for each team's workload.
- (c) The migration sequence: which workload migrates first, how parallel operation during the transition is managed, and how you validate that the migrated workload performs correctly.
- (d) How preemption should be configured between the nightly ETL jobs (MapReduce, 8-hour window) and the recommendation Spark jobs (4-hour window, started 2 hours before ETL completes).
- (e) The rollback plan if the unified cluster shows unexpected performance degradation for the Samza real-time workload.

Programming Exercises

PE5.1. YARN Capacity Planning Calculator (Python). Implement a YARN capacity planning tool that models a multi-queue cluster, computes maximum concurrent containers per queue, identifies the bottleneck resource (RAM vs. vCPU), and detects over-subscription.

```
1  from dataclasses import dataclass
2
3  @dataclass
4  class NodeSpec:
5      count: int
6      ram_gb: int
7      vcpus: int
8
9  @dataclass
10 class QueueSpec:
11     name: str
12     capacity_pct: float # fraction of cluster [0, 1]
13     container_ram_gb: int
14     container_vcpus: int
15
16 def plan_cluster(node: NodeSpec, queues: list[QueueSpec]):
```

```

17     total_ram_gb = node.count * node.ram_gb
18     total_vcpus  = node.count * node.vcpus
19
20     print(f"Cluster: {node.count} nodes x {node.ram_gb} GB / {
21           node.vcpus} vCPU")
22     print(f"Total:    {total_ram_gb:,} GB RAM | {total_vcpus:,}
23           vCPUs")
24     print()
25     print(f"'Queue':<14} {'Cap':>5} {'RAM alloc':>10} {'CPU
26           alloc':>10} "
27           f"'Max conts':>10} {'Bottleneck':>12}")
28     print("-" * 65)
29
30     for q in queues:
31         q_ram_gb = total_ram_gb * q.capacity_pct
32         q_vcpus  = total_vcpus  * q.capacity_pct
33         by_ram   = int(q_ram_gb / q.container_ram_gb)
34         by_cpu   = int(q_vcpus  / q.container_vcpus)
35         max_c    = min(by_ram, by_cpu)
36         bottle   = "RAM" if by_ram < by_cpu else "vCPU"
37         print(f"{q.name:<14} {q.capacity_pct*100:>4.0f}% "
38               f"{q_ram_gb:>9,.0f} GB {q_vcpus:>9,.0f} "
39               f"{max_c:>10,} {bottle:>12}")
40
41     # Detect container spec errors: container bigger than node
42     print()
43     for q in queues:
44         if q.container_ram_gb > node.ram_gb:
45             print(f"[ERROR] Queue {q.name}: container RAM {q.
46                   container_ram_gb} GB "
47                   f"> node RAM {node.ram_gb} GB")
48         if q.container_vcpus > node.vcpus:
49             print(f"[ERROR] Queue {q.name}: container vCPUs {q.
50                   container_vcpus} "
51                   f"> node vCPUs {node.vcpus}")
52
53     # LinkedIn 5,000-node cluster model
54     node = NodeSpec(count=5_000, ram_gb=64, vcpus=16)
55
56     queues = [
57         QueueSpec("production", 0.40, container_ram_gb=4,
58                  container_vcpus=2),
59         QueueSpec("spark",      0.25, container_ram_gb=8,
60                  container_vcpus=4),

```

```

54     QueueSpec("samza",      0.15, container_ram_gb=2,
55               container_vcpus=1),
56     QueueSpec("adhoc",      0.10, container_ram_gb=4,
57               container_vcpus=2),
58     QueueSpec("tez",        0.07, container_ram_gb=4,
59               container_vcpus=2),
60     QueueSpec("reserve",    0.03, container_ram_gb=4,
61               container_vcpus=2),
62 ]
63
64 plan_cluster(node, queues)

```

Listing 5.1: PE5.1: YARN capacity planning calculator

PE5.2. Parquet Layout and Predicate Pushdown Simulator (Python). Simulate the Parquet file structure—row groups, column chunks, footer statistics—and implement predicate pushdown to skip row groups for a simple filter predicate.

```

1  import random
2  import math
3
4  random.seed(42)
5
6  ROW_GROUP_ROWS = 1_000      # rows per row group (small for
7                               demo)
8  TOTAL_ROWS     = 10_000     # total table rows
9  COLUMNS       = ["user_id", "name", "country", "age", "score",
10                    "revenue"]
11  BYTES_PER_CELL = 8          # average bytes per cell
12
13 def generate_row_group(rg_id, start_row, rows):
14     """Simulate one Parquet row group with column stats."""
15     data = {
16         "user_id": list(range(start_row, start_row + rows)),
17         "score":   [random.randint(40, 100) for _ in range(rows)
18                     ],
19         "revenue": [random.randint(0, 1_000_000) for _ in range
20                     (rows)],
21         "country": [random.choice(["US", "GB", "DE", "JP", "FR"])
22                     for _ in range(rows)],
23         "age":     [random.randint(18, 80) for _ in range(rows)
24                     ],
25         "name":    [f"User_{i}" for i in range(start_row,

```

```

        start_row + rows)],
20     }
21     # Build footer stats per column
22     stats = {}
23     for col, vals in data.items():
24         if isinstance(vals[0], int):
25             stats[col] = {"min": min(vals), "max": max(vals),
26                           "null_count": 0}
27         else:
28             stats[col] = {"min": min(vals), "max": max(vals),
29                           "null_count": 0}
30     return {"rg_id": rg_id, "row_count": rows, "data": data, "
31             stats": stats}
32
33 # Build Parquet file (footer + row groups)
34 num_rg = math.ceil(TOTAL_ROWS / ROW_GROUP_ROWS)
35 rg_list = [generate_row_group(i, i * ROW_GROUP_ROWS,
36                               min(ROW_GROUP_ROWS,
37                                   TOTAL_ROWS - i *
38                                   ROW_GROUP_ROWS))
39             for i in range(num_rg)]
40
41 footer = {"num_row_groups": num_rg,
42           "schema": COLUMNS,
43           "row_group_stats": [rg["stats"] for rg in rg_list]}
44
45 def parquet_scan(row_groups, footer, predicate_col,
46                 predicate_op, threshold,
47                 select_cols):
48     """
49     Simulate Parquet scan with:
50     - predicate pushdown (skip row groups via footer stats)
51     - column pruning (read only select_cols)
52     """
53     total_rg = len(row_groups)
54     rg_read = 0
55     rows_returned = 0
56
57     for i, rg in enumerate(row_groups):
58         stats = footer["row_group_stats"][i]
59         col_stat = stats.get(predicate_col, {})
60
61         # Predicate pushdown: can we skip this row group?
62         if predicate_op == ">" and col_stat.get("max", float("

```



```

        inf")) <= threshold:
60         continue    # definitely no rows satisfy predicate
                    -> skip
61         if predicate_op == "<" and col_stat.get("min", float("-inf")) >= threshold:
62             continue
63
64         rg_read += 1
65         # Column pruning: only access select_cols
66         for row_i in range(rg["row_count"]):
67             row_val = rg["data"][predicate_col][row_i]
68             if predicate_op == ">" and row_val > threshold:
69                 rows_returned += 1
70             elif predicate_op == "<" and row_val < threshold:
71                 rows_returned += 1
72
73         # I/O calculation
74         full_scan_bytes = total_rg * ROW_GROUP_ROWS * len(COLUMNS)
75                         * BYTES_PER_CELL
76         actual_col_bytes = rg_read * ROW_GROUP_ROWS * len(
77                         select_cols) * BYTES_PER_CELL
78         io_reduction_pct = (1 - actual_col_bytes / full_scan_bytes)
79                         * 100
80
81         print(f"Query: SELECT {select_cols} WHERE {predicate_col} {
82               predicate_op} {threshold}")
83         print(f"  Row groups total : {total_rg}")
84         print(f"  Row groups read  : {rg_read} (skipped {total_rg
85               - rg_read} via pushdown)")
86         print(f"  Rows returned   : {rows_returned:,}")
87         print(f"  Full scan bytes  : {full_scan_bytes:,}")
88         print(f"  Actual I/O bytes : {actual_col_bytes:,}")
89         print(f"  I/O reduction   : {io_reduction_pct:.1f}%")
90
91         # Query: SELECT user_id, score WHERE score > 90
92         parquet_scan(rg_list, footer,
93                     predicate_col="score", predicate_op=">", threshold
94                     =90,
95                     select_cols=["user_id", "score"])
96     print()
97     # Query: SELECT user_id, revenue WHERE revenue > 800000
98     parquet_scan(rg_list, footer,
99                 predicate_col="revenue", predicate_op=">",
100                 threshold=800_000,

```

```
94 select_cols=["user_id", "revenue"])
```

Listing 5.2: PE5.2: Parquet predicate pushdown simulator

PE5.3. DRF Fairness Simulation (Python). Simulate Dominant Resource Fairness for a cluster with two resource dimensions (CPU and RAM) and three competing jobs, showing how DRF equalises dominant shares.

```
1 def drf_allocate(cluster_cpu, cluster_ram_gb, jobs, max_rounds
  =20):
2     """
3     Simulate DRF allocation.
4     Each job specifies: name, cpu_per_task, ram_per_task_gb,
      max_tasks
5     DRF grants one task at a time to the job with the lowest
      dominant share.
6     """
7     allocated = {j["name"]: {"tasks": 0, "cpu": 0.0, "ram":
      0.0}
8
9         for j in jobs}
9     used_cpu = 0.0
10    used_ram = 0.0
11
12    print(f"Cluster: {cluster_cpu} CPUs | {cluster_ram_gb} GB
      RAM")
13    print(f"\n{'Round':>5} {'Job':>12} {'Dominant':>10} "
14          f"{'Dom share':>10} {'Tasks':>7} {'CPU':>7} {'RAM
      ':>7}")
15    print("-" * 68)
16
17    for rnd in range(1, max_rounds + 1):
18        # Compute dominant share for each job
19        dom_shares = {}
20        for j in jobs:
21            a = allocated[j["name"]]
22            if a["tasks"] >= j["max_tasks"]:
23                dom_shares[j["name"]] = float("inf")
24                continue
25            cpu_share = a["cpu"] / cluster_cpu if cluster_cpu >
26                0 else 0
27            ram_share = a["ram"] / cluster_ram_gb if
28                cluster_ram_gb > 0 else 0
29            dom_res = "cpu" if cpu_share >= ram_share else "
      ram"
```

```

28         dom_shares[j["name"]] = (max(cpu_share, ram_share),
                                   dom_res)
29
30     # Select job with minimum dominant share
31     eligible = [j for j in jobs
32                 if isinstance(dom_shares[j["name"]], tuple)
33                 and used_cpu + j["cpu_per_task"] <=
34                     cluster_cpu
35                 and used_ram + j["ram_per_task_gb"] <=
36                     cluster_ram_gb]
37
38     if not eligible:
39         break
40
41     winner = min(eligible, key=lambda j: dom_shares[j["name"]][0])
42     w_name = winner["name"]
43     dom_val, dom_res = dom_shares[w_name]
44
45     allocated[w_name]["tasks"] += 1
46     allocated[w_name]["cpu"] += winner["cpu_per_task"]
47     allocated[w_name]["ram"] += winner["ram_per_task_gb"]
48     used_cpu += winner["cpu_per_task"]
49     used_ram += winner["ram_per_task_gb"]
50
51     print(f"{rnd:>5} {w_name:>12} {dom_res:>10} "
52           f"{dom_val*100:>9.1f}% "
53           f"{allocated[w_name]['tasks']:>7} "
54           f"{allocated[w_name]['cpu']:>7.1f} "
55           f"{allocated[w_name]['ram']:>7.1f}")
56
57     print("\nFinal allocations:")
58     for j in jobs:
59         a = allocated[j["name"]]
60         cpu_share = a["cpu"] / cluster_cpu * 100
61         ram_share = a["ram"] / cluster_ram_gb * 100
62         print(f" {j['name']:<14}: {a['tasks']} tasks | "
63               f"CPU: {a['cpu']:.1f} ({cpu_share:.1f}%) | "
64               f"RAM: {a['ram']:.1f} GB ({ram_share:.1f}%)")
65
66 # 3 jobs with different resource profiles
67 jobs = [
68     {"name": "MapReduce", "cpu_per_task": 2, "ram_per_task_gb":
69      4, "max_tasks": 8},
70     {"name": "Spark", "cpu_per_task": 4, "ram_per_task_gb":

```

```
67         "": 16, "max_tasks": 6},  
        {"name": "Samza", "cpu_per_task": 1, "ram_per_task_gb"  
68         "": 2, "max_tasks": 12},  
69     ]  
70     drf_allocate(cluster_cpu=32, cluster_ram_gb=128, jobs=jobs,  
        max_rounds=20)
```

Listing 5.3: PE5.3: Dominant Resource Fairness (DRF) simulator

Part IV

In-Memory Processing

Apache Spark: In-Memory Distributed Computing

“The most important question we faced was: what should be the fundamental programming model? We chose to expose distributed memory as a first-class abstraction, and that single decision determined everything else about Spark’s design.”

Matei Zaharia, co-creator of Apache Spark, 2016

Learning Objectives

After completing this chapter, you will be able to:

1. Identify the specific I/O bottleneck of MapReduce for iterative workloads and derive the disk-I/O cost formula that motivated the design of Spark.
2. Define a Resilient Distributed Dataset (RDD), explain its five defining properties, and describe how lineage-based fault tolerance differs from replication-based fault tolerance.
3. Trace the life cycle of a Spark job from `sc.textFile()` through the DAG Scheduler, Task Scheduler, and Executor, identifying the role of each component.
4. Distinguish narrow transformations from wide transformations, ex-

- plain why wide transformations introduce shuffle boundaries, and identify which RDD operations fall into each category.
5. Explain the performance difference between `groupByKey` and `reduceByKey`, and apply the correct operator to avoid unnecessary data movement.
 6. Describe the four phases of the Catalyst query optimizer (analysis, logical optimisation, physical planning, and code generation) and explain how each phase reduces query execution cost.
 7. Select the appropriate RDD persistence level from `MEMORY_ONLY`, `MEMORY_AND_DISK`, `MEMORY_ONLY_SER`, and `DISK_ONLY` for a given workload, cost, and fault-tolerance requirement.
 8. Describe the Spark MLlib Pipeline API—`Transformer`, `Estimator`, `Pipeline`, and `PipelineModel`—and construct a three-stage feature engineering and classification pipeline.
 9. Explain Spark’s unified memory management model, distinguish execution memory from storage memory, and calculate the available memory for each region given a heap size and configuration.
 10. Apply the salting technique to resolve data skew in a wide transformation, and explain why skew is undetectable by the Catalyst optimizer.

By 2010, Hadoop’s MapReduce framework had conquered the storage and batch-processing challenges that motivated its creation. Thousands of organisations processed petabytes per day with clusters of commodity servers, achieving the economics that Google had demonstrated internally with its MapReduce implementation (Dean and Ghemawat 2004). Yet practitioners who attempted to use MapReduce for machine learning training, graph analysis, and iterative data mining encountered a fundamental performance barrier: MapReduce was designed for data that is read once, processed once, and written once. Any algorithm that reads the same data repeatedly—which describes virtually every machine learning training algorithm—paid an enormous, repeated disk I/O penalty that made computation on even modest datasets prohibitively slow.

This chapter introduces Apache Spark, the distributed computing frame-

work that resolved the iterative-workload bottleneck by elevating distributed memory to a first-class programming abstraction. Spark’s central concept, the *Resilient Distributed Dataset* (RDD), allows the results of an expensive computation to be stored in the combined RAM of a cluster and reused across multiple algorithm iterations without a single disk read. For workloads such as logistic regression training, this in-memory approach achieved speedups of $20\times$ to $100\times$ over MapReduce in the original benchmarks (Zaharia, Chowdhury, Das, et al. 2012).

6.1 The MapReduce Iteration Problem

To understand why Spark was necessary, we must quantify precisely what MapReduce does poorly: iterative computation over the same dataset.

Consider training a logistic regression model on a 100 GB training set using gradient descent. The training algorithm proceeds in rounds, called *iterations* or *epochs*. In each iteration, the algorithm: (1) reads the entire training set; (2) computes the gradient of the loss function with respect to the current model parameters; (3) updates the parameters; and (4) checks a convergence criterion. A typical training run requires 50 to 200 iterations before the model converges.

The MapReduce implementation maps one iteration to one MapReduce job. Each job:

1. Reads the 100 GB training set from HDFS ($3\times$ replicated = 300 GB of disk reads).
2. Runs Map tasks to compute per-record gradient contributions.
3. Runs a Reduce task to aggregate the gradients.
4. Writes the updated model parameters to HDFS.
5. The next iteration reads HDFS again from the beginning.

Definition	MapReduce Iteration I/O Cost
For an iterative algorithm requiring I iterations over a dataset of size D	

bytes, stored on HDFS with replication factor r :

$$\text{Total disk I/O (MapReduce)} = I \times D \times r \quad (6.1)$$

For logistic regression with $I = 100$ iterations, $D = 100$ GB, $r = 3$:

$$\text{Total disk I/O} = 100 \times 100 \text{ GB} \times 3 = 30,000 \text{ GB} = 30 \text{ TB}$$

Processing 30 TB of disk I/O to train a model on 100 GB of unique data is a $300\times$ I/O amplification. On a cluster that sustains 1 GB/s aggregate disk read throughput per node, with 100 nodes, this represents 5 minutes of wall-clock time *per iteration*—8.3 hours total for 100 iterations.

The root cause is straightforward: MapReduce has no mechanism for retaining intermediate data in memory between jobs. Every job boundary is a forced write to and read from HDFS. The framework was designed for single-pass batch processing, not iterative refinement. The cluster’s RAM—which by 2010 already totalled hundreds of gigabytes across even a modest 100-node cluster—is completely wasted between iterations: it is cleared, and the entire dataset is re-read from disk for the next round.

Interactive analytical workloads suffer a related problem. A data analyst running ten queries over the same dataset against a Hive-on-MapReduce cluster must wait for the full input to be read from HDFS for each query, even if the dataset has already been read by a prior query. There is no shared memory pool between independent jobs that could serve as a cache.

Matei Zaharia and his collaborators at UC Berkeley identified both problems in 2010 and proposed a solution: expose the cluster’s aggregate RAM as a distributed, fault-tolerant storage layer that persists across job boundaries. This idea became Apache Spark (Zaharia, Chowdhury, Franklin, et al. 2010).

6.2 Resilient Distributed Datasets

The fundamental abstraction of Apache Spark is the *Resilient Distributed Dataset* (RDD), introduced formally by Zaharia et al. at NSDI 2012 (Zaharia, Chowdhury, Das, et al. 2012). An RDD is the unit of data in Spark in the same way that an HDFS block is the unit of data in Hadoop: every computation operates on RDDs, produces RDDs, and stores RDDs.

Definition	Resilient Distributed Dataset (RDD)
------------	-------------------------------------

An **RDD** is an immutable, partitioned, fault-tolerant collection of records that can be stored in memory across a cluster of nodes and computed in parallel. Formally, an RDD R is characterised by five properties:

1. **A set of partitions:** $R = \{P_1, P_2, \dots, P_k\}$, where each partition P_i is a subset of the data assigned to one executor.
2. **A set of dependencies:** a reference to the parent RDD(s) from which R was derived, forming a lineage graph (DAG).
3. **A function to compute each partition** from its parent partition(s).
4. **Partitioning metadata:** the scheme (e.g., hash or range) used to assign records to partitions.
5. **Preferred locations:** hints on which nodes hold the data locally (e.g., HDFS block locations).

6.2.1 Lineage-Based Fault Tolerance

The “Resilient” in RDD refers to Spark’s mechanism for recovering from executor failures: *lineage-based fault tolerance*. This is the most conceptually important property of RDDs, and it distinguishes Spark’s approach from both MapReduce (which writes intermediate results to disk) and traditional distributed shared memory (which replicates data in RAM).

When an executor fails and the partitions it held are lost, Spark does not need to restart the entire job. Instead, it consults the RDD’s *lineage graph*—the directed acyclic graph of transformations that produced the lost partition—and *recomputes only the lost partition* on a surviving executor.

This design trades replication overhead for recomputation cost. RDDs

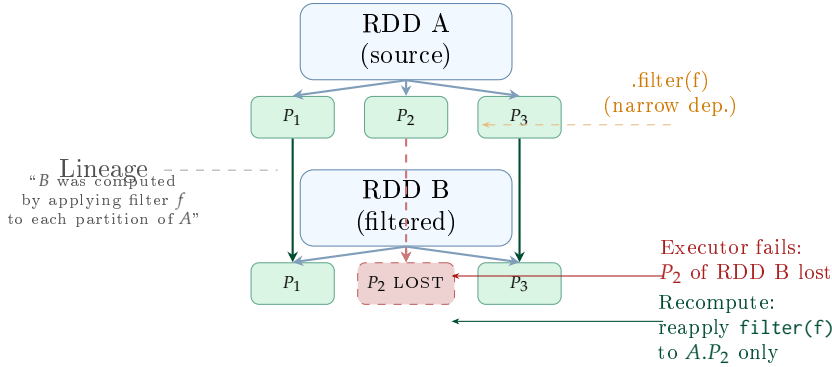


Figure 6.1: RDD lineage-based fault tolerance. When P_2 of RDD B is lost due to executor failure, Spark consults the lineage graph to determine that P_2 of B was produced by applying `filter(f)` to P_2 of RDD A, which is still available. Only the lost partition is recomputed—no full job restart is required.

backed by data on HDFS (the most common case) can always recompute any lost partition from its HDFS source data, which is itself replicated. For RDDs produced by long chains of transformations, Spark allows users to insert *checkpoints*—explicit writes of an RDD to HDFS that truncate the lineage graph—preventing unbounded recomputation cost in the event of failure after many transformation stages.

Key Insight | Lineage vs. Replication: A Design Trade-off

Lineage-based fault tolerance costs nothing in memory (no second copy of the data is maintained) but can be expensive in time if the lineage is long. Replication-based fault tolerance (as in HDFS) costs memory for every replica but recovers in constant time regardless of lineage length. Spark’s design assumes that executor failures are rare and datasets are large relative to available RAM—so paying memory for replication would defeat the purpose of in-memory computing. For streaming workloads where maintaining lineage across an unbounded stream is impractical, Spark Structured Streaming adds micro-batch replication.

6.2.2 Lazy Evaluation and the RDD DAG

RDDs are computed *lazily*: calling a transformation method such as `filter()` or `map()` does not immediately execute any computation. It only records the

transformation in the RDD's lineage graph. Actual computation is triggered only when the programmer calls an *action*—a method that returns a non-RDD result to the driver (such as `count()`, `collect()`, or `saveAsTextFile()`).

This laziness is not a limitation; it is a deliberate optimisation opportunity. When an action is called, Spark's DAG Scheduler can inspect the entire lineage graph of all RDDs involved in producing the result and optimise the computation plan. It can fuse adjacent narrow transformations into a single pass over the data, avoiding intermediate materialisation, and it can identify which partitions are already cached in memory and skip their recomputation.

6.2.3 RDD Persistence

The performance advantage of Spark over MapReduce materialises only when RDDs are *persisted*. When an RDD is used in multiple actions—the common case in iterative algorithms—the programmer must explicitly call `rdd.persist(StorageLevel.MEMORY_ONLY)` (or the equivalent `rdd.cache()`, which is shorthand for `MEMORY_ONLY`) to instruct Spark to retain the RDD's partitions in executor memory after the first computation. Without persistence, each action triggers a full recomputation from the lineage root.

Spark evicts persisted RDD partitions using a *least-recently-used* (LRU) policy when storage memory is exhausted. Under `MEMORY_AND_DISK`, evicted partitions spill to the local disk of the executor node rather than being dropped; they are read back from disk if needed by a subsequent action. Under `MEMORY_ONLY`, evicted partitions are simply recomputed from their lineage on demand.

6.3 Spark Architecture

A Spark application consists of a single *driver program* and one or more *executors* running on worker nodes. The driver and executors communicate through a *cluster manager*, which allocates the physical resources (CPU cores and RAM) that the application requires.

Table 6.1: RDD storage levels and their trade-offs

Storage level	Memory	Disk	When to use
MEMORY_ONLY	Yes (raw)	No	Fits in RAM; fast recompute from HDFS
MEMORY_AND_DISK	Yes (raw)	Overflow	Larger-than-RAM; avoid full recompute
MEMORY_ONLY_SER	Yes (Kryo)	No	Tight on RAM; CPU overhead acceptable
MEMORY_AND_DISK_SER	Yes (Kryo)	Overflow	Tight on RAM + large data
DISK_ONLY	No	Yes	Rarely accessed; fast recompute is slow
OFF_HEAP	Off- heap	No	Avoid GC pauses; Tungsten memory

6.3.1 The Driver Program and SparkSession

The *driver program* is the process that contains the user’s application code. It runs the `main()` function of the Spark application, constructs the *SparkSession* (the unified entry point since Spark 2.0, replacing the earlier `SparkContext` and `HiveContext`), and orchestrates the overall execution.

The driver performs three critical functions:

- **DAG construction:** as the application code calls transformation methods, the driver records each transformation in an RDD lineage DAG. No computation occurs yet.
- **Job submission:** when an action is called, the driver submits a job to the DAG Scheduler. The DAG Scheduler partitions the lineage DAG into *stages*—groups of transformations that can be pipelined without a data shuffle—and submits each stage as a set of *tasks* to the Task Scheduler.
- **Result collection:** for actions that return data to the driver (such as `collect()` or `first()`), the driver receives task results from executors and assembles the final result.

Common Misconception | The Driver Memory Trap

`rdd.collect()` transfers *all* partitions of an RDD to the driver's JVM heap. On a production dataset of even a few hundred gigabytes, this causes an `OutOfMemoryError` that crashes the driver process and terminates the entire application. The correct alternatives are `rdd.take(n)` (return only the first n records), `rdd.saveAsTextFile(path)` (write results to HDFS without collecting), or `rdd.sample(false, 0.01)` (return a 1% sample). Use `collect()` only for small result sets that the driver can safely hold in memory—such as the output of a final aggregation.

6.3.2 Executors

Executors are long-lived JVM processes launched on worker nodes at application start-up time. Each executor:

- Runs *tasks* assigned by the Task Scheduler, with multiple tasks running concurrently in separate threads (one thread per available vCPU core allocated to the executor).
- Maintains a local *block store*—the Spark Block Manager—that holds cached RDD partitions and shuffle output blocks.
- Reports task status, progress, and shuffle data locations back to the driver via a heartbeat mechanism.
- Runs for the entire lifetime of the application, enabling in-memory data sharing across multiple Spark jobs within the same application.

The executor's JVM heap is divided between *execution memory* (for active task computation: sort buffers, hash tables for aggregations, shuffle read/write buffers) and *storage memory* (for cached RDD partitions). This division is discussed in depth in Section 6.7.

6.3.3 Cluster Managers

Spark is *cluster-manager-agnostic*: it delegates the allocation of physical resources to a pluggable cluster manager. Three cluster managers are commonly used in production:

- **Spark Standalone:** Spark's built-in cluster manager, suitable for dedicated Spark clusters. Simple to configure but does not support multi-

tenant resource sharing or integration with other frameworks. Appropriate for development and single-purpose deployment.

- **YARN:** the most common production deployment (see Chapter 5). Spark submits an `ApplicationMaster` to YARN; the AM negotiates executor containers with the `ResourceManager`. All of YARN's scheduling policies (Capacity, Fair, DRF) apply to Spark executors. Enables multi-framework resource sharing with MapReduce, Tez, and Samza on the same cluster.
- **Kubernetes:** the preferred deployment model on cloud infrastructure. The driver runs as a Kubernetes Pod; each executor runs in its own Pod. Kubernetes manages container scheduling, scaling, and isolation using its standard infrastructure, enabling integration with cloud autoscaling.

6.3.4 DAG Scheduler and Task Scheduler

When an action triggers job execution, the driver's scheduler pipeline proceeds in two phases.

The *DAG Scheduler* receives the RDD lineage graph, identifies the *stage boundaries* (discussed in Section 6.4), and decomposes the computation into a DAG of stages. Each stage is a maximal set of consecutive narrow transformations that can be pipelined. Stage boundaries are placed wherever a wide transformation (shuffle) appears. The DAG Scheduler also applies *partition skipping*: any stage whose output partitions are already cached in executor storage memory is skipped entirely, and the cached data is used directly as input to the next stage.

The *Task Scheduler* receives the physical task list for each ready stage and submits tasks to executors via the cluster manager. The Task Scheduler applies *delay scheduling*: if the data-local executor is briefly unavailable, the Task Scheduler waits a configurable period (default 3 seconds) before falling back to a rack-local or any-node assignment, maximising data locality.

6.4 Transformations and Actions

Every RDD operation in Spark is classified as either a *transformation* or an *action*. Understanding this distinction, and the further sub-classification of

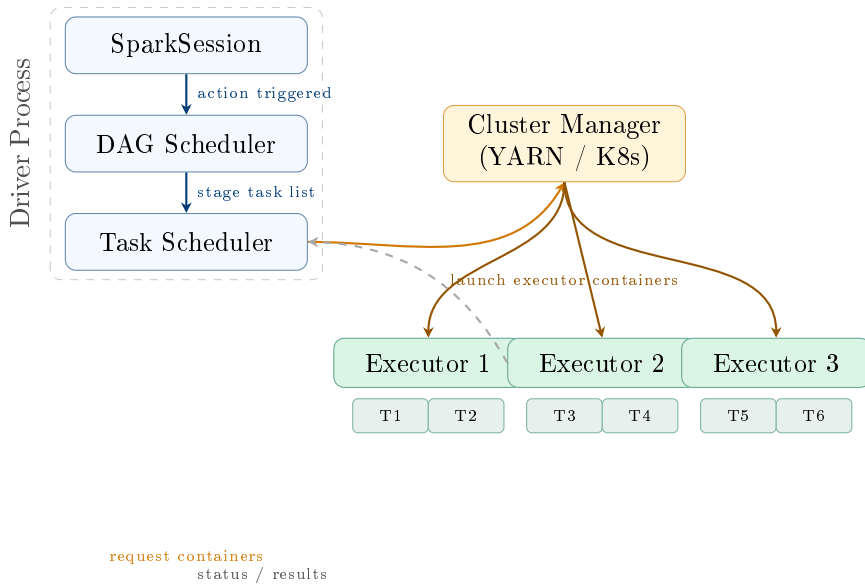


Figure 6.2: Spark architecture. The driver process contains the DAG Scheduler and Task Scheduler. The cluster manager (YARN, Kubernetes, or Standalone) allocates executor containers on worker nodes. Executors run tasks assigned by the driver and report results via heartbeat. Executors remain alive for the full application lifetime, enabling in-memory data sharing across jobs.

transformations into narrow and wide, is essential for writing correct and performant Spark applications.

6.4.1 Narrow Transformations

A *narrow transformation* is one in which each output partition depends on at most one input partition. No data movement across the network is required: each executor can compute its output partitions independently, working only on the input partitions it already holds locally. Consecutive narrow transformations are *pipelined* by the DAG Scheduler into a single stage: a task processes a record through all pipelined transformations without materialising any intermediate RDD.

The most important narrow transformations are:

- `map(f)`: applies function f to each record, producing one output record per input record.
- `filter(p)`: retains only records satisfying predicate p .

- `flatMap(f)`: applies f to each record and flattens the resulting sequences into one output RDD.
- `mapPartitions(f)`: applies f once per partition rather than once per record, which is more efficient when the function has per-call overhead (e.g., opening a database connection or loading a model).
- `union(other)`: concatenates two RDDs with the same schema, with no data movement (each partition of the result is exactly one partition of an input).
- `sample(fraction)`: samples each partition independently, with no cross-partition communication.

6.4.2 Wide Transformations and the Shuffle

A *wide transformation* is one in which each output partition may depend on multiple input partitions, potentially held on different executors. Producing the output requires *moving data across the network*—a *shuffle*. Every shuffle is a stage boundary: the DAG Scheduler inserts a synchronisation point before the downstream stage begins, ensuring all upstream tasks have completed and written their shuffle output before the downstream tasks begin reading it.

The shuffle is the most expensive operation in Spark. Each shuffle involves:

1. **Map side (shuffle write)**: each upstream task hash-partitions its output records by key and writes each partition's records to a local shuffle file on the executor's disk. This write is unavoidable: the data must survive until all downstream tasks have read it.
2. **Network transfer**: each downstream task reads the shuffle blocks it needs from remote executors via HTTP, crossing the network.
3. **Reduce side (shuffle read)**: each downstream task receives and potentially sorts its input before applying the aggregation or join function.

The most important wide transformations are:

- `groupByKey()`: collects all values for each key into a single iterable in one partition. Requires a full shuffle.
- `reduceByKey(f)`: applies the commutative and associative function f to reduce values for each key. Crucially, `reduceByKey` applies a partial aggre-

gation *within each partition* before the shuffle (analogous to the Combiner in MapReduce), dramatically reducing the volume of data transferred.

- `aggregateByKey(zeroValue)(seqOp, combOp)`: a generalisation of `reduceByKey` that supports different input and output types.
- `sortByKey(ascending)`: globally sorts the RDD by key, requiring a range-partitioned shuffle.
- `join(other)`: joins two RDDs on their keys, requiring both to be hash-shuffled by key to co-locate matching records.
- `distinct()`: removes duplicates, requiring a shuffle to co-locate all copies of each record.
- `repartition(n)`: redistributes the RDD into exactly n partitions, performing a full shuffle.

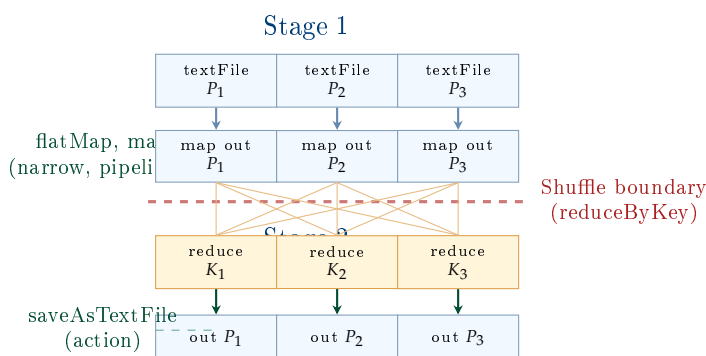


Figure 6.3: Spark stage and shuffle boundary for a word-count job. Stage 1 pipelines `textFile`, `flatMap`, and `map` (all narrow) into a single pass. The `reduceByKey` is a wide transformation, creating a shuffle boundary. Stage 2 performs the shuffle read and final aggregation. The all-to-all arrows represent network transfer of shuffle blocks.

6.4.3 `groupByKey` vs. `reduceByKey`: A Critical Distinction

Common Misconception | **`groupByKey` Transfers All Data; `reduceByKey` Does Not**

The most common performance anti-pattern in Spark is using `groupByKey()` when `reduceByKey()` is applicable. Consider computing the sum of values per key. `groupByKey()` sends *all* raw values for each key across the network to one executor, where the sum is com-

puted. If there are 10^8 records with key “DE” each holding a 16-byte value, that is 1.6 GB of network traffic *just for that one key*.

`reduceByKey(lambda a, b: a + b)` first sums values within each source partition independently (producing one sum per partition per key), then shuffles only these partial sums. If there are 200 partitions, the shuffle transfers 200 numbers for key “DE”—16-byte numbers—instead of 1.6 GB. The network amplification factor is reduced by the partition count ($200\times$ in this example).

The general rule: use `reduceByKey`, `aggregateByKey`, or `combineByKey` whenever the aggregation function is associative and commutative. Use `groupByKey` only when all raw values for a key must be available simultaneously (e.g., computing a per-key median).

6.4.4 Actions

Actions are the operations that trigger the execution of the accumulated RDD lineage DAG and return a concrete result. The most important actions are:

- `count()`: returns the number of records in the RDD as a Long.
- `collect()`: transfers all partitions to the driver and returns a local array (use only for small results).
- `take(n)`: returns the first n records from the first partition(s), with less data movement than `collect()`.
- `reduce(f)`: applies the associative and commutative function f to all records, returning a single value.
- `foreach(f)`: applies function f to each record on the executor side (e.g., to write records to an external system) without returning data to the driver.
- `saveAsTextFile(path)`: writes the RDD as plain text files to an HDFS path (one file per partition).
- `saveAsSequenceFile(path)`: writes as Hadoop SequenceFile format (key-value pairs).

6.5 Spark SQL: DataFrames, Datasets, and the Catalyst Optimizer

The RDD API provides low-level control over distributed computation but requires the programmer to reason explicitly about types, partitions, and data layouts. For SQL-style analytical workloads, this level of control is unnecessarily cumbersome. Spark SQL, introduced in Spark 1.0 and substantially redesigned through Spark 2.0 and 3.0, provides a structured, schema-aware API that allows the Spark engine to apply far more aggressive automatic optimisations.

6.5.1 DataFrames and Datasets

A *DataFrame* is an RDD of Row objects with a named, typed schema—conceptually a distributed relational table. The schema is declared explicitly or inferred from structured data sources (Parquet, ORC, JSON, Hive tables, JDBC). Unlike a raw RDD, a DataFrame carries column names and data types, enabling the query engine to reason about the computation without executing it.

A *Dataset* is a strongly typed, JVM-object-centric version of the DataFrame, available in Scala and Java. A `Dataset[Person]` stores JVM Person objects rather than generic Row objects, providing compile-time type safety at the cost of some JVM serialisation overhead. In Python and R, DataFrames are used exclusively (there is no separate Dataset API because these languages are dynamically typed).

The *SparkSession* is the unified entry point for all Spark SQL operations:

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, avg, count
3
4 spark = SparkSession.builder \
5     .appName("SalesAnalysis") \
6     .config("spark.sql.shuffle.partitions", "200") \
7     .getOrCreate()
8
9 # Load a Parquet table from HDFS
10 sales = spark.read.parquet("hdfs:///data/sales/dt=2024-*")
```

```

11
12 # DataFrame operations: filter, groupBy, aggregate
13 result = (sales
14     .filter(col("amount") > 100)
15     .groupBy("region", "product_id")
16     .agg(
17         count("*").alias("num_sales"),
18         avg("amount").alias("avg_amount")
19     )
20     .orderBy(col("avg_amount").desc())
21 )
22
23 result.write.mode("overwrite").parquet("hdfs:///output/
    sales_summary")

```

Listing 6.1: SparkSession creation and DataFrame operations

6.5.2 The Catalyst Optimizer

The key advantage of the DataFrame/SQL API over the raw RDD API is that every DataFrame operation passes through the *Catalyst optimizer* before execution. Catalyst is a tree-based query optimizer written in Scala that applies a cascade of rule-based and cost-based transformations to reduce query execution cost. It operates in four phases.

Phase 1 — Analysis. Catalyst resolves attribute names and data types by consulting the *Spark Catalog*—the metadata registry for tables, views, functions, and databases (backed by the Hive Metastore in production). Unresolved logical plans (trees with unresolved attribute references) become resolved logical plans (trees where every attribute has a confirmed name and data type).

Phase 2 — Logical Optimisation. Catalyst applies rule-based algebraic transformations to the resolved logical plan:

- **Predicate pushdown:** move WHERE filter conditions as close to the data source as possible, so that Parquet row groups can be skipped before data is read into memory.
- **Column pruning:** remove columns not referenced by the query from the scan; only referenced column chunks are read from Parquet files.
- **Constant folding:** evaluate expressions that involve only constants at

compile time (e.g., `WHERE year = 2020 + 4` becomes `WHERE year = 2024`).

- **Join reordering:** place smaller tables on the build side of hash joins (or as the broadcast side of broadcast hash joins).

Phase 3 — Physical Planning. Catalyst converts the optimised logical plan into one or more candidate physical plans—concrete execution strategies involving Spark operators. For a join, physical plans may include sort-merge join (for large tables), broadcast hash join (when one table fits in executor memory, default threshold 10 MB), or hash join. Catalyst applies a simple cost model (based on estimated row counts from Parquet file statistics) to select the lowest-cost plan.

Phase 4 — Code Generation (Tungsten). The selected physical plan is compiled to JVM bytecode by the *Tungsten* execution engine. Tungsten uses *whole-stage code generation*: rather than interpreting a query plan row-by-row through a generic operator tree, Tungsten generates a single JVM method that fuses all operators in a pipeline into a tight loop. This eliminates virtual function call overhead and enables the JIT compiler to produce CPU-efficient native code. Tungsten also uses off-heap *unsafe* memory for sort and aggregate buffers, bypassing JVM garbage collection overhead.

System Design Note | Catalyst and Parquet: An End-to-End Example

A query `SELECT product_id, SUM(amount) FROM sales WHERE region = 'EU' AND dt BETWEEN '2024-01-01' AND '2024-03-31' GROUP BY product_id` on a Parquet table partitioned by `dt` passes through Catalyst as follows:

1. **Partition pruning:** only the subdirectories `/sales/dt=2024-01-01/` through `/sales/dt=2024-03-31/` are scanned. All other partitions are eliminated before any data is read.
2. **Predicate pushdown:** the `region = 'EU'` filter is pushed into the Parquet reader. Row groups where `max(region) < 'EU'` or `min(region) > 'EU'` (lexicographically) are skipped via footer statistics.
3. **Column pruning:** only the `region`, `product_id`, and `amount` column chunks are read; all other column chunks are not opened.
4. **Aggregation:** Catalyst generates a two-phase aggregate plan: a partial sum within each partition (no shuffle), then a shuffle on `product_id`, then a final sum per partition in the reduce stage.

The combined effect of partition pruning, predicate pushdown, and column pruning can reduce the physical I/O from terabytes to gigabytes on a selective query—without any manual hint or configuration beyond the initial table partitioning.

6.5.3 SparkSQL and Hive Integration

Spark SQL integrates with the Hive Metastore through the Hive Catalog interface, allowing SparkSQL queries to operate on tables defined by Hive DDL statements, including partitioned Parquet and ORC tables (introduced in Chapter 5). This integration allows organisations to migrate from Hive-on-MapReduce to Spark SQL with no changes to table definitions, partition schemes, or ETL job logic—only the execution engine changes.

6.6 MLlib: Distributed Machine Learning

Apache Spark MLlib is Spark’s built-in machine learning library. Its central design goal is to express machine learning pipelines—sequences of data preprocessing, feature engineering, and model training steps—as *Pipeline* objects that can be fit to training data, saved to disk, and applied to new data with a single API call.

6.6.1 Core Abstractions: Transformer, Estimator, and Pipeline

The MLlib Pipeline API is built from three abstractions.

A *Transformer* is an object that implements a `transform(df)` method: it takes a `DataFrame` as input and produces a new `DataFrame` with one or more new columns added. Transformers are *stateless*—they do not learn parameters from data. Examples include `Tokenizer` (splits text into words), `HashingTF` (converts token lists to TF-IDF feature vectors), `StandardScaler` after fitting (normalises numerical features using pre-computed mean and standard deviation), and any trained model (which transforms a feature `DataFrame` into a predictions `DataFrame`).

An *Estimator* is an object that implements a `fit(df)` method: it trains a model on data and produces a *Transformer* (the trained model). Examples include `LogisticRegression`, `RandomForestClassifier`, `KMeans`, and `StandardScaler` (before fitting—the fitted version is a *Transformer*).

A *Pipeline* is an ordered sequence of *Transformers* and *Estimators*. `pipeline.fit(df)` calls `fit()` on each *Estimator* in sequence, using the training `DataFrame` and passing the output to the next stage. The result is a `PipelineModel`—a sequence of *Transformers*—which can be applied to new data via `model.transform(test_df)`.

```

1  from pyspark.ml import Pipeline
2  from pyspark.ml.feature import (Tokenizer, HashingTF, IDF,
3                                  StringIndexer, VectorAssembler
4                                  )
5  from pyspark.ml.classification import LogisticRegression
6  from pyspark.ml.evaluation import
7      MulticlassClassificationEvaluator
8
9  # Load labelled text data
10 df = spark.read.parquet("hdfs:///data/news_articles")
11
12 # Build pipeline stages
13 tokenizer = Tokenizer(inputCol="text", outputCol="tokens")
14 hashing_tf = HashingTF(inputCol="tokens", outputCol="
15     raw_features",
16                        numFeatures=20_000)
17 idf = IDF(inputCol="raw_features", outputCol="features")
18 indexer = StringIndexer(inputCol="label", outputCol="
19     label_idx")
20 lr = LogisticRegression(featuresCol="features",
21                        labelCol="label_idx",
22                        maxIter=100)
23
24 pipeline = Pipeline(stages=[tokenizer, hashing_tf, idf, indexer
25                             , lr])
26
27 # Train / test split
28 train, test = df.randomSplit([0.8, 0.2], seed=42)
29
30 # Fit: trains IDF, StringIndexer, LogisticRegression on train
31 model = pipeline.fit(train)

```

```

28 # Transform: applies all transformations to test
29 predictions = model.transform(test)
30
31 evaluator = MulticlassClassificationEvaluator(
32     labelCol="label_idx", predictionCol="prediction",
33     metricName="accuracy")
34 print(f"Test accuracy: {evaluator.evaluate(predictions):.4f}")
35
36 # Persist the trained model to HDFS for serving
37 model.save("hdfs:///models/news_classifier_v1")

```

Listing 6.2: MLlib Pipeline for text classification

6.6.2 Key MLlib Algorithms

Table 6.2: Representative MLlib algorithms by category

Category	Algorithm	Typical use case
Classification	LogisticRegression, RandomForestClassifier, GBTClassifier, LinearSVC	Spam detection, churn prediction, document classification
Regression	LinearRegression, RandomForestRegressor, GBTRRegressor	Price forecasting, demand estimation
Clustering	KMeans, GaussianMixture, BisectingKMeans	Customer segmentation, anomaly detection
Recommendation	ALS (Alternating Least Squares)	Collaborative filtering (Netflix, Spotify)
Feature Eng.	PCA, Word2Vec, MinHashLSH, QuantileDiscretizer	Dimensionality reduction, similarity search

Alternating Least Squares (ALS) deserves particular mention as it illustrates how Spark’s in-memory model enables iterative ML algorithms.

ALS is a matrix factorisation algorithm for collaborative filtering: given a user-item rating matrix with many missing entries, it alternately holds user factors fixed and solves for item factors (an embarrassingly parallel linear algebra problem), then holds item factors fixed and solves for user factors. Each alternation reads the full ratings matrix. On a dataset of 10^9 ratings, this read is performed 10–20 times per training run; with the ratings cached in Spark’s distributed memory, all subsequent reads are served from RAM at disk-free speed.

6.7 Memory Management in Spark

Spark’s performance advantage over MapReduce rests on its ability to use the executor’s JVM heap for both active computation and RDD caching. Understanding how Spark divides this memory between competing uses is essential for diagnosing and resolving the two most common production failure modes: spill-to-disk (which degrades performance) and OOM (OutOfMemory) errors (which crash executors).

6.7.1 The Unified Memory Model

Spark 1.6 introduced the *unified memory model*, which replaced the earlier static partitioning of execution and storage memory with a dynamic boundary. The executor JVM heap is divided into three regions:

1. **Reserved Memory** (300 MB, fixed): reserved for Spark’s internal objects. Not configurable.
2. **Spark Memory** (`spark.memory.fraction × (heap – 300 MB)`, default 0.6): the pool shared between execution and storage.
3. **User Memory** (remaining $1 - 0.6 = 0.4$ fraction): available for user-defined data structures, UDF objects, and the Java/Scala collections used in application code.

Within the Spark Memory pool, execution memory and storage memory share the space dynamically:

- **Storage Memory** (initially `spark.memory.storageFraction × Spark Memory`, default 0.5) caches persisted RDD partitions. Storage memory can

expand into unused execution memory, but cannot forcibly evict active execution memory.

- **Execution Memory** (the remainder) holds sort buffers, hash tables for aggregations, and shuffle read/write buffers. Execution memory can borrow from storage memory, and if it does, Spark evicts cached RDD partitions (via LRU) to make room.

Definition | Unified Memory Allocation

For an executor with H bytes of JVM heap:

$$M_{\text{spark}} = \text{fraction} \times (H - 300 \text{ MB}) \quad (6.2)$$

$$M_{\text{storage}} \approx \text{storageFraction} \times M_{\text{spark}} \quad (6.3)$$

$$M_{\text{exec}} = M_{\text{spark}} - M_{\text{storage}} \quad (\text{initial}) \quad (6.4)$$

For $H = 16 \text{ GB}$, $\text{fraction} = 0.6$, $\text{storageFraction} = 0.5$:

$$M_{\text{spark}} = 0.6 \times (16 \text{ GB} - 0.3 \text{ GB}) \approx 9.4 \text{ GB}$$

$$M_{\text{storage}} \approx 0.5 \times 9.4 \text{ GB} = 4.7 \text{ GB}$$

$$M_{\text{exec}} \approx 4.7 \text{ GB} \quad (\text{can grow up to } 9.4 \text{ GB})$$

6.7.2 Serialisation and Kryo

When RDD partitions are persisted with `MEMORY_ONLY_SER` or spill to disk, Spark must serialise Java/Scala objects into a byte stream. By default, Spark uses Java's native serialisation, which is flexible (handles arbitrary Java objects) but slow and space-inefficient.

Kryo serialisation (`spark.serializer = org.apache.spark.serializer.KryoSerializer`) is a third-party library that serialises registered classes 2–10× faster and produces 3–5× smaller byte representations than Java serialisation. For RDDs of simple, known types (such as `(String, Int)` pairs or custom case classes), Kryo is almost always the correct choice.

```
1 from pyspark import SparkConf, SparkContext
2
3 conf = SparkConf() \
4     .setAppName("KryoDemo") \
```

```

5      .set("spark.serializer",
6          "org.apache.spark.serializer.KryoSerializer") \
7      .set("spark.kryo.registrationRequired", "false") \
8      .set("spark.kryoserializer.buffer.max", "256m")
9
10     sc = SparkContext(conf=conf)

```

Listing 6.3: Enabling Kryo serialization and registering classes

6.7.3 Data Skew and Salting

Data skew occurs when the keys of a wide transformation are distributed unevenly: a small number of keys contain a very large fraction of the records. In a `reduceByKey`, all records for the same key must be sent to the same partition. If 40% of the dataset has the key “US”, one task will process 40% of the data while the other 199 tasks process the remaining 60% divided evenly. The entire stage cannot complete until the skewed task finishes. The stage latency is dominated by the slowest task, making the overall job wall-clock time proportional to the largest single-key volume.

The standard remedy is **salting**: appending a random integer suffix (the “salt”) to skewed keys to distribute their records across multiple partitions, then performing a two-phase aggregation.

```

1  import random
2
3  SALT_FACTOR = 50      # number of salt buckets for skewed keys
4  SKEWED_KEYS = {"US", "CN", "DE"} # keys known to be skewed
5
6  def add_salt(record):
7      key, value = record
8      if key in SKEWED_KEYS:
9          salted_key = (key, random.randint(0, SALT_FACTOR - 1))
10     else:
11         salted_key = (key, 0)
12     return (salted_key, value)
13
14  def remove_salt(record):
15      (key, _salt), value = record
16      return (key, value)
17
18  # Phase 1: shuffle by (key, salt) -> 50x more reducers for
    skewed keys

```

```

19 partial = (rdd
20     .map(add_salt)
21     .reduceByKey(lambda a, b: a + b))    # partial aggregation
22
23 # Phase 2: strip salt, re-aggregate to get final per-key totals
24 final = (partial
25     .map(remove_salt)
26     .reduceByKey(lambda a, b: a + b))    # final aggregation

```

Listing 6.4: Salting to resolve data skew in a PySpark RDD**Production Lesson | Skew is Invisible to the Optimizer**

Data skew is one of the most common causes of Spark job failure in production, and it is almost entirely invisible to the Catalyst optimizer. Catalyst can pushdown predicates, prune columns, and choose between sort-merge and broadcast joins—but it cannot detect that 40% of the data has the same key unless the user provides statistics or explicit hints. The symptoms are: a stage where 199 out of 200 tasks complete quickly but one task runs for hours, eventually causing the executor to OOM or timeout.

Diagnostic steps: examine the Spark UI “Stages” tab and look for tasks with anomalously large “Input Size” or “Shuffle Read Size.” Use `df.groupBy("key").count().orderBy(F.desc("count")).show(20)` to identify skewed keys before designing the aggregation.

6.8 Research Spotlights

Research Spotlight | Zaharia et al. (2010) — Spark: Cluster Computing with Working Sets

Citation: Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., and Stoica, I. (2010). Spark: Cluster Computing with Working Sets. *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 2010)*. (Zaharia, Chowdhury, Franklin, et al. [2010](#))

Key insight: The paper’s thesis is that MapReduce is fundamentally unsuited for two important and growing workload classes: *iterative algorithms* (machine learning, graph processing) and *interactive data analysis*

(exploratory SQL on cached datasets). Both require reading the same data multiple times, and MapReduce's forced disk materialisation between stages makes this prohibitively expensive. The solution is to treat distributed RAM as a first-class abstraction that persists across job boundaries.

Technical contributions:

- Introduced the concept of a *working set*—the subset of data that an application accesses repeatedly—and argued that a cluster computing framework should allow the working set to be stored in distributed memory across the cluster.
- Demonstrated that a logistic regression training job ran 10× faster on Spark than on Hadoop MapReduce by caching the training set in memory after the first iteration.
- Showed that interactive queries over a 39 GB Wikipedia dataset ran in 0.5–1 second on cached data vs. 20–60 seconds on Hadoop, enabling genuine interactive exploration at scale.

Impact today: The HotCloud 2010 paper introduced Spark to the research community and has been cited over 5,000 times. Within three years of this publication, Spark had been adopted by hundreds of organisations and had become the fastest-growing Apache project in history. The working-set insight—that most production ML and analytics workloads repeatedly access the same data—is now the primary justification for in-memory computing frameworks across the industry.

Research Spotlight | Zaharia et al. (2012) — Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing

Citation: Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., and Stoica, I. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2012)*, pp. 15–28. **Best Paper Award.** (Zaharia, Chowdhury, Das, et al. [2012](#))

Key insight: The central contribution is the formalization of the RDD abstraction and its lineage-based fault tolerance model. The paper

argues that existing distributed shared memory systems fail for large-scale data processing because they require fine-grained writes to be tracked for replication—an overhead that is prohibitive at petabyte scale. RDDs restrict mutations to coarse-grained transformations (each transformation applies the same function to all records in a partition), which makes it possible to describe any lost partition’s provenance with a short lineage graph rather than a full data log.

Technical contributions:

- Formally defined the RDD as a five-tuple (partitions, dependencies, compute function, partitioner, preferred locations) and proved that lineage-based fault tolerance is sufficient for the class of coarse-grained transformations that covers all common data analytics operations.
- Distinguished narrow from wide dependencies and showed that narrow-dependency lineage recomputation is always partition-local (no cross-executor coordination needed), while wide-dependency recomputation may require reading multiple parent partitions.
- Benchmarked Spark RDDs against Hadoop MapReduce on four workloads: logistic regression (25× speedup on second iteration), PageRank (7× speedup), *k*-means clustering (40× speedup), and interactive SQL queries (sub-second vs. tens of seconds).

Impact today: The NSDI 2012 paper is the most-cited Spark paper, with over 10,000 citations, and is considered one of the foundational papers of the systems era of big data processing. The RDD abstraction directly influenced the design of DataFrame/Dataset APIs, Apache Flink’s DataSet/DataStream APIs, TensorFlow’s distributed variable model, and Ray’s distributed object store. The narrow/wide dependency distinction is now standard vocabulary in distributed systems courses worldwide.

6.9 Case Study: Alibaba’s Spark Deployment for Singles’ Day (11.11)

Case Study | Alibaba — Apache Spark at 10,000 Nodes for the World's Largest Shopping

Background. Alibaba's Singles' Day (11.11) shopping festival—held annually on 11 November—is the world's largest single-day commerce event. In 2019, the event generated \$38.4 billion USD in gross merchandise value within 24 hours, with a peak transaction rate of 544,000 orders per second. Behind this scale lay one of the largest production deployments of Apache Spark in existence, integrated into Alibaba's MaxCompute (formerly ODPS) and E-MapReduce platforms.

Scale. Alibaba's internal Spark deployment for 11.11 2019 involved:

- Over **10,000 executor nodes**, each with 96 GB of RAM and 24 vCPUs—approximately 960 TB of aggregate RAM and 240,000 vCPUs available to Spark applications.
- Processing over **970 PB** of data in the 24-hour event window, primarily through Spark SQL queries over Parquet tables partitioned by buyer_id, seller_id, and event timestamp.
- Over **1 trillion individual user behaviour events** (page views, searches, cart additions, purchases, refunds) accumulated, processed, and fed into real-time recommendation engines during the event.

The recommendation challenge. The highest-impact use of Spark on 11.11 was powering Taobao's product recommendation system, which generates personalised product lists for 800 million active users. Recommendation model updates during the event must incorporate behaviour signals from the event itself: a user who viewed three laptops in the first hour of the event should be shown accessories in the second hour, not laptops she has already browsed.

This requirement—updating models on streaming data *during* the event, not just in nightly batch jobs—drove Alibaba's engineers to build a hybrid streaming-batch architecture:

1. **Behaviour ingestion** (Alibaba's Blink / Apache Flink): raw user events arrive from Taobao at approximately 11 million events per

second at peak. Flink processes them in real time, computing streaming feature updates (user-product affinity scores, click-through rates, conversion funnels) with a latency of under 500 milliseconds.

2. **Model refresh** (Spark MLlib): every 10 minutes during the 24-hour event, a Spark MLlib ALS collaborative filtering job is triggered, consuming the latest streaming feature snapshots from HDFS (written by Flink) as training data. The 10-minute ALS job runs on 2,000 executor nodes with the rating matrix (approximately 800 million users $\times$ 2 billion items, with 40 billion observed entries) cached in distributed memory across the executors. Retraining from scratch on this matrix would require 6–8 hours with disk-based MapReduce; with Spark and in-memory caching, the job completes in 8–12 minutes.
3. **Recommendation serving**: the refreshed model parameters (user and item latent factors, each a 128-dimensional vector) are pushed to Alibaba’s distributed key-value store and served to Taobao’s API layer within 2 minutes of the ALS job completing. Users browsing during the refresh window see recommendations based on model parameters that incorporate signals from as recently as 12 minutes ago.

Spark SQL for campaign analytics. Beyond recommendation, Alibaba’s marketing analytics team ran Spark SQL queries continuously during 11.11 to power the event’s live broadcast analytics dashboard—visible to 800 million TV viewers as a real-time counter of GMV, transaction volume, and category breakdowns. These queries joined three Parquet tables:

Table	Size	Update frequency
transactions	~200 TB at event end	Appended per minute
products	~3 GB (broadcast)	Static
sellers	~5 GB (broadcast)	Static

Catalyst automatically applied *broadcast hash join* for the small dimension tables (products, sellers), avoiding shuffles entirely for those joins.

The transactions table, partitioned by event_minute, was scanned with partition pruning: each dashboard query only read the most recent 5-minute partition (~600 MB) rather than the full 200 TB of accumulated transactions.

Skew management. A critical engineering challenge was the Taobao flagship stores: a single store operated by a major brand might account for hundreds of millions of transaction records in 24 hours. Grouping by seller_id without salting would have created severe skew, with one task processing >20% of the entire dataset. Alibaba's engineering team applied salting for known high-volume sellers, reducing the maximum single-task volume to approximately 2% of the dataset.

Outcomes and lessons. Alibaba's 11.11 2019 Spark deployment demonstrated three principles that have since become industry standards for large-scale event analytics:

- **In-memory iterative retraining is viable at billion-user scale.** The ALS rating matrix caching strategy reduced model refresh time by 40× compared to disk-based alternatives, enabling 144 model refreshes during the 24-hour event.
- **Broadcast joins for dimension tables eliminate shuffle.** Alibaba measured a 15× latency reduction for multi-table queries by broadcasting the 3 GB products table to all 10,000 executor nodes, compared to sort-merge joining it through a shuffle.
- **Partition pruning must be designed into the schema.** Partitioning the transactions table by event_minute rather than dt (which is standard for daily analytics) allowed dashboard queries to scan only the most recent partition. This was a deliberate schema design decision made 6 weeks before the event; retroactively repartitioning 200 TB during the event would have been impossible.

- MapReduce's disk I/O bottleneck for iterative algorithms is quantified by the formula: $\text{total I/O} = I \times D \times r$, where I is the number of iterations, D is the dataset size, and r is the HDFS replication factor. For 100 iterations on 100 GB with $r = 3$, this produces 30 TB of disk I/O.
- An **RDD** is an immutable, partitioned, lazily computed, fault-tolerant collection of records. Its five defining properties are: partitions, dependencies (lineage), compute function, partitioning metadata, and preferred locations.
- **Lineage-based fault tolerance** recomputes lost partitions from their lineage graph rather than maintaining replicas. This costs nothing in memory but may require recomputation after long lineage chains; **checkpointing** truncates the lineage to bound recomputation cost.
- **Narrow transformations** (map, filter, flatMap) have each output partition depending on at most one input partition and require no shuffle. **Wide transformations** (reduceByKey, groupByKey, join) require all-to-all data exchange (shuffle) and create stage boundaries.
- **reduceByKey** applies partial aggregation within each partition before the shuffle, reducing network traffic by approximately the partition count. **groupByKey** transfers all raw values across the network and should be avoided whenever an aggregation function is associative and commutative.
- The **Catalyst optimizer** processes DataFrame/SQL plans in four phases: analysis (name resolution), logical optimisation (predicate pushdown, column pruning, constant folding), physical planning (join strategy selection), and code generation (Tungsten whole-stage code generation).
- **RDD persistence levels** range from MEMORY_ONLY (fastest, no fault tolerance for evicted partitions) to DISK_ONLY (slowest, full fault tolerance). Kryo serialisation reduces the memory footprint and serialisation time of persisted RDDs by 3–10× over Java serialisation.
- The **unified memory model** divides executor heap into Reserved (fixed 300 MB), Spark (0.6 fraction), and User (0.4 fraction) regions. Within the Spark region, execution and storage memory share space dynamically, with execution memory able to evict cached RDD partitions.

- **Data skew** occurs when a small number of keys dominate the dataset. The **salting** remedy appends a random integer to skewed keys, distributing their records across multiple partitions, then removes the salt in a second aggregation pass.
- The **MLlib Pipeline API** expresses preprocessing and training as a sequence of Transformers and Estimators. A `Pipeline.fit()` call trains all Estimators and produces a `PipelineModel` that can be saved to HDFS and applied to new data.

Exercises

Short Questions

SQ6.1. Explain the MapReduce iteration I/O problem using the formula $\text{Total I/O} = I \times D \times r$. For a k -means clustering job that runs 50 iterations on a 200 GB dataset with $r = 3$, calculate the total disk I/O. How does Spark reduce this?

SQ6.2. Define a Resilient Distributed Dataset. List its five defining properties and explain the purpose of each.

SQ6.3. What is lineage-based fault tolerance? Compare it with replication-based fault tolerance on two dimensions: memory overhead and recovery latency.

SQ6.4. Explain the concept of lazy evaluation in Spark. What is the performance benefit of deferring computation until an action is called?

SQ6.5. Distinguish narrow transformations from wide transformations. Give two examples of each, and explain why wide transformations create stage boundaries in the DAG.

SQ6.6. Why is `reduceByKey` almost always preferable to `groupByKey` for aggregation? Give a concrete numerical example showing the difference in network traffic for a dataset with 1 billion records and 200 partitions, where 30% of records share the same key.

SQ6.7. What is a YARN ApplicationMaster for a Spark job? Which component of Spark architecture runs inside the AM container, and what does it do?

SQ6.8. Explain the four phases of the Catalyst optimizer. Which phase is responsible for choosing between a sort-merge join and a broadcast hash join, and what criterion determines the choice?

SQ6.9. What is the unified memory model in Spark? For an executor with 32 GB JVM heap, `spark.memory.fraction = 0.6`, and `spark.memory.storageFraction = 0.5`, calculate the available storage memory, execution memory, and user memory.

SQ6.10. What is Tungsten whole-stage code generation? How does it differ from the interpreted, row-at-a-time Volcano model used by traditional query engines?

SQ6.11. Explain the Spark MLlib abstraction of Transformer vs. Estimator. Give one concrete example of each and explain what `fit(df)` produces for an Estimator.

SQ6.12. What is data skew in a Spark wide transformation? Describe the observable symptom in the Spark UI that indicates a skew problem.

SQ6.13. What does `MEMORY_ONLY_SER` mean as an RDD storage level? Under what circumstances should you choose it over `MEMORY_ONLY`?

SQ6.14. Explain what a checkpoint does in Spark. When is checkpointing necessary, and what are its costs?

SQ6.15. A Spark job with 8 stages takes 4 hours. The Spark UI shows that Stage 3 ran for 3.5 hours, with 199 tasks completing in 2 minutes and one task running for 3.5 hours. What is the most likely cause, and how would you diagnose it?

Analytical Problems

AP6.1. Spark vs. MapReduce: Iterative Algorithm Cost Analysis. A genomics research lab runs a principal component analysis (PCA) on a SNP (single nucleotide polymorphism) dataset. The dataset is 500 GB, stored on HDFS with $r = 3$. PCA requires 80 power-iteration steps.

- (a) Calculate the total disk I/O for a MapReduce implementation (one job per iteration, each job reads from and writes to HDFS).
- (b) The lab's HDFS cluster sustains 2 GB/s aggregate disk read bandwidth. Estimate the wall-clock time for the MapReduce implementation.
- (c) With Spark and MEMORY_ONLY persistence, only the first iteration reads from HDFS. Subsequent iterations read from memory at 50 GB/s aggregate throughput. Estimate the total wall-clock time for the Spark implementation.
- (d) Calculate the speedup ratio T_{MR}/T_{Spark} .
- (e) The dataset grows to 2 TB. The cluster has 1 TB of aggregate executor RAM. Explain what happens to the Spark job when the RDD no longer fits in memory under MEMORY_ONLY and under MEMORY_AND_DISK.

AP6.2. Stage and Shuffle Analysis. A Spark job reads a log file from HDFS, parses each line into a (session_id, page_url, duration_seconds) tuple, filters records with duration > 5 seconds, computes the total duration per session, keeps sessions with total duration > 60 seconds, joins with a 200 MB session-metadata table (session_id, user_id, country), groups by country, and writes to HDFS.

- (a) Draw the RDD lineage DAG for this job. Label each RDD with the operation that produced it.
- (b) Identify all stage boundaries and explain why each boundary is placed where it is.
- (c) Identify any narrow transformations that can be pipelined into a single task pass, and explain the I/O savings this produces.
- (d) The session-metadata table is 200 MB. Explain how the Catalyst optimizer would handle the join with this table and why this avoids a shuffle.
- (e) The log file has 20 billion records and `spark.sql.shuffle.partitions = 200`. The join produces 150 billion output records. Explain whether 200 shuffle partitions is appropriate and how you would determine the correct value.

AP6.3. Unified Memory Sizing. An e-commerce company runs a Spark job on a YARN cluster. Each executor is allocated 24 GB JVM heap and 6

vCPUs. The job caches a 180 GB RDD (serialised with Kryo) that must be accessed by 20 iterative training passes. The remaining 30 GB of the dataset does not need caching.

- (a) With `spark.memory.fraction = 0.6` and `spark.memory.storageFraction = 0.5`, calculate the storage memory and execution memory per executor.
- (b) If the executor count is 40, calculate the total aggregate storage memory across the cluster. Does the 180 GB RDD fit?
- (c) If Kryo serialisation achieves a $3\times$ compression ratio over the raw RDD size, recalculate whether the RDD fits in aggregate storage memory.
- (d) If the RDD still does not fit, describe two configuration changes (with specific parameter names and values) that could increase the available storage memory.
- (e) Explain the performance consequence if the RDD does not fit in `MEMORY_ONLY` storage but the job uses `MEMORY_AND_DISK` instead.

AP6.4. Data Skew Diagnosis and Salting. A retail company runs a Spark job that joins a 5 TB transactions table (partitioned by `store_id`) with a 2 TB products table. The join key is `product_id`. Analysis of the data reveals:

<code>product_id</code>	Transaction share
IPHONE-15-PRO	28.3%
SAMSUNG-S24	11.7%
AIRPODS-PRO	6.2%
All other 4.2M products	53.8%

- (a) With 200 shuffle partitions, estimate the number of records assigned to the task handling IPHONE-15-PRO vs. the average task (the total dataset has 50 billion records).
- (b) The average task runs in 4 minutes. Estimate the stage latency with and without skew mitigation.
- (c) Design a salting strategy for the top-3 skewed keys with a salt factor of 30. Show the key before and after salting for a sample record.
- (d) Explain the two-phase aggregation required after salting and why it is necessary.

- (e) An alternative to salting for join skew is *skew join hint* (SKEW JOIN in Spark SQL). Explain conceptually how the optimizer would handle a skew hint differently from a standard sort-merge join.

Design Problems

DP6.1. Spark Architecture for a Real-Time Fraud Detection Platform. A global payment network processes 30,000 transactions per second. Each transaction has 45 features (amount, merchant category, cardholder history, location, time-since-last-transaction, etc.). A fraud detection model must score each transaction within 200 milliseconds of arrival. The ML team retrains the model weekly on 90 days of historical transactions (≈ 240 TB).

Design the Spark architecture for both the training pipeline and the serving infrastructure. Your design must specify:

- (a) The cluster topology for weekly model retraining: number of executor nodes, memory per executor, and YARN queue configuration. Justify the memory sizing based on the 240 TB dataset and the 10-iteration gradient boosting training algorithm.
- (b) The MLlib Pipeline stages for feature engineering: which Estimators and Transformers are needed, and in what order.
- (c) How the trained model is exported from the Spark cluster to the serving layer, given the 200-millisecond latency constraint (Spark jobs take seconds to initialise; model serving cannot invoke a Spark job per transaction).
- (d) The persistence strategy for the feature engineering pipeline: which intermediate RDDs should be cached, at what storage level, and why.
- (e) How the design changes if the retraining frequency is increased from weekly to hourly, incorporating the most recent 6 hours of transactions into the model on each run.

DP6.2. Spark SQL Migration from Hive-on-MapReduce. A telecommunications company's analytics team runs 400 Hive-on-MapReduce queries per day against a 500 TB ORC warehouse. Average query latency is 45

minutes; the SLA for analyst-facing queries is 15 minutes; and 50 queries (12.5%) exceed 2 hours. The data engineering team proposes migrating to Spark SQL.

Design the migration. Your design must address:

- (a) The Catalyst optimisations (predicate pushdown, column pruning, broadcast joins) that are most likely to reduce latency for queries over an ORC table partitioned by date and region. Estimate the I/O reduction for a query selecting 8 of 120 columns, filtering on a date range covering 5% of the total data, with an additional predicate that eliminates 70% of the remaining row groups.
- (b) The YARN queue configuration changes required to co-run Hive-on-MapReduce (existing nightly ETL) and Spark SQL (new analyst workload) on the same cluster without either workload starving the other.
- (c) A compatibility verification strategy: how to confirm that the Spark SQL query results match the Hive-on-MapReduce results for the 400 queries before switching production traffic.
- (d) The expected performance change for the 50 slow queries (currently >2 hours). Which of the following root causes would be fixed by Catalyst, and which would still require manual query rewriting: (i) full-table scans that could use partition pruning; (ii) GROUP BY on a column with extreme skew; (iii) a three-table join where the smallest table is 400 MB.
- (e) A rollback plan if Spark SQL produces incorrect results for a category of queries (e.g., NULL handling differences from Hive's default behaviour).

Programming Exercises

PE6.1. Word Frequency with RDD API (PySpark). Implement a word frequency counter using the Spark RDD API. The program must: (1) read a text file from HDFS; (2) tokenise each line into words (lowercase, strip punctuation); (3) count occurrences of each word; (4) return the top 20 most frequent words and their counts. Measure the performance difference between using `reduceByKey` and `groupByKey`.

```

1  import re
2  from pyspark.sql import SparkSession
3
4  spark = SparkSession.builder.appName("WordFreq").getOrCreate()
5  sc     = spark.sparkContext
6
7  def tokenize(line):
8      """Lower-case and split on non-alpha characters."""
9      return [w for w in re.split(r"^[a-z]+", line.lower()) if w]
10
11 # Read text from HDFS (replace with actual path)
12 text_rdd = sc.textFile("hdfs:///data/wikipedia_sample.txt")
13
14 # --- Implementation using reduceByKey (correct approach) ---
15 freq_rdd = (text_rdd
16             .flatMap(tokenize)
17             .map(lambda w: (w, 1))
18             .reduceByKey(lambda a, b: a + b)
19             .sortBy(lambda kv: kv[1], ascending=False)
20             )
21
22 top20 = freq_rdd.take(20)
23 print("Top 20 words (reduceByKey):")
24 for word, count in top20:
25     print(f"    {word:<20} {count:>8,}")
26
27 # --- Implementation using groupByKey (anti-pattern for
28 #      comparison) ---
29 # TODO: implement using groupByKey and compare memory/time
30 # groupby_rdd = (text_rdd
31 #               .flatMap(tokenize)
32 #               .groupBy(lambda w: w)
33 #               .mapValues(lambda vals: sum(1 for _ in vals))
34 #               ...
35 #               )
36
37 # Cache and measure second-pass performance
38 freq_rdd.persist()
39 print(f"\nTotal unique words: {freq_rdd.count():,}")

```

Listing 6.5: PE6.1: Word frequency with RDD API

PE6.2. Iterative k -Means with RDD Caching (PySpark). Implement k -

means clustering using the raw RDD API (not MLlib) to observe the effect of caching on iterative performance. The implementation must: (1) cache the input data RDD after the first iteration; (2) measure wall-clock time per iteration; (3) report the speedup of cached vs. uncached iterations.

```

1  import time
2  import random
3  import math
4  from pyspark.sql import SparkSession
5  from pyspark import StorageLevel
6
7  spark = SparkSession.builder.appName("KMeans").getOrCreate()
8  sc    = spark.sparkContext
9
10 def parse_point(line):
11     return [float(x) for x in line.strip().split(",")]
12
13 def distance(p, c):
14     return math.sqrt(sum((a - b)**2 for a, b in zip(p, c)))
15
16 def closest_centroid(point, centroids):
17     dists = [distance(point, c) for c in centroids]
18     return dists.index(min(dists))
19
20 def add_vectors(v1, v2):
21     return [a + b for a, b in zip(v1, v2)]
22
23 def mean_vector(total, count):
24     return [x / count for x in total]
25
26 K          = 10
27 MAX_ITERS  = 20
28 SEED       = 42
29
30 # Load data
31 data = sc.textFile("hdfs:///data/clustering_input.csv").map(
32     parse_point)
33
34 # Cache the input data -- this is the key optimisation
35 data.persist(StorageLevel.MEMORY_ONLY)
36 data.count()    # trigger caching (materialize into memory)
37
38 # Initialise centroids randomly from the first partition
39 centroids = data.takeSample(False, K, seed=SEED)

```

```
39
40 iter_times = []
41 for iteration in range(MAX_ITERS):
42     t_start = time.time()
43
44     # Assign each point to its nearest centroid
45     assignments = data.map(
46         lambda p: (closest_centroid(p, centroids),
47                    (p, 1))
48     )
49
50     # Sum vectors and counts per cluster
51     cluster_stats = (assignments
52                      .reduceByKey(lambda a, b: (add_vectors(a[0], b[0]),
53                                                         a[1] + b[1]))
54     )
55
56     # Compute new centroids
57     new_centroids_list = (cluster_stats
58                          .mapValues(lambda vs: mean_vector(vs[0], vs[1]))
59                          .collect())
60
61     new_centroids = {k: v for k, v in new_centroids_list}
62     centroids = [new_centroids.get(i, centroids[i]) for i in
63                  range(K)]
64
65     elapsed = time.time() - t_start
66     iter_times.append(elapsed)
67     print(f"Iteration {iteration+1:>2}: {elapsed:.2f}s")
68
69 print(f"\nFirst iteration : {iter_times[0]:.2f}s (HDFS read +
70       compute)")
71 print(f"Mean iter 2-20 : {sum(iter_times[1:])/len(iter_times
72       [1:]):.2f}s (cached)")
73 print(f"Speedup from cache: {iter_times[0]/sum(iter_times[1:])*
74       len(iter_times[1:]):.1f}x")
```

Listing 6.6: PE6.2: Iterative k-means with RDD caching

PE6.3. Salting Benchmark for Skew Mitigation (PySpark). Simulate a skewed dataset and measure the performance difference between a naive `reduceByKey` (with skew) and a salted two-phase aggregation. Report per-task statistics to demonstrate that salting eliminates the long tail.

```

1 import random
2 import time
3 from pyspark.sql import SparkSession
4
5 spark = SparkSession.builder.appName("SkewBenchmark") \
6     .config("spark.sql.shuffle.partitions", "50") \
7     .getOrCreate()
8 sc = spark.sparkContext
9
10 TOTAL_RECORDS = 5_000_000
11 SKEW_KEY = "IPHONE-15-PRO"
12 SKEW_FRACTION = 0.35 # 35% of all records
13 SALT_FACTOR = 20 # number of salt buckets
14
15 def generate_record(_):
16     """Generate (product_id, sale_amount) with skew."""
17     if random.random() < SKEW_FRACTION:
18         return (SKEW_KEY, random.randint(500, 1500))
19     products = [f"PROD-{i:06d}" for i in range(1, 1001)]
20     return (random.choice(products), random.randint(10, 500))
21
22 data = sc.parallelize(range(TOTAL_RECORDS), 50).map(
23     generate_record)
24 data.persist()
25 data.count() # materialise
26
27 # --- Approach 1: naive reduceByKey (skewed) ---
28 t0 = time.time()
29 naive_result = data.reduceByKey(lambda a, b: a + b).collect()
30 t_naive = time.time() - t0
31 print(f"Naive reduceByKey: {t_naive:.2f}s")
32
33 # --- Approach 2: salted two-phase aggregation ---
34 SKEWED_KEYS = {SKEW_KEY}
35
36 def salt_key(record):
37     key, val = record
38     if key in SKEWED_KEYS:
39         return ((key, random.randint(0, SALT_FACTOR - 1)), val)
40     return ((key, 0), val)
41
42 def desalt_key(record):
43     (key, _salt), val = record

```

```
43     return (key, val)
44
45 t1 = time.time()
46 salted_result = (data
47     .map(salt_key)
48     .reduceByKey(lambda a, b: a + b)
49     .map(desalt_key)
50     .reduceByKey(lambda a, b: a + b)
51     .collect()
52 )
53 t_saltd = time.time() - t1
54 print(f"Salted 2-phase : {t_saltd:.2f}s")
55
56 print(f"Speedup          : {t_naive/t_saltd:.2f}x")
57
58 # Verify correctness: totals should match
59 naive_dict = dict(naive_result)
60 salted_dict = dict(salted_result)
61 assert abs(naive_dict.get(SKEW_KEY, 0) -
62            salted_dict.get(SKEW_KEY, 0)) < 1e-6, "Results
63            differ!"
64 print("Correctness check: PASSED")
```

Listing 6.7: PE6.3: Salting benchmark for skew mitigation

SQL at Scale: HiveQL and the SQL-on-Hadoop Ecosystem

“Presto is the democratisation of data. Before Presto, only engineers could query our data warehouse. After Presto, every product manager, every analyst, every curious Facebooker could answer their own questions.”

Dain Sundstrom, Presto co-creator, Meta Engineering Blog, 2013

Learning Objectives

After completing this chapter, you will be able to:

1. Use HiveQL complex data types—ARRAY, MAP, and STRUCT—to model semi-structured data, and apply LATERAL VIEW EXPLODE to unnest array and map columns into relational rows.
2. Write SQL window function queries using OVER, PARTITION BY, ORDER BY, ROWS BETWEEN, and the ranking functions ROW_NUMBER, RANK, LAG, and LEAD.
3. Distinguish the three Hive UDF categories—scalar UDF, User-Defined Aggregate Function (UDAF), and User-Defined Table-Generating Function (UDTF)—and identify the appropriate category for a given extension.

4. Explain how Apache Tez improves on Hive-on-MapReduce by replacing the chain of MapReduce jobs with a single DAG of vertices and edges, and identify which types of intermediate materialisation Tez eliminates.
5. Describe the Hive LLAP architecture—the persistent daemon, in-memory columnar cache, and shared executor threads—and explain why LLAP enables sub-second interactive query latency.
6. Explain how the Cost-Based Optimizer (CBO) uses table and column statistics from the Hive Metastore to choose join order and join strategy, and describe the difference between rule-based and cost-based optimisation.
7. Compare Hive’s four join execution strategies—Common Join, Map Join, Bucket Map Join, and Sort-Merge Bucket Join—and select the correct strategy for a given table size and bucketing configuration.
8. Describe the Presto/Trino architecture—Coordinator, Workers, and Connectors—and explain how the connector model enables federated SQL queries across heterogeneous data sources.
9. Explain the key design differences between Hive and Presto that make Presto suitable for interactive queries (<1 minute) and Hive suitable for long-running batch ETL (hours).
10. Compare the four major SQL-on-Hadoop engines—Hive, Presto/Trino, Spark SQL, and Apache Impala—on latency, scale, fault tolerance, and data source support, and select the appropriate engine for a given workload.

Chapter 4 introduced Apache Hive as the SQL layer that made Hadoop accessible to analysts without MapReduce expertise, covering the Hive Metastore, partitioning, bucketing, and the basic HiveQL-to-MapReduce compilation pipeline. Chapter 6 showed how Spark SQL and the Catalyst optimizer brought a faster, in-memory SQL engine to the same Hive Metastore and Parquet ecosystem.

This chapter completes the SQL-at-scale picture by examining what sits between those two endpoints: the advanced features of HiveQL that make it a production-grade analytical SQL dialect; the successive execu-

tion engines (Tez, LLAP) that have reduced Hive query latency from hours to seconds; the cost-based optimizer that brings intelligent query planning to Hive; and the family of alternative SQL engines—most importantly Presto/Trino—that address the interactive analytics use case that neither Hive-on-MapReduce nor batch Spark optimally serves.

The central theme is *trade-off*: no single SQL engine is optimal for all workloads. Understanding what each engine optimises for—and what it sacrifices in return—is the core competency this chapter develops.

7.1 HiveQL: Advanced Language Features

HiveQL extends ANSI SQL with features designed specifically for semi-structured data and large-scale analytical workloads. Three categories of extensions—complex data types, window functions, and user-defined functions—are essential for production data engineering work.

7.1.1 Complex Data Types: ARRAY, MAP, and STRUCT

Real-world data rarely arrives in fully normalised form. A web event may carry a variable-length list of product categories, an e-commerce order may embed a nested shipping address, and a user profile may contain a key-value map of arbitrary feature flags. Hive models these patterns with three native complex types.

An **ARRAY** is an ordered sequence of values of the same type. An **MAP** is an unordered collection of key-value pairs. A **STRUCT** is a named tuple of fields with potentially different types, analogous to a JSON object:

```

1 CREATE TABLE user_events (
2     event_id    BIGINT,
3     user_id     BIGINT,
4     event_ts    TIMESTAMP,
5     -- ARRAY: a user may view 0..N product categories
6     categories  ARRAY<STRING>,
7     -- MAP: arbitrary key-value feature flags
8     features    MAP<STRING, DOUBLE>,
9     -- STRUCT: nested shipping address
10    ship_addr    STRUCT<
11                street: STRING,
```

```

12         city:    STRING,
13         postal:  STRING,
14         country: STRING
15     >
16 )
17 STORED AS ORC
18 PARTITIONED BY (dt STRING);
19
20 -- Accessing nested fields
21 SELECT event_id,
22        categories[0]           AS first_category,
23        features['ctr_score']   AS ctr_score,
24        ship_addr.city          AS city,
25        ship_addr.country       AS country
26 FROM   user_events
27 WHERE  dt = '2024-03-01'
28 AND    size(categories) > 0;

```

Listing 7.1: HiveQL: table schema with complex data types

7.1.2 LATERAL VIEW and EXPLODE

Aggregating or filtering over the elements of an ARRAY or MAP column requires first *exploding* the collection into separate rows. **LATERAL VIEW EXPLODE** generates one output row per element of the collection, joining it back to the containing row's other columns:

```

1  -- Count page views by category across all events
2  SELECT cat,
3         COUNT(*)           AS views,
4         COUNT(DISTINCT user_id) AS unique_users
5  FROM   user_events
6  LATERAL VIEW EXPLODE(categories) cat_table AS cat
7  WHERE  dt BETWEEN '2024-03-01' AND '2024-03-31'
8  GROUP BY cat
9  ORDER BY views DESC
10 LIMIT 20;

```

Listing 7.2: HiveQL: LATERAL VIEW EXPLODE to unnest an array

Without LATERAL VIEW EXPLODE, this query would require a user-defined function to iterate over the array. With it, the HiveQL compiler expands each user_events row into as many rows as there are elements in its categories

array, making the standard GROUP BY aggregation applicable directly.

Definition | LATERAL VIEW EXPLODE

LATERAL VIEW EXPLODE(col) generates a virtual table by unnesting an ARRAY or MAP column. For an ARRAY<T> column, each element produces one row. For a MAP<K, V> column, each key-value pair produces one row with two virtual columns (key and value). The virtual table is cross-joined to the source row via the lateral view construct.

For MAP columns, EXPLODE produces a two-column virtual table:

```

1  -- Find the average feature value by feature name
2  SELECT  feature_name,
3          AVG(feature_val) AS avg_val
4  FROM    user_events
5  LATERAL VIEW EXPLODE(features) feat_table AS feature_name,
          feature_val
6  WHERE   dt = '2024-03-01'
7  GROUP BY feature_name;
```

Listing 7.3: HiveQL: EXPLODE on a MAP column

7.1.3 Window Functions

Window functions compute aggregate values over a sliding window of rows around each output row, without collapsing the result set the way GROUP BY does. They are declared with the OVER clause and are essential for time-series analysis, ranking, sessionisation, and cohort analytics.

The general syntax is:

function_name(args) OVER ([PARTITION BY cols] [ORDER BY cols] [frame])

```

1  SELECT
2      region,
3      product_id,
4      sale_date,
5      revenue,
6
7      -- Running total within region, ordered by date
8      SUM(revenue) OVER (
9          PARTITION BY region
```

```
10      ORDER BY sale_date
11      ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
12  ) AS running_total,
13
14  -- 7-day moving average
15  AVG(revenue) OVER (
16      PARTITION BY region, product_id
17      ORDER BY sale_date
18      ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
19  ) AS ma7,
20
21  -- Rank product by revenue within region (ties get same
22      rank)
23  RANK() OVER (
24      PARTITION BY region
25      ORDER BY revenue DESC
26  ) AS revenue_rank,
27
28  -- Row number (no ties; deterministic ordering)
29  ROW_NUMBER() OVER (
30      PARTITION BY region
31      ORDER BY revenue DESC
32  ) AS row_num,
33
34  -- Revenue of the previous day for the same product-region
35  LAG(revenue, 1, 0) OVER (
36      PARTITION BY region, product_id
37      ORDER BY sale_date
38  ) AS prev_day_revenue,
39
40  -- Revenue of the next day
41  LEAD(revenue, 1, 0) OVER (
42      PARTITION BY region, product_id
43      ORDER BY sale_date
44  ) AS next_day_revenue
45 FROM daily_sales
46 WHERE dt = '2024-Q1';
```

Listing 7.4: HiveQL: window functions for sales ranking and trends

Table 7.1: Key HiveQL window functions

Function	Description
ROW_NUMBER()	Unique sequential integer per row within partition; no ties
RANK()	Rank with gaps for ties (e.g., 1, 2, 2, 4)
DENSE_RANK()	Rank without gaps (e.g., 1, 2, 2, 3)
NTILE(<i>n</i>)	Divides partition into <i>n</i> approximately equal buckets
PERCENT_RANK()	$(rank - 1) / (rows - 1)$; range [0, 1]
LAG(<i>col</i> , <i>n</i> , <i>def</i>)	Value of <i>col</i> from <i>n</i> rows before the current row
LEAD(<i>col</i> , <i>n</i> , <i>def</i>)	Value of <i>col</i> from <i>n</i> rows after the current row
FIRST_VALUE(<i>col</i>)	First value in the window frame
LAST_VALUE(<i>col</i>)	Last value in the window frame
SUM/AVG/MIN/MAX	Standard aggregates over the window frame

7.1.4 User-Defined Functions

When built-in HiveQL functions are insufficient, analysts can extend HiveQL through three categories of *User-Defined Functions* (UDFs):

Scalar UDF: maps one input row to one output row (one-to-one). The most common category: URL parsing, custom hashing, text normalisation, geolocation lookups. Implemented by extending Hive’s `org.apache.hadoop.hive.ql.exec` class in Java, or defined inline as a temporary function from a Python/R script via HiveQL’s `TRANSFORM ... USING` clause.

User-Defined Aggregate Function (UDAF): maps many input rows to one output row per group (many-to-one). Use when the built-in aggregates (SUM, AVG, COUNT, COLLECT_LIST) are insufficient—for example, computing a custom weighted median or a sketch (HyperLogLog approximate distinct count). UDAFs must implement both a partial phase (within the Map task,

analogous to a Combiner) and a final phase (within the Reduce task).

User-Defined Table-Generating Function (UDTF): maps one input row to zero or more output rows (one-to-many). This is the category that `EXPLODE` belongs to: it takes one row with an array column and produces one row per array element. Custom UDTFs are appropriate for parsing XML or JSON strings embedded in a column, splitting a delimited string into multiple rows, or expanding a compressed binary event log.

7.2 HiveServer2 and Concurrent Access

Original Hive (version 0.x) ran as a command-line tool that executed one query at a time. For a production data warehouse serving hundreds of concurrent analysts, this is untenable. *HiveServer2*, introduced in Hive 0.11 (2013), is a long-running service that exposes Hive’s query capabilities over JDBC and ODBC interfaces, supporting concurrent sessions from multiple clients simultaneously.

HiveServer2’s architecture separates query compilation from execution. A pool of *HiveServer2 daemon threads* each manage one client session, compiling the client’s HiveQL into a query plan and submitting it to the cluster’s execution engine. Multiple threads may submit queries to YARN simultaneously, with the Fair Scheduler or Capacity Scheduler arbitrating cluster resources between them.

System Design Note | HiveServer2 in Production: Beeline and JDBC

The standard command-line client for HiveServer2 is beeline, which connects over JDBC:

```
beeline -u "jdbc:hive2://hs2-host:10000/analytics_db" \  
-n analyst_user --password=... \  
-e "SELECT COUNT(*) FROM web_events WHERE dt \  
    = '2024-03-01' "
```

Business intelligence tools (Tableau, Power BI, Apache Superset) connect to HiveServer2 via their Hive JDBC/ODBC drivers, making the full Hive warehouse accessible through standard BI tooling without any Hadoop-specific configuration on the analyst’s workstation.

HiveServer2 also supports Kerberos authentication for secure enterprise deployments.

7.3 Execution Engine Evolution: MapReduce, Tez, and LLAP

The HiveQL language has remained stable since Hive 1.0, but the execution engines that compile HiveQL into distributed computation have undergone three successive architectural generations, each eliminating a specific performance bottleneck of its predecessor.

7.3.1 Hive-on-MapReduce: The Baseline

Chapter 4 established that a HiveQL query compiles to a sequence of MapReduce jobs. A join followed by an aggregation compiles to two jobs: Job 1 performs the join (Map reads input tables, Reduce co-locates matching rows); Job 2 performs the aggregation (Map reads the join output, Reduce computes the aggregate). Between Job 1 and Job 2, the entire join result is materialised to HDFS.

This HDFS materialisation is the core performance problem. For a pipeline with k intermediate stages, each transition writes and reads the full intermediate result from HDFS: $2 \times (k - 1)$ HDFS operations. For a complex ETL query with 5 stages, that is 8 full HDFS read/write cycles before the final result is produced. At 1 TB of intermediate data per stage, this amounts to 8 TB of unnecessary disk I/O in addition to the actual computation.

7.3.2 Apache Tez: DAG-Based Execution

Apache Tez, released in 2014 and developed primarily by Hortonworks, replaces the MapReduce execution model with a *directed acyclic graph (DAG)* of computation vertices connected by data edges. Tez allows Hive to express a complex multi-stage query as a single DAG rather than a chain of separate MapReduce jobs.

Tez introduces three types of edges between vertices that control how data flows between them:

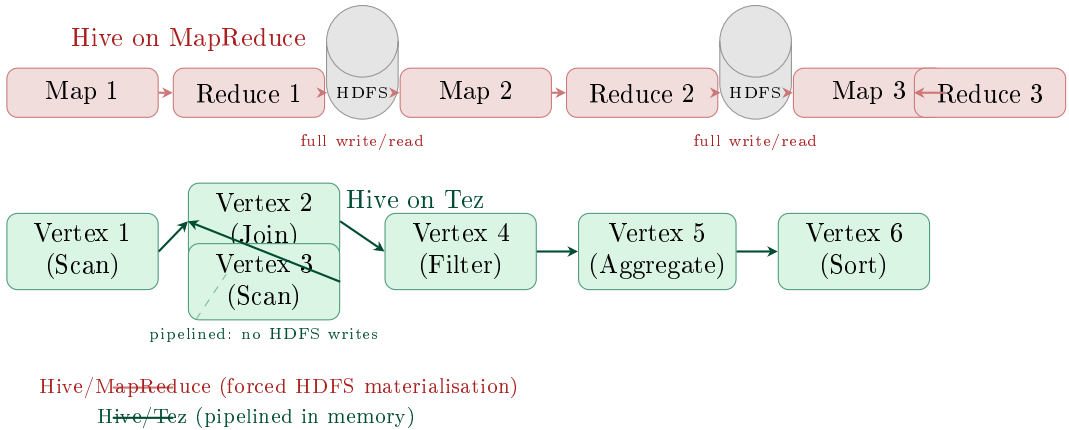


Figure 7.1: Hive-on-MapReduce vs. Hive-on-Tez for a three-stage query. MapReduce forces a full HDFS write and read at each stage boundary (shown as cylinders). Tez represents the same query as a DAG of vertices connected by pipelined edges; intermediate results flow in memory without HDFS materialisation.

- **One-to-one edge:** each task of the downstream vertex reads from exactly one task of the upstream vertex—no data movement required. Used for pipelined filtering and projection.
- **Broadcast edge:** all tasks of the downstream vertex receive a full copy of all data produced by the upstream vertex. Used for broadcasting a small dimension table to all join tasks (equivalent to Spark’s broadcast hash join).
- **Scatter-gather edge:** the standard shuffle—each upstream task partitions its output by key, and each downstream task reads one partition from every upstream task. Used for GROUP BY and sort-merge joins.

The practical speedup of Hive-on-Tez over Hive-on-MapReduce is 2–5× for simple queries and up to 100× for complex multi-stage pipelines, because the HDFS write-read cost of each stage boundary is eliminated. By 2015, most Hadoop distributions had configured Hive-on-Tez as the default execution engine.

7.3.3 Hive LLAP: In-Memory Interactive Queries

Even with Tez, a Hive query incurs several sources of latency that prevent sub-minute interactive response times. First, each YARN container must be launched fresh for every query—a process that takes 5–30 seconds of JVM startup and class-loading overhead. Second, every query must read its input data from HDFS, even if a prior query just read the same data seconds ago. Third, thread-level parallelism within a YARN container is limited to the cores allocated to that container, preventing fine-grained sharing of executor resources between concurrent queries.

Hive LLAP (Live Long And Process), introduced in Hive 2.0 (2016), resolves all three bottlenecks through a fundamentally different execution model.

Definition | Hive LLAP Architecture

LLAP replaces per-query YARN containers with **persistent long-running daemon processes** running on each worker node. Each daemon:

1. Maintains a **columnar in-memory cache** of recently read ORC/Parquet data, organised at the column chunk granularity.
2. Runs a **shared thread pool** that executes tasks from multiple concurrent queries simultaneously, without per-query JVM startup overhead.
3. Performs **vectorized execution**: processes data in batches of 1,024 rows using tight CPU loops rather than the row-at-a-time interpreted execution model.

The in-memory cache is the key feature that enables interactive latency. A workload in which analysts repeatedly query different slices of the same large table—typical in a business intelligence environment—will find that after the first query warms the cache, subsequent queries across the same table read from RAM at 40–100 GB/s rather than from HDFS at 1–5 GB/s. Combined with eliminated container startup overhead, LLAP delivers sub-second latency for queries over cached data, and 5–30 second latency for queries requiring HDFS reads.

7.4 Query Optimisation in Hive

7.4.1 Rule-Based vs. Cost-Based Optimisation

In Chapter 4, the Hive compilation pipeline applied *rule-based optimisation* (RBO): fixed logical rules such as “push predicates below joins” and “eliminate unused columns.” These rules improve query plans in many cases but are blind to data characteristics—they cannot know whether a join’s left input has 100 rows or 100 billion rows.

Cost-Based Optimisation (CBO), introduced in Hive 0.14 (2014) using Apache Calcite as the optimisation framework, extends the compilation pipeline with statistics-driven decisions. The CBO reads table and column statistics from the Hive Metastore (collected by `ANALYZE TABLE` commands or auto-gathered on partition creation) and uses them to estimate the cardinality and selectivity of each operator in the logical plan. With these estimates, the CBO can:

- **Join reordering:** place the smallest table as the build side of a hash join, minimising the memory footprint and build time. For a three-table join, the optimal ordering reduces hash table size from the cross product of all tables to the smallest table’s size.
- **Join strategy selection:** choose between Common Join (sort-merge, for large tables), Map Join (broadcast, for small tables), and Sort-Merge Bucket Join (for pre-bucketed tables) based on estimated sizes.
- **Partition pruning with statistics:** estimate the fraction of rows eliminated by a predicate and use this to decide whether to perform a partition scan or a full-table scan.

Enabling the CBO requires first collecting statistics:

```
1  -- Collect table-level and column-level statistics
2  ANALYZE TABLE orders COMPUTE STATISTICS;
3  ANALYZE TABLE orders COMPUTE STATISTICS FOR COLUMNS
4      customer_id, order_date, amount, region;
5
6  -- Verify statistics in the Metastore
7  DESCRIBE FORMATTED orders;
8  -- Output includes: numRows, numFiles, totalSize,
```

```

9  --                column-level: min, max, numNulls, numDVs,
    avgLen

```

Listing 7.5: Collecting Hive statistics for CBO

7.4.2 Join Execution Strategies

Hive implements four distinct join strategies, each suited to different combinations of table sizes and data organisation. Selecting the wrong join strategy is one of the most common causes of avoidable performance degradation in Hive ETL pipelines.

Common Join (Reduce-Side Join): the default for large tables. Both tables are shuffled on the join key to co-locate matching rows on the same reducer. For two tables of size A and B , the shuffle transfers $A + B$ bytes of data across the network. The shuffle is the bottleneck: for terabyte-scale tables, it can require hours.

Map Join (Broadcast Join): applicable when one table is small enough to fit in executor memory (default threshold: 25 MB in Hive, 10 MB in Spark SQL). The small table is loaded into a hash table in memory at each Map task. The large table is scanned without any shuffle: each Map task probes its local hash table to find matching rows. No reducer is needed. For a join of a 10 TB fact table against a 20 MB dimension table, Map Join eliminates the entire 10 TB shuffle.

Bucket Map Join: applicable when both tables are bucketed on the join key with the same number of buckets (or one is a multiple of the other). Each Map task processes only the corresponding bucket of each table, without loading the entire small table. Extends Map Join to cases where the “small” table is too large for a full broadcast but is well-organised.

Sort-Merge Bucket Join (SMB Join): the highest-performance join for large tables that are both bucketed *and* sorted on the join key. Because matching rows in both tables are co-located (same bucket) and pre-sorted, the join can be performed by a single linear scan of each bucket pair—analogueous to the merge step in merge sort—without any shuffle whatsoever. SMB Join requires upfront investment in table design (bucketing and sorting at write time) but delivers the best possible join performance at read time.

Table 7.2: Hive join strategy comparison

Strategy	Shuffle?	Requirements	Optimal use case
Common Join	Yes (full)	None	Large–large join; no pre-processing
Map Join	No	Small table < threshold	Large table × small dimension
Bucket Map Join	No	Both bucketed on join key	Medium–large join; data pre-bucketed
SMB Join	No	Both bucketed + sorted	Largest tables; highest write cost

7.4.3 Vectorized Execution

Traditional SQL execution engines process one row at a time through a tree of operator nodes—the *Volcano model* (or pull-based model). For each row, the query engine invokes virtual function calls at each operator, navigating the operator tree once per record. At billions of rows, the overhead of these function calls and the corresponding CPU branch mispredictions becomes significant.

Vectorized execution, enabled in Hive with `SET hive.vectorized.execution.enabled = true`, processes data in batches of 1,024 rows (one *vector batch*) rather than one row at a time. Each operator receives a batch of column vectors and applies a tight loop over all 1,024 rows without virtual function calls. The CPU can predict the loop branch perfectly and pipeline the arithmetic within each column chunk, making effective use of SIMD (Single Instruction Multiple Data) instructions on modern hardware.

For CPU-bound operations (arithmetic on numerical columns, string comparisons, hash computations), vectorized execution provides a 2–10× throughput improvement over the row-at-a-time model. ORC files (Chap-

ter 5) are particularly well-suited for vectorized execution because their columnar layout naturally delivers data to the engine in column-major order, matching the layout expected by the vector batch operators.

7.5 Presto and Trino: SQL on Everything

Presto was created at Facebook in 2012 to solve a specific problem: the Facebook data warehouse team had hundreds of analysts running queries on petabyte-scale data, and Hive-on-MapReduce was too slow for the interactive ad-hoc analytical queries that characterise analyst workflows. A query that answers the question “how many users in Germany clicked on ad campaign 4421 in the last 7 days?” should return in seconds, not hours.

After open-sourcing Presto in 2013, it was adopted by hundreds of organisations including LinkedIn, Twitter, Airbnb, and Uber. In 2019, the original Presto creators left Facebook and forked the project as *Trino* (2020). Both Presto (now maintained by the Presto Foundation) and Trino remain actively developed, with largely compatible SQL dialects. In this chapter, “Presto” refers to both unless the distinction matters.

(Sethi et al. 2019)

7.5.1 Presto Architecture

A Presto cluster consists of one *Coordinator* node and multiple *Worker* nodes, communicating over HTTP/2.

The *Coordinator* is the query planning and scheduling process:

- **Parser:** converts the SQL string into an abstract syntax tree.
- **Analyser:** resolves table names and column types by querying the registered catalogs (each catalog is backed by a connector’s metadata interface).
- **Query Planner:** produces a distributed execution plan as a tree of *plan fragments*. Each fragment corresponds to a set of tasks that can run concurrently on multiple workers. Plan fragments are connected by *exchange operators*: the upstream fragment’s output is buffered in the upstream workers’ memory and pulled by the downstream fragment over HTTP.

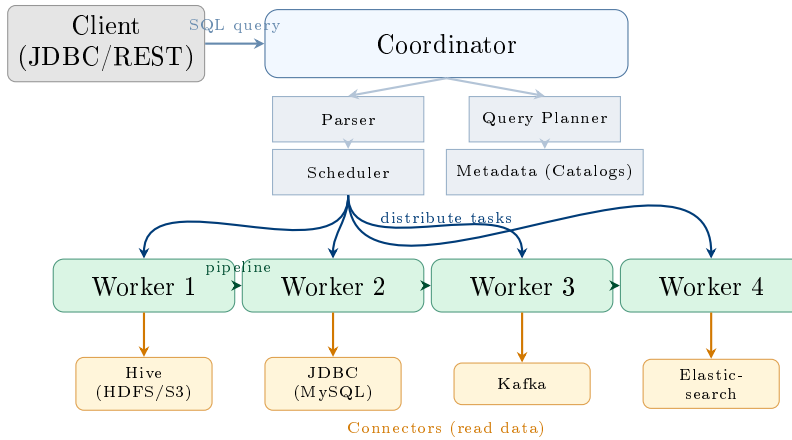


Figure 7.2: Presto cluster architecture. The Coordinator receives SQL queries from clients, parses and plans them, then distributes tasks to Worker nodes. Workers pipeline results between themselves in memory. Each worker reads data through a Connector that abstracts the underlying data source—HDFS, JDBC databases, Kafka, Elasticsearch, or others.

- **Scheduler:** assigns fragments to specific workers, taking data locality (HDFS block locations) into account for the scan fragments.

Workers execute the tasks assigned by the coordinator. Each worker processes a subset of the data by reading through its connector and applying the plan fragment’s operators (filter, project, aggregate, join). Workers communicate intermediate results to each other via in-memory buffers over HTTP/2. *There is no disk I/O between plan fragments:* all intermediate data flows in memory. This is the primary architectural difference from Hive-on-MapReduce.

7.5.2 The Connector Model: SQL on Everything

The most distinctive feature of Presto is its *connector model*: a pluggable interface that allows Presto to read from (and in some cases write to) any data source that implements the connector API. Each connector provides:

- **Metadata API:** listing catalogs, schemas, tables, columns, and column statistics.
- **Data location API:** describing how data is split into independently-readable chunks (HDFS splits, Kafka partition offsets, JDBC pagination

ranges).

- **Record cursor / Page source API:** reading actual data records from the underlying storage.

This abstraction enables *federated queries*—SQL queries that join data from multiple different data sources in a single statement:

```

1  -- Join HDFS order data (Hive connector)
2  -- with a MySQL reference table (JDBC connector)
3  -- in a single Presto query
4  SELECT  h.customer_id,
5           m.customer_name,
6           m.country,
7           SUM(h.order_amount) AS total_spent,
8           COUNT(*)           AS order_count
9  FROM    hive.analytics.orders AS h           -- HDFS/Parquet
          via Hive connector
10 JOIN    mysql.crm.customers AS m           -- MySQL via JDBC
          connector
11  ON      h.customer_id = m.customer_id
12  WHERE   h.order_date >= DATE '2024-01-01'
13  AND     m.country IN ('US', 'GB', 'DE')
14  GROUP BY h.customer_id, m.customer_name, m.country
15  ORDER BY total_spent DESC
16  LIMIT 100;

```

Listing 7.6: Presto federated query: joining HDFS data with a MySQL table

This query reads the orders table from Hive (backed by Parquet files on HDFS) and the customers table from a MySQL database, joining them in Presto’s memory without any ETL pipeline or intermediate data movement. In a Hive-only environment, this query would require first exporting the MySQL table to HDFS and creating a Hive external table—a process taking hours.

7.5.3 Presto vs. Hive: Design Philosophy

The fundamental design difference between Presto and Hive reflects a deliberate trade-off between *query latency* and *fault tolerance*.

Presto’s design choices:

- **No intermediate disk spill (by default).** All data processing is in-memory. A query that exceeds the cluster’s total memory fails imme-

diately with an “exceeded memory limit” error rather than spilling to disk and completing slowly.

- **No task retry.** If a worker node fails during query execution, the entire query fails. The coordinator does not retry failed tasks on different workers (though more recent Presto/Trino versions have added limited retry capabilities).
- **Low startup overhead.** Workers are persistent processes; there is no container-launch overhead. A query begins executing within milliseconds of submission.
- **Push-based pipelining.** Results flow from upstream to downstream operators without waiting for the upstream stage to complete—the first results are produced before the last input row is read.

Hive’s design choices:

- **Disk-based fault tolerance.** Intermediate results are materialised to HDFS at each stage boundary (MapReduce) or can be spilled to local disk (Tez). A task can be re-executed if it fails, without restarting the entire query.
- **Unbounded query size.** A Hive query can process any volume of data, regardless of cluster RAM, because it spills to HDFS. A 100 TB sort that would exhaust Presto’s memory completes in Hive—slowly, but completely.
- **Higher startup overhead.** YARN container launches add 5–30 seconds to every query. LLAP partially mitigates this with persistent daemons.

Common Misconception | Presto is Not a Hive Replacement for ETL

A common mistake is deploying Presto as a general-purpose replacement for Hive and then running large ETL pipelines through it. A 50 TB nightly ETL pipeline that produces many intermediate materialised results belongs in Hive-on-Tez (or Spark), where disk-based fault tolerance and HDFS spill prevent out-of-memory failures. Presto is optimised for the interactive analytics use case: queries that return in seconds or minutes, over data that fits in the cluster’s RAM. Running multi-hour ETL jobs through Presto risks losing all progress if any worker fails. Use the right engine for each workload.

7.6 The SQL-on-Hadoop Ecosystem

Hive and Presto are the two most widely deployed SQL-on-Hadoop engines, but the ecosystem includes several other significant systems. Understanding their design points helps data engineers match engine to workload.

7.6.1 Apache Impala

Apache Impala, developed by Cloudera and released in 2012, is a massively parallel processing (MPP) SQL engine designed for interactive analytics. Unlike Hive, which is built on top of MapReduce/Tez, Impala is implemented entirely in C++ and runs as a native process on each cluster node—bypassing the JVM and YARN container model entirely.

Impala's key design characteristics:

- **No JVM.** C++ avoids JVM garbage collection pauses, which can add hundreds of milliseconds of latency to individual query tasks—unacceptable for sub-second interactive queries.
- **Shared metadata.** Impala reads the Hive Metastore for table definitions, enabling analysts to switch between Hive and Impala queries against the same tables.
- **In-memory, no fault tolerance.** Like Presto, Impala does not retry failed tasks. Queries that exceed memory fail.
- **HDFS and Kudu.** Impala supports both HDFS (Parquet, ORC) and Apache Kudu (a columnar storage engine optimised for fast random reads and updates), making it suitable for near-real-time analytics over mutable data.

7.6.2 Comparative Analysis

The selection principle is workload-driven:

- **Nightly batch ETL over terabytes:** Hive-on-Tez or Spark. Fault tolerance and unlimited scale matter; latency does not.
- **BI dashboards with repeated queries:** Hive LLAP. The cache warms on the first query; subsequent queries are served from memory at sub-second latency.

- **Ad-hoc analyst queries (<5 minutes):** Presto/Trino. Federated query capability and low startup latency are critical.
- **Fastest interactive queries on Parquet, low cardinality:** Apache Impala. C++ execution with no JVM overhead.
- **ML training on the same data as SQL analytics:** Spark SQL. A single framework covers both SQL and MLlib.

Research Spotlight | Venkataraman et al. (2019) — Presto: SQL on Everything

Citation: Sethi, R., Traverso, M., Sundstrom, D., Phillips, D., Xie, W., Sun, Y., Yigitbasi, N., Jin, H., Hwang, E., Shingte, N., and Berner, C. (2019). Presto: SQL on Everything. *Proceedings of the 35th IEEE International Conference on Data Engineering (ICDE 2019)*, pp. 1802–1813. (Sethi et al. 2019)

Key insight: The paper’s thesis is that the connector model—a clean separation between the SQL execution engine and the data source—is the enabling abstraction for “SQL on everything.” By standardising how the engine asks a data source about its schema and data layout, Presto can treat any data source (HDFS, RDBMS, Kafka, Elasticsearch, custom REST APIs) identically from the query execution perspective. The paper calls this property *data source agnosticism*.

Technical contributions:

- Described the Coordinator/Worker/Connector architecture and the push-based distributed execution model, showing how pipelined query plans eliminate the HDFS materialisation bottleneck of Hive-on-MapReduce.
- Introduced the *Connector SPI* (Service Provider Interface)—the five interfaces (metadata, data location, data source, record cursor, and transaction) that a connector must implement to integrate a new data source into Presto without modifying the core engine.
- Reported Facebook’s production deployment: Presto ran on thousands of worker nodes, processing petabytes per day, serving 30,000+ queries per day from 1,000+ internal Facebook users with a median query latency of under 1 minute and a P95 latency of under 10 minutes.

Impact today: Presto/Trino is deployed at hundreds of organisations and has become the de facto standard engine for interactive SQL analytics on cloud data lakes. Amazon Athena (serverless SQL on S3) is built on Presto. Databricks SQL Warehouse and Snowflake adopted similar connector architectures. The Presto SPI was the direct inspiration for Apache Arrow's DataFusion connector model and the Delta Sharing protocol's reader interface.

7.7 Case Study: Airbnb's Migration to a Unified SQL Platform

Case Study | Airbnb — From Hive Monolith to Hive-Plus-Presto Tiered SQL Platform

Background. By 2015, Airbnb had grown to over 2 million listings and 40 million users, with a data team of approximately 100 engineers and analysts generating more than 10,000 Hive queries per day. The company's data warehouse ran on a 500-node Hadoop cluster with approximately 15 PB of data, processed exclusively by Hive-on-MapReduce. The data platform was suffering from three interconnected problems: analyst query latency averaged 45 minutes (with some queries taking 8+ hours), cluster utilisation was chronically at 95%+ because long-running batch ETL jobs and short analyst queries shared the same YARN queue, and analyst adoption of data-driven decision-making was limited because the 45-minute query latency discouraged exploratory iteration.

Phase 1 (2016): Hive Tez Migration. Airbnb's first performance initiative migrated all analyst-facing Hive queries from MapReduce to Tez. The migration was transparent to analysts: no query rewrites were required, as the HiveQL syntax is identical regardless of execution engine. The results were significant:

- Median analyst query latency dropped from 45 minutes to 18 minutes.

- P95 latency dropped from >8 hours to 75 minutes.
- Cluster CPU utilisation dropped from 95% to 70% for the same analytical workload, because Tez eliminated the HDFS materialisation I/O that had been consuming a disproportionate share of disk bandwidth.

However, even 18 minutes remained too slow for the exploratory, iterative analysis style that Airbnb’s product and growth teams required. An analyst testing hypotheses about booking conversion rates might run 15–20 queries in sequence; at 18 minutes each, this required a full day.

Phase 2 (2017): Presto Deployment for Interactive Analytics. Airbnb deployed a dedicated 150-node Presto cluster alongside the existing Hive cluster. The Presto cluster shared the Hive Metastore, so all tables defined for Hive were immediately queryable through Presto without any schema migration.

The workload routing strategy divided queries by expected duration:

Expected duration	Engine	Queue
<5 minutes (interactive)	Presto	analyst_interactive
5–60 minutes (analytical)	Hive/Tez	analyst_batch
>60 minutes (ETL/batch)	Hive/Tez	production_etl

Routing was implemented in Airbnb’s internal query routing service, which classified incoming queries based on a set of heuristics—estimated data scanned (from partition statistics), presence of CROSS JOIN or unconstrained JOIN patterns, and explicit analyst preference flags. The routing service directed queries to the appropriate cluster transparently.

The booking funnel analysis pipeline. A representative workload that benefited most from Presto was the booking funnel analysis: understanding how users progress from search to booking across Airbnb’s platform. The analysis requires joining four tables:

1. **search_events** (~2 TB/day): one row per guest search, with search parameters, result count, and filters applied.

2. **listing_views** (~500 GB/day): one row per listing viewed from a search results page.
3. **booking_attempts** (~50 GB/day): one row per booking initiation, including payment method and price.
4. **dim_listings** (~4 GB, daily snapshot): listing metadata including location, price tier, and amenity flags.

The booking funnel query computes conversion rates at each step of the funnel, segmented by geography and listing price tier. On Hive-on-MapReduce, this query ran in 4 hours. On Hive-on-Tez, it ran in 55 minutes. On Presto, after Catalyst-equivalent optimisations by the Presto analyser (broadcast join for `dim_listings`, which at 4 GB fits within each worker's broadcast buffer), the query ran in 4 minutes.

The 4-minute latency transformed analyst behaviour: instead of running the funnel analysis once per day and committing to the results, analysts began running it hourly during A/B test periods, checking whether a new feature was affecting conversion rates in near real time.

Phase 3 (2019): LLAP for Dashboards. Airbnb's engineering dashboards required even lower latency for recurring queries: the homepage executive dashboard refreshed every 5 minutes, pulling the same suite of KPIs (bookings, revenue, host activations, guest satisfaction scores) from Hive tables. On Presto, these queries took 20–60 seconds depending on cluster load.

Deploying Hive LLAP for the dashboard tier cached the relevant table partitions in the LLAP daemons' memory after the first query of each 5-minute cycle. Subsequent queries in the same cycle served from the in-memory cache, reducing dashboard query latency to 1–5 seconds.

Outcomes. The three-tier SQL architecture—LLAP for dashboards, Presto for interactive analytics, Hive/Tez for batch ETL—produced measurable improvements across every operational dimension:

- **Analyst query latency:** P50 reduced from 45 minutes (Hive/MR) to 3.5 minutes (Presto), a 13× improvement.
- **Data-driven decisions:** analyst-submitted queries per day increased from 10,000 to 47,000 within 18 months of deploying Presto—a 4.7×

increase, attributed to the lower iteration cost of interactive query response times.

- **Cluster efficiency:** separating interactive (Presto) from batch ETL (Hive/Tez) eliminated the queue starvation problem. Long-running ETL jobs no longer delayed analyst queries; the Presto cluster's dedicated allocation guaranteed sub-5-minute response for all routed queries.
- **Engineering savings:** the number of "bespoke pre-computed aggregate tables" (summary tables created by engineers specifically to speed up slow analyst queries) dropped by 60%, because analysts could run the original queries directly on Presto at acceptable latency.

The architectural lesson. Airbnb's platform evolution illustrates the principle that no single SQL engine is optimal for all workloads. The cost of this differentiation—operating three distinct query engines against the same Hive Metastore—is engineering complexity in routing, monitoring, and SLA management. The benefit is that each engine operates within its optimal design envelope: LLAP for cached sub-second BI, Presto for interactive exploration, Hive/Tez for batch ETL that requires fault tolerance and unlimited scale.

Chapter Summary

- HiveQL's **complex data types** (ARRAY, MAP, STRUCT) model semi-structured data natively. **LATERAL VIEW EXPLODE** unnests array and map columns into relational rows, enabling standard GROUP BY aggregation over nested data without UDFs.
- **Window functions** (OVER, PARTITION BY, ORDER BY, ROWS BETWEEN) compute running totals, moving averages, and rankings without collapsing the result set. Key functions: ROW_NUMBER, RANK, LAG, LEAD, NTILE.
- Hive **UDF** categories: scalar UDF (one-to-one), **UDAF** (many-to-one, must implement partial + final phases), and **UDTF** (one-to-many, used for EXPLODE-style expansion).

- **Apache Tez** replaces MapReduce's chain of jobs with a single DAG of vertices, eliminating forced HDFS materialisation at each stage boundary. Speedup: 2–5× for simple queries, up to 100× for complex multi-stage pipelines.
- **Hive LLAP** adds a persistent in-memory columnar cache and shared executor threads, eliminating per-query JVM startup overhead and enabling sub-second latency for cached data.
- Hive's **Cost-Based Optimizer** (CBO, powered by Apache Calcite) uses Metastore statistics (`ANALYZE TABLE`) to choose join order and join strategy. Without statistics, the CBO falls back to rule-based optimisation.
- The four Hive **join strategies**: Common Join (shuffle, always applicable), Map Join (no shuffle, small table broadcast), Bucket Map Join (no shuffle, both bucketed), and Sort-Merge Bucket Join (no shuffle, both bucketed and sorted — highest performance, highest write cost).
- **Vectorized execution** processes 1,024 rows per batch rather than one row at a time, eliminating virtual function call overhead and enabling SIMD parallelism. Compatible with ORC's columnar layout. Typical speedup: 2–10× for CPU-bound operations.
- **Presto/Trino** uses a Coordinator + Worker architecture with fully pipelined, in-memory execution and no fault tolerance. The **Connector SPI** enables federated SQL across HDFS, JDBC databases, Kafka, and arbitrary data sources in a single query.
- Key Presto vs. Hive trade-off: Presto sacrifices fault tolerance and unbounded scale for low latency and federated access. Use Presto for interactive queries (<5 minutes); use Hive for batch ETL (hours) that requires fault tolerance and disk spill.

Exercises

Short Questions

SQ7.1. What is the `LATERAL VIEW EXplode` construct in HiveQL? A table

orders has a column `tags ARRAY<STRING>`. Write a HiveQL query that counts the total number of times each tag appears across all orders.

SQ7.2. What is the difference between `RANK()` and `DENSE_RANK()`? Give a concrete example with five rows where the two functions produce different output.

SQ7.3. Explain the three categories of Hive UDFs: UDF, UDAF, and UDTF. For each, give one real-world use case and explain why it falls into that category.

SQ7.4. What problem does Apache Tez solve that Hive-on-MapReduce does not? For a HiveQL query that compiles to three MapReduce jobs with 500 MB of intermediate data at each stage boundary, calculate the HDFS I/O saved by using Tez instead.

SQ7.5. What is Hive LLAP? Describe the three components of LLAP architecture and explain why LLAP enables sub-second latency for repeated queries over the same table.

SQ7.6. What is the difference between rule-based optimisation (RBO) and cost-based optimisation (CBO) in Hive? Give one example where RBO would produce a suboptimal plan that CBO would avoid.

SQ7.7. Explain Hive's Sort-Merge Bucket Join. What preconditions must both tables satisfy, and why does this join require no shuffle at query time?

SQ7.8. What is the Presto Connector SPI? Name three data sources that Presto can query through connectors, and explain what interface the connector must implement for each source.

SQ7.9. A data engineer argues that Presto should replace Hive entirely because Presto is always faster. Write a detailed counter-argument using a specific workload scenario where Hive is the correct choice.

SQ7.10. HiveServer2 replaced the original Hive command-line tool. What two problems with the original tool did HiveServer2 address, and what interface (protocol) does HiveServer2 expose to clients?

SQ7.11. What is vectorized execution in Hive? Explain the difference from the Volcano (row-at-a-time) execution model, and identify which file format is best suited for vectorized execution in Hive.

SQ7.12. A Presto query fails with "Query exceeded maximum memory."

Explain: (a) why this error occurs; (b) why the same query would not fail in Hive-on-Tez; (c) what you should do to fix the problem without switching engines.

SQ7.13. Explain Hive's Map Join strategy. What is the default table size threshold for a Map Join, and what happens during query execution that eliminates the shuffle?

SQ7.14. What is a Presto federated query? Write the SQL statement structure (not actual data) for a query that joins a Hive table with a MySQL table. What does the Presto Coordinator need to execute this query?

SQ7.15. Compare Apache Impala and Presto on three dimensions: implementation language, JVM usage, and approach to data source support.

Analytical Problems

AP7.1. Window Function Design. A subscription streaming service wants to analyse user churn. The table `user_daily_activity` has columns: `user_id`, `activity_date`, `minutes_streamed` (0 if not active), `country`.

- (a) Write a HiveQL window function query that computes, for each user and date, the 7-day rolling average of `minutes_streamed`.
- (b) Write a query using `LAG` to compute the number of days since each user last had a non-zero `minutes_streamed` value.
- (c) Write a query using `NTILE(10)` to divide users into 10 deciles by their total minutes streamed in the past 30 days, and compute the churn rate for each decile.
- (d) Explain how the window function queries in (a)–(c) would be compiled by Hive-on-Tez compared to Hive-on-MapReduce. How many jobs/stages would each require?

AP7.2. Join Strategy Selection. A retail analytics system has the following tables:

Table	Rows	Compressed size	Bucketed on
transactions	80 billion	12 TB	customer_id (500 buckets)
customers	200 million	80 GB	customer_id (500 buckets)
dim_country	250	40 KB	None
dim_category	8,000	2 MB	None

- For a join between transactions and dim_country, identify the optimal join strategy and explain why. Estimate the network traffic for the alternative (Common Join) approach.
- For a join between transactions and customers, both bucketed on customer_id with 500 buckets, identify the optimal join strategy. What additional condition must hold for Sort-Merge Bucket Join to be applicable?
- For a three-table join: transactions JOIN customers JOIN dim_category, propose the join order and strategy for each join. Justify the order using the CBO's cost minimisation objective.
- Explain what Hive statistics (output of ANALYZE TABLE) the CBO would use to automatically determine the join strategies in (a)–(c) without manual hints.

AP7.3. Presto vs. Hive Workload Assignment. A data analytics team runs the following five daily workloads:

- A nightly ETL job that reads 200 TB of raw clickstream logs, filters and aggregates them, and writes the result to a Hive table (runtime: 4–6 hours).
- Ad-hoc analyst queries over the last 7 days of clickstream data (~50 GB) to investigate a product anomaly (target: <3 min).
- A weekly cohort analysis joining three tables totalling 800 GB with 12 join stages and complex window functions (runtime target: <30 minutes).
- A live executive dashboard refreshing every 5 minutes, running the same 5 KPI queries over the last hour of data (~2 GB).
- A machine learning feature engineering pipeline that generates 1,000 features per user from 60 days of history, followed by a logistic regression training run (runtime: 2–3 hours).

For each workload, specify: (a) which engine (Hive/MapReduce, Hive/Tez, Hive/LLAP, Presto, Spark SQL) is most appropriate and why; (b) the specific property of the workload that drives the engine choice; (c) the risk of using the wrong engine.

AP7.4. LLAP Cache Sizing. An analytics team deploys Hive LLAP on a 50-node cluster. Each node has 128 GB of RAM. The LLAP daemon on each node is allocated 80 GB, of which 60 GB is dedicated to the in-memory columnar cache and 20 GB to the executor thread pool.

The team's most frequently queried table is `sales_events`: partitioned by day, each day's partition is 400 GB compressed (ORC) but 1.2 TB uncompressed. Analysts typically query the last 30 days.

- (a) Calculate the total LLAP cache capacity across the cluster.
- (b) The uncompressed size of 30 days of data is 36 TB. What fraction of the 30-day working set fits in the LLAP cache?
- (c) ORC's columnar layout allows the cache to store only the columns accessed by queries rather than full rows. If analyst queries reference an average of 8 of the 120 columns in `sales_events`, recalculate the fraction of the working set that fits in cache.
- (d) Propose two changes to the LLAP configuration or table design that would increase the cache hit rate, and explain the trade-off of each.

Design Problems

DP7.1. Multi-Engine SQL Platform for a Media Company. A digital media company operates a 1,000-node Hadoop cluster with 400 TB of data. Three teams use the cluster:

- **Data Engineering:** runs nightly ETL pipelines (Hive/MapReduce), producing 20 curated tables from 150 TB of raw log data.
- **Analytics:** runs 500 ad-hoc queries per day, targeting <5-minute response time, on the curated tables (~15 TB).
- **Business Intelligence:** runs 10 recurring dashboard queries every 15 minutes on 5 summary tables (~500 GB total).

Design a three-tier SQL platform for this company. Your design must specify:

- (a) The SQL engine and execution environment (YARN queues, cluster sizing) for each team's workload.
- (b) How the three engines share the Hive Metastore and Parquet/ORC data on HDFS without data duplication.
- (c) A query routing service: how incoming queries are classified and directed to the correct engine. What metadata does the routing service inspect to make this classification?
- (d) The LLAP daemon configuration for the BI dashboard tier: how many nodes, how much cache, and how to ensure the most-queried partitions are warmed in the cache before the first dashboard refresh of the day.
- (e) A monitoring and SLA framework: which metrics to track for each tier, and the alerting thresholds that indicate a tier is degrading.

DP7.2. HiveQL to Presto SQL Migration. Your company has 2,000 HiveQL queries (stored as .hql files in a Git repository) that need to be migrated to Presto SQL to reduce query latency from 30–45 minutes to <5 minutes.

Design the migration. Your design must address:

- (a) The four most common SQL dialect differences between HiveQL and Presto SQL that require query rewrites. Give one example of each.
- (b) An automated compatibility scanning tool: how to programmatically detect which of the 2,000 queries require manual rewriting vs. which can be migrated automatically.
- (c) A testing strategy to verify that the migrated Presto queries produce results identical to the original HiveQL queries. What is the acceptable tolerance for numerical differences?
- (d) The migration sequence: which 20% of the 2,000 queries should be migrated first, and why? What criterion determines priority?
- (e) A rollback plan if a migrated query produces incorrect results in production (e.g., a finance report that a business user flags as wrong).

Programming Exercises

PE7.1. Window Function Analytics Pipeline (HiveQL). Write a HiveQL script that analyses a table `daily_sales` (columns: `region` STRING, `product_id` STRING, `sale_date` STRING, `revenue` DOUBLE) and produces a report containing: (a) 7-day moving average revenue per region-product; (b) revenue rank within region for each date; (c) day-over-day revenue growth percentage; (d) cumulative revenue per region from the start of the month.

```

1  -- Create the source table (run once to set up)
2  CREATE TABLE IF NOT EXISTS daily_sales (
3      region      STRING,
4      product_id  STRING,
5      sale_date   STRING,
6      revenue     DOUBLE
7  )
8  STORED AS ORC
9  PARTITIONED BY (dt STRING);
10
11 -- Main analytics query: 4 window metrics in one pass
12 SELECT
13     region,
14     product_id,
15     sale_date,
16     revenue,
17
18     -- (a) 7-day moving average
19     ROUND(
20         AVG(revenue) OVER (
21             PARTITION BY region, product_id
22             ORDER BY sale_date
23             ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
24         ), 2
25     ) AS ma7_revenue,
26
27     -- (b) Revenue rank within region for each date
28     RANK() OVER (
29         PARTITION BY region, sale_date
30         ORDER BY revenue DESC
31     ) AS daily_rank_in_region,
32
33     -- (c) Day-over-day growth %
34     ROUND(
35         100.0 * (revenue - LAG(revenue, 1, revenue) OVER (
36             PARTITION BY region, product_id

```

```

37         ORDER BY sale_date
38     )) /
39     NULLIF(LAG(revenue, 1, revenue) OVER (
40         PARTITION BY region, product_id
41         ORDER BY sale_date
42     ), 0),
43     2
44 ) AS dod_growth_pct,
45
46 -- (d) Month-to-date cumulative revenue per region
47 SUM(revenue) OVER (
48     PARTITION BY region, SUBSTR(sale_date, 1, 7)
49     ORDER BY sale_date
50     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
51 ) AS mtd_revenue
52
53 FROM     daily_sales
54 WHERE    dt BETWEEN '2024-01-01' AND '2024-03-31'
55 ORDER BY region, product_id, sale_date;

```

Listing 7.7: PE7.1: Window function analytics pipeline

PE7.2. Presto Query Profiler (Python). Implement a Python tool that connects to a Presto/Trino coordinator via its REST API, submits a SQL query, polls for completion, and reports execution statistics: elapsed time, bytes scanned, rows processed, number of splits, and peak memory usage.

```

1  import time
2  import requests
3  import json
4
5  PRESTO_HOST = "http://presto-coordinator:8080"
6  HEADERS     = {
7      "X-Presto-User":     "analyst",
8      "X-Presto-Catalog": "hive",
9      "X-Presto-Schema":  "analytics",
10     "Content-Type":      "application/json",
11 }
12
13 def submit_query(sql: str) -> str:
14     """Submit a query and return the query ID."""
15     resp = requests.post(
16         f"{PRESTO_HOST}/v1/statement",
17         headers=HEADERS,

```



```

18         data=sql
19     )
20     resp.raise_for_status()
21     data = resp.json()
22     return data.get("nextUri"), data.get("id")
23
24 def poll_query(next_uri: str) -> dict:
25     """Poll until the query completes; return final stats."""
26     while next_uri:
27         resp = requests.get(next_uri, headers=HEADERS)
28         resp.raise_for_status()
29         data = resp.json()
30         state = data.get("stats", {}).get("state", "UNKNOWN")
31
32         if state in ("FINISHED", "FAILED", "CANCELED"):
33             return data
34
35         # Print progress
36         stats = data.get("stats", {})
37         pct = stats.get("progressPercentage", 0)
38         print(f" [{state}] {pct:.1f}% complete "
39               f"({stats.get('completedSplits',0)}/{stats.get('totalSplits',0)} splits)",
40               end="\r")
41
42         next_uri = data.get("nextUri")
43         time.sleep(0.5)
44
45     return {}
46
47 def profile_query(sql: str):
48     """Submit, poll, and report stats for a Presto SQL query."""
49
50     print(f"Submitting: {sql[:80]}{'...' if len(sql)>80 else ''}")
51     t_start = time.time()
52     next_uri, query_id = submit_query(sql)
53     result = poll_query(next_uri)
54     elapsed = time.time() - t_start
55
56     stats = result.get("stats", {})
57     state = stats.get("state", "UNKNOWN")
58     print(f"\n{'='*60}")
59     print(f"Query ID      : {query_id}")

```

```

59     print(f"Final state : {state}")
60     print(f"Elapsed      : {elapsed:.2f}s")
61
62     if state == "FINISHED":
63         print(f"Rows read      : {stats.get('processedRows',0):,}")
64         print(f"Bytes scanned: {stats.get('processedBytes',0)/1e9:.2f} GB")
65         print(f"Splits done   : {stats.get('completedSplits',0):,}")
66         print(f"Peak mem      : {stats.get('peakMemoryBytes',0)/1e9:.2f} GB")
67         print(f"CPU time      : {stats.get('cpuTimeMillis',0)/1000:.2f}s")
68     elif state == "FAILED":
69         err = result.get("error", {})
70         print(f"Error          : {err.get('errorName','')}: {err.get('message','')}")
71
72     # --- Run a sample query ---
73     SQL = """
74     SELECT    region,
75              COUNT(*)          AS num_sales,
76              SUM(revenue)      AS total_revenue,
77              AVG(revenue)      AS avg_revenue
78     FROM      hive.analytics.daily_sales
79     WHERE     dt BETWEEN '2024-01-01' AND '2024-03-31'
80     GROUP BY  region
81     ORDER BY  total_revenue DESC
82     """
83
84     profile_query(SQL)

```

Listing 7.8: PE7.2: Presto query profiler via REST API

PE7.3. HiveQL Complex Type Analytics (HiveQL + Python). A table `session_events` stores user session data with an `ARRAY<STRING>` column `pages_visited` (ordered list of page URLs per session) and a `MAP<STRING,DOUBLE>` column `engagement_scores` (per-page engagement metrics). Write: (a) a HiveQL query using `LATERAL VIEW EXplode` to compute the top-10 most visited page URL patterns; (b) a Python script that reads the result and visualises the URL frequency distribution.

```

1  -- Top 10 most visited pages (unnesting the pages_visited array
   )
2  SELECT  page_url,
3          COUNT(*)                               AS visit_count,
4          COUNT(DISTINCT session_id)             AS unique_sessions,
5          ROUND(AVG(eng_score), 4)              AS avg_engagement
6  FROM    session_events
7  LATERAL VIEW EXPLODE(pages_visited)    pv_table AS page_url
8  LATERAL VIEW EXPLODE(engagement_scores) es_table AS eng_page,
          eng_score
9  WHERE   dt = '2024-03-01'
10 AND     eng_page = page_url              -- correlate map key with
          array element
11 GROUP BY page_url
12 HAVING COUNT(*) > 1000                  -- filter low-frequency
          pages
13 ORDER BY visit_count DESC
14 LIMIT 10;

```

Listing 7.9: PE7.3a: LATERAL VIEW EXPLODE on session pages

```

1  import pandas as pd
2  import matplotlib
3  matplotlib.use("Agg")
4  import matplotlib.pyplot as plt
5
6  # Simulate result of the HiveQL query above
7  results = {
8      "page_url": ["/home", "/search", "/listing/123", "/booking",
9                  "/messages", "/profile", "/wishlist", "/help",
10                 "/explore", "/trips"],
11      "visit_count": [9_812_441, 7_234_190, 4_112_030, 2_981_200,
12                    1_822_441, 1_654_022, 1_244_330, 987_441,
13                    872_301, 641_200],
14      "avg_engagement": [0.72, 0.65, 0.81, 0.89, 0.77, 0.68,
15                        0.74,
16                        0.45, 0.58, 0.83],
17  }
18  df = pd.DataFrame(results).sort_values("visit_count", ascending
19                                         =True)
20
21  fig, axes = plt.subplots(1, 2, figsize=(14, 6))

```

```
21 # Bar chart: visit counts
22 axes[0].barh(df["page_url"], df["visit_count"] / 1e6,
23             color="#003c78", alpha=0.8)
24 axes[0].set_xlabel("Visit count (millions)")
25 axes[0].set_title("Top 10 Pages by Visit Count")
26 axes[0].grid(axis="x", alpha=0.3)
27
28 # Scatter: engagement vs. volume
29 sc = axes[1].scatter(
30     df["visit_count"] / 1e6,
31     df["avg_engagement"],
32     s=120, c=df["visit_count"] / 1e6,
33     cmap="Blues", alpha=0.85, edgecolors="k", linewidths=0.5
34 )
35 for _, row in df.iterrows():
36     axes[1].annotate(row["page_url"].split("/")[-1] or "home",
37                     (row["visit_count"]/1e6, row["
38                         avg_engagement"]]),
39                     textcoords="offset points", xytext=(5, 3),
40                     fontsize=8)
41 axes[1].set_xlabel("Visit count (millions)")
42 axes[1].set_ylabel("Avg engagement score")
43 axes[1].set_title("Engagement vs.\ Volume by Page")
44 axes[1].grid(alpha=0.3)
45
46 plt.tight_layout()
47 plt.savefig("page_analytics.png", dpi=150)
48 print("Saved: page_analytics.png")
```

Listing 7.10: PE7.3b: Visualise URL frequency from Hive results

Table 7.3: SQL-on-Hadoop engine comparison

Engine	Latency	Scale	Fault Tol.	Data sources	Best use case
Hive/MapReduce	Hours	Unlimited	Full	HDFS (ORC/Parquet/CSV)	Multi-TB batch ETL
Hive/Tez	Minutes	Unlimited	Full	HDFS	Complex batch analytics
Hive/LLAP	Seconds	Large	Full	ORC/Parquet on HDFS	BI dashboards; repeated queries
Presto/Trino	Seconds	RAM-bound	Query-fail	Federated (Hive, JDBC, Kafka, . . .)	Ad-hoc interactive analytics
Spark SQL	Seconds	Large	Partial	HDFS, S3, JDBC, Kafka	ML pipelines; unified batch+stream
Apache Impala	Sub-second	RAM-bound	Query-fail	HDFS, Kudu	Fastest interactive on Parquet

Part V

Data Pipelines and Modern Integration

Data Pipelines, Data Lakes, and Real-Time Integration

“I believe that a unified log is the central data structure that can solve the coherency problem in a distributed, heterogeneous data system — a structure so simple that we take it for granted, and so fundamental that we underestimate it.”

Jay Kreps, *The Log: What Every Software Engineer Should Know About Real-Time Data's Unifying Abstraction*, LinkedIn Engineering, 2013

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why traditional point-to-point ETL produces an $O(N^2)$ integration complexity problem and describe how a central message bus reduces this to $O(N)$.
2. Compare the five major data integration tools—Apache Sqoop, Apache Flume, Apache Kafka, Apache NiFi, and Spark Streaming—on their primary use case, data direction, and throughput characteristics.
3. Describe how Apache Sqoop uses MapReduce to parallelise bulk data transfer between an RDBMS and HDFS, and configure an in-

- cremental import with `-incremental append`.
4. Explain the three components of an Apache Flume agent—Source, Channel, and Sink—and distinguish between Memory Channel and File Channel in terms of durability guarantees.
 5. Define the five core Kafka concepts—Topic, Partition, Offset, Broker, and Consumer Group—and explain how they combine to deliver durability, parallelism, and exactly-once-capable messaging.
 6. Explain how Kafka achieves high throughput through sequential disk writes, zero-copy transfer via the Linux `sendfile()` system call, producer batching, and partition-level parallelism.
 7. Describe the three layers of the Lambda architecture—Batch Layer, Speed Layer, and Serving Layer—and identify its primary engineering disadvantage: the dual-code-path maintenance burden.
 8. Explain how the Kappa architecture eliminates the batch layer by using Kafka log replay for historical reprocessing, and identify workloads for which Kappa is *not* a viable replacement for Lambda.
 9. Describe the four-layer architecture of a data lake—Ingestion, Storage, Processing, and Catalogue/Governance—and explain what turns a data lake into a data swamp.
 10. Explain how table formats such as Delta Lake and Apache Iceberg add ACID transaction semantics to object-store data lakes, and describe the role of the transaction log in enabling time-travel queries.

Chapter 6 introduced Apache Spark as an in-memory distributed compute engine and Chapter 7 extended Spark SQL into the interactive analytics domain with Hive, Presto, and LLAP. Both chapters addressed *how to process* data that is already resident in HDFS or an object store. This chapter asks the prior question: *how does data arrive there in the first place, and how can we ensure it arrives continuously, at low latency, and with sufficient quality to be trusted downstream?*

The shift from nightly batch ETL to continuous data pipelines is one of the most consequential architectural transitions in enterprise data engineering. That transition was enabled by a deceptively simple idea: replace the web of point-to-point connections between systems with a single, durable,

append-only log. Apache Kafka, introduced by LinkedIn engineers in 2011, is the canonical implementation of that idea, and it has become the backbone of modern event-driven architecture at companies ranging from Netflix and Uber to Grab and Revolut.

The chapter covers the full pipeline picture: bulk batch transfer with Sqoop and Flume; real-time streaming with Kafka; the two competing system architectures—Lambda and Kappa—that organise batch and stream processing into coherent pipelines; and the data lake as the storage substrate that absorbs all of this data at petabyte scale. The chapter closes with the governance problem that makes data lakes fail in practice, and a preview of the table formats (Delta Lake, Apache Iceberg) that are beginning to solve it.

8.1 From Batch ETL to Streaming Pipelines

For two decades, the standard approach to moving data between enterprise systems was the *ETL pipeline*: a scheduled job that **Extracts** data from a source system, **Transforms** it into the target schema, and **Loads** it into a destination (typically a data warehouse). ETL pipelines are straightforward to build and reason about—a single job has a defined source, a defined transformation, and a defined target—but they scale poorly as the number of systems in an organisation grows.

8.1.1 The N^2 Integration Problem

In a point-to-point ETL architecture, each pair of systems that needs to exchange data requires a dedicated ETL job. With N systems, the number of direct connections in the worst case is $N(N - 1)/2 = O(N^2)$.

A bank with 50 operational systems (core banking, CRM, risk management, fraud detection, mobile wallet, settlement, audit, compliance. . .) may require up to $50 \times 49/2 = 1,225$ separate ETL jobs. This creates four interconnected problems:

1. **Brittleness:** When the schema of source system A changes, every ETL job that reads from A must be updated simultaneously. A column rename cascades into a multi-day incident.

2. **No replay:** Traditional ETL jobs delete data from the source queue after consumption. If a downstream system is offline when a job runs, that data is lost unless the ETL job is re-run from scratch.
3. **High latency:** All jobs are scheduled nightly (or hourly at best). Real-time fraud detection, live dashboards, and event-driven personalisation are architecturally impossible in this model.
4. **Tight coupling:** Source and consumer are tightly bound. Adding a new consumer of an existing source requires modifying or duplicating the source-side ETL job.

Key Insight | The Event Bus Reduces $O(N^2)$ to $O(N)$

Replacing direct ETL connections with a central *event bus* eliminates the coupling between producers and consumers. Each of the N systems connects to the bus once: producers write events to named topics; consumers subscribe to the topics they care about. Adding a new consumer requires **zero changes** to any producer. The total number of connections is $2N$ (one publish connection and one subscribe connection per system), reducing integration complexity from $O(N^2)$ to $O(N)$.

The logical architecture of a central event bus is shown in Figure 8.1.

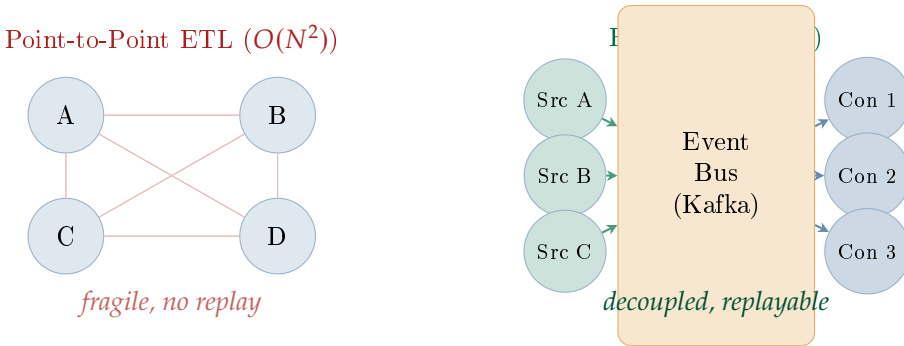


Figure 8.1: Point-to-point ETL (left) versus a central event bus (right). With $N = 4$ systems, point-to-point requires up to six direct connections; the event bus requires only $2N = 8$ connections, and each source is oblivious to the number of consumers.

8.1.2 The Data Integration Tool Landscape

The transition from batch ETL to streaming pipelines did not happen overnight: a family of tools emerged to fill specific niches on the spectrum between pure batch and pure stream processing.

Tool	Direction	Type	Primary Use Case
Apache Sqoop	RDBMS ↔ HDFS	Batch	Nightly bulk transfer of data between relational tables
Apache Flume	Logs → HDFS	Batch/micro-batch	Web server log ingestion; multi-source aggregation
Apache Kafka	Any → Any	Real-time stream	Centralised event bus for fraud, clickstream, IoT
Apache NiFi	Any → Any	Visual pipeline	No-code data routing from IoT edge to cloud
Spark Streaming	Kafka → HDFS/DB	Stream processing	Stateful aggregation; enrichment; stream joins

Rule of thumb: Sqoop = bulk DB sync; Flume = log collection; Kafka = real-time events; NiFi = visual pipeline

Table 8.1: Data integration tool landscape. No single tool is optimal for all scenarios; the correct choice depends on data direction, latency requirement, and operational complexity budget.

8.2 Batch Integration: Apache Sqoop and Apache Flume

Before real-time messaging systems like Kafka were available, two Hadoop-ecosystem tools dominated data ingestion: Apache Sqoop for bulk database transfer and Apache Flume for log collection. Both remain in production at organisations with legacy Hadoop deployments, and understanding their design illuminates why Kafka eventually displaced them for most use cases.

8.2.1 Apache Sqoop: Bulk RDBMS–HDFS Transfer

Apache Sqoop (*SQL-to-Hadoop*) is a command-line tool that transfers data in bulk between a relational database—MySQL, Oracle, PostgreSQL, SQL Server, or any JDBC-compliant store—and HDFS (or Hive, HBase). Its design insight is deceptively simple: convert a table scan into a MapReduce job, where each Map task reads a non-overlapping range of rows from the database in parallel using JDBC.

Import pipeline. When a Sqoop import command is submitted, Sqoop:

1. Connects to the database via JDBC and queries the table metadata (column names, types, primary key range).
2. Determines the number of mappers (m , default 4). It divides the primary key range $[k_{\min}, k_{\max}]$ into m equal-width sub-ranges: mapper i reads rows where $k \in [k_{\min} + (i - 1) \cdot \delta, k_{\min} + i \cdot \delta)$.
3. Submits a MapReduce job. Each Map task opens its own JDBC connection, executes the bounded SELECT, and writes its partition to HDFS as CSV, Avro, Parquet, or SequenceFile.
4. There is no Reduce phase: each mapper writes an independent part file, and HDFS assembles them into the output directory.

```
# Full import: entire table to HDFS as Parquet, 8 parallel
mappers
sqoop import \
  --connect jdbc:mysql://db-host:3306/finance \
  --username analyst \
  --password-file /user/analyst/.sqoop-pass \
  --table transactions \
  --as-parquetfile \
  --num-mappers 8 \
  --target-dir /data/raw/transactions \
  --delete-target-dir

# Incremental import: only rows newer than the last checkpoint
# Use this for nightly appends to an existing HDFS dataset
sqoop import \
  --connect jdbc:mysql://db-host:3306/finance \
  --username analyst \
  --password-file /user/analyst/.sqoop-pass \
  --table transactions \
  --as-parquetfile \
```

```
--num-mappers 4 \
--target-dir /data/raw/transactions \
--incremental append \
--check-column txn_timestamp \
--last-value '2024-06-11 23:59:59'
```

Listing 8.1: Sqoop: full import and incremental import**Definition | Sqoop Incremental Import**

An **incremental import** transfers only rows added since the previous run, identified by a monotonically increasing column (`-check-column`) and a checkpoint value (`-last-value`). Two modes are supported: `append` (new rows, identified by value greater than the checkpoint) and `lastmodified` (rows whose modification timestamp exceeds the checkpoint). Sqoop saves the new checkpoint to a *saved job* so the next run picks up where the previous left off.

Export pipeline. Sqoop can also export data *from* HDFS *to* a relational database—for example, after a Spark batch job produces a summary table on HDFS, Sqoop exports it back to a MySQL reporting database for a business intelligence tool to query.

```
sqoop export \
--connect jdbc:mysql://db-host:3306/reports \
--username loader \
--password-file /user/loader/.sqoop-pass \
--table daily_summary \
--export-dir /data/processed/daily_summary \
--input-fields-terminated-by '\t' \
--update-mode allowinsert \
--update-key summary_date,region
```

Listing 8.2: Sqoop: export from HDFS to MySQL**Production Lesson | Why Sqoop Is Being Retired in Modern Stacks**

Sqoop's dependency on MapReduce makes it incompatible with YARN-free deployments and cloud-native object stores where MapReduce locality is irrelevant. Its JDBC-per-mapper approach hammers source databases with concurrent connections, often requiring a dedicated read replica. In modern data platforms, Kafka Connect with a JDBC

source connector has replaced Sqoop for most use cases: it supports streaming incremental ingestion (capturing changes via CDC), requires no MapReduce infrastructure, and integrates with Kafka’s replay semantics. Sqoop reached End of Life in 2021 and is no longer receiving Apache releases, though it remains in production at many Hadoop-era deployments.

8.2.2 Apache Flume: Log Ingestion Pipelines

Apache Flume is a distributed, reliable service for streaming log data from multiple sources into HDFS. Its primary use case is collecting semi-structured logs—web server access logs, application error logs, IoT sensor readings—and storing them in HDFS for subsequent batch analysis.

The Flume agent model. The fundamental unit of Flume is the *agent*: a JVM process with three components.

Definition	Flume Agent: Source, Channel, Sink
A Flume agent consists of three components that form a directed pipeline: • Source: receives events from an external system. Types include Exec source (<code>tail -F logfile</code>), Syslog source (UDP/TCP syslog), Avro source (from another Flume agent), and HTTP source (REST push). • Channel: buffers events between Source and Sink. <i>Memory Channel</i> (fast, non-durable) or <i>File Channel</i> (writes to local disk; survives agent restart at the cost of throughput). • Sink: writes buffered events to the destination. HDFS Sink (writes Avro or text files, rolling by time or size), Kafka Sink (forwards events to a Kafka topic), HBase Sink. Events flow Source → Channel → Sink. The channel decouples source read speed from sink write speed, absorbing bursts.	

Multiple agents can be chained together: a *fan-in* topology aggregates logs from many web servers into a single *collector agent* before writing to HDFS, reducing the number of small files that would result from each server writing directly to HDFS.


```
# flume-agent.conf
# Source: tail the nginx access log
agent.sources = nginx_src
agent.channels = file_channel
agent.sinks = hdfs_sink

agent.sources.nginx_src.type = exec
agent.sources.nginx_src.command = tail -F /var/log/nginx
    /access.log
agent.sources.nginx_src.channels = file_channel

agent.channels.file_channel.type = file
agent.channels.file_channel.checkpointDir = /var/flume/
    checkpoint
agent.channels.file_channel.dataDirs = /var/flume/data

agent.sinks.hdfs_sink.type = hdfs
agent.sinks.hdfs_sink.hdfs.path = hdfs://namenode:9000/
    logs/%Y/%m/%d
agent.sinks.hdfs_sink.hdfs.fileType = DataStream
agent.sinks.hdfs_sink.hdfs.rollInterval= 300
agent.sinks.hdfs_sink.hdfs.rollSize = 134217728
agent.sinks.hdfs_sink.channel = file_channel
```

Listing 8.3: Flume: agent configuration for web log collection**Common Misconception | Small Files and the HDFS Namespace Tax**

Flume's rolling policy controls how often a new HDFS file is created. A short roll interval (e.g., every 10 seconds) produces thousands of small files per day, each consuming a NameNode metadata entry and degrading NameNode heap. Best practice is to use a roll interval of 300–600 seconds and a roll size of 128 MB (one HDFS block), so each rolled file is exactly one block. Alternatively, a downstream compaction job (Spark or Hive `INSERT OVERWRITE . . . SELECT . . .`) merges small files nightly into large Parquet files.

System Design Note | Flume Fan-In: Aggregating Logs from Many Servers

A typical Flume deployment for a web application with 20 application servers uses two tiers: twenty *edge agents* (one per server) each with an Exec or Syslog source and a Memory Channel writing to an Avro Sink;

and two *collector agents* each with an Avro Source (receiving from 10 edge agents) and a File Channel writing to an HDFS Sink. The collector tier provides durability (File Channel survives collector restart) and aggregation (reducing 20 HDFS writers to 2, greatly reducing small file pressure). This fan-in topology is specified entirely in the Flume agent configuration files—no code is required.

8.3 Apache Kafka: The Distributed Append-Only Log

Apache Kafka was created by Jay Kreps, Neha Narkhede, and Jun Rao at LinkedIn in 2010–2011 to solve the N^2 integration problem that LinkedIn faced with its rapidly growing activity data pipeline. It was open-sourced to the Apache Software Foundation in 2012 and has since become the dominant event streaming platform in the industry.

Kafka’s central innovation is not a clever algorithm or a novel hardware optimisation: it is a *rethinking of what a message broker should be*. Traditional message queues (ActiveMQ, RabbitMQ) model messages as transient objects: once a consumer acknowledges receipt, the message is deleted. Kafka models messages as *records in a durable, immutable, ordered log*: they are retained for a configurable period regardless of whether they have been consumed, and any consumer can read them at any time by specifying an offset.

8.3.1 The Core Log Abstraction

A *log*, in the computer science sense, is the simplest possible storage abstraction: an append-only, totally ordered sequence of records. Every database transaction log, every write-ahead log, every file system journal is a log in this sense. Kafka’s insight is that this same abstraction—which databases use for durability—can serve as the backbone of an entire distributed data infrastructure.

Definition | Kafka Log Abstraction

In Kafka, a **log** is a durable, append-only sequence of records. Each record has:

- A **key** (optional): a byte array used to determine partition assignment (all records with the same key go to the same partition, preserving key-level ordering).
- A **value**: the payload bytes (serialised JSON, Avro, Protobuf, or raw bytes).
- A **timestamp**: producer-assigned or broker-assigned event time.
- An **offset**: a monotonically increasing integer assigned by the broker that uniquely identifies the record within its partition.

Records are **never modified or deleted** within the retention period. The log is the single source of truth; all consumers derive their view from it by tracking their own read offset.

Figure 8.2 contrasts the traditional queue model with Kafka’s log model. In a traditional queue, a message is removed after it is consumed; a late-joining consumer has no access to past messages. In Kafka, all messages are retained for a configurable period, enabling any consumer—including one that joins weeks after the messages were written—to read from offset 0 and replay the full history.



Figure 8.2: Traditional queue versus Kafka’s append-only log. In the queue model, consumed messages are deleted. In Kafka, all messages are retained; each consumer independently tracks its read offset and can replay history by resetting to any past offset.

8.3.2 Kafka Architecture: Topics, Partitions, Offsets, and Brokers

Kafka’s architecture has five foundational concepts that work together to deliver durability, parallelism, and horizontal scalability.

1. **Topic.** A *topic* is a named, durable, append-only log for a category of events. Topics are analogous to database tables but append-only: payment-events, rideshare-gps-pings, and city-weather-sensors might each be a Kafka topic. Producers write to a topic; consumers read from it.
2. **Partition.** Each topic is split into $P \geq 1$ ordered *partitions*. A partition is an independent ordered log residing on a single broker. Partitions provide two things: *parallelism* (multiple producers can write to different partitions simultaneously; multiple consumers in a group can each process a distinct partition) and *key-based ordering* (all messages with the same key are routed to the same partition via key hashing, guaranteeing that a given customer’s transactions are always read in order).
3. **Offset.** Each message within a partition has a unique, monotonically increasing integer *offset*—its position in the partition log. Offsets within a partition are dense and ordered; they are not globally unique across partitions. Each consumer independently tracks its read offset; the broker does not maintain per-consumer state (offsets are stored by consumers in a special Kafka topic named `__consumer_offsets`).
4. **Broker.** A *broker* is a Kafka server process that stores partitions on its local disk and serves producers and consumers over the Kafka wire protocol. A Kafka cluster contains B brokers. Each partition has one *leader replica* (handles all reads and writes) and $R - 1$ *follower replicas* (copy data from the leader asynchronously; stand in as leader on failure). The default replication factor is $R = 3$, tolerating the simultaneous failure of any two brokers without data loss.
5. **ZooKeeper / KRaft.** Kafka versions prior to 3.0 used Apache ZooKeeper for cluster metadata and leader election. Kafka 3.0 introduced KRaft (Kafka Raft), a self-managed metadata quorum built directly into the Kafka brokers, removing the ZooKeeper dependency and simplifying deployment.

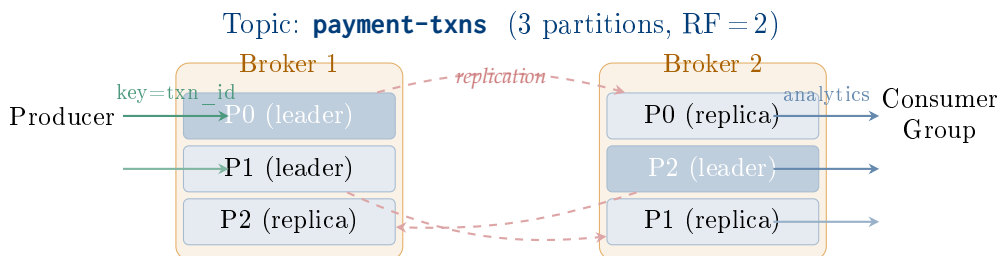


Figure 8.3: Kafka cluster with two brokers, one topic, three partitions, and replication factor 2. Partition leaders handle all reads and writes; followers maintain copies for fault tolerance. If Broker 1 fails, Broker 2 promotes its P0 and P1 replicas to leaders.

8.3.3 Producers, Consumers, and Consumer Groups

A *Kafka producer* is any application that publishes messages to a topic. The producer selects the target partition either by hashing the message key ($\text{hash}(\text{key}) \% P$) or by round-robin if no key is specified. The producer batches messages in memory and sends a batch to the broker's partition leader in a single network request, trading latency for throughput via the `batch.size` and `linger.ms` configuration parameters.

A *Kafka consumer* is any application that reads messages from one or more topics by polling the broker and advancing its read offset. The consumer never *pushes* messages back to acknowledge deletion; it simply records that it has processed up to a given offset.

Consumer groups are the key mechanism for parallel consumption. A consumer group is a named set of consumer processes that collectively consume all partitions of a topic. Kafka's partition-assignment guarantee ensures that each partition is assigned to *exactly one consumer* in the group at any point in time. This means:

- Adding consumers to a group (up to P consumers) increases parallelism proportionally.
- Two independent applications (e.g., a fraud-detection service and an analytics service) reading the same topic use *different* consumer groups. Each group maintains its own set of offsets, so both applications receive every message independently.
- When a consumer joins or leaves the group, Kafka triggers a *rebalance*:

partitions are reassigned among the remaining consumers. Modern Kafka (2.4+) uses cooperative incremental rebalancing to minimise the disruption.

```
1 import json, random, time
2 from kafka import KafkaProducer, KafkaConsumer
3
4 BROKER = "localhost:9092"
5 TOPIC = "payment-events"
6
7 # ----- Producer -----
8 producer = KafkaProducer(
9     bootstrap_servers=BROKER,
10     key_serializer=lambda k: k.encode(),
11     value_serializer=lambda v: json.dumps(v).encode(),
12     # batch.size=16384 (16 KB), linger.ms=5 -- trade latency
13     # for throughput
14     batch_size=16384,
15     linger_ms=5,
16     acks='all',          # wait for all in-sync replicas
17     retries=5,
18 )
19 for i in range(10_000):
20     txn = {
21         "txn_id": f"TXN{i:08d}",
22         "sender": random.randint(1, 50_000),
23         "receiver": random.randint(1, 50_000),
24         "amount": round(random.uniform(10, 5000), 2),
25         "currency": "USD",
26         "timestamp": int(time.time() * 1000),
27     }
28     # key = str(sender): ensures all transactions from the same
29     # sender land in the same partition, preserving send-order
30     producer.send(TOPIC, key=str(txn["sender"]), value=txn)
31
32 producer.flush() # block until all buffered records are sent
33 print("Produced 10,000 transactions")
34
35 # ----- Consumer (fraud-detection group) -----
36 consumer = KafkaConsumer(
37     TOPIC,
38     bootstrap_servers=BROKER,
```

```

39     group_id="fraud-detection-v1",
40     auto_offset_reset="earliest",          # start from offset 0
        if new_group
41     enable_auto_commit=True,
42     auto_commit_interval_ms=5000,
43     value_deserializer=lambda m: json.loads(m.decode()),
44     key_deserializer=lambda k: k.decode() if k else None,
45 )
46
47 for record in consumer:
48     txn = record.value
49     # Simple heuristic: flag large transactions for review
50     if txn["amount"] > 4500:
51         print(f"[ALERT] Large txn {txn['txn_id']}: "
52               f"USD {txn['amount']:.2f} "
53               f"(partition={record.partition}, offset={record.
                    offset})")

```

Listing 8.4: Kafka producer and consumer in Python (kafka-python)

8.3.4 Kafka Throughput: Sequential I/O and Zero-Copy Transfer

The Kreps et al. (2011) paper reports Kafka producer throughput of 505,000 messages/sec—roughly 11× that of ActiveMQ and 3.4× that of RabbitMQ on equivalent hardware. Four engineering decisions explain this performance advantage.

1. **Sequential disk writes.** Kafka never updates messages in place. Partition logs are written by appending to the active segment file. Sequential writes on a spinning hard disk achieve 500–600 MB/s; random writes to the same disk achieve only 50–100 MB/s due to seek latency. Kafka exploits this difference fully. The OS page cache prefetches the next disk blocks automatically, making Kafka’s effective write speed close to raw disk bandwidth even without SSDs.
2. **Zero-copy transfer.** When a consumer fetches messages, a traditional broker copies data along the following path: disk → kernel buffer → user-space buffer → socket buffer → NIC (four copies, two context switches). Kafka uses the Linux `sendfile()` system call to transfer data from the kernel buffer directly to the NIC, bypassing the user-space copy entirely (two copies, zero context switches). This halves CPU usage for read-

heavy workloads.

3. **Batch compression.** Producers compress a *batch* of messages as a single unit (Snappy, LZ4, Zstandard, or GZIP) before transmitting to the broker. The broker stores the compressed batch without decompressing it; consumers decompress on receipt. For structured log data, batch compression ratios of 5–10× are typical, reducing both network and disk I/O proportionally.
4. **Partition parallelism.** Throughput scales linearly with partition count: P partitions allow P parallel producer threads and up to P consumers in a group to operate simultaneously. Increasing partitions from 1 to 32 increases end-to-end throughput by a factor approaching 32, bounded by broker and network capacity.

Key Insight | Kafka's Disk Is Faster Than Memory for Sequential I/O

A common assumption is that memory-based messaging (e.g., keeping all messages in RAM) is faster than disk-based messaging. This is true for *random* access. For the *sequential* access pattern that Kafka's append-only log produces, the OS page cache makes sequential disk access *faster than a heap-allocated data structure* at large scale: sequential disk reads saturate the memory bus rather than L3 cache, and the page cache is managed by the kernel's LRU policy which is better tuned than a custom in-process cache for this workload.

8.3.5 Retention, Replay, and Exactly-Once Semantics

Retention policy. Kafka retains messages for a configurable period (`retention.ms`, default 7 days) or up to a configurable total log size (`retention.bytes`). When both conditions are reached, the oldest segment files are deleted. A consumer that has not read a partition before its messages expire will miss those messages—a design choice that prioritises throughput over guaranteed delivery to slow consumers.

Replay. Because offsets are persistent and messages are not deleted on consumption, any consumer can reset its offset to any point in the retention window and re-read messages. This enables:

- **Backfilling:** a new downstream service can be launched and immedi-

ately process historical data without re-ingesting from the source.

- **Bug recovery:** if a consumer has a bug that corrupts its state, it can reset to the last known-good offset and reprocess.
- **Multi-consumer independence:** a second consumer group reading the same topic starts fresh at offset 0 without affecting the first group's offset position.

Exactly-once semantics. Kafka 0.11 (2017) introduced two mechanisms for exactly-once message delivery:

- **Idempotent producer.** The producer assigns a monotonically increasing sequence number to each batch. If the broker receives a duplicate batch (e.g., due to a network retry), it deduplicates using the sequence number. This prevents duplicate writes caused by producer retries.
- **Transactional API.** A producer can open a transaction, send messages to multiple partitions, and either commit (making all messages visible atomically) or abort (rolling back). Combined with `isolation.level=read_committed` on the consumer side, this provides exactly-once semantics across the full produce-consume-produce pipeline.

Production Lesson | Schema Governance at Scale: The Schema Registry

At small scale, Kafka consumers can deserialize message bytes using any format they agree upon with producers. At scale—4,000 topics and 400 microservices, as at Netflix—schema evolution breaks consumers: a producer that adds a field to its Avro schema without warning causes all consumers to crash on the next schema change.

The Confluent Schema Registry (open-source) solves this by registering every Avro, Protobuf, or JSON Schema version under a subject (typically `{topic}-value`). Producers embed the schema ID (an integer) in each message; consumers look up the schema by ID on receipt. The Registry enforces a *compatibility policy* (BACKWARD, FORWARD, FULL) that rejects schema changes which would break existing consumers. At Netflix and LinkedIn, deploying a schema registry is considered non-negotiable for production Kafka usage.

8.4 Lambda Architecture

As real-time stream processing systems matured alongside batch processing frameworks, practitioners faced a design dilemma: how to serve queries that require both low latency (seconds) *and* high accuracy (based on complete historical data), when stream processors are fast but approximate and batch processors are accurate but slow?

Nathan Marz formulated the *Lambda architecture* in 2011 as a systematic answer to this dilemma. The name is drawn from the Greek letter λ , reflecting the three-way branching of the data path.

8.4.1 The Three-Layer Design

Definition	Lambda Architecture
------------	---------------------

The **Lambda architecture** is a data pipeline pattern that serves both accurate historical batch views and low-latency real-time views from the same input data, without choosing one at the expense of the other. It has three layers:

1. **Batch Layer:** stores the complete, immutable master dataset (typically on HDFS or S3). Periodically recomputes *batch views* by running MapReduce, Spark, or Hive jobs over the entire dataset. Output is *accurate* but *high-latency*: views are 1 hour to 24 hours stale.
2. **Speed Layer:** processes new data arriving since the last batch run using a stream processor (Kafka + Spark Streaming or Flink). Produces *real-time views* covering the most recent window. Output is *low-latency* (<1 second to seconds) but may be *approximate* (bounded by available state in the stream processor).
3. **Serving Layer:** merges batch views and real-time views to answer user queries. A query result equals *batch view (up to N hours ago)* $\cup$ *real-time view (last N hours)*. Apache Druid, Cassandra, or a custom merge index typically implements the serving layer.

8.4.2 Strengths and the Dual-Code-Path Problem

The Lambda architecture has four significant strengths:

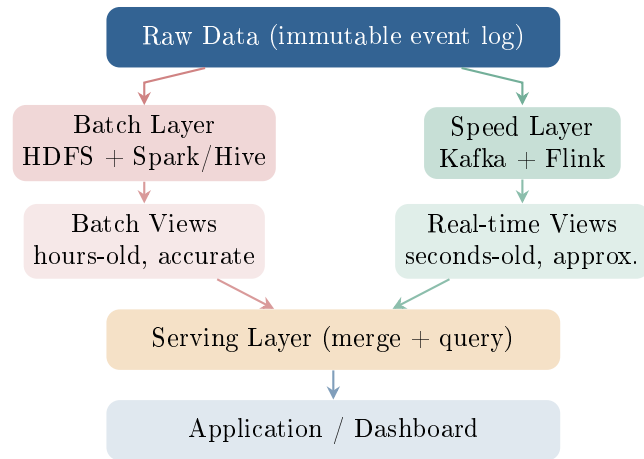


Figure 8.4: Lambda architecture. Raw data flows into both the batch layer (for accurate periodic recomputation) and the speed layer (for low-latency real-time views). The serving layer merges both view types to answer queries.

- **Accuracy from immutability.** The batch layer always recomputes from the raw, immutable dataset. A bug in a past batch job can be corrected by fixing the logic and re-running the job—the historical truth is never lost.
- **Low latency for recent data.** The speed layer delivers query results for the most recent window within seconds, regardless of how large the historical dataset is.
- **Fault tolerance.** No derived view is irreplaceable: batch views and real-time views can both be recomputed from the master dataset or from Kafka’s retained log.
- **Separation of concerns.** Batch and speed components can be built with the most appropriate tools for each: Hive for batch SQL, Flink for streaming CEP, for example.

The Lambda architecture’s critical weakness is the **dual-code-path problem**: the same business logic must be implemented *twice*—once in the batch layer (SQL or Spark batch) and once in the speed layer (streaming SQL or Flink operators).

Common Misconception | The Dual-Code-Path Maintenance Burden

Consider a data engineering team computing “daily active users per region” for a fraud monitoring system. In a Lambda deployment this requires:

1. A Spark batch job (running nightly) that aggregates the full historical table and writes batch views for periods up to midnight.
2. A Flink streaming job that maintains a rolling count of active users from midnight to now (the real-time view).

When the definition of “active user” changes (e.g., the business adds a minimum session-duration threshold), *both jobs must be updated simultaneously*. Industry reports indicate that 30–40% of data engineering effort in Lambda deployments is spent keeping the two implementations in sync—a tax that grows with the number of metrics and the frequency of business logic changes.

8.5 Kappa Architecture

In 2014, Jay Kreps (Kafka co-creator) proposed the *Kappa architecture* as a simplification of Lambda. The core argument: if a stream processing system is powerful enough to reprocess historical data at the same rate as live data, the batch layer is unnecessary.

8.5.1 Stream-Only Processing and Log Replay

Definition | Kappa Architecture

The **Kappa architecture** replaces Lambda’s three-layer design with two components:

1. **Log storage** (Kafka with long-term retention, or an object store like S3 with Apache Iceberg providing table-level access): stores the complete immutable event history. This replaces the HDFS batch layer.
2. **Stream processing engine** (Apache Flink or Spark Structured

Streaming): the *single code path* that handles both live events and historical replay. To reprocess history, a new consumer reads from Kafka offset 0 and processes events in order until it catches up to the live stream.

The **reprocessing workflow** when business logic changes:

1. Deploy a new version of the stream job alongside the current version (they write to different output topics or tables).
2. Start the new job at Kafka offset 0; it reprocesses the full retained history.
3. When the new job catches up to the live stream, atomically swap the serving layer to read from the new output.
4. Shut down the old version.

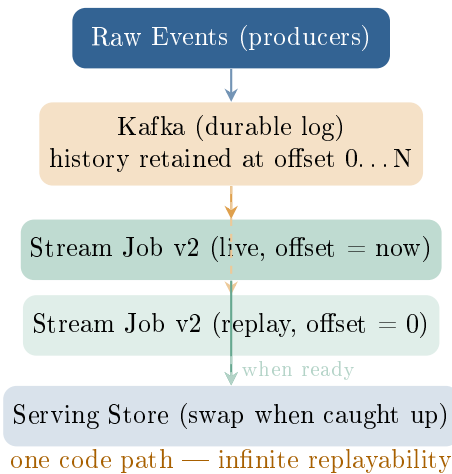


Figure 8.5: Kappa architecture. A single stream processing job handles both live events and historical replay. When business logic changes, a new job version is started at offset 0, reprocesses history, and atomically replaces the old serving store.

Property	Lambda	Kappa
Code paths	Two (batch + stream), must stay in sync	One (stream only)
Historical reprocessing	Re-run batch job on HDFS	Replay Kafka log from offset 0
Latency	Low (speed layer) + High (batch layer)	Uniformly low
Complexity	High: two systems, merge serving layer	Lower: single pipeline
Batch throughput	Very high (Spark/Hive, full dataset in parallel)	Bounded by stream engine throughput
Long-term storage cost	HDFS (commodity storage, dense packing)	Kafka retention (more expensive per GB than HDFS)
Fault tolerance	Disk-based; individual task retries	Checkpoint-based; replay on failure
Best when	Business logic is too complex for streaming SQL; regulatory batch reconciliation required; training datasets for ML must be reproducible	Stream engine is fast enough to reprocess history; single code path is worth the stream-storage cost; schema/logic evolves frequently

Table 8.2: Lambda versus Kappa architecture. Neither is universally superior; the correct choice depends on the organisation’s latency requirements, operational capabilities, and the pace of business logic change.

8.5.2 Lambda vs. Kappa: Comparison and Selection

Common Misconception | When Kappa Is Not Suitable

Kappa is *not* a universal Lambda replacement. Three scenarios favour Lambda:

1. **Terabyte-scale reprocessing is too slow for the stream engine.** A Flink job reprocessing 5 PB of Kafka history at 1 GB/s takes ~58 days. A Spark batch job over the same data on a 200-node cluster completes in hours.
2. **Batch SQL analytics dominate the workload.** If the primary consumers are data analysts running ad-hoc HiveQL or Spark SQL queries over months of history, a Kafka-backed Kappa architecture cannot serve these workloads without first landing the data to a queryable store—effectively recreating the batch layer.
3. **Regulatory requirement mandates independent batch reconciliation.** EU PSD2 Article 94 and equivalent national payment regulations require that all electronic payment transactions be reconciled in a separate batch system to detect discrepancies between the real-time ledger and the daily settlement—a process that cannot be replaced by a streaming job.

8.6 Data Lakes and the Data Swamp Risk

Both Lambda and Kappa architectures produce large volumes of raw and processed data that must be stored somewhere accessible to the processing layer. The dominant storage abstraction for this purpose in the 2010s was the *data lake*: a centralised repository that stores raw data at any scale and in any format—structured, semi-structured, and unstructured—without requiring upfront schema definition.

The term was coined by James Dixon, then CTO of Pentaho, in 2010, in contrast to the *data mart*: “If you think of a data mart as a store of bottled water—cleansed and packaged and structured for easy consumption—the data lake is a large body of water in a more natural state.”

8.6.1 Data Lake Architecture

A well-designed data lake has four layers that transform raw ingested bytes into query-ready, governed datasets.

Definition	Data Lake Four-Layer Architecture
	Ingestion Layer. Pulls data from all sources and writes raw files to the storage layer without transformation. Tools: Apache Sqoop (RDBMS bulk), Apache Flume (log streaming), Apache Kafka (event streaming), Apache NiFi (visual pipelines), REST API connectors. Storage Layer. Holds all data in its raw and processed forms. Organised into <i>zones</i>: • Raw zone: byte-for-byte copy of source data; never modified after ingest. • Processed zone: cleaned, de-duplicated, schema-normalised data; typically Parquet or ORC. • Curated zone: aggregated, business-ready datasets; typically Hive tables or Delta Lake tables. Underlying storage: HDFS (on-premise) or S3/Azure ADLS (cloud-native). Processing Layer. Transforms and queries data on demand. Tools: Apache Spark (batch ETL and ML), Apache Hive (SQL analytics), Apache Presto/Trino (interactive queries), Apache Flink (stream processing). Catalogue and Governance Layer. Tracks schemas, data lineage, ownership, and access policies across all datasets. Tools: Apache Atlas, AWS Glue Data Catalog, Hive Metastore, Apache Ranger (access control).

8.6.2 The Data Swamp Problem and Governance

A data lake without active governance degrades rapidly into a *data swamp*: a repository where data accumulates without metadata, owners, or quality guarantees, rendering it unusable in practice despite being physically present.

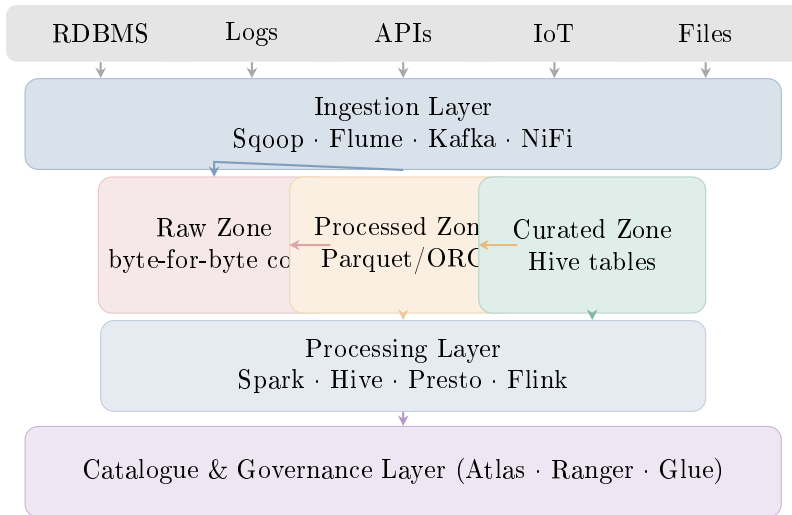


Figure 8.6: Data lake four-layer architecture. Raw data is ingested without schema enforcement, then progressively transformed through raw, processed, and curated zones. The governance layer enforces data quality, lineage tracking, and access control across all zones.

Data swamp symptoms are predictable:

- **Undiscoverability.** Analysts cannot find the dataset they need because there is no searchable catalogue. Directories like `/data/raw/2019/export_v3_FINAL_FINAL` proliferate without documentation.
- **Unknown freshness.** Without a timestamp or freshness SLA in the catalogue, an analyst cannot tell whether `/data/curated/customers` was updated today or six months ago.
- **Schema drift.** Multiple copies of the same dataset accumulated over time with inconsistent column names and types, because different teams ran their own transformations without a shared schema registry.
- **Privacy leakage.** Personally identifiable information (PII) mixes with non-sensitive data because ingest bypassed the governance layer. Under GDPR Article 83(5) and equivalent data-protection frameworks worldwide, this can result in significant regulatory penalties and mandatory breach notification.
- **Unreliable quality.** Without automated quality checks at ingest time, downstream ML models silently train on corrupted or incomplete data.

System Design Note | Data Lake Governance in Practice

Preventing a data swamp requires enforcing governance at *ingest time*, not as a remediation activity. Four controls are foundational:

1. **Mandatory catalogue registration.** Every ingest pipeline must register its output dataset in the data catalogue (Apache Atlas or AWS Glue) with at minimum: owner, source system, update frequency, schema version, and PII classification. Ingest pipelines that fail to register are blocked by a catalogue gateway.
2. **Schema enforcement at the boundary.** Use Apache Avro with a Schema Registry for Kafka-sourced data, and enforce schema compatibility checks before writing to the processed zone. A schema-breaking change triggers a Slack alert to the owning team.
3. **Automated data quality checks.** Deploy Apache Griffin or Great Expectations to run null-rate, outlier, and referential integrity checks on every batch landed in the processed zone. Datasets failing checks are quarantined in a /data/quarantine/ zone and not promoted to curated.
4. **Column-level access control.** Use Apache Ranger to define column-level access policies: analysts see masked customer names and phone numbers; only the compliance team sees raw PII. Row-level security restricts access to regional data.

8.6.3 Table Formats: Delta Lake and Apache Iceberg

Object stores (HDFS, S3) do not natively support ACID transactions: two concurrent Spark jobs writing to the same directory can corrupt each other's output (the *lost-update* problem). Traditional partitioned Hive tables suffer from further limitations: schema evolution requires ALTER TABLE operations that can break existing readers; time-travel (querying the state of a table at a past timestamp) is not supported; and partition discovery is an expensive NameNode scan for tables with thousands of partitions.

Open table formats—specifically Delta Lake (Databricks, 2019) and Apache Iceberg (Netflix, 2020)—solve these problems by adding a *transaction log* layer on top of Parquet files in an object store.

Definition	Transaction Log in Open Table Formats
------------	---------------------------------------

An **open table format** such as Delta Lake or Apache Iceberg maintains a *transaction log*: an ordered list of committed operations (add file, remove file, update schema, change partition spec). Each operation is an atomic JSON or Avro record written to a separate log directory. Properties enabled by the transaction log:

- **ACID writes.** A Spark job writes new Parquet files to a staging location, then atomically commits a “add files” entry to the transaction log. If the job fails, no partial state is visible to readers.
- **Snapshot isolation.** Readers always see a consistent snapshot of the table as of the latest committed transaction; concurrent writers do not interfere.
- **Time travel.** Querying the table AS OF TIMESTAMP '2024-01-01' reads the transaction log backward to find the snapshot valid at that time, then reads the corresponding Parquet files.
- **Schema evolution.** New columns, renamed columns, and changed nullability are recorded in the log and propagated to readers automatically.

```

1 from delta.tables import DeltaTable
2 from pyspark.sql import SparkSession
3 from pyspark.sql.functions import col, current_timestamp
4
5 spark = (SparkSession.builder
6         .appName("DeltaLakeDemo")
7         .config("spark.sql.extensions",
8               "io.delta.sql.DeltaSparkSessionExtension")
9         .config("spark.sql.catalog.spark_catalog",
10               "org.apache.spark.sql.delta.catalog.
11                 DeltaCatalog")
12         .getOrCreate())
13
14 TABLE_PATH = "s3://data-lake/curated/transactions"
15
16 # --- Write (ACID-guaranteed) ---
17 df = spark.read.parquet("s3://data-lake/raw/transactions
18                          /2024-06-28/")
19 df.write

```

```

18     .format("delta")
19     .mode("append")
20     .partitionBy("txn_date", "region")
21     .save(TABLE_PATH))
22
23 # --- Merge (upsert: update existing rows, insert new ones) ---
24 target = DeltaTable.forPath(spark, TABLE_PATH)
25 updates = spark.read.parquet("s3://data-lake/raw/corrections/")
26
27 (target.alias("t")
28     .merge(updates.alias("u"),
29           "t.txn_id = u.txn_id")
30     .whenMatchedUpdateAll()
31     .whenNotMatchedInsertAll()
32     .execute())
33
34 # --- Time travel: read the table as it was yesterday ---
35 yesterday = spark.read.format("delta") \
36     .option("timestampAsOf", "2024-06-27") \
37     .load(TABLE_PATH)
38 yesterday.count()
39
40 # --- Inspect transaction history ---
41 (DeltaTable.forPath(spark, TABLE_PATH)
42     .history()
43     .select("version", "timestamp", "operation", "
44           operationMetrics")
45     .show(10, truncate=False))

```

Listing 8.5: Delta Lake: ACID writes, time travel, and schema evolution

Key Insight | Delta Lake and Iceberg Converge the Lake and Warehouse

The combination of (1) object-store economics (\$0.02/GB/month on S3 vs. \$2–\$5/GB/month for proprietary data warehouse storage), (2) ACID transaction semantics from Delta Lake or Iceberg, and (3) SQL query engines like Presto and Spark SQL creates a new architecture called the *Lakehouse* (Armbrust et al., 2020)—a data lake with the ACID guarantees and performance characteristics of a data warehouse, without the data movement and duplication costs of maintaining separate systems. The Lakehouse is the subject of Chapter 11.

8.7 Research Spotlight

Research Spotlight | Kreps, Narkhede & Rao (2011) — Kafka: A Distributed Messaging S

Full citation: Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Aggregation. In *Proceedings of the NetDB Workshop at VLDB 2011*. Seattle, WA. Cited by >3,500 (Google Scholar, 2024).

Problem context. By 2010, LinkedIn was generating billions of activity events per day (profile views, connection requests, job searches, feed impressions). The existing pipeline—custom ad-hoc processes copying log files over NFS to Hadoop—was fragile, could not replay past data, and could not scale to the growing event volume. Existing message brokers (ActiveMQ, RabbitMQ) were designed for transactional messaging at thousands of messages/sec, not streaming at 100 MB/s per broker with multi-day retention.

Key insight. “A log is the most primitive form of storage. If you model your data infrastructure as a distributed, replicated, append-only log, you get durability, replayability, and decoupling for free.” This insight—that the database write-ahead log primitive could be generalised into a distributed messaging infrastructure—is the paper’s central contribution.

Technical contributions.

1. **Topic-partition-offset data model.** The paper formalises the abstraction of a topic (named log), partition (ordered, independent sub-log for parallelism), and offset (consumer-tracked position). This model provides ordered delivery within a partition, unlimited consumer fan-out (each consumer group tracks its own offsets independently), and log-time ordering without a global coordinator.
2. **Sequential disk I/O and zero-copy.** The paper quantifies the performance advantage of append-only log files over B-tree indexed stores: sequential disk bandwidth on spinning media is 600 MB/s versus 100 MB/s for random access. Kafka’s `sendfile()` zero-copy transfer reduces consumer-side CPU to near zero at peak through-

put. The benchmark reports 505,000 producer msgs/sec and 220,000 consumer msgs/sec on a single broker, outperforming ActiveMQ by 11.7× and 5.5× respectively.

3. **Pull-based consumers.** Unlike push-based brokers that must track per-consumer queue state, Kafka's brokers maintain only the log itself. Consumers pull messages at their own rate; the broker never needs to know which consumers exist or what they have consumed. This eliminates a significant bottleneck: a slow consumer cannot slow down the broker.

Impact. Kafka was open-sourced to the Apache Software Foundation in 2012. By 2021, LinkedIn's Kafka cluster processed 1.8 trillion messages/day across 4,400 brokers; production deployments at Netflix, Uber, Airbnb, Twitter/X, and thousands of enterprises handle aggregate event volumes exceeding 10 trillion messages/day. The paper triggered a rethinking of data infrastructure: Jay Kreps's 2013 essay *The Log* (which extends the paper's ideas) became one of the most widely shared technical articles in the data engineering community. The paper directly inspired the Dataflow model (Akidau et al., 2015) and the Kappa architecture (Kreps, 2014), both covered in Chapter 11.

8.8 Case Study: Netflix Keystone Real-Time Data Pipeline

Case Study | Netflix Keystone Pipeline — 100 Billion Events per Day

Context and scale. Netflix serves 200 million subscribers (as of 2021) across 190 countries. Every user action—play, pause, seek, quality switch, UI click, search query, content impression—is recorded as an event. At 1.3 million events/second at peak (Friday at 9 pm US Eastern) and 100 billion events/day at average load, Netflix's data infrastructure operates at a scale few organisations approach. The Keystone pipeline is the internal name for the real-time event ingestion and routing system that processes all of these events before they reach analytics, personalisation, A/B testing, and operational monitoring systems.

Problem. Before Keystone, Netflix used a custom log-based pipeline where applications wrote events to local disk, a daemon collected them and pushed to a queue, and batch jobs processed the queue nightly. This pipeline could not support real-time personalisation (the home screen must reflect what a user watched 10 minutes ago, not yesterday) or same-day A/B test results.

Architecture.

Netflix's Keystone architecture is a hybrid Lambda-Kappa system:

Layer	Tool	Latency	Output
Event bus	Apache Kafka (4,000+ topics)	<100 ms	All raw events
Speed layer	Apache Flink (150+ jobs)	<1 min	Real-time dashboards
Batch layer	Kafka → S3 → Spark (EMR) + Iceberg	~1 hour–1 day	ML datasets, billing
Serving	Cassandra (user state), Druid (OLAP)	<10 ms	Personalisation, analytics

Every device (TV, mobile, web browser) sends events to Kafka topics via a client SDK. Kafka acts as the *single event bus*: no downstream system receives events directly from devices. Flink jobs subscribe to Kafka topics and compute real-time aggregates (per-user play counts, per-title impression rates, A/B test counters). A Kafka Connect S3 Sink connector writes all raw events to Amazon S3 in Avro format, partitioned by hour. Spark jobs on Amazon EMR read from S3, compute higher-fidelity aggregates using Apache Iceberg ACID tables, and write results to the curated S3 zone.

A/B testing pipeline. Netflix runs >200 concurrent A/B tests at any time—different thumbnail artwork, autoplay delays, recommendation algorithm variants, and UI layouts. Each test requires:

1. An *assignment event* when the user is bucketed into control or treatment (stored in Cassandra keyed by user ID).
2. An *exposure event* logged to Kafka when the user actually sees the tested feature.
3. *Outcome events* (play, click, subscription) logged to Kafka within seconds.

A Flink job performs a three-way event-time window join over the assignments, exposures, and outcomes Kafka topics, computing conversion rates with a 30-second watermark lag to handle late-arriving mobile events. Experiment dashboards are updated every 5 minutes, enabling engineers to ship winning variants the same day instead of waiting for a nightly batch run.

Engineering lessons.

1. **Kafka as the single source of truth eliminates inconsistency.** By having both the speed layer (Flink) and the batch layer (Spark on S3) read from the same Kafka topics, Netflix avoids the classic Lambda problem of two separate ingestion paths diverging. The S3 landing is just a materialised snapshot of the Kafka log.
2. **7-day retention enables operational recovery.** When a Flink job has a bug, engineers reset the consumer group offset to the time before the bug was introduced and reprocess—without contacting source teams or rebuilding input datasets.
3. **Schema Registry is non-negotiable at 4,000 topics.** Netflix co-developed the Confluent Schema Registry and mandates Avro schema registration for every Kafka topic. A schema change that would break downstream consumers is rejected at deployment time.
4. **Backpressure is the production problem, not throughput.** During Super Bowl commercial breaks, event rates spike 5× in 30 seconds. Kafka absorbs the burst; Flink consumers catch up within minutes because they are stateless with respect to the Kafka log. Designing for backpressure—not peak throughput—is the key operational insight.
5. **Iceberg enables schema evolution without downtime.** Netflix’s batch pipeline manages 5,000+ Iceberg tables, many of which evolve weekly. Iceberg’s schema evolution support allows columns to be added or renamed without rewriting existing Parquet files or pausing consuming Spark jobs.

Open-source contributions. Netflix open-sourced Metacat (a unified metadata store and catalogue for Hive, Iceberg, S3, and Druid), and

was a co-developer of Apache Iceberg, both driven by the requirements of the Keystone pipeline. Netflix’s engineering blog posts on Kafka, Flink, and Iceberg (2015–2022) are widely used reference material in the data engineering community.

Chapter Summary

This chapter traced the evolution of data integration from fragile point-to-point ETL to the continuous, event-driven pipelines that underpin modern data infrastructure.

The N^2 coupling problem of direct ETL connections is resolved by a central *event bus*, of which Apache Kafka is the dominant implementation. Kafka’s performance advantage—10× throughput over traditional message brokers—derives from two engineering decisions: sequential-write-only disk access and zero-copy transfer via `sendfile()`. Its durability advantage—consumers can replay any historical window—derives from the append-only log abstraction that treats messages as records in a database, not as transient queue entries.

Two architectural patterns organise batch and stream processing around Kafka: the *Lambda architecture* (Marz, 2011), which serves both accurate batch views and low-latency real-time views at the cost of maintaining two code paths; and the *Kappa architecture* (Kreps, 2014), which eliminates the batch layer by using Kafka log replay for historical reprocessing, at the cost of higher storage requirements and the constraint that the stream processing engine must be capable of reprocessing history at acceptable speed.

Data lakes provide the petabyte-scale storage substrate for pipeline output. Without active governance—schema registration, data quality checks, ownership tracking, and access control—a data lake degrades into a data swamp. Open table formats (Delta Lake, Apache Iceberg) add ACID transaction semantics to object-store data lakes, enabling time travel, schema evolution, and concurrent writer safety. They form the foundation of the Lakehouse pattern explored in Chapter 11.

Exercises

Short Questions

SQ8.1. What is the N^2 problem in point-to-point ETL integration? With 30 systems, how many direct connections are required in the worst case? How does an event bus reduce this number?

SQ8.2. Describe the three components of an Apache Flume agent. What is the difference between a Memory Channel and a File Channel in terms of durability?

SQ8.3. What is the difference between an Apache Kafka *topic* and a Kafka *partition*? Why must messages with the same key be routed to the same partition?

SQ8.4. Explain what a Kafka *offset* is. Who is responsible for tracking offsets in the Kafka data model—the broker or the consumer? What is the benefit of this design?

SQ8.5. What is a Kafka consumer group? If a topic has 6 partitions and a consumer group has 4 consumers, how many partitions does each consumer process (assuming uniform assignment)?

SQ8.6. Explain the Linux `sendfile()` system call and its role in Kafka's consumer throughput. How many memory copies does a traditional message broker require compared to Kafka?

SQ8.7. Describe the Lambda architecture's three layers. What is the purpose of the Serving Layer?

SQ8.8. What is the "dual-code-path problem" in the Lambda architecture? Why does it create a maintenance burden?

SQ8.9. In the Kappa architecture, how is historical reprocessing accomplished? Describe the step-by-step reprocessing workflow when business logic changes.

SQ8.10. What is a data lake? List four symptoms that indicate a data lake has become a data swamp.

SQ8.11. What is an open table format (e.g., Delta Lake or Apache Iceberg)? Name three features that open table formats add to a standard Parquet-on-S3 data lake.

SQ8.12. Compare Apache Sqoop and Apache Kafka on: (a) data direction, (b) latency, (c) replay capability, and (d) the underlying parallelism mechanism.

Analytical Problems

AP8.1. Kafka Partition Sizing. A mobile payment company expects the following workload on its wallet-transactions Kafka topic:

- Peak producer throughput: 80,000 transactions/second
- Average message size: 512 bytes
- Required consumer throughput per partition: 10 MB/s (broker limit)
- Replication factor: 3
- Required retention: 14 days
- Average message rate: 20,000 transactions/second

(a) Calculate the minimum number of partitions required to sustain the peak producer throughput. (b) Calculate the total storage required on the Kafka cluster for 14-day retention (assuming average rate, before replication). (c) Calculate the total storage *after* replication. (d) If each broker node has 12 TB of usable disk, how many brokers are needed?

AP8.2. Lambda vs. Kappa Architecture Selection. An international commercial bank is designing a data pipeline for three workloads:

1. Real-time fraud detection: flag suspicious card-swipe transactions within 200 ms with a rolling 24-hour transaction history per card.
2. Monthly regulatory capital report: aggregates 36 months of transaction data into statutory metrics as required by Basel III / EU CRR capital adequacy reporting rules.
3. Live risk dashboard: shows current total exposure per counterparty updated every 30 seconds, combining live transactions (from the past 5 minutes) with a settled position snapshot from the previous close of business.

For each workload, identify whether Lambda, Kappa, or a different architecture is most appropriate. Justify each choice with reference to latency requirement, reprocessing cost, and regulatory constraints.

AP8.3. Consumer Group Rebalancing. A Kafka topic has 12 partitions. A consumer group starts with 6 consumers, each processing 2 partitions.

- (a) Consumer 3 fails (crashes without a graceful shutdown). Describe what Kafka does: which consumers take over its partitions, and how long does the rebalance take (Kafka's default `session.timeout.ms` is 45 seconds)?
- (b) Two new consumers join the group simultaneously. How are the 12 partitions reassigned under the default `RangeAssignor` strategy?
- (c) A new version of the consumer is deployed with 12 instances (one per partition). What happens if 4 additional instances are accidentally started, making the total 16?

AP8.4. Data Lake Zone Design. Design the zone structure for a regional healthcare data lake that will store:

- Patient records from 80 regional hospitals (structured, RDBMS)
- Doctor appointment audio recordings (unstructured, MP3)
- Lab result CSV uploads from 200 diagnostic centres
- Real-time vital signs from 1,000 ICU monitoring devices (Kafka)

For each data source: (a) identify which ingestion tool to use (Sqoop, Flume, Kafka, NiFi); (b) specify the landing zone (raw, processed, or curated); (c) identify the format in the processed zone; (d) describe which governance controls are mandatory given that patient records are sensitive health data under HIPAA (US) or GDPR Article 9 (EU).

Design Problems

DP8.1. Mobile Money Real-Time Fraud Pipeline. Design a complete Kafka-based data pipeline for a mobile financial service (e.g., M-Pesa Kenya, 30+ million users, 10 million transactions/day) that must:

- (a) Detect fraudulent transactions in real time (<200 ms decision latency)

using velocity checks (more than 5 transactions from the same wallet in 60 seconds).

- (b) Power a live operations dashboard showing total transaction volume and flagged transactions per minute, updated every 30 seconds.
- (c) Provide a daily reconciliation file to the financial regulator containing all flagged transactions and final dispositions (confirmed fraud vs. false positive).
- (d) Allow the fraud detection rules to be updated and back-tested against 30 days of historical transactions without re-ingesting from the source database.

Your design must specify: Kafka topic structure (topic names, partition count, key, retention); the processing framework for each workload (Kafka Streams, Flink, Spark batch); which architecture pattern (Lambda or Kappa) fits each requirement and why; and the serving layer technology for the dashboard and the reconciliation file.

DP8.2. University Data Lake Design. A mid-size university wishes to build a data lake to unify its operational data for academic analytics, student counselling, and research reporting. Data sources include: student RDBMS (Oracle, 50K student records), LMS (Moodle, JSON event logs), library system (MySQL, book borrowing records), exam results (Excel uploads by faculty), and Wi-Fi access logs (syslog, 100K events/day).

- (a) Design the ingestion pipeline for each source.
- (b) Define the zone structure and the schema for the curated zone `student_engagement` table (columns, types, partitioning).
- (c) Describe three data quality checks that must pass before a dataset is promoted from raw to processed zone.
- (d) Identify which datasets contain PII under FERPA (US) or GDPR (EU) and describe the access control policies (who can see what, in what form) to be enforced by Apache Ranger.

Programming Exercises

PE8.1. Kafka Producer and Consumer (Python). Using the `kafka-python`

library and a local single-broker Docker container:

- (a) Implement a producer that generates 50,000 synthetic mobile payment events (fields: `txn_id`, `sender`, `receiver`, `amount_usd`, `timestamp_ms`). Key each message by sender to ensure all transactions from a given sender land in the same partition.
- (b) Implement a consumer in group `analytics-v1` that reads all messages and computes: (i) total transaction volume in USD; (ii) top-10 senders by count; (iii) average transaction amount per minute over the message timestamps.
- (c) Reset the consumer offset to the beginning and re-run the computation. Verify that results are identical (idempotency test).
- (d) Create a second consumer group `fraud-v1`. Verify that both groups independently read all messages from offset 0, with no offset interference between them.

```
1 import json, random, time, uuid
2 from kafka import KafkaProducer
3
4 BROKER = "localhost:9092"
5 TOPIC = "mobile-payments"
6
7 producer = KafkaProducer(
8     bootstrap_servers=BROKER,
9     key_serializer=str.encode,
10    value_serializer=lambda v: json.dumps(v).encode(),
11    acks='all',
12    retries=3,
13 )
14
15 COUNTRY_CODES = ["+1", "+44", "+49", "+33", "+81", "+61"]
16 SENDERS = [f"{random.choice(COUNTRY_CODES)}{random.randint(
17     2000000000, 9999999999)}"
18             for _ in range(1000)]
19
20 for _ in range(50_000):
21     sender = random.choice(SENDERS)
22     event = {
23         "txn_id": str(uuid.uuid4()),
24         "sender": sender,
25         "receiver": random.choice(SENDERS),
```

```

25         "amount_usd": round(random.uniform(1.00, 5_000.00),
26                               2),
27         "timestamp_ms": int(time.time() * 1000),
28     }
29     producer.send(TOPIC, key=sender, value=event)
30 producer.flush()
31 print("Done: 50,000 events produced")

```

Listing 8.6: PE8.1: Kafka producer skeleton

PE8.2. Lambda Architecture Simulator (PySpark). Simulate a Lambda architecture for a ride-sharing app (e.g., Uber or Grab) using PySpark and local files:

- (a) **Batch layer:** Read a 1-million-row Parquet file of historical ride events (columns: ride_id, driver_id, pickup_zone, fare_usd, ride_date). Compute the hourly fare total per pickup zone and write as a batch view Parquet file.
- (b) **Speed layer:** Generate 10,000 synthetic “live” events for the current hour (using `datetime.now()`). Compute the same fare-total-per-zone aggregate as an in-memory DataFrame.
- (c) **Serving layer:** Union the batch view with the real-time view on (pickup_zone, hour) and resolve overlaps (the current hour appears in both; the batch view value should be discarded in favour of the real-time value).
- (d) Report the total number of rides and fare sum per zone for the most recent 24-hour period.

```

1  from pyspark.sql import SparkSession
2  from pyspark.sql.functions import col, sum as _sum, hour, lit
3  from datetime import datetime, timedelta
4
5  spark = SparkSession.builder.appName("LambdaSim").getOrCreate()
6
7  # --- Batch layer view (pre-computed) ---
8  batch_view = spark.read.parquet("data/batch_views/
9                                  fare_by_zone_hour/")
10 # Schema: pickup_zone, hour_ts, total_fare_usd, source='batch'
11
12 # --- Speed layer (in-memory simulation) ---

```

```

12 import random
13
14 live_events = spark.createDataFrame([
15     {
16         "ride_id":      f"R{i:06d}",
17         "driver_id":    random.randint(1, 5000),
18         "pickup_zone":  random.choice(["Downtown", "Midtown", "
19                                     Airport",
20                                     "Suburbs", "University", "
21                                     Harbour"]),
22         "fare_usd":     round(random.uniform(5.00, 80.00), 2),
23         "ride_ts":      datetime.now() - timedelta(seconds=
24                                     random.randint(0, 3599)),
25     }
26     for i in range(10_000)
27 ])
28
29 real_time_view = (live_events
30     .groupBy("pickup_zone")
31     .agg(_sum("fare_usd").alias("total_fare_usd"))
32     .withColumn("hour_ts", lit(datetime.now().replace(minute=0,
33                                                         second=0,
34                                                         microsecond
35                                                         =0)))
36     .withColumn("source", lit("realtime")))
37
38 # --- Serving layer: union, prefer real-time for current hour
39 ---
40 current_hour = datetime.now().replace(minute=0, second=0,
41                                       microsecond=0)
42 batch_historical = batch_view.filter(col("hour_ts") < lit(
43     current_hour))
44
45 served = batch_historical.unionByName(real_time_view)
46 served.orderBy("pickup_zone", "hour_ts").show(20, truncate=
47     False)

```

Listing 8.7: PE8.2: Lambda serving layer union in PySpark

PE8.3. Data Quality Gate (Python + Great Expectations). Implement an automated data quality gate that must pass before a dataset is promoted from the raw zone to the processed zone of a data lake:

- (a) Load a CSV file of customer records (fields: `customer_id`, `mobile`, `name`, `region`, `balance_usd`, `account_opened`).
- (b) Using the `great_expectations` library, define and run the following expectations:
 - `customer_id` must not be null and must be unique.
 - `mobile` must match the E.164 international format regex `^\+[1-9]\d{7,14}$`.
 - `balance_usd` must be between 0 and 10,000,000.
 - `region` must be one of the values in a reference catalogue (e.g., ISO 3166-2 region codes for the country of operation).
 - No more than 1% of rows may have a null name.
- (c) If all expectations pass, write the dataset to the processed zone as Parquet. If any expectation fails, write the dataset to a quarantine directory and raise an alert (print a summary report).

Further Reading

- **Kreps, Narkhede, and Rao 2011** — The original Kafka paper. Short and readable; the benchmark section is directly relevant to understanding why Kafka displaced existing message brokers.
- **Kleppmann 2017** — Chapters 10 and 11 cover batch processing (Lambda batch layer) and stream processing (speed layer and Kappa) at graduate level. The “unified log” discussion expands Kreps’s original insight into a full architectural theory.
- **White 2015** — Chapters 14 (Flume) and 15 (Sqoop) provide implementation details not covered in this chapter, including Flume interceptors, channel selectors, and Sqoop’s `scoop-import-all-tables` command.
- **Kreps, J. (2013). “The Log: What Every Software Engineer Should Know About Real-Time Data’s Unifying Abstraction.” LinkedIn Engineering Blog.** Freely available online; the most important essay in data engineering of the last decade.
- **Arun, M. et al. (2016). “Kafka Inside Keystone Pipeline.” Netflix Tech Blog.** The primary Netflix source describing the Keystone architecture, Kafka configuration, and engineering challenges at 1.3 million

events/second.

Part VI

Insight Delivery

Data Visualization at Scale

"Above all else, show the data."

Edward R. Tufte, *The Visual Display of Quantitative Information*, Graphics Press, 1983

Learning Objectives

After completing this chapter, you will be able to:

1. Explain why Anscombe's Quartet demonstrates that summary statistics alone are insufficient for data understanding, and describe the role of visualization as a primary data quality check.
2. Define Tufte's data-ink ratio, compute it for a given chart, and identify specific chartjunk elements (heavy grids, 3-D effects, gradients) that should be removed to maximise it.
3. Calculate Tufte's lie factor for a chart with a truncated axis and explain the distortion it introduces in the viewer's perception of the data effect.
4. Describe Tufte's small multiples and sparklines and explain why each is preferable to animation for comparing trends across many subsets.
5. Apply Shneiderman's three-step visual information seeking mantra (overview first; zoom and filter; details on demand) to the design of an interactive big data dashboard.

6. Use Shneiderman’s seven data type taxonomy to select the appropriate visualization type for a given analytical question (temporal, 2D geographic, multi-dimensional, tree, and network data).
7. Describe three techniques for addressing overplotting in million-point scatterplots—alpha transparency, hexbin aggregation, and kernel density estimation—and explain what each preserves that the overplotted chart loses.
8. Explain how pre-aggregated data cubes enable sub-second interactive query response on billion-row fact tables, and identify the latency versus freshness trade-off.
9. Design a streaming visualization architecture using Kafka, Spark Structured Streaming, a time-series database, and Grafana, and explain when to use WebSocket push versus periodic poll.
10. Compare six visualization tools—Tableau, Power BI, D3.js, Grafana, Apache Superset, and Kibana—on audience, data source, scale, and real-time capability, and select the appropriate tool for a given scenario.

Chapter 8 established how data arrives in the platform — through Kafka pipelines, data lakes, and stream processing architectures. This chapter asks what happens at the other end: once data has been ingested, stored, and aggregated, how should it be communicated to the humans who must act on it?

The answer is not as obvious as it might appear. A petabyte-scale analytical result compressed into a single number is not trustworthy; a table of ten million rows is unreadable; and a chart that presents data incorrectly is worse than no chart at all. The discipline of data visualization provides a rigorous body of principles—grounded in perceptual psychology, information theory, and graphic design—that governs how data should be encoded visually so that human pattern recognition can operate effectively.

This chapter covers the three intellectual pillars of that discipline: Edward Tufte’s *data-ink* theory of minimal graphical design; Ben Shneiderman’s *task-by-data-type taxonomy* and interaction model; and Mike Bostock’s *D3.js*, which made both principles implementable in interactive web graph-

ics. It then addresses the three specific engineering challenges that big data introduces for visualization—overplotting, latency, and streaming—and maps the tool landscape across which practitioners must choose.

9.1 Why Visualization Matters: From Summary to Insight

The case for visualization is not merely aesthetic. It rests on a demonstrable limitation of summary statistics: different data with radically different structures can produce identical numerical summaries.

9.1.1 Anscombe's Quartet

In 1973, statistician Francis Anscombe constructed four datasets—now called *Anscombe's Quartet*—each with 11 observations, each sharing the following summary statistics to two decimal places: mean of $x = 9.00$; mean of $y = 7.50$; variance of $x = 11.00$; variance of $y = 4.12$; correlation $r = 0.816$; regression line $\hat{y} = 3.00 + 0.500x$.

Despite being statistically indistinguishable, the four datasets describe four qualitatively different phenomena:

- **Dataset I** is a linear relationship with normally distributed residuals—the “textbook” regression case.
- **Dataset II** is a perfect curved (quadratic) relationship; a linear fit is systematically wrong at every point.
- **Dataset III** is a near-perfect linear relationship with one outlier that drags the fitted slope away from the true slope.
- **Dataset IV** has all x -values equal to 8 except one; a vertical cluster of points with a single leverage point.

Anscombe's point was that no data analyst should trust a regression result without first plotting the data. The quartet has been the standard opening of every serious statistics course since 1973 and remains the canonical argument for prioritising visualization over numerical summary.

Key Insight | Visualization as the First Quality Check

At big data scale, Anscombe's problem becomes more severe, not less. A billion-row table may contain multiple overlapping sub-populations with opposing trends that cancel in the aggregate mean. Log-scale distributions (income, city populations, web traffic volumes) make arithmetic means uninformative. Seasonal effects can be exactly cancelled by counter-seasonal confounders, leaving a flat year-over-year mean that masks both effects. Visualization is therefore not a presentation step that happens after analysis—it is the first and most important analysis step.

9.1.2 Visualization in the Big Data Context

Big data introduces challenges that do not arise with small datasets:

1. **Scale:** rendering 10 million points in a standard scatter plot produces an uninformative solid blob. Visualization must be redesigned for scale.
2. **Velocity:** streaming data generates new observations every second. Charts must update continuously without requiring the user to manually refresh.
3. **Variety:** heterogeneous data types—geographic coordinates, time series, network graphs, free text—require different chart types with different interaction paradigms.
4. **Latency:** a business analyst querying a 100-billion-row Hive table expects a dashboard filter response in under one second. The underlying computation may take minutes.

The remainder of this chapter addresses each challenge in turn, building from foundational principles to engineering solutions.

9.2 Tufte's Principles of Analytical Design

Edward R. Tufte is Professor Emeritus of Statistics, Graphic Design, and Political Economy at Yale University. His 1983 book *The Visual Display of Quantitative Information*—rejected by every publisher and self-published

after Tufte mortgaged his house to pay for printing—sold 40,000 copies in its first year and has been cited over 20,000 times. It is required reading at Harvard, Stanford, Yale, and MIT in data science and statistics courses, and its principles are embedded in every modern dashboard tool.

9.2.1 The Data-Ink Ratio

Tufte's central concept is the *data-ink ratio*:

Definition | Data-Ink Ratio (Tufte, 1983)

The **data-ink ratio** of a graphic is the proportion of the total ink used in the graphic that encodes actual data:

$$\text{Data-ink ratio} = \frac{\text{ink devoted to data}}{\text{total ink used in graphic}}$$

Principle: maximise the data-ink ratio. The maximum is 1: every mark encodes data and no mark is decorative. Erasing a non-data mark that does not reduce information is called *erasing chartjunk*.

Chartjunk is Tufte's umbrella term for ink that does not encode data. He identifies three categories:

- **Vibrations:** optical illusions caused by dense cross-hatching, competing patterns, or high-contrast decorative elements.
- **Grids:** heavy gridlines that compete with the data. Tufte recommends light, thin gridlines or none at all; the data should dominate the visual field, not the scaffolding.
- **Ducks:** graphic elements that represent themselves rather than data. A bar chart drawn as a stack of coins, a temperature chart drawn as a thermometer, or any 3-D perspective added to a 2-D bar chart are all "ducks" — they encode the same information as the underlying simple chart but add ink without adding data.

Practical application. When designing or reviewing a chart, ask: "If I erased this element, would the viewer lose information?" If the answer is no, erase it. Specific elements that almost always fail this test:

- 3-D effects on bar charts, pie charts, or line charts. Three-dimensional perspective distorts quantitative comparison: the apparent depth makes

a bar appear larger than its height warrants.

- Gradient fills. A bar coloured dark-at-bottom and light-at-top implies a quantitative dimension that does not exist.
- Redundant legends when the chart can be directly labelled.
- Background colours, shadows, and rounded decorative borders.

9.2.2 The Lie Factor

Definition | Lie Factor (Tufte, 1983)

The **lie factor** of a graphic measures how much the chart exaggerates or understates the effect it purports to show:

$$\text{Lie factor} = \frac{\text{size of effect shown in graphic}}{\text{size of effect in data}}$$

A lie factor of 1 is truthful. A lie factor > 1 overstates the data effect; a lie factor < 1 understates it. Lie factors greater than 1.05 or less than 0.95 are considered deceptive by Tufte.

The most common source of a high lie factor is a *truncated axis*: a bar chart whose y -axis starts at 80 rather than 0. If two bars represent values of 82 and 90, the true ratio is $90/82 = 1.10$ (a 10% difference). If the y -axis starts at 80, the bars appear to have heights of 2 and 10—a ratio of 5—making the same 10% difference look like a 5 $\times$ difference. The lie factor is $5/1.1 \approx 4.5$.

Common Misconception | The Truncated Axis in Political and Business Reporting

Truncated axes are ubiquitous in political advertising, corporate earnings presentations, and cable news graphics. They are technically not incorrect—the tick marks do state the true values—but they exploit the human perceptual system’s tendency to judge bar charts by height rather than by reading the axis labels. In big data dashboards, the most common violation is a live metrics chart where the y -axis autoscales to the data range: a stable metric that fluctuates between 99.98% and 99.99% availability appears to swing dramatically between $\sim 0\%$ and $\sim 100\%$ of the chart’s visual height. Always verify that availability and ratio charts start their y -axes at zero.

9.2.3 Small Multiples and Sparklines

Two of Tufte's most practically useful design patterns address the problem of comparing many subsets simultaneously without animation.

Definition | Small Multiples (Tufte, 1983)

Small multiples (also called *trellis charts* or *facet charts*) are a grid of charts that repeat the same visual structure—same axes, same scale, same chart type—across different subsets of the data (different time periods, geographic regions, product categories, or experimental conditions). The viewer's eye detects differences and patterns across the grid instantaneously, leveraging the perceptual system's ability to perform parallel comparison. The critical requirement is that **all panels share the same scale**; if panels have different scales, comparison is invalid.

Definition | Sparklines (Tufte, 2006)

A **sparkline** is a small, word-sized data graphic embedded directly in text or in a table cell alongside the numeric value it contextualises. A sparkline adds temporal context to a number without requiring a separate figure. Grafana's stat panel sparkline, the GitHub contribution heatmap, and Bloomberg terminal trend indicators are all direct applications of this concept. The design rule is minimal: no axis labels, no legend, no gridlines—just the trend shape in a space no larger than the surrounding text.

Small multiples are preferable to animation for the following reason: animation requires the viewer to *remember* the previous frame to compare it with the current frame. A small-multiples grid allows simultaneous comparison of all subsets with no memory burden. A classic example is the New York Times COVID-19 tracker, which showed 50-state case charts in a small-multiples grid—allowing readers to compare New York, Texas, and California at a glance—rather than an animated map that required readers to hold the April 2020 pattern in working memory while watching the July 2020 pattern unfold.

9.3 Shneiderman's Visual Information Seeking Mantra

Ben Shneiderman is Professor of Computer Science at the University of Maryland and the inventor of the treemap (1991). His 1996 paper “The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations,” presented at the IEEE Symposium on Visual Languages, has been cited over 5,000 times and provides the standard framework for designing interactive visualisation systems.

9.3.1 The Three-Step Mantra

Shneiderman's central contribution is the *visual information seeking mantra*:

Key Insight | Shneiderman's Visual Information Seeking Mantra

“Overview first, zoom and filter, then details on demand.”

This three-step model describes the **universal interaction sequence** for any well-designed interactive visualisation:

1. **Overview first.** Present the complete dataset or the highest-level aggregation as the initial view. The user must be able to see the whole picture before selecting a subset.
2. **Zoom and filter.** Allow the user to narrow the view to a region of interest—either spatially (zoom on a geographic map, zoom on a time-axis), or by attribute (filter to a specific category, date range, or threshold).
3. **Details on demand.** Allow the user to inspect the raw record for any visible item by clicking or hovering. The tooltip or drill-down panel reveals the full record without switching context.

Every major dashboard tool implements this mantra: Tableau's drill-down, Grafana's time-range zoom, Google Maps' scroll-to-zoom, and Kibana's faceted search all instantiate the same three-step pattern.

9.3.2 Seven Data Types and Their Visualizations

Shneiderman identifies seven data types, each with a natural family of visualization forms and a natural set of interaction tasks.

Data Type	Typical Analytical Question	Recommended Visualizations
1-D (linear)	Which category is largest? How are values distributed?	Bar chart, histogram, dot plot
2-D (planar/geographic)	How is X distributed across space?	Choropleth map (rate), proportional bubble map (absolute count)
3-D (volumetric)	What is the internal structure of a volume?	Volume rendering, 3-D scatter (scientific visualization)
Temporal	How did X change over time?	Line chart, area chart, calendar heatmap
Multi-dimensional	Any pattern across >3 variables?	Scatter plot matrix, parallel coordinates, heatmap with clustering
Tree	What is the hierarchical breakdown?	Treemap, sunburst diagram, dendrogram
Network	How are entities connected?	Force-directed graph, chord diagram, adjacency matrix

Table 9.1: Shneiderman's seven data types with recommended visualisations. Matching the chart type to the data type is the most consequential design decision in dashboard construction.

The most abused chart. The pie chart should only be used when there are at most four categories and the question is “what fraction of the whole does each category represent?” For any comparison question, any ranking question, any time-series question, or any analysis with more than four categories, a horizontal bar chart ordered by value is always more readable. Human perception judges the length of a bar far more accurately than the

angle or arc length of a pie segment—a finding confirmed by Cleveland and McGill’s perceptual accuracy studies (1984), which are the empirical foundation of both Tufte’s and Shneiderman’s recommendations.

Data Type	Question Type	Best Visualization	Avoid
Temporal	Trend over time	Line chart, area chart, calendar heatmap	Pie chart
Categorical	Which is largest?	Horizontal bar chart (>4 cats); grouped bar	3-D pie
Geographic	Spatial distribution	Choropleth (rate); bubble map (count)	Plain bar
Relational	Entity connections	Force-directed graph; chord diagram	Table
Distribution	Shape and spread	Histogram; box plot; violin plot	Line chart
Correlation	Does X predict Y?	Scatterplot; scatterplot matrix	Bar chart
Hierarchical	Part-of-whole breakdown	Treemap; sunburst	Nested pie

Table 9.2: Visualization selection by data type and question. The “Avoid” column lists common mistakes that produce low-information charts for the given combination of data type and question.

9.4 Big Data Visualization Challenges

9.4.1 Challenge 1: Overplotting at Scale

A standard scatterplot renders one point per data record. At ten million records, points overlap completely, producing a solid blob that conveys no more information than “data exists in this region.” This is the *overplotting* problem, and it appears whenever visualising a large dataset with more than ~10,000 points in a 2-D projection.

Three solutions exist in increasing sophistication.

1. Alpha transparency. Set each point's opacity to a small value (e.g., $\alpha = 0.01$). Regions where points overlap appear darker because the opacities stack multiplicatively. This is simple to implement and reveals gross density differences, but fails at very high densities: once >100 points overlap, all regions appear equally saturated at $\alpha = 0.01$.

2. Hexbin aggregation. Divide the 2-D plot area into a regular hexagonal grid (hexagonal cells tile a plane more uniformly than square cells). Count the number of data points falling in each hexagonal cell. Colour each cell by its count using a sequential colour scale (e.g., light-to-dark blue). The result converts a 10-million-point scatter into a grid of $\sim 10,000$ hexagonal cells, each representing a local density estimate—interactive and readable.

```

1  import numpy as np
2  import matplotlib
3  matplotlib.use("Agg")
4  import matplotlib.pyplot as plt
5
6  rng = np.random.default_rng(42)
7
8  # Simulate 5 million GPS coordinates around New York City
9  x = rng.normal(-73.95, 0.15, 5_000_000) # longitude
10 y = rng.normal(40.75, 0.10, 5_000_000) # latitude
11
12 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
13
14 # Left: raw scatter (overplotted)
15 axes[0].scatter(x, y, s=0.01, alpha=0.003, color="steelblue")
16 axes[0].set_title("Scatterplot (5M points -- overplotted)")
17 axes[0].set_xlabel("Longitude")
18 axes[0].set_ylabel("Latitude")
19
20 # Right: hexbin (density readable)
21 hb = axes[1].hexbin(x, y, gridsize=80, cmap="Blues", mincnt=1)
22 fig.colorbar(hb, ax=axes[1], label="Trip count")
23 axes[1].set_title("Hexbin (same 5M points -- density readable
24                    )")
25 axes[1].set_xlabel("Longitude")
26 axes[1].set_ylabel("Latitude")
27
28 plt.tight_layout()
29 plt.savefig("nyc_gps_hexbin.png", dpi=150, bbox_inches="tight")
30 print("Saved nyc_gps_hexbin.png")

```

Listing 9.1: Hexbin scatterplot in Python (matplotlib)

3. Kernel Density Estimation (KDE). A *kernel density estimate* places a smooth kernel function (typically Gaussian) on each data point and sums the kernels to produce a continuous estimate of the probability density over the 2-D domain. The density surface is then visualised as filled contours or as a heatmap. KDE reveals the shape, modes (peaks), and tails of the point distribution more clearly than hexbin, but is more computationally expensive: $O(N \cdot G)$ where N is the number of points and G is the number of grid cells used to evaluate the density.

```

1  from scipy.stats import gaussian_kde
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import matplotlib
5  matplotlib.use("Agg")
6
7  rng = np.random.default_rng(0)
8  x = rng.normal(-73.95, 0.12, 200_000)
9  y = rng.normal(40.75, 0.08, 200_000)
10
11 # KDE on a random subsample (full dataset is too large for
    naive KDE)
12 idx = rng.choice(len(x), size=50_000, replace=False)
13 kde = gaussian_kde(np.vstack([x[idx], y[idx]]),
14                    bw_method=0.05)
15
16 # Evaluate on a grid
17 xi, yi = np.mgrid[x.min():x.max():200j,
18                  y.min():y.max():200j]
19 zi = kde(np.vstack([xi.ravel(), yi.ravel()])).reshape(xi.
    shape)
20
21 fig, ax = plt.subplots(figsize=(8, 6))
22 cf = ax.contourf(xi, yi, zi, levels=15, cmap="Blues")
23 fig.colorbar(cf, ax=ax, label="Density")
24 ax.set_title("KDE Density Contour -- NYC GPS Pings")
25 ax.set_xlabel("Longitude")
26 ax.set_ylabel("Latitude")
27 plt.tight_layout()
28 plt.savefig("nyc_kde.png", dpi=150)

```


Listing 9.2: 2-D KDE density plot (scipy + matplotlib)

9.4.2 Challenge 2: Latency and Pre-Aggregation

A business analyst using Tableau expects a dashboard filter (“show Q1 2024, North-East region only”) to respond in under one second. But the underlying fact table may contain 100 billion rows in Hive on HDFS. A Hive query over the full fact table takes minutes to complete, making interactive filtering impossible.

Two engineering solutions exist.

1. Pre-aggregated data cubes. A *data cube* (also called an *OLAP cube* or *aggregate table*) materialises the results of all combinations of GROUP BY queries over a set of dimensions at ingest time. If the fact table has three dimensions—date, region, and product category—the data cube pre-computes eight subtotals: one for each subset of the three dimensions (the *power set* of dimensions, sometimes called the *cuboid*).

Definition | Data Cube

A **data cube** over a fact table with d dimensions contains 2^d *cuboids*: one for each subset of dimensions. The base cuboid aggregates over all dimensions (producing one row); the apex cuboid contains the full fact table (no aggregation). Materialising the full cube requires $O(N \cdot 2^d)$ space where N is the fact table row count, but in practice only a subset of cuboids are materialised based on query frequency—a process called *selective materialisation*.

Queries served by the data cube read from a materialised cuboid rather than the raw fact table, reducing I/O by a factor of $N/|\text{cuboid}|$ —often 10^4 – $10^6\times$ for typical analytical workloads.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, sum as _sum, count, avg
3
4 spark = SparkSession.builder.appName("DataCube").getOrCreate()
5
6 # Raw fact table: 100 billion rows on HDFS
7 fact = spark.read.parquet("hdfs:///data/curated/transactions/")
```

```

8
9 # --- Materialise a data cube with GROUPING SETS ---
10 # This one Spark job produces all 8 cuboids for 3 dimensions
11 cube_df = (fact
12     .cube("txn_date", "region", "product_category")
13     .agg(
14         _sum("amount_usd").alias("total_usd"),
15         count("*").alias("txn_count"),
16         avg("amount_usd").alias("avg_usd"),
17     ))
18
19 # Write the cube as Parquet -- millions of rows, not billions
20 (cube_df.write
21     .mode("overwrite")
22     .partitionBy("txn_date")
23     .parquet("hdfs:///data/curated/txn_cube/"))
24
25 # Tableau / Superset query: reads from cube (ms), not fact
26   table (min)
27 region_q1 = spark.read.parquet("hdfs:///data/curated/txn_cube/"
28     ) \
29     .filter(
30         (col("txn_date").between("2024-01-01", "2024-03-31")) &
31         col("region").isNotNull() &
32         col("product_category").isNull()
33     ) \
34     .orderBy("total_usd", ascending=False)
35 region_q1.show(10)

```

Listing 9.3: Pre-aggregating a data cube in Spark SQL

2. Approximate query processing (AQP). Rather than pre-computing exact aggregates, *approximate query processing* returns a statistically correct estimate of the aggregate in milliseconds by sampling the table. Apache DataSketches (formerly Yahoo DataSketches) implements a family of streaming algorithms—HyperLogLog for cardinality, Theta sketch for set operations, KLL for quantiles—that provide guaranteed error bounds with sub-linear space. The trade-off: AQP is appropriate for exploratory dashboards where a 5% error is acceptable, but not for financial reconciliation or regulatory reporting where exactness is mandatory.

9.4.3 Challenge 3: Streaming Visualization

Many big data applications require visualisations that update continuously as new events arrive:

- An operations dashboard showing the number of active Uber drivers by city zone, updated every 5 seconds.
- A payment fraud operations centre showing transaction volume per minute and active alert count.
- A CDC / NHS disease surveillance dashboard showing new case reports as hospitals submit their hourly returns.

Two delivery mechanisms exist for updating a browser chart with new data.

Definition | Poll vs. WebSocket Push

- **Periodic poll (Grafana default):** The browser makes an HTTP request to the backend on a configured interval (e.g., every 5 seconds). The backend queries the time-series database and returns the latest value. Simplest to implement; adds latency of up to one poll interval.
- **WebSocket push:** The server establishes a persistent bidirectional connection to the browser. New data points are pushed to the browser as they arrive, without waiting for a poll. Sub-second update latency; required for real-time financial tickers, live election results, and sensor dashboards.

The standard streaming visualization architecture is:

1. **Kafka:** receives events from producers in real time.
2. **Spark Structured Streaming or Flink:** subscribes to Kafka topic; applies a tumbling or sliding window aggregation (e.g., count of transactions per minute per region).
3. **Time-series database** (InfluxDB, Prometheus, TimescaleDB): stores the window aggregates; indexed by timestamp.
4. **Grafana or Kibana:** polls the TSDB on a configured interval (Grafana: minimum 5 seconds) and re-renders the chart. For sub-second updates, a custom D3.js frontend with WebSocket replaces Grafana.

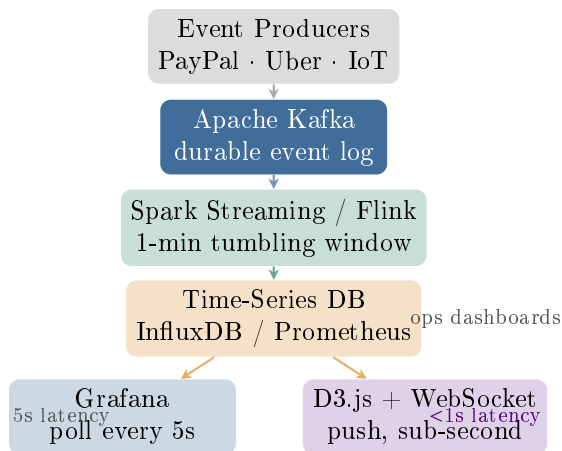


Figure 9.1: Streaming visualization architecture. Events flow from producers through Kafka and a stream processor to a time-series database. Grafana polls the TSDB every 5 seconds; a WebSocket-based D3.js frontend receives pushed updates for sub-second latency.

9.5 The Visualization Tools Landscape

The choice of visualization tool determines what is possible, what requires custom development, and how well the tool integrates with the rest of the data stack. Six tools dominate the landscape for big data visualisation.

9.5.1 Tableau and Power BI: Business Intelligence Platforms

Tableau (acquired by Salesforce in 2019) is the dominant business intelligence platform for analyst-facing dashboards. Its drag-and-drop interface allows non-programmers to build interactive charts, maps, and cross-tabs from SQL databases, Hive tables, Spark dataframes, and flat files. Tableau implements Shneiderman’s mantra through a drill-down hierarchy (Year → Quarter → Month → Day), its filter shelf (dynamic queries), and detail-on-demand tooltips.

Tableau’s primary limitation is scale: for tables exceeding ~50 million rows, Tableau either operates in *extract* mode (a cached in-memory copy of the data, updated on a schedule) or through a *live connection* that pushes each query to the underlying database. For Hive or Presto queries, even a simple filter can take 30–60 seconds on very large tables, breaking the

Tool	Best For	Data Sources	Scale	Real-time
Tableau	Business dashboards, drag-and-drop BI	SQL, Spark, Excel	Hive, CSV, Millions of rows	Limited
Power BI	Microsoft ecosystem, executive reports	Azure, SQL REST	Excel, Server, Millions of rows	Limited
D3.js	Custom interactive web graphics	JSON, REST APIs	CSV, Flexible (SVG/Canvas)	Yes (WebSoc)
Grafana	Ops metrics, real-time monitoring	InfluxDB, Prometheus, Elasticsearch	Millions of data points	Yes (5s poll)
Apache Superset	Open-source BI on data lakes	Presto, Spark SQL, PG	Hive, Billions of rows	Limited
Kibana	Log analytics, search exploration	Elasticsearch only	Billions of documents	Yes

Table 9.3: Visualization tool landscape. Each tool occupies a distinct niche; selecting the wrong tool forces expensive workarounds.

interactive experience. Pre-aggregated data cubes in Presto or Spark SQL are the standard mitigation.

Microsoft Power BI is the dominant choice in organisations standardised on the Microsoft Azure ecosystem (Azure Synapse Analytics, Azure Data Factory, Office 365). Its DAX formula language for calculated columns and measures is more expressive than Tableau for certain analytical patterns, but its connector ecosystem is smaller and its performance at big data scale is similar to Tableau's.

9.5.2 D3.js: Data-Driven Documents

D3.js (**Data-Driven Documents**) is a JavaScript library for binding data to DOM (Document Object Model) elements in a web browser and applying transformations based on data values. Created by Mike Bostock, Vadim Ogievetsky, and Jeffrey Heer at Stanford and released in 2011, D3 is the foundation of the New York Times interactive graphics, The Guardian's data journalism, and the Reuters COVID-19 tracker.

D3's design philosophy is minimal abstraction: it provides general geometric and visual primitives (SVG shapes, scales, axes, transitions) but does not prescribe chart types. Every element of a D3 chart is created by writing JavaScript that explicitly binds a data array to SVG elements:

```
1  /* Tufte-compliant D3 bar chart -- data-ink ratio maximised
2     No gridlines, no background, direct data labels, 0-based y-
        axis */
3
4  const data = [
5    { region: "New York",    cases: 4217 },
6    { region: "London",     cases: 1803 },
7    { region: "Berlin",     cases: 892 },
8    { region: "Sydney",     cases: 671 },
9    { region: "Toronto",    cases: 544 },
10 ];
11
12 const margin = { top: 20, right: 60, bottom: 40, left: 90 };
13 const W = 560 - margin.left - margin.right;
14 const H = 320 - margin.top - margin.bottom;
15
16 const svg = d3.select("#chart")
17   .append("svg")
```

```

18     .attr("width", W + margin.left + margin.right)
19     .attr("height", H + margin.top + margin.bottom)
20     .append("g")
21     .attr("transform", `translate(${margin.left},${margin.top})`);
22
23 const x = d3.scaleLinear()
24     .domain([0, d3.max(data, d => d.cases)])
25     .nice()
26     .range([0, W]);
27
28 const y = d3.scaleBand()
29     .domain(data.map(d => d.region))
30     .range([0, H])
31     .padding(0.35);
32
33 // Bars -- single flat colour (no gradient, no shadow)
34 svg.selectAll(".bar")
35     .data(data)
36     .join("rect")
37     .attr("class", "bar")
38     .attr("x", 0)
39     .attr("y", d => y(d.region))
40     .attr("width", d => x(d.cases))
41     .attr("height", y.bandwidth())
42     .attr("fill", "#003c78"); // DEBlue
43
44 // Direct data labels (no legend needed)
45 svg.selectAll(".label")
46     .data(data)
47     .join("text")
48     .attr("x", d => x(d.cases) + 4)
49     .attr("y", d => y(d.region) + y.bandwidth() / 2
50         + 4)
51     .text(d => d.cases.toLocaleString())
52     .attr("font-size", 11)
53     .attr("fill", "#505050");
54
55 // Minimal y-axis: region names only
56 svg.append("g").call(d3.axisLeft(y).tickSize(0));
57
58 // Minimal x-axis: no domain line, light ticks only
59 svg.append("g")
    .attr("transform", `translate(0,${H})`)

```

```

60     .call(d3.axisBottom(x).ticks(5).tickSizeOuter(0));
61
62  /* Tooltip (Shneiderman: details-on-demand) */
63  const tooltip = d3.select("body").append("div")
64    .style("position", "absolute")
65    .style("background", "#fff")
66    .style("border", "1px solid #ccc")
67    .style("padding", "6px 10px")
68    .style("font-size", "12px")
69    .style("pointer-events", "none")
70    .style("opacity", 0);
71
72  svg.selectAll(".bar")
73    .on("mouseover", (event, d) => {
74      tooltip.style("opacity", 1)
75        .html(`<b>${d.region}</b><br>Cases: ${d.cases.
76          toLocaleString()}`);
77    })
78    .on("mousemove", event => {
79      tooltip.style("left", (event.pageX + 10) + "px")
80        .style("top", (event.pageY - 28) + "px");
81    })
82    .on("mouseout", () => tooltip.style("opacity", 0));

```

Listing 9.4: D3.js: Tufte-style bar chart (data-driven SVG)

9.5.3 Grafana and Apache Superset: Open-Source Platforms

Grafana is the dominant open-source tool for operations and infrastructure monitoring dashboards. It natively integrates with InfluxDB (time-series database optimised for metrics), Prometheus (pull-based monitoring used in Kubernetes deployments), and Elasticsearch (for log aggregates). Grafana implements Shneiderman’s zoom-and-filter via its time-range selector (click-drag on any chart to zoom into a sub-interval) and variable dropdowns (dynamic query parameters). Its stat panel widget renders a number with a sparkline—a direct implementation of Tufte’s sparkline concept.

Apache Superset (open-source, Apache Software Foundation) is the standard open-source BI tool for organisations that run their data platform on Presto, Trino, Apache Hive, or Spark SQL. Its SQL Lab feature allows analysts to write arbitrary SQL and render the results as charts; its dashboard

builder allows interactive filter boxes and cross-chart drill-down. For organisations that cannot afford or prefer not to use Tableau, Superset provides comparable BI functionality with direct connectivity to the Hadoop/Spark ecosystem.

9.6 Research Spotlight

Research Spotlight | Tufte (1983) · Shneiderman (1996) · Bostock, Ogievetsky & Heer (2011)

Three works that together define the field:

Work 1: Tufte, E. R. (1983). *The Visual Display of Quantitative Information*. Graphics Press. (>20,000 citations.)

Tufte analysed 4,000 charts from science, business, and journalism publications and derived the data-ink ratio principle: of all the ink used in a graphic, maximise the fraction that encodes data. He coined “chartjunk” to describe ink that encodes no data (grids, 3-D effects, gradients, redundant legends) and documented the lie factor—a metric for quantifying graphic deception from axis truncation or scale distortion. His concept of small multiples described a grid of same-scale same-axes charts as the most efficient form of comparative display, and his sparklines (formalised in his 2006 follow-up) described word-sized trend indicators now ubiquitous in Grafana, Bloomberg terminals, and spreadsheets. Tufte’s book is unique in data engineering in that it both states the principle and implements it beautifully in the book’s own typography and layout: the book itself has a high data-ink ratio. Every modern dashboard framework—from Tableau’s clean minimal default theme to D3’s blank-canvas philosophy—is a direct application of Tufte’s principles.

Work 2: Shneiderman, B. (1996). “The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations.” *Proc. IEEE Visual Languages*, pp. 336–343. (>5,000 citations.)

Shneiderman identified seven types of data (linear, 2-D planar, 3-D volumetric, temporal, multi-dimensional, tree, network) and seven tasks that users perform on visualisations (overview, zoom, filter, details-

on-demand, relate, history, extract). He proposed the “visual information seeking mantra”—overview first, zoom and filter, then details on demand—as a universal three-step design pattern for interactive information visualisation. He also invented the *treemap* (1991) and the *dynamic query* concept (sliders and checkboxes that update the visualisation in <100 ms). The 1996 paper’s taxonomy directly shapes how modern BI platforms are designed: Tableau’s hierarchy drill-down implements “overview → zoom”; Grafana’s variable dropdowns implement “filter”; hover tooltips implement “details-on-demand.” The mantra is the single most widely implemented design pattern in data visualisation software.

Work 3: Bostock, M., Ogievetsky, V., & Heer, J. (2011). “D³: Data-Driven Documents.” *IEEE Transactions on Visualization and Computer Graphics*, 17(12), 2301–2309. (>3,000 citations.)

Bostock, Ogievetsky, and Heer introduced D3.js—a JavaScript library that binds data arrays to DOM elements (SVG shapes, HTML elements) and applies visual transformations based on data values. The key innovation over previous web charting libraries (Protovis, Raphaël) was *general selections*: any set of DOM elements can be bound to any array; the enter-update-exit pattern handles data-driven creation, modification, and removal of elements. D3 does not prescribe chart types; instead it provides general primitives (scales, axes, path generators, force simulations, geographic projections) from which any chart can be assembled. This philosophy—maximum expressive power at the cost of requiring more code—directly reflects Tufte’s principle that the graphic designer must control every pixel of ink. D3 became the production tool of the New York Times Graphics Desk, The Guardian, Reuters, the FT’s Visual Journalism team, and the US 2020 Census Bureau’s interactive data explorer. It is the most widely cited visualisation systems paper of the 2010s and is used in the graduate visualisation curriculum at Harvard (CS171), MIT (6.859), and Stanford (CS448B).

9.7 Case Study: The New York Times Graphics Desk

Context. The New York Times Graphics Desk is a team of approximately 50 journalists, designers, and engineers who produce interactive data visualisations for nytimes.com—reaching 6 million readers per day. The desk is responsible for some of the most widely read and most cited data visualisations of the past decade, including the 2012 US presidential election maps, the 2016 election needle, and the COVID-19 tracker that was loaded 800 million times in 2020.

Mike Bostock—co-creator of D3.js—worked at the NYT Graphics Desk from 2010 to 2015, during which time he created D3.js to replace the desk’s previous tool (Protovis). D3.js was born at the NYT not as a research project but as a production engineering need: the desk required pixel-level control over chart layout, custom geographic projections, and animated transitions between data states that no existing library provided.

Technical stack.

Component	Technology
Data collection and cleaning	Python (pandas, requests, BeautifulSoup)
Geographic processing	PostGIS, GDAL, Mapbox
Statistical analysis	R (ggplot2, tidyverse)
Interactive web charts	D3.js (SVG and Canvas)
Static print graphics	Adobe Illustrator + ai2html
Collaborative prototyping	Observable Notebooks
Data publication	GitHub CSV repository
Content delivery	nytimes.com CDN (AWS CloudFront)

COVID-19 Tracker (2020–2022): a big data visualisation at journalism scale.

From March 2020 the desk built and maintained the most-read COVID-19 data tracker in the English-speaking world. The data engineering challenges were representative of any big data pipeline at scale:

1. **Data acquisition at scale.** The tracker aggregated case and death data from the US Centers for Disease Control (CDC), 50 state health departments, and 3,000 county health departments —each with different update schedules, different column names, and different definitions of “confirmed case.” Python scrapers ran hourly; a pandas pipeline normalised schemas and detected anomalies (negative daily case counts from retroactive corrections; impossible date values; duplicate state-county records).
2. **The rolling average as a design decision.** Raw daily counts exhibited strong day-of-week artefacts: fewer cases were reported on weekends because fewer testing sites operated. The desk chose a 7-day rolling average to smooth these artefacts, encoding the average as the primary line and the raw counts as a lighter bar chart behind it. This is a *journalistic* choice with statistical implications: the 7-day window delays the detection of a surge by 3–4 days compared to the raw count.
3. **Tufte principles in production.** The 50-state small-multiples chart (all states on a single page, same scale, same axes) allowed readers to compare New York, Texas, and California in one glance without animation. No gridlines appeared on the case charts; only a single light horizontal reference line at the prior peak provided context. Hover tooltips provided exact date, raw count, and 7-day average (Shneiderman’s details-on-demand). The colour scale was tested for colour-blindness (deuteranopia and protanopia) using the `chroma.js` simulator.
4. **Accessibility at 800 million page loads.** Every chart had a data-table alternative for screen readers. SVG title and desc elements described each chart’s content to assistive technology. This was not optional: Section 508 of the US Rehabilitation Act requires federal government and major news organisations to make digital content accessible.
5. **GitHub as data infrastructure.** The NYT published its cleaned CSV data daily on GitHub (`nytimes/covid-19-data`). Within 30 days of the tracker’s launch, over 1,000 derivative dashboards had been built by universities, local governments, and independent develop-

ers worldwide—all using the NYT’s cleaned dataset. This transformed the NYT from a single publisher into a data infrastructure provider.

Engineering lessons.

1. **Data quality is the invisible half of visualisation.** The hardest engineering work on the COVID tracker was schema normalisation across 50 health departments—not SVG rendering. A visualisation pipeline without a robust data quality gate produces misleading charts with high production values.
2. **Small multiples beat animation for comparative analysis.** The 50-state grid was more actionable than an animated spreading-map because it allowed simultaneous comparison of all states without requiring readers to hold past frames in working memory.
3. **Accessibility is an engineering requirement, not a nicety.** At 6 million readers per day, even 1% of readers with visual impairments represents 60,000 people per day who would be excluded by inaccessible charts.
4. **Rolling averages are editorial decisions.** The choice of window size changes the story the chart tells. A 3-day window reveals surges earlier but amplifies noise; a 14-day window is more stable but delays detection. These decisions have public health consequences and must be documented transparently in the chart’s methodology notes.

Chapter Summary

Effective data visualization is not a presentation afterthought: it is the first quality check on any analytical result and the primary interface between data infrastructure and human decision-making.

Tufte’s data-ink ratio provides an actionable metric for graphical efficiency: maximise the fraction of ink that encodes data; erase everything else. His lie factor quantifies graphical deception from axis truncation. Small multiples and sparklines solve the comparative-display problem without

animation. Shneiderman’s mantra (overview → zoom/filter → details-on-demand) is the universal interaction design pattern for interactive information visualisation, implemented in every major dashboard platform.

Big data introduces three engineering challenges not present with small datasets: overplotting (solved by hexbin aggregation and KDE contours), latency (solved by pre-aggregated data cubes and approximate query processing), and streaming (solved by the Kafka → stream processor → TSDB → Grafana/WebSocket architecture). Tool selection—Tableau for analyst-facing BI, D3.js for custom interactive journalism and production web graphics, Grafana for real-time operational monitoring, Apache Superset for open-source lake-native analytics—determines what is achievable without custom development.

The New York Times COVID-19 tracker demonstrated that at journalistic scale the data quality pipeline is harder than the rendering pipeline, and that responsible visualization design—rolling-average window selection, colour-blindness testing, accessibility requirements—carries ethical weight proportional to audience size.

Exercises

Short Questions

SQ9.1. What is Anscombe’s Quartet? What property of summary statistics does it illustrate, and what does this imply about the role of visualization in data analysis?

SQ9.2. Define Tufte’s data-ink ratio. Identify three specific chart elements that commonly lower the data-ink ratio and should be removed.

SQ9.3. A bar chart shows four annual revenue figures. The data changed from \$82M in 2022 to \$90M in 2023—a 9.8% increase. The chart’s y -axis starts at \$80M. The bar for 2023 appears five times taller than the bar for 2022. Calculate Tufte’s lie factor.

SQ9.4. What is a “small multiples” chart? Give an example of a small

multiple grid that would be more effective than an animated chart for communicating the same information.

SQ9.5. State Shneiderman’s visual information seeking mantra in full. Give one example of how each of the three steps is implemented in Tableau, in Grafana, and in a D3.js chart.

SQ9.6. What is overplotting? At what approximate number of data points does it typically become a problem in a standard scatterplot?

SQ9.7. Describe the hexbin technique for addressing overplotting. What does it show that alpha transparency cannot?

SQ9.8. What is a data cube? If a fact table has four dimensions—date, region, category, and channel—how many cuboids does the full data cube contain?

SQ9.9. Explain the difference between a *periodic poll* and a *WebSocket push* architecture for streaming visualization. Give a use case where each is appropriate.

SQ9.10. Which visualization tool is most appropriate for: (a) a real-time operations dashboard monitoring Kafka consumer lag; (b) a business analyst building a quarterly sales report in a company that uses Azure; (c) a data journalist building a custom interactive choropleth map for publication on a news website?

SQ9.11. What is the Shneiderman treemap? Explain how area and colour are used to encode two variables simultaneously.

SQ9.12. Why does Tufte argue that 3-D effects on bar charts lower the data-ink ratio *and* increase the lie factor?

Analytical Problems

AP9.1. Data-Ink Ratio Audit. A data engineer builds a Grafana dashboard for a Kafka pipeline showing hourly message throughput per topic. The current chart has the following elements: a 3-D bar chart; a y -axis that starts at 500,000 messages/hour (when the true minimum is 480,000); a thick black border around the chart area; a gradient fill on each bar (dark blue at bottom, light blue at top); a legend listing all 8 topic names in

a box overlapping the chart; and heavy horizontal gridlines every 10,000 messages.

- (a) Identify every element that violates Tufte’s data-ink principle, classify each as chartjunk or a lie-factor violation, and explain what information is lost by removing it (if any).
- (b) Compute the approximate lie factor for the truncated y -axis if the true range is 480,000–560,000 messages/hour but the chart displays 500,000–560,000. The highest bar appears to be $5\times$ the height of the lowest bar on screen.
- (c) Redesign the chart: describe each element of the corrected version, applying Tufte’s principles.

AP9.2. Shneiderman Dashboard Design. A ride-sharing company data scientist needs to build an operations dashboard for the city fleet management team showing:

- City-wide active driver count by zone (20 city zones)
- Trip completion rate per hour over the past 7 days
- Distribution of trip duration (in minutes) for completed trips
- Geographic density of trip pickups (GPS coordinates)

For each requirement: (a) identify Shneiderman’s data type; (b) choose the appropriate chart type, citing the data-type taxonomy; (c) specify how overview, zoom/filter, and details-on-demand are implemented; (d) identify which big data challenge (overplotting, latency, or streaming) applies and how to address it.

AP9.3. Streaming Visualization Latency Budget. A mobile payment fraud operations dashboard must update its fraud alert rate chart within 10 seconds of an event arriving at the source system. The pipeline is: Mobile app → Kafka (1 partition, 50 MB/s) → Spark Structured Streaming (1-minute tumbling window) → InfluxDB → Grafana (WebSocket push).

Allocate the 10-second latency budget across the pipeline components. For each component: (a) state the typical latency contribution; (b) identify one configuration parameter that can reduce it; (c) identify the trade-off of reducing it.

AP9.4. Data Cube Design. A national electricity utility has a 5-year fact

table with 120 billion rows and four dimensions: `meter_id` (10 million distinct values), `hour_ts` (43,800 hours over 5 years), `region` (50 regions), and `tariff_band` (5 bands).

- (a) How many cuboids are in the full data cube? How many are practically useful for a dashboard (explain which dimensions are too high cardinality to materialise)?
- (b) Design the materialised cuboids for a dashboard that must serve: (i) hourly consumption per region for any 30-day window; (ii) monthly total consumption per tariff band; (iii) peak-hour demand by region.
- (c) Estimate the storage size (in GB) of the hourly-by-region cuboid assuming 30 bytes per row.

Design Problems

DP9.1. Public Health Surveillance Dashboard. A national public health agency (e.g., CDC or NHS) needs a disease surveillance dashboard communicating:

- (a) National COVID-19 confirmed case counts by region over 24 months, using a 7-day rolling average.
- (b) Real-time hospitalisation rate updated every 5 minutes from hospital daily reporting forms submitted via a REST API.
- (c) Demographic breakdown (age group $\times$ gender) of confirmed cases as a proportion of population.
- (d) A “situation map” showing the 7-day incidence rate per 100,000 population for all regions.

Design the complete dashboard: specify the chart type and justification for each panel; describe how Shneiderman’s mantra is implemented for each panel; describe the data pipeline (ingestion, aggregation, storage, and rendering tools); identify and address the big data visualization challenge for each panel; and describe two accessibility requirements.

DP9.2. Streaming Payment Operations Dashboard. Design a real-time operations dashboard for a mobile payment platform (10 million transactions/day, peak 500 transactions/second) that shows:

- (a) Total transaction volume (USD) per minute, last 60 minutes, updated every 5 seconds.
- (b) Number of active fraud alerts, updated every 30 seconds.
- (c) Geographic heatmap of transaction density across a metro area (GPS coordinates from merchant terminal locations).
- (d) Top-5 error codes by frequency in the past 15 minutes.

Specify: the Kafka topic structure and partition count for each stream; the Spark Structured Streaming window and aggregation for each metric; the TSDB schema for each metric; the Grafana panel type and refresh interval; and any pre-aggregation required to achieve the specified latency.

Programming Exercises

PE9.1. Overplotting Solutions in Python. Given a simulated dataset of 2 million ride-sharing GPS trip origin points (longitude, latitude) distributed across five city zones:

- (a) Plot the raw scatterplot with $\alpha = 0.005$. Describe the overplotting effect observed.
- (b) Create a hexbin plot with `gridsize=100` and a sequential blue colour scale. Describe what information is now visible.
- (c) Compute and plot a 2-D KDE contour using `scipy.stats.gaussian_kde` on a 10,000-point random subsample. Identify the location of the three highest-density zones.
- (d) Create a 2×2 panel figure (overview in panel 1; one city zone zoomed in panels 2–4 with hexbin). Label each panel with its Shneiderman interaction level (overview, zoom, detail).

```
1 import numpy as np
2 import matplotlib
3 matplotlib.use("Agg")
4 import matplotlib.pyplot as plt
5 from scipy.stats import gaussian_kde
6
7 rng = np.random.default_rng(2024)
8
9 # Five NYC zones: (lon_center, lat_center, sigma, n_points)
```

```

10 zones = [
11     (-73.970, 40.785, 0.020, 600_000), # Upper West Side
12     (-73.990, 40.730, 0.015, 500_000), # Greenwich Village
13     (-73.940, 40.760, 0.018, 400_000), # Midtown East
14     (-73.950, 40.820, 0.012, 300_000), # Harlem
15     (-74.010, 40.710, 0.010, 200_000), # Lower Manhattan
16 ]
17
18 lons, lats = [], []
19 for lon_c, lat_c, sigma, n in zones:
20     lons.append(rng.normal(lon_c, sigma, n))
21     lats.append(rng.normal(lat_c, sigma, n))
22
23 lons = np.concatenate(lons)
24 lats = np.concatenate(lats)
25
26 fig, axes = plt.subplots(2, 2, figsize=(14, 12))
27
28 # Panel 1 -- Overview: raw scatter (overplotted)
29 axes[0,0].scatter(lons, lats, s=0.003, alpha=0.004, c="
    steelblue")
30 axes[0,0].set_title("(a) Scatterplot -- overplotted (
    Shneiderman: overview)")
31
32 # Panel 2 -- Zoom: hexbin full city
33 hb = axes[0,1].hexbin(lons, lats, gridsize=100, cmap="Blues",
    mincnt=1)
34 fig.colorbar(hb, ax=axes[0,1], label="Trip count")
35 axes[0,1].set_title("(b) Hexbin -- density readable (
    Shneiderman: zoom)")
36
37 # Panel 3 -- Zoom: KDE contour on subsample
38 idx = rng.choice(len(lons), 10_000, replace=False)
39 kde = gaussian_kde(np.vstack([lons[idx], lats[idx]]),
    bw_method=0.08)
40 xi, yi = np.mgrid[lons.min():lons.max():150j, lats.min():lats.
    max():150j]
41 zi = kde(np.vstack([xi.ravel(), yi.ravel()])).reshape(xi.
    shape)
42 cf = axes[1,0].contourf(xi, yi, zi, levels=12, cmap="Blues")
43 fig.colorbar(cf, ax=axes[1,0], label="Density")
44 axes[1,0].set_title("(c) KDE contour -- shape revealed (
    Shneiderman: zoom)")
45

```

```

46 # Panel 4 -- Detail: Upper West Side zone only
47 mask = (lons > -73.99) & (lons < -73.95) & (lats > 40.77) & (
    lats < 40.80)
48 axes[1,1].hexbin(lons[mask], lats[mask], gridsize=60, cmap="
    Reds", mincnt=1)
49 axes[1,1].set_title("(d) Upper West Side detail (Shneiderman:
    detail)")
50
51 for ax in axes.ravel():
52     ax.set_xlabel("Longitude")
53     ax.set_ylabel("Latitude")
54
55 plt.tight_layout()
56 plt.savefig("nyc_overplotting.png", dpi=150, bbox_inches="tight
    ")
57 print("Saved: nyc_overplotting.png")

```

Listing 9.5: PE9.1: GPS overplotting simulation

PE9.2. Pre-Aggregated Data Cube (PySpark). Using PySpark, build a pre-aggregated data cube for a simulated 100 million-row e-commerce fact table with columns: `order_date`, `region`, `category`, and `revenue_usd`.

- (a) Generate 100 million synthetic rows using Spark’s built-in random functions.
- (b) Use `DataFrame.cube()` to materialise all cuboids for the three dimensions (`order_date`, `region`, `category`).
- (c) Write the cube as Parquet, partitioned by `order_date`.
- (d) Simulate three Tableau/Superset queries (filter by date range, filter by region, group by category) against the cube and measure their execution time using `time.perf_counter()`. Compare to the same queries against the raw fact table.
- (e) Report the size of the materialised cube in MB and the speedup factor for each query.

PE9.3. Streaming Dashboard Simulation (Python + InfluxDB). Build a simulated streaming pipeline using Python threads:

- (a) Thread 1 (“producer”): generates 100 synthetic payment transaction events per second, writing them to a Python queue.
- (b) Thread 2 (“stream processor”): reads from the queue; aggregates over

- a 10-second tumbling window; computes total USD, transaction count, and fraud flag count per window.
- (c) Thread 3 (“TSDB writer”): writes each window’s aggregates to an InfluxDB local instance (using the influxdb-client Python library) with a payment_metrics measurement and fields total_usd, txn_count, fraud_count.
 - (d) Configure a local Grafana instance to display the three metrics as a time-series chart with a 10-second auto-refresh interval.
 - (e) Run the pipeline for 5 minutes and capture a screenshot showing the live chart updating.

```

1  import queue, time, random, threading
2  from datetime import datetime, timezone
3  from influxdb_client import InfluxDBClient, Point
4  from influxdb_client.client.write_api import SYNCHRONOUS
5
6  INFLUX_URL      = "http://localhost:8086"
7  INFLUX_TOKEN    = "mytoken"
8  INFLUX_ORG      = "cse4345"
9  INFLUX_BUCKET   = "payment_metrics"
10 WINDOW_SEC      = 10
11
12 event_queue: queue.Queue = queue.Queue(maxsize=5000)
13
14 def producer():
15     """Generates 100 synthetic events/sec."""
16     while True:
17         for _ in range(100):
18             event_queue.put({
19                 "amount":      round(random.uniform(50, 5000), 2)
20                 ,
21                 "is_fraud":    random.random() < 0.003,    # 0.3%
22                             fraud_rate
23                 "ts":         time.time(),
24             })
25             time.sleep(1)
26
27 def stream_processor(write_api):
28     """Aggregates events in 10-second tumbling windows."""
29     window_start = time.time()
30     total_usd, txn_count, fraud_count = 0.0, 0, 0

```

```

29
30 while True:
31     try:
32         event = event_queue.get(timeout=0.05)
33         total_usd += event["amount"]
34         txn_count += 1
35         fraud_count += int(event["is_fraud"])
36     except queue.Empty:
37         pass
38
39     now = time.time()
40     if now - window_start >= WINDOW_SEC:
41         # Write window aggregate to InfluxDB
42         point = (Point("payment_metrics")
43                 .field("total_usd", total_usd)
44                 .field("txn_count", txn_count)
45                 .field("fraud_count", fraud_count)
46                 .time(datetime.now(timezone.utc)))
47         write_api.write(bucket=INFLUX_BUCKET, org=
48                        INFLUX_ORG, record=point)
49         print(f"Window: {txn_count} txns | "
50               f"USD {total_usd:,.0f} | "
51               f"Fraud {fraud_count}")
52         total_usd, txn_count, fraud_count = 0.0, 0, 0
53         window_start = now
54
55 if __name__ == "__main__":
56     client = InfluxDBClient(url=INFLUX_URL, token=
57                            INFLUX_TOKEN, org=INFLUX_ORG)
58     write_api = client.write_api(write_options=SYNCHRONOUS)
59
60     t_prod = threading.Thread(target=producer, daemon=True)
61     t_proc = threading.Thread(target=stream_processor,
62                              args=(write_api,), daemon=True)
63
64     t_prod.start()
65     t_proc.start()
66
67     print("Pipeline running. Press Ctrl+C to stop.")
68     time.sleep(300) # run for 5 minutes

```

Listing 9.6: PE9.3: Streaming aggregation and InfluxDB write

Further Reading

- **Tufte 1983** — Self-published masterpiece that founded the analytical visualization discipline. Chapters 1–4 cover data-ink ratio, chartjunk, and the lie factor with hundreds of printed examples.
- **Shneiderman 1996** — The original mantra paper. Five pages. Read it in full; every page has a principle directly applicable to dashboard design.
- **Bostock, Ogievetsky, and Heer 2011** — The D3 paper. Section 3 (Design Considerations) explains why D3 provides general primitives rather than chart types, and how the enter-update-exit pattern works. Reading this alongside the D3 API documentation is the fastest path to production D3 fluency.
- **Acharya and Chellappan 2015** — Chapter 10 covers data visualisation tools for big data, including Tableau, R, and Python with a practical emphasis suited to the course level.
- **Wilke, C. O. (2019). *Fundamentals of Data Visualization*. O'Reilly.** A modern, freely available book that extends Tufte's principles with colour theory, perceptual accuracy research, and R ggplot2 implementation examples. (Available at clauswilke.com/dataviz.)
- **Bostock, M. (2022). *Observable Plot documentation*.** Bostock's successor to D3 for exploratory data analysis in the browser; simpler API while preserving the data-binding philosophy.

Domain Applications: Social Networks, Time Series, and Healthcare

“Big data hubris is the often implicit assumption that big data are a substitute for, rather than a supplement to, traditional data collection and analysis.”

David Lazer, Ryan Kennedy, Gary King & Alessandro Vespignani, *Science*, 343(6176), 2014

Learning Objectives

After completing this chapter, you will be able to:

1. Explain four structural properties of social media graphs—power-law degree distribution, small-world diameter, asymmetric directed edges, and community structure—and state the implications of each for analysis at big data scale.
2. Describe the Louvain community detection algorithm: its modularity objective, two-phase greedy procedure, and $O(n \log n)$ complexity, and explain why it is preferred over spectral clustering for billion-node graphs.
3. Define influence maximisation, state Kempe, Kleinberg & Tardos’s NP-hardness result and their greedy $(1 - 1/e)$ approximation guarantee, and identify which nodes to seed for maximum information

spread.

4. Apply VADER sentiment analysis to social media text, interpret the compound score, and describe how VADER handles capitalisation, punctuation, degree modifiers, and negation differently from bag-of-words approaches.
5. Explain what makes time series data fundamentally different from i.i.d. tabular data: autocorrelation, seasonality, trend, and non-stationarity, and describe how $\text{ARIMA}(p, d, q)$ addresses each.
6. Describe Facebook Prophet's additive decomposition $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$, explain how automatic changepoint detection and Fourier seasonality work, and explain why the model is preferred over ARIMA for business forecasting at scale.
7. Describe two anomaly detection methods for time series—STL decomposition with residual IQR thresholding and Isolation Forest—and explain the circumstances under which each is appropriate.
8. Design a vectorised batch forecasting pipeline using PySpark's `applyInPandas` that fits one Prophet model per partition across thousands of time series simultaneously.
9. Identify and contrast four healthcare data modalities—EHR/FHIR, genomics (VCF/Parquet), wearables (Kafka streaming), and medical imaging (DICOM)—and select the appropriate storage, processing, and privacy technique for each.
10. Explain three privacy-preserving techniques for healthcare analytics—safe harbour de-identification, differential privacy, and federated learning—and identify which regulatory framework (HIPAA, GDPR, or CCPA) mandates each.

The preceding nine chapters built the complete data engineering stack: distributed storage (Chapter 3), batch processing (Chapter 4), resource management and file formats (Chapter ??), in-memory processing with Spark (Chapter 6), SQL at scale (Chapter 7), streaming pipelines and data lakes (Chapter 8), and visualization (Chapter 9). This chapter applies the complete stack to three high-impact application domains that collectively define where big data systems deliver the most measurable value: social media

analytics, time series forecasting, and healthcare.

Each domain presents a distinct combination of data characteristics (graph-structured, temporally ordered, and privacy-sensitive respectively) and a distinct set of algorithmic techniques. All three domains share the same underlying infrastructure—HDFS or object storage for persistence, Kafka for event ingestion, Spark for distributed processing, and Grafana or Superset for visualization—demonstrating that the engineering stack built in previous chapters is not domain-specific but universally applicable.

The chapter closes with three research spotlights and a case study that illustrates both the power and the peril of big data in public health: the rise and fall of Google Flu Trends, and the more robust Prophet-based syndromic surveillance architecture that replaced it.

10.1 Three Domains of Big Data Application

The same infrastructure serves radically different application domains. Table 10.1 summarises the characteristics, tools, and primary analytical tasks for the three domains covered in this chapter.

10.2 Domain A: Social Media Analytics

Social media platforms are among the largest-scale data systems humans have ever built. Twitter (now X) processes 500 million posts per day; Facebook logs 100 billion interactions; TikTok serves 1 billion video views. In many markets, Facebook remains the primary channel for crisis communication, political discourse, and commercial marketing. The data they generate is graph-structured, temporally indexed, linguistically complex, and at a scale that requires the full Hadoop/Spark stack to process.

10.2.1 Graph Structure and Scale

Social platforms are fundamentally *directed graphs*: users are nodes; following, friending, retweeting, and replying are edges. The properties of

Domain	Data Characteristics	Primary Algorithms	Algorithms	Domain Tools
Social Media	Graph-structured, temporal, text-heavy, multimedia	Louvain, PageRank, influence maximisation, VADER, LSH		GraphX, NetworkX, VADER, Neo4j,
Time Series	Ordered, autocorrelated, seasonal, non-stationary	ARIMA, Prophet, LSTM, STL, Isolation Forest		statsmodels, Prophet, Spark MLlib
Healthcare	Semi-structured EHR, genomic, streaming wearables, binary imaging	Random Forest, Gradient Boosting, CNN, AlphaFold		FHIR, GATK, TensorFlow, HAPI

Shared infrastructure: HDFS / S3 · Kafka · Spark · Hive · Grafana / Superset

Table 10.1: Three big data application domains. All three share the same distributed infrastructure; domain-specific tools sit above the common stack.

these graphs differ dramatically from the random graphs assumed by naive statistical models.

Definition | Social Network Graph Properties

A **social network graph** $G = (V, E)$ has four structural properties that distinguish it from random graphs:

1. **Power-law degree distribution:** the fraction of nodes with degree k follows $P(k) \propto k^{-\alpha}$ for some $\alpha \in (2, 3)$. A small number of “hub” nodes (celebrities, media accounts) have millions of followers; the vast majority have tens. This is called a *scale-free network* after Barabási & Albert (1999).
2. **Small-world property:** despite billions of nodes, the average shortest path between any two nodes is very short—Kwak et al. (2010) found 4.12 hops on Twitter’s 41.7-million-node graph. Milgram’s “six degrees of separation” (1967) observed this empirically in 1967; it is now a mathematical consequence of power-law degree distributions.
3. **Asymmetric directed edges:** on Twitter, only 22.1% of following relationships are reciprocal. On Facebook (undirected, mutual friendship), reciprocity is 100%. The directedness of the graph determines how information propagates.
4. **Community structure:** nodes cluster into densely interconnected groups (communities) that are sparsely connected to other communities. Political echo chambers, professional networks, and fan communities are all examples.

Storage and processing. A social graph with N nodes and M edges requires $O(N + M)$ storage. Twitter’s 2010 graph had $N = 41.7$ million users and $M = 1.47$ billion directed edges, storing roughly 40 GB of compressed adjacency data. At 2024 scale ($N = 500$ million users), the full following graph is ~500 GB compressed. Storing and querying this graph requires distributed systems:

- **Apache GraphX** (part of Spark) for large-scale iterative graph algorithms (PageRank, connected components, triangle count) running on the full graph in Spark’s in-memory model.

- **Neo4j** for online, low-latency graph queries (“show me the mutual friends of user X and user Y”) where Spark’s batch latency would be unacceptable.
- **Apache Hive / Parquet** for offline analytical queries over the graph’s edge list (“how many edges were added in Q1 2024?”).

10.2.2 Community Detection: The Louvain Algorithm

A **community** in a graph is a group of nodes that are more densely connected to each other than to the rest of the graph. Detecting communities reveals interest groups, political factions, professional clusters, or coordinated bot networks. It is one of the most practically important operations in social network analysis.

Definition | Modularity and the Louvain Algorithm (Blondel et al., 2008)

Modularity Q measures the fraction of edges that fall within communities minus the expected fraction under a null model of random edge placement:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

where A_{ij} is the adjacency matrix, k_i is node i ’s degree, m is the total number of edges, and $\delta(c_i, c_j) = 1$ if nodes i and j are in the same community.

The **Louvain algorithm** maximises Q in two phases repeated iteratively:

1. **Local optimisation:** each node is moved to the neighbour community that maximises the modularity gain ΔQ . A node stays in its own community if no move improves Q . This passes over all nodes until no further improvement is possible.
2. **Network aggregation:** communities become super-nodes. Edges between communities become weighted edges between super-nodes. The algorithm is then applied to the aggregated network.

Complexity: $O(n \log n)$ on sparse graphs—scalable to graphs with hundreds of millions of nodes.

```
1 import networkx as nx
```

```

2  import community as community_louvain    # pip install python-
    louvain
3  import matplotlib
4  matplotlib.use("Agg")
5  import matplotlib.pyplot as plt
6
7  # Construct a simple directed follow graph (Twitter-style)
8  # In production: load edge list from HDFS Parquet via Spark
9  G_directed = nx.DiGraph()
10 edges = [
11     (1, 2), (2, 1), (1, 3), (3, 1),
12     (2, 3), (3, 4), (4, 5), (5, 4),
13     (4, 6), (6, 5), (5, 7), (7, 5),
14     (1, 8), (8, 4),    # bridge nodes
15 ]
16 G_directed.add_edges_from(edges)
17
18 # Louvain requires undirected input
19 G = G_directed.to_undirected()
20
21 # --- Apply Louvain ---
22 partition = community_louvain.best_partition(G)
23 modularity = community_louvain.modularity(partition, G)
24 print(f"Modularity Q = {modularity:.4f}")
25
26 communities = {}
27 for node, comm_id in partition.items():
28     communities.setdefault(comm_id, []).append(node)
29
30 for comm_id, members in sorted(communities.items()):
31     print(f"    Community {comm_id}: nodes {sorted(members)}")
32
33 # Visualise
34 pos = nx.spring_layout(G, seed=42)
35 colors = [partition[n] for n in G.nodes()]
36 fig, ax = plt.subplots(figsize=(8, 6))
37 nx.draw_networkx(G, pos=pos, node_color=colors,
38                 cmap=plt.cm.Set1, node_size=500,
39                 font_color="white", ax=ax)
40 ax.set_title(f"Louvain Communities (Q = {modularity:.3f})")
41 ax.axis("off")
42 plt.tight_layout()
43 plt.savefig("louvain_communities.png", dpi=150)
44 print("Saved: louvain_communities.png")

```

Listing 10.1: Community detection with Louvain in Python (networkx + python-louvain)

System Design Note | Scaling Community Detection to Billion-Node Graphs

For graphs with hundreds of millions of nodes, Python’s python-louvain library is too slow. Production implementations use:

- **Apache Spark GraphX:** the Pregel API implements iterative vertex-program algorithms including Louvain phases; the full graph is partitioned across executors and edges are shuffled only when communities cross partition boundaries.
- **Neo4j Graph Data Science (GDS) library:** implements Louvain natively with a streaming result API; suitable for online, low-latency community refresh.
- **GraphSAGE / Graph Neural Networks** (Hamilton et al., 2017): for graphs where node feature attributes (text, image embeddings) should inform community assignment, GNNs learn community representations end-to-end.

For the *full* Twitter graph at 500 million nodes, Louvain in Spark GraphX requires approximately 3–5 iterations to converge, with each iteration involving one full shuffle of the edge list. On a 100-executor Spark cluster with 16 GB RAM per executor, this completes in ~20–40 minutes.

10.2.3 Influence Maximisation

Given a social network and a budget of k “seed” users, the *influence maximisation* problem asks: which k users should receive an initial message so that the maximum number of users are ultimately reached through organic sharing?

This problem has commercial applications (viral marketing campaigns, product seeding), public health applications (vaccination campaign targeting), and disinformation applications (coordinated bot networks that seed false narratives at high-PageRank nodes).

Definition | Influence Maximisation: NP-Hard with $(1-1/e)$ Approximation (Kempe, Kleinberg & Tardos, 2003)

Under the **independent cascade model**, each edge (u, v) has a probability p_{uv} that user u activates user v in a single time step. The influence $\sigma(S)$ of a seed set S is the expected number of activated nodes after cascades run to completion.

Problem: Find $S^* = \arg \max_{|S| \leq k} \sigma(S)$.

Hardness: The problem is NP-hard; $\sigma(S)$ is #P-hard to compute exactly.

Greedy approximation: The function $\sigma(\cdot)$ is *monotone submodular*: adding a node to a larger set provides no more benefit than adding it to a smaller set. Nemhauser's theorem guarantees that the greedy algorithm—at each step, add the node that maximises the marginal gain $\sigma(S \cup \{v\}) - \sigma(S)$ —achieves

$$\sigma(S_{\text{greedy}}) \geq \left(1 - \frac{1}{e}\right) \sigma(S^*) \approx 0.632 \cdot \sigma(S^*)$$

This is the best polynomial-time approximation ratio possible unless $P = NP$.

Key Insight | PageRank vs. Follower Count for Seeding

Kwak et al. (2010) demonstrated that users ranked by *follower count* and users ranked by *PageRank* are substantially different populations. A user with 1 million followers who is followed by many high-PageRank users (e.g., other celebrities and media accounts) has higher influence than a celebrity with 5 million followers who is followed by low-influence accounts. For influence maximisation campaigns, seeding at high-PageRank nodes produces measurably more cascade depth than seeding at high-follower-count nodes. This is because PageRank correctly captures the *recursive* nature of influence: being followed by influential people makes you more influential.

10.2.4 Sentiment Analysis: VADER

Sentiment analysis assigns a polarity (positive, negative, or neutral) to a piece of text. At social media scale—millions of tweets per hour—it pro-

vides real-time measures of public opinion, brand sentiment, and crisis emotion.

Standard bag-of-words and TF-IDF approaches fail on social media text because they ignore the conventions of online communication: capitalisation for emphasis, punctuation for intensity, emojis for emotion, and degree modifiers (“very”, “kind of”, “absolutely”) that shift sentiment magnitude.

Definition | VADER: Valence Aware Dictionary and sEntiment Reasoner (Hutto & Gilbert, 2014)

VADER is a lexicon-and-rule-based sentiment analyser specifically designed for social media text. Its lexicon assigns a sentiment valence score to 7,517 lexical features (words, emojis, slang terms). Five heuristic rules modify these base scores:

1. **Capitalisation:** GREAT is scored higher than great (amplification factor: +0.733).
2. **Punctuation:** each exclamation mark adds +0.292 to a positive score up to a maximum of three.
3. **Degree modifiers:** “very good” > “good” > “kind of good”. Boosters add +0.293; dampeners subtract.
4. **Negation:** a negation word within three positions of a sentiment word inverts and attenuates the score by multiplying by −0.74: “not good” → negative.
5. **Contrastive conjunctions:** “but” signals that the clause following it is more salient than the clause preceding.

Output: a **compound score** $\in [-1, +1]$. Convention: > 0.05 is positive; < -0.05 is negative; otherwise neutral.

```

1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import udf, col, window, avg
3 from pyspark.sql.types import FloatType
4 from vaderSentiment.vaderSentiment import
   SentimentIntensityAnalyzer
5
6 spark = SparkSession.builder.appName("TwitterSentiment").
   getOrCreate()
7 spark.sparkContext.setLogLevel("WARN")
8

```

```

9  # Broadcast the VADER analyser -- one object reused per
    executor
10  sid_broadcast = spark.sparkContext.broadcast(
    SentimentIntensityAnalyzer())
11
12  @udf(returnType=FloatType())
13  def vader_compound(text: str) -> float:
14      if text is None:
15          return 0.0
16      sid = sid_broadcast.value
17      return float(sid.polarity_scores(text)["compound"])
18
19  # Load Twitter stream from Kafka (see ch08 for Kafka source
    config)
20  tweets_df = (spark.readStream
21      .format("kafka")
22      .option("kafka.bootstrap.servers", "kafka:9092")
23      .option("subscribe", "twitter-bd-2024")
24      .load()
25      .selectExpr("CAST(value AS STRING) AS text",
26          "timestamp"))
27
28  # Apply VADER UDF
29  sentiment_df = tweets_df.withColumn(
30      "compound", vader_compound(col("text")))
31
32  # Aggregate: average sentiment per 5-minute tumbling window
33  agg_df = (sentiment_df
34      .groupBy(window(col("timestamp"), "5 minutes"))
35      .agg(
36          avg("compound").alias("avg_sentiment"),
37          count("*").alias("tweet_count")
38      ))
39
40  # Stream to console (replace with InfluxDB sink for Grafana)
41  query = (agg_df.writeStream
42      .outputMode("complete")
43      .format("console")
44      .option("truncate", False)
45      .start())
46
47  query.awaitTermination()

```

Listing 10.2: VADER sentiment in a PySpark pipeline (UDF on tweet DataFrame)

10.2.5 Trend Detection and the Twitter News-Media Finding

Kwak et al. (2010) analysed 106 million tweets and found that 85% of Twitter’s trending topics were *headline news events*—not personal conversations or social memes. The median duration of a trending topic was 17.5 hours, matching the lifespan of a news cycle. Breaking news topics reached their peak tweet rate within 15 minutes of the event—faster than most professional news agencies.

This finding established Twitter as a *real-time news medium* rather than a social network, with profound implications for public health surveillance: if Twitter reflects real-world events within minutes, then flu-related tweets should correlate with actual flu incidence faster than physician-reported surveillance data.

Trend detection algorithm. Twitter’s trending topics algorithm detects sudden relative spikes in phrase frequency using a variant of Locality-Sensitive Hashing (LSH) on character n -grams. A phrase that normally appears 10 times per hour and suddenly appears 10,000 times per hour is “trending”—regardless of its absolute frequency. This *relative* thresholding is what distinguishes a trend from simply a very popular phrase.

Common Misconception | Ethical Risks of Social Media Analytics

The same techniques that enable flu surveillance and brand monitoring enable political surveillance and targeted manipulation:

- Sentiment analysis on political posts can be weaponised to identify and suppress dissent—flagging negative sentiment about government policy for content removal.
- Influence maximisation can be used by coordinated inauthentic behaviour campaigns (bot networks) to artificially amplify misinformation by seeding it at high-PageRank nodes.
- Community detection can expose political affiliations, religious identities, or sexual orientations of users who have not consented to dis-

close them—violating privacy through *inference*.

Any deployment of social media analytics must include an explicit purpose-limitation policy (stating what the analysis will and will not be used for) and an independent algorithmic audit.

10.3 Domain B: Time Series Analysis

Time series data is fundamentally different from i.i.d. tabular data. A transaction table can be analysed by shuffling its rows without loss of information. A time series cannot: the temporal order of observations is itself the information. This section covers the properties that make time series special, the classical ARIMA model, Facebook Prophet’s modern approach to large-scale forecasting, and two anomaly detection methods.

10.3.1 Properties of Time Series Data

Definition | Time Series: Five Structural Properties

A **time series** $\{y_t\}_{t=1}^T$ is a sequence of observations indexed by time. Five structural properties must be understood before modelling:

1. **Autocorrelation:** y_t is correlated with $y_{t-1}, y_{t-2}, \dots$ at multiple lags. Standard regression that assumes $\text{Cov}(y_t, y_{t-k}) = 0$ for $k > 0$ is invalid—its parameter estimates are unbiased but its standard errors and p -values are wrong.
2. **Trend:** a long-run upward or downward drift. Trend must be removed or modelled explicitly; a model that ignores a positive trend will persistently under-predict future values.
3. **Seasonality:** regular cyclical patterns that repeat with a fixed period—daily (web traffic peaks in the evening), weekly (retail sales peak on Friday), or annual (flu season peaks in January–February). Multiple simultaneous seasonalities are common.
4. **Non-stationarity:** the mean or variance of the series changes over time. Most classical models (ARIMA, regression) require *stationarity*

(constant mean and variance). Non-stationarity is typically removed by *differencing*: $\nabla y_t = y_t - y_{t-1}$.

5. **Irregularity (Holiday Effects)**: one-off events (public holidays, national elections, product launches, natural disasters) cause sharp deviations from the seasonal pattern. These cannot be captured by a Fourier seasonality model and must be encoded explicitly.

10.3.2 Classical Forecasting: ARIMA

Definition | ARIMA(p, d, q) — AutoRegressive Integrated Moving Average

The **ARIMA**(p, d, q) model has three components:

- **AR**(p): the current value is a linear function of the past p values:
 $y_t = \phi_1 y_{t-1} + \dots + \phi_p y_{t-p} + \varepsilon_t$.
- **I**(d): the series is differenced d times to achieve stationarity. First-order differencing ($d = 1$) removes linear trend; second-order ($d = 2$) removes quadratic trend.
- **MA**(q): the current value depends on the past q forecast errors:
 $y_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}$.

Parameter selection uses the **Akaike Information Criterion (AIC)**; `auto.arima` (R) and `pmdarima.auto_arima` (Python) automate this search.

When ARIMA is appropriate: short stationary series with no strong seasonality, requiring exact confidence intervals. Small datasets ($T < 500$) where Prophet’s Bayesian MAP estimation overfits. Regulatory contexts (financial forecasting) that require documented statistical assumptions.

When ARIMA is *not* appropriate: multiple overlapping seasonalities (ARIMA’s SARIMA extension handles only one); 10,000+ parallel series requiring automated fitting without expert tuning of p, d, q for each; irregular calendar effects (holidays, special events) that fall outside the seasonal period.

10.3.3 Facebook Prophet: Decomposable Forecasting at Scale

Prophet (Taylor and Letham 2018) was developed at Facebook’s Core Data Science team to address a concrete business need: daily forecasts for thousands of internal metrics (daily active users, ad impressions, revenue) that had to be produced automatically, without a statistician tuning ARIMA parameters for each series.

Definition | Prophet’s Additive Decomposition (Taylor & Letham, 2018)

Prophet models a time series as an additive composition of four interpretable components:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$$

- **Trend** $g(t)$: piecewise linear (or logistic) growth. Structural trend changes (*changepoints*) are detected automatically by placing potential changepoints at regular intervals and imposing a sparse prior $\delta_j \sim \text{Laplace}(0, \tau)$ on the slope changes. The hyperparameter τ (called `changepoint_prior_scale`) controls trend flexibility: small values produce smooth trends; large values produce flexible trends that may overfit.
- **Seasonality** $s(t)$: a Fourier series captures periodic patterns at multiple resolutions simultaneously:

$$s(t) = \sum_{n=1}^N \left(a_n \cos \frac{2\pi nt}{P} + b_n \sin \frac{2\pi nt}{P} \right)$$

where P is the period (365.25 for annual; 7 for weekly). Multiple periods (annual *and* weekly) are summed additively. The order N controls smoothness.

- **Holiday effects** $h(t)$: a user-specified table of one-off events (Black Friday, national elections, sporting finals, bank holidays) is encoded as indicator variables that absorb the effect of each event. Events with multi-day impact (e.g., a long weekend) are represented with a window.
- **Noise** $\varepsilon(t)$: assumed i.i.d. Gaussian; the residual not captured by the

three structural components.

Parameters are estimated via maximum a posteriori (MAP) estimation using the probabilistic programming language **Stan**, which is fast enough to fit thousands of series in parallel.

```

1 import pandas as pd
2 import numpy as np
3 from prophet import Prophet
4 import matplotlib
5 matplotlib.use("Agg")
6 import matplotlib.pyplot as plt
7
8 rng = np.random.default_rng(2024)
9
10 # Simulate 3 years of daily e-commerce sales volume
11 dates = pd.date_range("2021-01-01", periods=3*365, freq="D")
12 t      = np.arange(len(dates))
13
14 trend      = 1_000_000 + 500 * t                # growing
15     trend
16 weekly_s   = 200_000 * np.sin(2*np.pi * t / 7)  # weekly
17     seasonality
18 annual_s   = 500_000 * np.sin(2*np.pi * t / 365 - np.pi/2)
19 noise      = rng.normal(0, 80_000, len(t))
20 y          = trend + weekly_s + annual_s + noise
21
22 df = pd.DataFrame({"ds": dates, "y": y})
23
24 # Define shopping holidays (volume spikes during Black Friday /
25     Cyber Monday)
26 holidays = pd.DataFrame({
27     "holiday": "BlackFriday",
28     "ds": pd.to_datetime([
29         "2021-11-26", "2021-11-29",
30         "2022-11-25", "2022-11-28",
31         "2023-11-24", "2023-11-27",
32     ]),
33     "lower_window": -1, # 1 day before
34     "upper_window": 2,  # 2 days after
35 })
36
37 m = Prophet(

```



```

35     yearly_seasonality = True,
36     weekly_seasonality = True,
37     daily_seasonality = False,
38     holidays = holidays,
39     changepoint_prior_scale = 0.05,      # conservative trend
40     seasonality_prior_scale = 10,
41 )
42 m.fit(df)
43
44 # Forecast 90 days ahead
45 future = m.make_future_dataframe(periods=90)
46 forecast = m.predict(future)
47
48 fig1 = m.plot(forecast)
49 fig1.axes[0].set_title("E-commerce Daily Sales Volume --
    Prophet Forecast")
50 fig1.savefig("sales_forecast.png", dpi=150, bbox_inches="tight"
    )
51
52 fig2 = m.plot_components(forecast)
53 fig2.savefig("sales_components.png", dpi=150, bbox_inches="
    tight")
54 print("Saved: sales_forecast.png and sales_components.png")

```

Listing 10.3: Prophet on one time series: e-commerce daily sales volume with Black Friday holiday effect

10.3.4 Anomaly Detection in Time Series

An *anomaly* in a time series is an observation that does not conform to the expected pattern—either a transient spike, a structural break, or a slow drift outside expected bounds. Anomaly detection is a critical component of fraud monitoring, network operations, and predictive maintenance.

Method 1: STL Decomposition. STL (Seasonal-Trend decomposition using LOESS) decomposes the time series into trend, seasonal, and residual components using locally weighted regression (LOESS) for each. Residuals that exceed $k \times \text{IQR}$ of all residuals are flagged as anomalies. STL is interpretable (the analyst can inspect each component), handles missing data gracefully, and works well when the seasonal pattern is stable.

Method 2: Isolation Forest. An *Isolation Forest* is an ensemble of random trees that *isolates* anomalies by recursively splitting the feature

space at randomly chosen thresholds. An anomaly requires fewer splits to isolate than a normal observation, because it lies in a sparse region of the feature space. The anomaly score is the average *path length* across all trees: short path length indicates an anomaly. Isolation Forest makes no distributional assumptions, handles high-dimensional feature spaces, and scales to millions of observations in Spark MLlib.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, hour, dayofweek, lag
3 from pyspark.sql.window import Window
4 from pyspark.ml.feature import VectorAssembler
5 from pyspark.ml.classification import RandomForestClassifier
6
7 # NOTE: Spark MLlib does not have a native IsolationForest (as
8   # of Spark 3.5).
9 # Use the spark-iforest package or scikit-learn on Pandas
10   # partitions.
11 # Here we show the applyInPandas approach with sklearn.
12
13 from pyspark.sql.types import StructType, StructField,
14   DoubleType, IntegerType
15 from sklearn.ensemble import IsolationForest
16 import pandas as pd
17
18 spark = SparkSession.builder.appName("AnomalyDetection").
19   getOrCreate()
20
21 # Example: payment agent hourly transaction counts
22 df = spark.read.parquet("hdfs:///data/payments/agent_hourly_txn
23   /")
24
25 # Feature engineering: add lag features
26 w = Window.partitionBy("agent_id").orderBy("hour_ts")
27 df = (df
28   .withColumn("lag1", lag("txn_count", 1).over(w))
29   .withColumn("lag24", lag("txn_count", 24).over(w))
30   .withColumn("hour_of_day", hour(col("hour_ts")))
31   .withColumn("dow", dayofweek(col("hour_ts")))
32   .na.drop())
33
34 # Schema for the output of our Pandas UDF
35 output_schema = StructType([
36   StructField("agent_id", df.schema["agent_id"].dataType,
```

```

    True),
32     StructField("hour_ts", df.schema["hour_ts"].dataType,
    True),
33     StructField("txn_count", DoubleType(), True),
34     StructField("anomaly_score", DoubleType(), True),
35     StructField("is_anomaly", IntegerType(), True),
36 ])
37
38 def detect_anomalies(pdf: pd.DataFrame) -> pd.DataFrame:
39     """Fit Isolation Forest per agent partition."""
40     features = ["txn_count", "lag1", "lag24", "hour_of_day", "dow"]
41     X = pdf[features].fillna(0).values
42
43     if len(X) < 10:          # too few observations
44         pdf["anomaly_score"] = 0.0
45         pdf["is_anomaly"]    = 0
46         return pdf[["agent_id", "hour_ts", "txn_count",
47                    "anomaly_score", "is_anomaly"]]
48
49     clf = IsolationForest(n_estimators=100,
50                           contamination=0.02, # expect 2%
51                           anomalies
52                           random_state=42)
53
54     pdf["anomaly_score"] = -clf.score_samples(X) # higher =
55     pdf["is_anomaly"]    = (clf.predict(X) == -1).astype(int)
56     return pdf[["agent_id", "hour_ts", "txn_count",
57                "anomaly_score", "is_anomaly"]]
58
59 # One IsolationForest per agent; runs in parallel across Spark
60 # partitions
61 anomalies = (df.groupBy("agent_id")
62              .applyInPandas(detect_anomalies, schema=
63                             output_schema))
64
65 anomalies.filter(col("is_anomaly") == 1) \
66           .orderBy("anomaly_score", ascending=False) \
67           .show(20, truncate=False)

```

Listing 10.4: Anomaly detection with Isolation Forest in PySpark MLlib

10.3.5 Vectorised Batch Forecasting in Spark

A retail chain needs 90-day demand forecasts for 5,000 store–product SKU combinations, each with its own seasonal pattern and promotional calendar. Fitting 5,000 separate Prophet models sequentially on a single machine would take 8+ hours. Fitting them in parallel across a Spark cluster reduces this to ~12 minutes on a 20-executor cluster —a 40× speedup.

The pattern is called *vectorised batch forecasting*: each SKU’s time series is assigned to one Spark partition; a Pandas UDF (applyInPandas) runs the full Prophet fit-and-predict cycle inside that partition; results are written as Parquet for serving.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col
3 from pyspark.sql.types import (StructType, StructField,
4     StringType, DateType, DoubleType)
5 from prophet import Prophet
6 import pandas as pd
7 import time
8
9 spark = SparkSession.builder \
10     .appName("VectorisedProphet") \
11     .config("spark.sql.execution.arrow.pyspark.enabled", "true"
12         ) \
13     .getOrCreate()
14
15 # Load historical sales: schema (sku_id, date, units_sold)
16 sales = spark.read.parquet("hdfs:///data/retail/daily_sales/")
17
18 # Output schema: what each partition will return
19 forecast_schema = StructType([
20     StructField("sku_id", StringType(), True),
21     StructField("ds", DateType(), True),
22     StructField("yhat", DoubleType(), True),
23     StructField("yhat_lower", DoubleType(), True),
24     StructField("yhat_upper", DoubleType(), True),
25 ])
26
27 def fit_and_forecast(pdf: pd.DataFrame) -> pd.DataFrame:
28     """Fit Prophet on one SKU's history; return 90-day forecast
29     """
30     sku_id = pdf["sku_id"].iloc[0]
```

```

30     # Prophet requires columns named 'ds' and 'y'
31     train = pdf.rename(columns={"date": "ds", "units_sold": "y"
32                               })
32     train["ds"] = pd.to_datetime(train["ds"])
33     train = train[["ds", "y"]].sort_values("ds")
34
35     if len(train) < 14:      # need at least 2 weeks of data
36         return pd.DataFrame(columns=["sku_id", "ds", "yhat",
37                                     "yhat_lower", "yhat_upper"
38                                     ])
39
40     m = Prophet(
41         yearly_seasonality      = True,
42         weekly_seasonality      = True,
43         changepoint_prior_scale = 0.05,
44         seasonality_prior_scale = 10.0,
45     )
46     m.fit(train)
47
48     future = m.make_future_dataframe(periods=90)
49     forecast = m.predict(future)
50
51     result = forecast[["ds", "yhat", "yhat_lower", "yhat_upper"
52                      ]].copy()
53     result["sku_id"] = sku_id
54     result["ds"] = result["ds"].dt.date
55     return result[["sku_id", "ds", "yhat", "yhat_lower", "
56                   yhat_upper"]]
57
58 t0 = time.perf_counter()
59
60 forecasts = (sales
61              .repartition(200, "sku_id")  # one or more SKUs per
62              .partition
63              .groupBy("sku_id")
64              .applyInPandas(fit_and_forecast, schema=forecast_schema))
65
66 # Write forecasts to Parquet for the inventory management
67 # dashboard
68 (forecasts.write
69  .mode("overwrite")
70  .partitionBy("sku_id")
71  .parquet("hdfs:///data/retail/sku_forecasts_90d/"))

```

```
68 forecasts.count()      # trigger execution
69 elapsed = time.perf_counter() - t0
70 print(f"5,000 SKU forecasts completed in {elapsed/60:.1f}
    minutes")
```

Listing 10.5: Vectorised Prophet forecasting: 5000 SKUs in parallel with `applyInPandas`

10.4 Domain C: Healthcare Big Data

Healthcare is the domain where big data analytics is most consequential and most constrained. The potential benefits—earlier disease detection, personalised treatment, pandemic preparedness—are matched by the sensitivity of the data and the severity of the consequences of errors.

10.4.1 Healthcare Data Modalities

Healthcare generates four structurally distinct data modalities, each requiring different storage, processing, and privacy approaches.

1. Electronic Health Records (EHR). EHR systems contain structured data (ICD-10 diagnosis codes, medication orders, laboratory results, vital signs), semi-structured data (HL7 FHIR resources, as covered in Chapter 2), and unstructured data (clinical notes written by physicians in natural language). A typical hospital generates 2–5 GB of new EHR data per day. At the national level, large healthcare systems such as the NHS (UK) or Kaiser Permanente (US) hold 15+ years of digitised records, representing sufficient scale for population-level clinical analytics.

2. Genomics. A single whole-genome sequence at 30× coverage produces ~200 GB of raw sequencing reads (FASTQ format), which compress to ~50 GB as an aligned BAM file and ~1 GB as a called variant VCF file. A clinical genomics study of 10,000 patients generates 2 PB of raw data—requiring the HDFS-class distributed storage introduced in Chapter 3 and the GATK (Genome Analysis Toolkit) pipeline ported to Spark for distributed variant calling.

3. Wearables and IoT. Continuous monitoring devices (smartwatches, continuous glucose monitors, cardiac monitors, hospital bedside vital-sign

monitors) generate time-series data at per-second resolution. A single ICU patient generates ~1 GB of monitoring data per day across all bedside devices. This is precisely the streaming ingestion scenario from Chapter 8: Kafka receives the device events, Spark Structured Streaming applies anomaly detection in real time, and an InfluxDB time-series database stores the per-second aggregates for Grafana dashboards on the nursing station.

4. Medical Imaging. CT scans (200–500 MB per study), MRI scans (200 MB–2 GB), digital pathology slides (2–10 GB per slide), and chest X-rays (50 MB) are stored in *DICOM* format—a binary file format with embedded metadata headers. DICOM is fundamentally unstructured: there is no predefined row-column schema that SQL can query. Analysis requires deep learning models (convolutional neural networks for radiology, vision transformers for pathology slides) running on GPU infrastructure. PACS (Picture Archiving and Communication Systems) store these files; the Hadoop stack is not the primary processing environment for imaging analytics.

Modality	Format		Storage		Processing		Privacy Risk
EHR / FHIR	HL7 JSON, relational	FHIR relational	Hive / PostgreSQL	PostgreSQL	Spark NLP	SQL, on	High
Genomics	VCF / FASTQ	BAM / events	HDFS Parquet	HDFS	GATK Spark	on	Very high
Wearables	JSON (Kafka)	events	InfluxDB, HDFS	HDFS	Spark Streaming	on	Medium
Medical Imaging	DICOM binary	bi-nary	PACS / S3	S3	CNN / ViT GPU	on	High

Table 10.2: Healthcare data modalities and their infrastructure requirements. Genomic data carries the highest privacy risk: a person’s genome is permanently identifying and cannot be de-identified by removing names and dates.

10.4.2 Clinical Prediction Models

(Obermeyer and Emanuel 2016) argued in a landmark New England Journal of Medicine paper that machine learning models trained on EHR data at scale could systematically outperform physician heuristics for three critical clinical tasks.

ICU mortality prediction. A random forest trained on vital signs, laboratory values (serum lactate, creatinine, bilirubin, platelet count), and prior diagnosis codes achieves an AUROC greater than 0.90 for predicting which ICU patients will die within 24 hours. Conventional ICU scoring systems (APACHE-IV, SOFA) achieve AUROC ≈ 0.74 –0.82. The difference in AUROC translates to 6–8 hours of earlier identification of deteriorating patients—time in which palliative care discussions can occur or escalation of treatment can be initiated.

30-day hospital readmission. Gradient boosting on structured discharge records (diagnosis codes, procedure codes, length of stay, laboratory trends, medication count) identifies high-risk patients for post-discharge follow-up interventions. When these interventions are implemented, readmission rates fall by 15–20%, reducing both patient suffering and hospital readmission penalties under value-based care contracts.

Sepsis early warning. Sepsis is a life-threatening organ dysfunction caused by infection; each hour of delayed treatment increases mortality by 7%. A continuous streaming model fed by Kafka (receiving bedside vital sign updates every 60 seconds) and Spark Structured Streaming can detect sepsis onset 6 hours before the standard clinical criteria (SIRS or qSOFA) are met, enabling prophylactic antibiotic administration.

Key Insight | Data Scale Requirements for Clinical ML

Obermeyer and Emanuel’s meta-analysis found that clinical prediction models trained on fewer than 10,000 patient records are generally unreliable for clinical deployment: the models overfit to local population characteristics and fail to generalise to patients with comorbidities not well-represented in the training data. Reliable generalisation requires 100,000+ patient records for common diagnoses (heart failure, pneumonia, sepsis) and 1,000,000+ for rare conditions. Large national health systems (NHS England, Kaiser Permanente, VA Health System) record

tens of millions of inpatient encounters per year—if integrated and de-identified, sufficient for mortality and readmission modelling.

10.4.3 Privacy: HIPAA, GDPR, and the CCPA

Healthcare data is the most sensitive personal data category recognised by law. Three regulatory frameworks govern its handling.

Definition	Healthcare Privacy Regulatory Frameworks
	• HIPAA (USA) — Health Insurance Portability and Accountability Act (1996): requires de-identification of 18 specific “protected health information” (PHI) identifiers (name, date of birth, National ID number, zip code, telephone, email, account numbers, etc.) before re-search use. Two de-identification methods are defined: the <i>Safe Harbour</i> method (remove all 18 identifiers) and the <i>Expert Determination</i> method (a qualified statistician certifies that re-identification risk is very small). • GDPR (EU) — General Data Protection Regulation (2018): treats health data as a “special category” requiring explicit consent for processing. Data minimisation (collect only what is needed), purpose limitation (use only for the stated purpose), and right to erasure (the “right to be forgotten”) apply. GDPR’s €20 million or 4% of global revenue fines have made it the effective global standard. • CCPA (California) — California Consumer Privacy Act (2020): grants residents the right to know what personal data is collected, to opt out of its sale, and to request deletion. Health data collected outside HIPAA-covered entities (e.g., by consumer wellness apps) falls under CCPA rather than HIPAA. Many US states are enacting similar legislation, and CCPA serves as a model for comprehensive state-level privacy law.

Technical mitigation techniques. Four engineering approaches exist for enabling analytics on sensitive healthcare data without exposing individual records.

1. **Safe harbour de-identification.** Remove all 18 HIPAA-specified

identifiers. After safe harbour, the data is no longer legally PHI and can be shared for research without patient consent. Limitation: dates must be generalised to year only; zip codes must be generalised to the first 3 digits—changes that reduce the data’s utility for geospatial and temporal analyses.

2. Differential privacy. Add calibrated Gaussian or Laplace noise to the output of any aggregate query such that no individual record can be inferred from the query result. The privacy guarantee is formal: a mechanism $\mathcal{M}$ is (ϵ, δ) -differentially private if adding or removing any single individual’s record changes the output distribution by at most a factor e^ϵ with probability $1 - \delta$. Smaller ϵ (stronger privacy) requires more noise (less accuracy). Apple, Google, and the US Census Bureau use differential privacy for aggregate statistics release.

3. Federated learning. Rather than centralising patient data in a single data lake, each hospital trains the model locally on its own data. Only the model’s gradient updates (parameter changes, not raw data) are sent to a central coordinator, which aggregates them using *federated averaging* (McMahan et al., 2017). The raw patient records never leave the hospital. Federated learning is the standard approach for multi-site clinical trials and cross-hospital model training in GDPR-regulated environments.

4. Synthetic data generation. Generative models (Variational Autoencoders, Generative Adversarial Networks) trained on real patient records produce *synthetic* patient records that preserve the statistical properties of the original data while containing no real individuals. Synthetic data can be shared publicly without legal restriction. Limitation: the generative model itself must be trained on real data, so data access remains necessary at the training stage.

10.4.4 AlphaFold: Big Data and Deep Learning in Science

AlphaFold (Jumper et al. 2021) is the most dramatic example of big data combined with deep learning transforming an experimental science. For 50 years, predicting a protein’s three-dimensional structure from its amino acid sequence was considered one of biology’s hardest problems—the “protein folding problem.” Experimental determination by X-ray crystallography or cryo-electron microscopy takes months per protein and costs thou-

sands of dollars. As of 2020, the Protein Data Bank (PDB) contained approximately 170,000 experimentally determined structures—a tiny fraction of the ~250 million known protein sequences in the UniProt database.

AlphaFold 2 (DeepMind, 2020) solved this problem with:

- A transformer-based architecture (*Evoformer*) that operates jointly on the amino acid sequence and a *multiple sequence alignment (MSA)*—a matrix of thousands of evolutionarily related sequences from 250 million protein sequences in UniProt.
- Training on all 170,000 PDB structures using 128 TPU v3 cores over several weeks.
- Prediction accuracy: median GDT score >92 on the CASP14 benchmark—the first computational method to achieve expert-level accuracy.

In 2022, DeepMind released the *AlphaFold Protein Structure Database*: predicted structures for 200 million proteins across all known organisms, covering effectively the entire known proteome—a resource that took experimental biology 70 years to partially populate. The database is freely accessible and has been used in vaccine design, drug discovery, and fundamental biology research.

Why this is a big data problem. The key input to AlphaFold is not just the query sequence—it is the MSA. Generating an MSA for a single query requires searching 250 million sequences (200+ GB compressed), aligning the top hits, and encoding the result as a 2-D attention matrix. This search alone takes 3–5 minutes on a high-memory CPU server. Training required petabyte-scale data access across the PDB and UniRef90. AlphaFold is in this sense less a breakthrough in neural network architecture than a breakthrough in what becomes possible when a deep learning model is trained on a complete and expertly curated scientific dataset at scale.

Production Lesson | AlphaFold's Limitation: Dynamics and Complexes

AlphaFold predicts *static* protein structures in isolation. Two fundamental limitations remain for practical drug discovery:

1. Many drug targets are *intrinsically disordered proteins* (IDPs) that have no fixed structure—they fold only upon binding their interaction partner. AlphaFold's confidence scores for IDPs are low and should not be trusted as structural predictions.

2. AlphaFold predicts monomeric (single-chain) structures. Many proteins function as *complexes* (multi-chain assemblies). AlphaFold-Multimer (2021) and AlphaFold 3 (2024) address this, but with lower accuracy than the monomer model.

In production use for drug discovery pipelines, AlphaFold predictions are treated as *starting hypotheses* for experimental validation, not as definitive structures.

10.5 Research Spotlight

Research Spotlight | Kwak et al. (2010) · Taylor & Letham (2018) · Obermeyer & Emanuel (2019)

Work 1: Kwak, H., Lee, C., Park, H., & Moon, S. (2010). “What is Twitter, a Social Network or a News Media?” *Proceedings of WWW 2010*, pp. 591–600. ACM. (>7,000 citations; KAIST.)

The paper’s central question was deceptively simple: is Twitter’s graph structure more like a social network (mutual relationships, dense local communities, information spreading through friendship chains) or a news medium (one-to-many broadcasting, shallow cascades, content driven by events rather than social ties)?

To answer it, Kwak, Lee, Park, and Moon crawled Twitter’s full public graph in June 2009—41.7 million users, 1.47 billion directed following edges—using breadth-first search via the Twitter API over three weeks. The resulting 40 GB compressed adjacency dataset was one of the first big data social graph studies.

Three findings reshaped social media analytics:

1. **Twitter is a news medium.** 85% of trending topics were headline news events, not personal conversations. The median trending topic lasted 17.5 hours. Breaking news reached peak tweet rate within 15 minutes—faster than news wire services.
2. **Asymmetric structure.** Only 22.1% of following relationships are reciprocal—far lower than Facebook’s 100%. The in-degree distribution follows a power law with exponent $\alpha \approx 2.276$: a small number of

celebrity accounts have millions of followers, while the vast majority have tens. The average shortest path across the full 41.7-million-node graph is 4.12 hops.

3. **PageRank diverges from follower count.** Users ranked by PageRank (recursive influence in the directed graph) differ substantially from those ranked by raw follower count. A user followed by many high-PageRank accounts outranks a celebrity followed by low-influence accounts.

Impact. The news-medium finding directly enabled social media epidemiology: CDC, WHO, and academic groups built influenza, COVID-19, and mental health surveillance systems on the Twitter graph model. The directed-cascade model of retweets is the reference network structure for misinformation propagation studies (Vosoughi, Roy & Aral, 2018, *Science*: “False news spreads faster, farther, and more broadly than true news”). Twitter’s own Trends algorithm redesigns were shaped by the paper’s finding that trending topics are news events.

Work 2: Taylor, S. J., & Letham, B. (2018). “Forecasting at Scale.” *The American Statistician*, 72(1), 37–45. (>1,500 citations; Facebook Core Data Science.)

The Prophet paper addresses a practical industrial problem: Facebook needed daily forecasts for thousands of internal business metrics (daily active users, ad impressions, revenue per cohort)—automatically, without a statistician manually tuning ARIMA parameters for each series. Classical approaches (ARIMA, exponential smoothing) had three failure modes at this scale: they required expert parameter selection (p , d , q) that could not be automated reliably across heterogeneous time series; they struggled with complex multi-period seasonality (annual and weekly simultaneously); and they had no mechanism for handling irregular calendar effects (holidays that fall on different dates each year). Prophet’s key innovations were:

1. **Interpretable decomposition:** the additive $g(t) + s(t) + h(t) + \varepsilon(t)$ structure lets a domain analyst (not a statistician) inspect and correct each component independently.
2. **Automatic changepoint detection:** a sparse Laplace prior on trend

slope changes detects structural breaks without over-segmenting the trend.

3. **Fourier seasonality:** annual, weekly, and daily seasonality are represented as overlapping Fourier series, capturing all simultaneously without the SARIMA workaround of a single fixed period.
4. **Stan-based MAP estimation:** fast enough to fit thousands of models per day; calibrated uncertainty intervals outperform ARIMA's theoretical confidence intervals on real data.

Impact. Prophet became the de facto industry standard for business time series forecasting within two years: adopted by Airbnb, Uber, Walmart, and most major tech companies. The `applyInPandas` + Prophet vectorisation pattern is now the canonical PySpark design pattern for parallel time series forecasting. In healthcare, Prophet is used for ICU capacity forecasting, hospital admission nowcasting, and CDC FluSight flu season projections.

Work 3: Obermeyer, Z., & Emanuel, E. J. (2016). "Predicting the Future—Big Data, Machine Learning, and Clinical Medicine." *New England Journal of Medicine*, 375(13), 1216–1219. (>2,000 citations; NEJM; foundational clinical ML position paper.)

Obermeyer and Emanuel's paper is not a primary research study—it is a perspective article written for the clinical audience of the NEJM, arguing on the basis of existing evidence that machine learning models trained on EHR big data can and should be integrated into clinical practice. Its influence is exceptional because it appeared in the world's most prestigious medical journal and was written in language accessible to practising physicians rather than computer scientists.

Three clinical ML results they synthesised:

1. **ICU mortality:** random forest on vitals and labs achieves AUROC > 0.90 vs. APACHE's ≈ 0.74 —earlier identification of deteriorating patients.
2. **30-day readmission:** gradient boosting on discharge records identifies high-risk patients for post-discharge follow-up, reducing readmission by 15–20%.

3. **Sepsis:** continuous streaming models detect sepsis 6 hours before clinical criteria—time enough for prophylactic antibiotics.

The paper also warned of three failure modes: biased training data (if certain patient populations are underrepresented, predictions are unreliable for those groups); overfitting to local population characteristics that do not generalise to other hospitals; and the “black-box problem”—clinicians distrust predictions they cannot explain. These warnings have proved prescient: subsequent audits of deployed clinical ML models have found racial and socioeconomic bias, poor cross-site generalisation, and physician distrust leading to models being ignored in practice even when technically accurate.

10.6 Case Study: CDC Syndromic Surveillance and the Google Flu Trends Lesson

Case Study | CDC BioSense and the Prophet-Based Flu Surveillance Architecture (2009–2013)

Context: Traditional Surveillance Lag. The CDC’s traditional influenza surveillance system, ILINet, relies on >3,000 sentinel physicians across all 50 US states reporting weekly counts of influenza-like illness (ILI) patients—defined as fever > 100°F plus cough or sore throat. The data flow is: clinic → state health department → CDC weekly report. The lag between a patient’s first symptoms and the CDC receiving the report is **1–2 weeks**. By the time CDC identifies a surge, it may already have passed its peak.

Google Flu Trends: The Promise and the Failure. In 2008, Ginsberg et al. at Google published in *Nature* a system called Google Flu Trends (GFT): by correlating 45 flu-related search queries (“flu symptoms”, “how to treat fever”, etc.) with ILINet surveillance data, GFT could *predict* flu incidence two weeks before CDC’s official reports with $r^2 = 0.97$ on 5 years of training data. GFT was cited as the canonical example of “big data surpassing traditional surveillance.”

In the 2012–2013 flu season, GFT over-predicted flu incidence by a

factor of 2×. Lazer et al. (*Science*, 2014) (Lazer et al. 2014) diagnosed three causes:

1. **Big Data hubris:** $r^2 = 0.97$ on a 5-year training window does not guarantee generalisation when the data-generating process is non-stationary. Search behaviour changes as the internet population and search interface evolve.
2. **Algorithm confounding:** Google’s autocomplete algorithm (opaque to GFT’s developers) suggested flu-related search terms to users during flu news coverage, amplifying search volume *independently* of actual incidence.
3. **Media amplification:** heavy news coverage of a flu season drove flu-related searches even among people who were not ill, creating a spurious correlation between media attention and predicted (but not actual) incidence.

GFT was discontinued as a primary source and eventually taken offline in 2015.

BioSense: The Better Architecture. The CDC’s BioSense Platform represents the mature, post-GFT approach to syndromic surveillance:

- **Primary signal: direct medical observation.** BioSense receives near-real-time electronic health data from >3,500 emergency departments, urgent care centres, and hospitals via HL7 FHIR messages. Syndromes (influenza-like, gastrointestinal, neurological) are automatically extracted from chief complaints using NLP. Lag: <72 hours vs. ILINet’s 1–2 weeks.
- **Secondary signal: Twitter/social media.** Twitter flu-related keywords are monitored as a *leading indicator* supplementing the direct medical signal—not replacing it. The Kwak et al. finding that Twitter reflects real-world events within minutes makes it valuable as a 3–5 day leading signal. But unlike GFT, the Twitter signal is an input to a model alongside clinical data, not the sole prediction basis.
- **Forecasting: Prophet + Ensemble.** CDC’s FluSight forecasting challenge (2013–present) pools submissions from academic teams worldwide. The top performers are always ensemble methods. Prophet

is a strong component: it models the annual flu season’s characteristic double-peaked shape (a primary peak in January–February, a secondary peak in March) as overlapping seasonal components, and its changepoint detection identifies season-start changepoints within 1–2 weeks.

Scale: 2019–2020 Season.

Metric	Value
Emergency department records processed	38 million visits
FHIR messages ingested per day	~250,000
HHS regions with independent Prophet models	60
Spark executors for Prophet batch inference	20
Twitter keyword stream volume monitored	~500,000 flu-related tweets/day
FluSight ensemble models combined	38 (2019–2020 season)

Engineering Lessons.

1. **Proxy data amplifies noise, not signal.** GFT’s failure was that search queries capture *health anxiety and media attention* rather than health incidence. Proxy signals require causal validation before policy use; correlation is not sufficient.
2. **Algorithm transparency is a public health issue.** GFT failed partly because Google’s internal autocomplete changes were opaque to CDC. When a public health surveillance system depends on a third party’s algorithm, that algorithm’s changes become a public health risk.
3. **Decomposability improves trust.** Prophet’s $g + s + h$ structure lets epidemiologists inspect each component—an epidemiologist can identify which changepoint corresponds to the season start, which holiday effect captures school opening, and whether the trend component reflects a genuine long-term burden change. Black-box deep learning forecasters produce comparable accuracy but generate less physician and public health official trust.
4. **Ensemble beats any single model.** The CDC FluSight top perform-

ers from 2013 to 2022 are always ensemble methods. No single model—Prophet, ARIMA, LSTM, or human expert—wins every season. The value of Prophet in the ensemble is its interpretability: when the ensemble disagrees with a Prophet component, epidemiologists can inspect the component and understand why.

5. **Twitter adds lead time, not accuracy.** Social media signals are available 3–5 days earlier than BioSense ED data but with 30–40% higher error. The trade-off is latency versus precision—an acceptable trade-off for an early warning signal but not for a primary surveillance measure.

Chapter Summary

Three high-impact application domains demonstrate that the data engineering stack built in previous chapters is universally applicable.

Social media analytics treats platforms as directed graphs and applies community detection (Louvain, $O(n \log n)$, modularity maximisation), influence maximisation (NP-hard, $(1 - 1/e)$ -approximable by greedy), and real-time sentiment analysis (VADER: compound score handles capitalisation, punctuation, degree modifiers, and negation) to problems ranging from viral marketing to public health surveillance. Kwak et al. (2010) established that Twitter is a news medium, not a social network—a finding that unlocked social media epidemiology.

Time series analysis begins with ARIMA's classical approach (differencing for stationarity, autoregressive and moving average components for autocorrelation) and extends to Facebook Prophet's decomposable $g + s + h$ model (piecewise linear trend with automatic changepoint detection, Fourier seasonality, holiday tables). Vectorised batch forecasting via `applyInPandas` parallelises one Prophet model per series across thousands of series simultaneously in Spark. STL and Isolation Forest address the anomaly detection problem.

Healthcare big data combines four data modalities (EHR/FHIR, genomics, wearables, DICOM imaging), three privacy frameworks (HIPAA,

GDPR, CCPA), and four privacy-engineering techniques (safe harbour, differential privacy, federated learning, synthetic data). Clinical prediction models at scale outperform physician heuristics for ICU mortality, readmission, and sepsis detection. AlphaFold demonstrated that a 50-year scientific grand challenge could be solved by training a transformer on a complete curated database at scale.

The CDC/Google Flu Trends case study provides the chapter's most important cautionary lesson: high in-sample correlation does not guarantee out-of-sample accuracy when the data-generating process is non-stationary, and proxy data captures anxiety and media attention as much as the phenomenon it purports to measure.

Exercises

Short Questions

SQ10.1. What are the four structural properties of social network graphs described in this chapter? For each, state one implication for how information spreads through the network.

SQ10.2. Describe the two phases of the Louvain community detection algorithm. What is the modularity objective Q , and why does maximising it detect communities?

SQ10.3. Why is PageRank a better measure of user influence on Twitter than raw follower count? Describe a specific scenario where the two measures would rank users in a different order.

SQ10.4. VADER outputs a "compound score" for social media text. For each of the following tweets, state whether the compound score will be higher or lower than the score for the plain form, and explain which VADER rule causes the difference: (a) "GREAT concert" vs. "great concert"; (b) "not good" vs. "good"; (c) "absolutely fantastic!!!" vs. "fantastic".

SQ10.5. What is autocorrelation in a time series, and why does it invalidate standard regression's standard errors? Give an example from a business

time series (e.g., daily e-commerce transactions) where strong autocorrelation would be expected.

SQ10.6. Describe each component of Prophet’s model $y(t) = g(t) + s(t) + h(t) + \varepsilon(t)$. What hyperparameter controls the flexibility of the trend component, and what happens if it is set too high?

SQ10.7. A Prophet model for weekly hospital outpatient visits shows an unexplained spike every year in late April and late June. Which Prophet component is responsible for these spikes? What information would you add to the model to capture them explicitly?

SQ10.8. What are STL decomposition and Isolation Forest? For each, describe what type of anomaly it is best suited to detect in a time series.

SQ10.9. What is the difference between a *safe harbour* de-identification and *differential privacy* for healthcare data? Give one scenario where safe harbour is sufficient and one where differential privacy would also be needed.

SQ10.10. What was the core argument of Obermeyer & Emanuel (2016)? List two specific clinical tasks where they argued ML models outperform physician heuristics, and state the metric used to measure outperformance.

SQ10.11. Explain why Google Flu Trends over-predicted flu incidence in 2012–2013. Name all three causes identified by Lazer et al. (2014) and describe which is the most fundamental.

SQ10.12. What is *federated learning*? Why is it used in multi-hospital clinical machine learning, and what is the primary limitation of the federated averaging algorithm?

Analytical Problems

AP10.1. Influence Maximisation Calculation. A Facebook group for a CDC dengue alert campaign has 1 million users. The following-graph has a power-law degree distribution with exponent $\alpha = 2.3$. The campaign budget allows seeding $k = 5$ initial posts.

- (a) Under the independent cascade model with $p_{uv} = 0.05$ on each edge, estimate (qualitatively) whether seeding at the 5 users with the highest follower count or the 5 users with the highest PageRank will achieve

- greater spread. Justify your answer using the Kwak et al. finding on PageRank vs. follower count.
- (b) The Kempe et al. greedy algorithm guarantees a $(1 - 1/e) \approx 0.632$ approximation ratio. If the optimal seed set S^* achieves influence $\sigma(S^*) = 200,000$ users, what is the worst-case influence of the greedy seed set? Is this a meaningful guarantee for a public health campaign? Discuss.
- (c) Kempe et al.'s greedy algorithm requires evaluating $\sigma(S)$ by Monte Carlo simulation (running the cascade 10,000 times). For a graph of 1 million nodes, $k = 5$ seed nodes, and 100 candidate nodes to evaluate at each greedy step, estimate the total number of cascade simulations required. Is this tractable on a single machine?

AP10.2. Prophet Model Diagnostics. A Prophet model was fitted to 3 years of weekly dengue case data for a tropical metropolitan region. The fitted components show:

- Trend $g(t)$: an increasing linear trend with two changepoints: at week 52 (slope increase) and week 130 (slope decrease).
 - Seasonality $s(t)$: a strong annual peak centred on weeks 32–38 (August–September) coinciding with the post-monsoon season.
 - Residuals $\varepsilon(t)$: mostly near zero except for weeks 88, 89, and 90, where residuals are +150, +220, and +180 cases above the seasonal expected value.
- (a) What real-world events might explain the two changepoints at weeks 52 and 130? (Hint: consider vector control interventions, climate events, and reporting system changes.)
- (b) The large positive residuals in weeks 88–90 were not captured by the seasonal component. What type of event would cause a 3-week anomaly in dengue cases, and how would you modify the Prophet model to capture it in future forecasts?
- (c) The public health director asks whether the model can predict a dengue surge 8 weeks in advance. Describe the specific uncertainty that makes 8-week forecasts unreliable in a dengue context (not a general statistical argument—specific to dengue epidemiology).

AP10.3. Healthcare Privacy Analysis. A health system researcher wants to analyse 5 years of inpatient records (500,000 patients) from 10 hospitals to build a 30-day readmission prediction model. The data includes: patient name, national ID number, date of birth, home region, ICD-10 diagnosis codes, procedure codes, admission date, length of stay, and discharge outcome.

- (a) List all fields that would be removed under HIPAA's safe harbour method. Which field is most uniquely identifying and why?
- (b) The researcher wants to release a public version of the dataset for academic use. Under GDPR's data minimisation principle, which fields should be retained and which should be removed or generalised? Justify each decision.
- (c) A second research team at a partner university wants to train on their own 50,000-patient dataset and combine insights via federation. Why would federated learning be preferable to data centralisation? Describe the federated averaging protocol and identify one risk.
- (d) The final deployed model has AUROC 0.82. The Chief Data Officer asks: "Is the model equally accurate for patients from rural regions as for patients from urban centres?" Describe the analysis you would run to answer this question and the corrective action if the answer is no.

AP10.4. Vectorised Forecasting Performance Analysis. A telecom company needs to forecast daily call-drop rates for 6,000 cell towers, each with 2 years of daily measurements.

- (a) Design the Spark job using `applyInPandas`. Specify: the `DataFrame` schema, the partition key, the Pandas UDF signature, and the output schema for 30-day forecasts.
- (b) A single Prophet fit on 730 daily observations takes approximately 2 seconds. Estimate the wall-clock time to fit all 6,000 tower models on:
 - (i) a single machine with 8 cores; (ii) a Spark cluster with 20 executors each having 8 cores.
- (c) The Prophet models for 300 towers (5%) produce $\text{MAPE} > 30\%$ on a holdout set. Describe three diagnostic checks you would perform to identify why these towers have poor forecast accuracy.
- (d) Propose a strategy for handling towers with fewer than 60 days of data

(e.g., new towers installed in the past 2 months).

Design Problems

DP10.1. National Disease Surveillance System. Design a big data disease surveillance system analogous to the CDC BioSense platform. The system should detect outbreaks of dengue fever, influenza, and COVID-19 up to 7 days before the health authority receives clinical confirmation.

- (a) **Data sources:** identify four non-traditional data sources analogous to the CDC's syndromic surveillance inputs (social media, pharmacy, ER, search). For each, identify the entity that controls the data and the access mechanism.
- (b) **Ingestion pipeline:** specify the Kafka topic structure (topic name, partition count, key, value format) for each source. Explain why Kafka is preferable to a direct database write for multi-source signal aggregation.
- (c) **Signal processing:** describe the Spark Structured Streaming window aggregation for the social media signal (window size, slide, output metric). Describe the Prophet model configuration for the weekly hospitalization series.
- (d) **Alert logic:** specify the threshold that triggers an outbreak alert. Should the threshold be a fixed case count, a percentage above the Prophet forecast, or a combination? Justify.
- (e) **Privacy safeguards:** the social media signal includes geotagged posts and the pharmacy signal includes named individuals' medication purchases. Specify the de-identification steps before these signals enter the pipeline, citing GDPR Article 9 and HIPAA Safe Harbour requirements.

DP10.2. Multi-Hospital Clinical ML Pipeline. A regional health authority wants to deploy a 30-day readmission prediction model trained on data from five hospitals in a metropolitan region. Data sharing between hospitals is restricted by HIPAA / GDPR.

- (a) Design a federated learning architecture: specify which component

runs at each hospital, what is transmitted to the central server, and how the federated averaging step works.

- (b) Specify the minimum feature set that can be computed from ICD-10 codes, length of stay, and admission month without transmitting any PHI identifiers. Explain the trade-off between feature richness and privacy risk.
- (c) The central model achieves AUROC 0.79 after 10 federated rounds. Hospital A's local model (50,000 patients) achieves AUROC 0.84. The hospital director asks whether to use the global model or the local model. Analyse the trade-offs and provide a recommendation.
- (d) Describe two bias risks in the federated model arising from the dataset heterogeneity across five hospitals (e.g., different patient populations, different documentation practices) and propose a mitigation for each.

Programming Exercises

PE10.1. Social Network Analysis with NetworkX and Louvain. Construct a social network from a synthetic dataset of 1,000 Twitter users and 15,000 directed following edges, then analyse its structure.

- (a) Generate the synthetic directed graph using networkx with a preferential attachment model (`barabasi_albert_graph`). Compute: number of nodes, number of edges, average in-degree, maximum in-degree, reciprocity rate, and average shortest path length (on the largest connected component).
- (b) Apply the Louvain algorithm (`python-louvain`). Report: number of communities detected, modularity Q , and the size of the five largest communities.
- (c) Compute PageRank for all nodes. Identify the top-5 nodes by PageRank and the top-5 by in-degree. Are they the same nodes? Calculate the Spearman rank correlation between PageRank and in-degree rankings.
- (d) Create a visualisation: draw the network with nodes coloured by community assignment (Louvain partition), node size proportional to PageRank, and inter-community edges drawn as dashed grey lines.

PE10.2. Prophet Forecasting with Holiday Effects. Using the Prophet library, build a demand forecasting model for an international retailer with strong year-end seasonal spikes.

- (a) Generate 3 years of synthetic weekly sales data with: an upward linear trend, strong annual seasonality (peak in late November / December holiday season), moderate weekly seasonality (peak on Saturday), and random Gaussian noise.
- (b) Fit two Prophet models: one without holidays and one with Black Friday, Cyber Monday, and Christmas dates explicitly encoded with a window of $[-3, +3]$ days. Compare the MAPE on a 12-week holdout set.
- (c) Plot the component decomposition for the model with holidays. Identify the estimated holiday effect size (in sales units) for Black Friday.
- (d) Change `changepoint_prior_scale` from 0.01 to 0.5 in steps. Plot how the fitted trend changes. At what value does the model begin to overfit to noise in the training data?

PE10.3. Healthcare Data Pipeline (FHIR + Spark). Build a Spark pipeline that simulates processing EHR data from multiple hospitals for a 30-day readmission prediction task.

- (a) Generate 100,000 synthetic patient records with fields: `patient_id` (UUID), `admission_date`, `region` (50 values), `primary_icd10` (50 codes), `los_days`, `readmitted_30d` (binary label, 15% positive rate).
- (b) Apply safe harbour de-identification: generate a surrogate `anon_id` (SHA-256 hash of patient UUID), generalise `admission_date` to `admission_year_month` and group `region` into 10 macro-regions.
- (c) Using Spark MLlib, train a Gradient Boosted Tree classifier on the de-identified features. Report AUROC, precision, recall, and F1 on a 20% holdout set.
- (d) Stratify the AUROC by region. Identify the three regions with the lowest AUROC and hypothesise (in 2–3 sentences) why the model performs worse in those regions.

Further Reading

- **Kwak et al. 2010** — Five pages; read it in full. The network topology findings (Sections 3–4) and the trending topics analysis (Section 5) are the most cited. The data collection section (Section 2) is worth reading for its description of rate-limit-aware BFS crawling—a practical lesson for any large-scale API data collection.
- **Taylor and Letham 2018** — Open access at peerj.com/preprints/3190. Read Sections 2 (model) and 3 (analyst-in-the-loop). The Appendix contains the Stan code for the full model—useful if you want to extend Prophet’s priors for a specific domain.
- **Obermeyer and Emanuel 2016** — Two pages in NEJM. Read alongside Caruana et al.’s 2015 KDD paper on “Intelligible Models for Healthcare” for the interpretability counterargument: high-AUROC models that physicians cannot explain are frequently ignored in clinical practice.
- **Lazer et al. 2014** — Three pages in *Science*. A masterclass in the epistemology of big data: why $r^2 = 0.97$ on training data means almost nothing, and why algorithm opacity is a scientific problem.
- **Jumper et al. 2021** — Read the introduction and the “Results” first paragraph. The Evoformer architecture section (Methods) is deep and requires familiarity with attention mechanisms; save it for after studying transformers.
- **Leskovec, Rajaraman & Ullman. *Mining of Massive Datasets*, 3rd ed. Chapter 10:** social network structure, community detection, influence propagation. Freely available at mmds.org.
- **Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*, 3rd ed.** Freely available at otexts.com/fpp3. The definitive modern time series forecasting textbook; Chapters 9–10 cover ARIMA and Prophet in depth.

Part VII

The Modern Data Stack

The Lakehouse, MLOps, and Emerging Architectures

“The exciting thing about the Lakehouse is that it is not a compromise — it is actually better than either a data warehouse or a data lake at their own jobs.”

Matei Zaharia, co-creator of Apache Spark and
Delta Lake

Learning Objectives

After completing this chapter, you will be able to:

1. Describe the four eras of big data technology (2003–present), explain the engineering problem that drove each transition, and identify which era’s infrastructure remains relevant in modern production deployments.
2. Explain the Lakehouse architecture: how an open table format (Delta Lake, Apache Iceberg, Apache Hudi) stored on cheap object storage achieves ACID transactions via optimistic concurrency control, time travel via a transaction log, and schema enforcement at the storage layer.
3. Compare Delta Lake’s transaction log mechanism against the traditional data warehouse approach and the raw data lake approach on dimensions of ACID compliance, open format, BI support, ML

- support, time travel, and storage cost.
4. Explain Apache Iceberg’s hidden partitioning and partition evolution capabilities and identify the scenario where Iceberg is preferred over Delta Lake.
 5. Describe the four principles of the Data Mesh paradigm (domain ownership, data as a product, self-serve platform, federated computational governance) and explain which engineering bottleneck each principle addresses.
 6. Identify the seven components of an MLOps pipeline (ingestion, feature engineering, feature store, model training, experiment tracking, model registry, serving and monitoring), describe the tool used for each, and explain what “hidden technical debt” each component eliminates.
 7. Describe the three-tier edge–cloud architecture for IoT analytics (device, edge gateway, cloud) and explain what processing happens at each tier and why.
 8. Define (ϵ, δ) -differential privacy formally, explain the Laplace noise mechanism, and describe the accuracy–privacy trade-off controlled by the privacy budget ϵ .
 9. Describe the Dataflow Model’s four questions (What, Where, When, How), explain how batch processing is a special case of streaming under this model, and explain how Apache Beam eliminates the Lambda architecture’s dual code path.
 10. Apply Delta Lake’s core operations (create, append, update, time travel, DESCRIBE HISTORY) using PySpark and the `delta-spark` library.

Every chapter in this book has described a layer of the data engineering stack as it existed at a specific moment in the field’s development. Chapter 3 described HDFS and the assumption that storage and compute were co-located on-premise. Chapter 4 described MapReduce’s batch processing model. Chapter 6 described how Spark replaced MapReduce with in-memory processing. Chapter 8 described Kafka, streaming pipelines, and the Lambda architecture’s attempt to unify batch and stream.

This final chapter completes the arc. It describes the fourth era of big data technology—the *Lakehouse*—which resolves the central tension between data lakes (cheap, flexible, but unreliable) and data warehouses (reliable, fast, but expensive and closed). It also covers the organisational architecture (*Data Mesh*) that resolves the bottleneck created when a single platform team owns all data pipelines; the operational discipline (*MLOps*) that bridges the gap between a trained model and a deployed, monitored production system; the computational pattern (*edge analytics*) that extends distributed processing to IoT devices; and the theoretical framework (the *Dataflow Model*) that eliminates the Lambda architecture’s fundamental complexity by treating batch as a special case of streaming.

11.1 The Technology Evolution Arc

Big data technology has evolved through four identifiable eras, each driven by a specific engineering bottleneck that the previous era’s tools could not resolve.

Key Insight | Technologies Stratify, They Do Not Die

HDFS and MapReduce from Era 1 are still running in production at legacy installations across banking, telecommunications, and government. A big data architect in 2025 must understand all four eras because they will encounter systems from each era in real deployments. Era 1 systems have often been running for 10+ years and cannot be migrated without significant business risk. The progression in Table 11.1 is not a replacement sequence—it is an accretion sequence. Each era added a new layer without fully eliminating the previous one.

11.2 Lakehouse Architecture

The Lakehouse resolves the central architectural tension of the 2010s: organisations needed both the flexibility and cost-efficiency of a data lake *and* the reliability and query performance of a data warehouse, but running both

Era	Period	Defining Tools	Key Innovation	Bottleneck Resolved
1	2003–2011	GFS, MapReduce, Hadoop, Hive	Commodity hardware cluster replaces supercomputer	Storage cost; single-machine limit
2	2011–2017	Spark, Kafka, YARN, Parquet	In-memory processing; persistent event log	MapReduce latency; batch-only processing
3	2017–2022	AWS S3, EMR, Dataproc, Snowflake	Compute and storage separation; managed services	On-premise hardware ops; cluster idle cost
4	2022–now	Delta Lake, Iceberg, MLflow, Apache Beam	ACID on object stores; unified BI+ML; batch=stream	Data lake unreliability; Lambda complexity; ML ops debt

Table 11.1: Four eras of big data technology. Tools from earlier eras are still running in production at large enterprises — technologies stratify into legacy tiers rather than disappearing.

simultaneously doubled infrastructure cost and created data consistency problems.

11.2.1 The Two-System Problem

By 2018, the standard enterprise architecture for big data analytics comprised two separate systems:

The data lake (HDFS or Amazon S3 with Spark): raw data in Parquet or ORC format, accessible to ML frameworks (Spark MLlib, PyTorch) for model training. Cheap storage (~\$23/TB-month on S3). No ACID guarantees: concurrent writes could corrupt data; there was no schema enforcement to prevent malformed records from entering the lake; and deleting a specific record (required by GDPR’s right to erasure) required reading and rewriting every Parquet file containing that record.

The data warehouse (Amazon Redshift, Google BigQuery, Snowflake): ACID-compliant SQL tables, fast query performance, schema enforcement. Expensive proprietary storage (~\$200–250/TB-month). Closed format: ML frameworks could not read warehouse data directly, so data had to be extracted as CSV or Parquet and loaded into the lake—a nightly ETL copy that created a 12-hour freshness gap and a second copy of all data.

The result was three compounding problems:

1. **Cost:** two systems, two storage bills, two compute bills.
2. **Consistency:** analysts queried the warehouse; data scientists queried the lake. They worked with different copies of the data and frequently produced inconsistent results.
3. **Lambda complexity:** streaming workloads required maintaining two code paths (one for the lake’s batch processing, one for the real-time layer) that had to be kept in sync.

11.2.2 Delta Lake: ACID on Object Stores

(Armbrust, Das, et al. 2020) introduced Delta Lake as an open-source storage layer that adds ACID transactions, scalable metadata handling, and audit history to data lakes on object stores. Its core mechanism is the *Delta Log*: a series of JSON files stored inside the table’s directory in S3 that form a serialisable transaction log.

Definition | Delta Lake Transaction Log (Delta Log)

A **Delta table** on S3 consists of two components stored in the same S3 prefix:

1. **Data files:** immutable Parquet files containing the actual row data. They are never modified in place; instead, new files are written and old files are logically deleted.
2. **Delta Log** (`_delta_log/`): a sequence of JSON files (one per committed transaction: `00000.json`, `00001.json`, ...) each recording the set of files *added* and *removed* by that transaction, plus schema changes and metadata. Every 10 commits, a *checkpoint* Parquet file condenses the full current state of the table for fast reader initialisation.

ACID properties:

- **Atomicity:** S3's PUT operation is atomic. A transaction either appends its JSON entry completely or not at all. A partial write has no effect because readers only follow committed log entries.
- **Isolation (Optimistic Concurrency Control):** writers read the latest committed version, make their changes, and attempt to write the next log entry. If two writers simultaneously attempt to write the same log entry number, S3's conditional PUT ensures only one succeeds; the other detects the conflict and either retries or fails with a `ConcurrentModificationException`.
- **Consistency:** the Delta Log is the single source of truth. Readers always reconstruct the latest committed table state from the log before scanning data files; they can never see a partial write.
- **Durability:** all committed log entries persist in S3 indefinitely. No in-memory coordinator holds state; the log is self-contained.

Time travel. Because every version of the table is preserved in the Delta Log, Delta Lake supports querying any past snapshot:

```

1  -- Read table as of a specific version
2  SELECT * FROM transactions VERSION AS OF 42;
3
4  -- Read table as of a specific timestamp
5  SELECT *
6  FROM   transactions

```

```

7  TIMESTAMP AS OF '2025-01-01 00:00:00';
8
9  -- Inspect the full transaction history
10 DESCRIBE HISTORY transactions;
11
12 -- Roll back to a previous version (overwrites current version)
13 RESTORE TABLE transactions TO VERSION AS OF 42;

```

Listing 11.1: Delta Lake time travel queries

Time travel enables: (a) compliance audit trails (“what did the data look like on December 31?”); (b) reproducible ML experiments (lock training data to a specific version, ensuring identical results on rerun); (c) data recovery from accidental deletes or overwrites.

Schema enforcement and evolution. Delta Lake rejects any write that does not conform to the table’s declared schema—including new columns, missing columns, or type mismatches—unless schema evolution is explicitly enabled with the `mergeSchema` option. Schema enforcement eliminates the “data swamp” failure mode where successive writes from different teams gradually make the table’s schema inconsistent.

Z-Ordering. Delta Lake supports *Z-ordering*: a data locality optimization that co-locates rows with similar values in the same Parquet files, reducing the number of files scanned for multi-column filter queries. A traditional Hive table partitioned by date still requires scanning all files for a query that filters by (`user_id`, `date`). Z-ordering on (`user_id`, `date`) co-locates the data so that up to 95% of files can be skipped.

```

1  from pyspark.sql import SparkSession
2  from delta import configure_spark_with_delta_pip
3  from pyspark.sql.functions import col
4
5  # --- Session setup ---
6  builder = (SparkSession.builder
7      .appName("DeltaLakeDemo")
8      .config("spark.sql.extensions",
9          "io.delta.sql.DeltaSparkSessionExtension")
10     .config("spark.sql.catalog.spark_catalog",
11         "org.apache.spark.sql.delta.catalog.DeltaCatalog"))
12
13  spark = configure_spark_with_delta_pip(builder).getOrCreate()
14

```

```

15 TABLE_PATH = "hdfs:///lakehouse/payments/transactions"
16
17 # --- 1. Create initial Delta table ---
18 from pyspark.sql.types import StructType, StructField, \
19     StringType, DoubleType, TimestampType
20
21 schema = StructType([
22     StructField("txn_id", StringType(), False),
23     StructField("sender", StringType(), True),
24     StructField("receiver", StringType(), True),
25     StructField("amount_usd", DoubleType(), True),
26     StructField("ts", TimestampType(), True),
27     StructField("region", StringType(), True),
28 ])
29
30 df_initial = spark.createDataFrame([
31     ("T001", "user_a1b2c3", "user_d4e5f6", 500.0, None, "
32         Northeast"),
33     ("T002", "user_g7h8i9", "user_j0k1l2", 1200.0, None, "West"
34         ),
35 ], schema=schema)
36
37 (df_initial.write
38     .format("delta")
39     .mode("overwrite")
40     .partitionBy("region")
41     .save(TABLE_PATH))
42
43 # --- 2. Append new batch (transaction 2 -- a new commit) ---
44 df_new = spark.createDataFrame([
45     ("T003", "user_m3n4o5", "user_a1b2c3", 250.0, None, "
46         Northeast"),
47     ("T004", "user_p6q7r8", "user_s9t0u1", 750.0, None, "South"
48         ),
49 ], schema=schema)
50
51 (df_new.write
52     .format("delta")
53     .mode("append")
54     .save(TABLE_PATH))
55
56 # --- 3. UPDATE -- raise a specific amount ---
57 spark.sql(f"""
58     UPDATE delta.`{TABLE_PATH}`

```

```

55     SET      amount_usd = amount_usd * 1.10
56     WHERE   txn_id = 'T001'
57     """
58
59 # --- 4. DELETE -- GDPR right to erasure ---
60 spark.sql(f"""
61     DELETE FROM delta.`{TABLE_PATH}`
62     WHERE   sender = 'user_g7h8i9'
63     """)
64
65 # --- 5. Time travel: read state before DELETE ---
66 v2 = (spark.read
67     .format("delta")
68     .option("versionAsOf", 2)    # version 0=create, 1=append,
69     .load(TABLE_PATH))
70 print("State at version 2 (before DELETE):")
71 v2.show()
72
73 # --- 6. Inspect transaction history ---
74 spark.sql(f"DESCRIBE HISTORY delta.`{TABLE_PATH}`").show(
75     truncate=False)
76
77 # --- 7. Z-ordering for faster multi-column queries ---
78 spark.sql(f"""
79     OPTIMIZE delta.`{TABLE_PATH}`
80     ZORDER BY (sender, region)
81     """)
82
83 # --- 8. Schema enforcement: this will FAIL with
84 #       AnalysisException
85 try:
86     bad = spark.createDataFrame([(999,)], ["txn_id"])    # wrong
87     bad.write.format("delta").mode("append").save(TABLE_PATH)
88 except Exception as e:
89     print(f"Schema enforcement blocked write: {type(e).__name__}
90           ")

```

Listing 11.2: Delta Lake core operations in PySpark

11.2.3 Apache Iceberg and Apache Hudi

Delta Lake is not the only open table format. Two alternatives address different production requirements.

Apache Iceberg (Netflix, open-sourced 2020) uses a *manifest list* approach to the transaction log—a tree of JSON and Avro files that records which data files belong to each snapshot. Iceberg’s key differentiating features are:

- **Hidden partitioning:** partition columns are computed automatically from data columns and stored in table metadata, not exposed in SQL queries. A table partitioned by `months(event_time)` requires no changes to queries when the partition scheme changes—contrast with Hive’s explicit partition column requirement.
- **Partition evolution:** the partitioning scheme can change without rewriting existing data. Old partitions are read under the old scheme; new data uses the new scheme. This is impossible in Hive.
- **Multi-engine support:** Iceberg tables can be read by Spark, Trino, Flink, Dremio, and Presto simultaneously without format conversion. Delta Lake (as of 2024) requires a compatibility layer (UniForm) for non-Spark engines.

Apache Hudi (Uber, open-sourced 2019) is optimised for *record-level upserts*: updating or deleting individual rows in a large Parquet dataset without rewriting entire files. Hudi is the natural choice when the primary workload is CDC (Change Data Capture)—streaming database change events from MySQL or PostgreSQL into the lake and applying them as upserts in near-real time.

Definition	Lakehouse Architecture (Zaharia et al., CIDR 2021)
A Lakehouse is a data management system with the following properties:	
1. Cheap, open-format storage: data is stored as Parquet or ORC files on commodity object storage (S3, Azure Data Lake Storage, Google Cloud Storage) at ~\$23/TB-month. 2. Metadata and governance layer: an open table format (Delta Lake, Iceberg, or Hudi) adds ACID transactions, schema enforcement, time	

Feature	Delta Lake	Apache Iceberg	Apache Hudi
Origin	Databricks	Netflix	Uber
Transaction log	JSON files	Manifest list (Avro)	Timeline (Avro)
Best for	Spark-native workloads	Multi-engine; partition evolution	Record-level CDC upserts
Hidden partitioning	No	Yes	Partial
Time travel	Yes	Yes	Yes
Partition evolution	Limited	Full	Partial
Z-Ordering / Sort	Yes	Sort order (Puffin)	Clustering
Primary ecosystem	Databricks / Spark	Trino / Spark / Flink	Spark / Flink

Table 11.2: Comparison of the three major open table formats. Delta Lake is the default choice for Spark-centric shops; Iceberg is preferred when multiple query engines must read the same tables; Hudi is preferred for high-throughput CDC upsert workloads.

travel, and audit logging on top of the object storage—without moving the data to a proprietary format.

3. **Unified query engine:** a single query engine (Spark SQL, Trino, or DuckDB) serves both BI dashboards (interactive SQL) and ML training jobs (DataFrame reads) from the *same ACID- consistent copy* of the data. There is no ETL copy between a lake and a warehouse.
4. **Declarative access control:** a unified catalogue and governance layer (Databricks Unity Catalog, Apache Atlas, AWS Glue Data Catalog) applies column-level masking, row-level filtering, and audit logging automatically for every query.

11.3 Data Mesh: Decentralised Domain Ownership

The Lakehouse solves the technical problem of unreliable, expensive, fragmented data storage. The Data Mesh (Dehghani 2022) addresses a different class of problem: the *organisational* bottleneck created when a central data platform team is responsible for serving every business domain’s data needs.

11.3.1 The Central Platform Bottleneck

In the traditional model, a central data engineering team:

1. Ingests raw data from every business domain (sales, finance, operations, marketing) into a centralised data lake.
2. Builds transformation pipelines to produce clean, curated datasets.
3. Publishes datasets for consumption by analysts and data scientists.

At 200+ teams in a large enterprise, this creates a predictable bottleneck:

- **Ownership mismatch:** the pipeline team does not understand the domain semantics (what does a “cancelled order” mean in the sales domain?); the domain team that understands the data cannot fix the pipeline.
- **Queue bottleneck:** 200 teams compete for 20 data engineers. New dataset requests take weeks or months to fulfil.

- **Quality degradation:** no single team is accountable for downstream data correctness. When a dashboard shows incorrect numbers, three teams blame each other.

11.3.2 The Four Data Mesh Principles

Dehghani proposed Data Mesh as a *sociotechnical* paradigm—meaning it changes both the technology architecture and the organisational structure simultaneously. It has four principles.

Definition	Data Mesh: Four Principles (Dehghani, 2019/2022)
1. Domain ownership: each business domain (sales, finance, logistics, marketing) owns, builds, operates, and is SLA-accountable for its own data products. The sales team is responsible for the quality of sales_events; the logistics team is responsible for shipment_records. There is no central data team that intermediates. 2. Data as a product: each domain’s dataset is treated as a product with explicit quality contracts. A <i>data product</i> is discoverable (findable in a catalogue), addressable (accessible via a stable, versioned API or URI), trustworthy (with documented SLAs on freshness, accuracy, and completeness), and self-describing (with schema, lineage, and ownership metadata). 3. Self-serve data platform: domain teams are not data engineers. The central platform team’s role changes from <i>owning pipelines</i> to <i>providing infrastructure abstractions</i> so that domain teams can build data products without Spark or Kafka expertise. The Lakehouse is the natural storage substrate for a self-serve data mesh. 4. Federated computational governance: global policies (GDPR compliance, CCPA data minimisation, HIPAA de-identification, data retention limits) are enforced <i>automatically</i> at the platform level for all data products. Domain teams cannot opt out of global policies, but retain local flexibility in how they structure and process their data.	

Common Misconception | Data Mesh Is Not a Technology

Data Mesh is frequently misunderstood as a distributed storage architecture or a multi-cloud strategy. It is neither. It is an organisational design pattern for managing data responsibility. You can implement Data Mesh on a single cloud with a single Lakehouse. Conversely, building a distributed multi-cloud storage system does not give you a Data Mesh if the data ownership and accountability structure is still centralised. The transformation is primarily social (who is accountable for data quality?) and secondarily technical (how does the infrastructure enforce that accountability?).

11.4 MLOps: Operationalising Machine Learning

A trained machine learning model that exists only in a Jupyter notebook delivers zero business value. The discipline of *MLOps* (Machine Learning Operations) covers the engineering required to reliably deploy, monitor, and retrain models in production. The first paper to formalise this engineering gap was (Sculley et al. 2015).

11.4.1 Hidden Technical Debt in ML Systems

Sculley et al. identified a fundamental structural problem with production ML systems: the ML code itself—the model training loop—is only a tiny fraction of the total codebase. The surrounding infrastructure is vast, complex, and poorly understood.

Key Insight | Hidden Technical Debt (Sculley et al., NeurIPS 2015)

“Only a small fraction of real-world ML systems is composed of the ML code. The required surrounding infrastructure is vast and complex.” The paper identified nine categories of technical debt specific to ML systems:

1. **Entanglement:** changing one input feature changes the behaviour of all others (“Changing Anything Changes Everything”).
2. **Hidden feedback loops:** a model’s predictions influence the data it

is next trained on—a source of instability invisible to offline evaluation.

3. **Undeclared consumers:** other systems silently depend on a model's output format; schema changes break them invisibly.
4. **Data dependency debt:** stale, underutilised, or epsilon-feature dependencies accumulate and become expensive to remove.
5. **Configuration debt:** hyperparameters, thresholds, and pipeline configurations are not version-controlled.
6. **Fixed thresholds in dynamic worlds:** a fraud threshold set in 2020 may be inappropriate in 2025 as transaction distributions shift.
7. **Monitoring gaps:** the model trains on historical data but serves live data; distribution drift goes undetected until a business metric degrades.
8. **Training-serving skew:** the features computed at training time use different code than the features computed at serving time, producing silent prediction errors.
9. **Pipeline jungle:** ad-hoc data pipelines accumulate into an unmaintainable collection of scripts.

MLOps is the systematic response to this debt: a set of tools and practices that makes each failure mode visible, measurable, and preventable.

11.4.2 The MLOps Pipeline

A complete MLOps pipeline has seven components, each addressing a specific category of technical debt.

1. Data ingestion (Kafka + Spark): reproducible, versioned data pipelines replace ad-hoc scripts. Data is written to the Lakehouse as a Delta table, providing immutable, versioned input to all downstream stages.

2. Feature engineering (Apache Airflow DAGs): feature computation logic is expressed as versioned, tested Airflow DAGs rather than one-off notebook cells. A DAG that computes “average transaction amount per user over the past 30 days” runs on the same code path in both the training pipeline and the real-time serving pipeline, eliminating training-serving skew.

3. Feature store (Feast, Tecton): a *feature store* is a centralised repository that stores pre-computed features and serves them to both training jobs (historical feature values for a training window) and inference services (real-time feature values for a prediction request). Without a feature store, different teams independently compute the same features with subtly different logic, creating inconsistent model inputs. The feature store also prevents *point-in-time correctness* violations: a training example for a customer's state at time t must use only feature values known at time t , not future values ("data leakage").

4. Model training (Spark MLlib, PyTorch, scikit-learn): the model training code reads features from the feature store, trains the model, and records all inputs (dataset version, feature set version, hyperparameters) and outputs (model artefacts, evaluation metrics) in an experiment tracker.

5. Experiment tracking (MLflow): MLflow's `mlflow.log_param()`, `mlflow.log_metric()`, and `mlflow.log_artifact()` API records every experiment run's configuration and output. The MLflow Tracking UI allows comparing runs by metric and reproducing any past result from its logged configuration.

6. Model registry (MLflow Model Registry): trained models are registered in a version-controlled catalogue with stage labels (Staging, Production, Archived). A model transitions from Staging to Production only after passing automated integration tests and a manual approval step. A failing production model can be rolled back to a previous version in one command.

7. Serving and monitoring (KServe, Seldon, Evidently): the model is deployed as a REST API endpoint with canary deployment (10% of traffic goes to the new model; 90% stays on the old model until the new model proves itself). Evidently or Arize monitors the live feature distribution against the training distribution, alerting when *data drift* is detected.

```

1 import mlflow
2 import mlflow.spark
3 from pyspark.sql import SparkSession
4 from pyspark.ml.classification import GBTCClassifier
5 from pyspark.ml.feature import VectorAssembler
6 from pyspark.ml.evaluation import BinaryClassificationEvaluator
7
8 spark = SparkSession.builder.appName("MLflowDemo").getOrCreate()

```

```

9  mlflow.set_tracking_uri("http://mlflow-server:5000")
10 mlflow.set_experiment("payment-fraud-detection-v2")
11
12 # Load feature-engineered training data from Delta Lake
13 train = spark.read.format("delta") \
14         .load("hdfs:///lakehouse/features/fraud_train/") \
15         .na.drop()
16
17 feature_cols = ["amount_usd", "txn_per_hour",
18                 "avg_txn_30d", "is_new_receiver",
19                 "hour_of_day", "day_of_week"]
20 assembler = VectorAssembler(inputCols=feature_cols,
21                             outputCol="features")
22 train_vec = assembler.transform(train)
23
24 with mlflow.start_run(run_name="gbt_maxDepth5_lr01"):
25
26     # --- Hyperparameters ---
27     max_depth = 5
28     lr = 0.1
29     n_trees = 100
30     mlflow.log_param("maxDepth", max_depth)
31     mlflow.log_param("learningRate", lr)
32     mlflow.log_param("numTrees", n_trees)
33     mlflow.log_param("feature_set", "v2.3")
34     mlflow.log_param("training_version", 47) # Delta Lake
35                                             version
36
37     # --- Train ---
38     gbt = GBTClassifier(
39         labelCol = "is_fraud",
40         featuresCol = "features",
41         maxDepth = max_depth,
42         stepSize = lr,
43         maxIter = n_trees,
44     )
45     model = gbt.fit(train_vec)
46
47     # --- Evaluate ---
48     test = assembler.transform(
49         spark.read.format("delta")
50         .load("hdfs:///lakehouse/features/fraud_test/")
51         .na.drop())
52     preds = model.transform(test)

```

```

51     eval_    = BinaryClassificationEvaluator(labelCol="is_fraud"
52         )
53     auroc    = eval_.evaluate(preds)
54     mlflow.log_metric("auroc", auroc)
55     print(f"AUROC = {auroc:.4f}")
56
57     # --- Log model artefact ---
58     mlflow.spark.log_model(model, "gbt_fraud_model",
59                             registered_model_name="payment-fraud
60                             -gbt")
61
62     # --- Transition to Staging via Model Registry API ---
63     client = mlflow.MlflowClient()
64     latest = client.get_latest_versions("payment-fraud-gbt",
65                                         stages=["None"])[0]
66     client.transition_model_version_stage(
67         name      = "payment-fraud-gbt",
68         version   = latest.version,
69         stage     = "Staging",
70     )
71     print(f"Model v{latest.version} promoted to Staging")

```

Listing 11.3: MLflow experiment tracking and model registration in PySpark

11.4.3 Feature Stores and Training–Serving Skew

Training–serving skew is the most common silent failure mode in production ML systems: the code that computes a feature at training time differs from the code that computes the same feature at serving time, because they were written by different teams at different times. The result is a model that performs well on the training and test sets but degrades mysteriously in production—because the features it sees during inference are not the same features it was trained on.

A feature store eliminates this skew by maintaining a *single* implementation of each feature that is called by both the training pipeline (to build the historical feature matrix) and the serving pipeline (to compute real-time features for an incoming prediction request).

System Design Note | Feature Store Architecture with Feast

Feast is an open-source feature store with two stores:

- **Offline store** (Delta Lake / BigQuery / Redshift): stores the full historical record of each feature, indexed by `entity_id` and `event_timestamp`. Training jobs retrieve a point-in-time correct feature matrix by joining the entity's label events to the offline store at the exact timestamps of those events—preventing future data leakage.
- **Online store** (Redis / DynamoDB): stores only the *latest* value of each feature for each entity. Inference requests look up the online store in <5 milliseconds. The offline store is periodically materialised to the online store (“feature materialisation”).

Both stores share the same feature transformation code, guaranteeing that training and serving use identical feature values.

11.5 Edge Analytics and Cloud-Native Big Data

11.5.1 The Three-Tier Edge–Cloud Architecture

IoT devices generate data at a rate that cannot economically be transmitted in full to a central cloud cluster. A DESCO power grid substation generates 100,000 voltage readings per second; a factory floor with 500 vibration sensors generates 10 million samples per second. Transmitting all this raw data to the cloud would require terabytes per day of bandwidth—prohibitively expensive and subject to network latency that makes real-time anomaly response impossible.

Edge analytics solves this by placing computation at three tiers.

Definition | Three-Tier Edge–Cloud Architecture

1. **Tier 1: Device** (microcontroller, FPGA, SBC). At the data source, lightweight models (TensorFlow Lite, ONNX Runtime) perform pre-processing, filtering, and anomaly detection. Only *events* (anomalies detected, thresholds exceeded) are forwarded upstream—not raw

sensor readings. A cardiac monitor runs a QRS peak detection algorithm on-device and transmits only the detected heartbeat events, not the raw 500 Hz ECG waveform.

2. **Tier 2: Edge Gateway** (local server, AWS Greengrass, Azure IoT Edge, NVIDIA Jetson). The edge gateway runs pre-trained inference models and windowed aggregation on the stream of device events from multiple devices. It forwards only *summary alerts* or *windowed aggregates* to the cloud. Typical data reduction ratio: 1000× (from raw 100,000 readings/sec to ~100 alerts/sec). A factory's Greengrass gateway runs Isolation Forest on vibration sensor events to detect bearing wear; it sends one alert per anomalous 5-minute window rather than 30 million raw readings.
3. **Tier 3: Cloud Lakehouse** (AWS S3 + Delta Lake, Azure ADLS + Iceberg). The cloud receives the aggregated alerts and periodically batched raw samples. It retrains global models on aggregated data from all edge nodes; long-term historical storage enables trend analysis and root-cause investigation. Updated models are pushed back down to the edge gateways.

The upstream data flow is **Kafka** or **MQTT** between tiers. The downstream model push is typically a CI/CD pipeline triggered by MLflow model registry promotion.

Cloud-native big data. On-premise Hadoop clusters are being replaced by *managed cloud services* that decouple storage from compute:

- **Storage:** Amazon S3, Azure Data Lake Storage (ADLS), Google Cloud Storage (GCS) — ~\$23/TB-month, 11 nines durability, no cluster to maintain.
- **Compute:** AWS EMR (Elastic MapReduce), Google Dataproc, Azure HDInsight — ephemeral Spark clusters that start in 5 minutes, process the job, and terminate automatically (scaling to zero when idle). A batch pipeline that previously required a 100-node always-on cluster now runs on a 20-node cluster for 3 hours per day—a 60–80% cost reduction.

The Lakehouse is the natural convergence of cloud-native storage and managed compute: Delta tables live on S3; ephemeral Spark clusters read

them directly, without copying data to a warehouse.

11.6 Privacy-Preserving Analytics

Chapter 10 introduced differential privacy and federated learning in the healthcare context. This section formalises both concepts and extends them to the general data engineering setting.

11.6.1 Differential Privacy

Definition | (ϵ, δ) -Differential Privacy

A randomised mechanism $\mathcal{M} : \mathcal{D} \rightarrow \mathcal{R}$ is (ϵ, δ) -**differentially private** if for all pairs of neighbouring datasets D, D' that differ in exactly one record, and for all subsets of outputs $S \subseteq \mathcal{R}$:

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \cdot \Pr[\mathcal{M}(D') \in S] + \delta$$

Interpretation: the probability that the mechanism's output changes when one individual's record is added or removed is bounded by e^ϵ (multiplicatively) with probability $1 - \delta$. Smaller ϵ is stronger privacy. $\epsilon = 0$ means perfect privacy (the output reveals nothing about any individual); $\epsilon = \infty$ means no privacy guarantee.

The Laplace mechanism: for a real-valued function $f : \mathcal{D} \rightarrow \mathbb{R}$ with sensitivity $\Delta f = \max_{D, D'} |f(D) - f(D')|$, adding Laplace noise achieves $(\epsilon, 0)$ -differential privacy:

$$\mathcal{M}(D) = f(D) + \text{Lap}\left(\frac{\Delta f}{\epsilon}\right)$$

The Gaussian mechanism: adding Gaussian noise achieves (ϵ, δ) -differential privacy, which is practically preferable for machine learning (gradient perturbation).

```
1 import numpy as np
2
3 def laplace_mechanism(true_count: int,
```

```

4         sensitivity: float,
5         epsilon: float,
6         rng: np.random.Generator) -> float:
7     """Add Laplace noise to a count query."""
8     scale = sensitivity / epsilon
9     noise = rng.laplace(loc=0.0, scale=scale)
10    # Counts must be non-negative; clip to 0
11    return max(0.0, true_count + noise)
12
13    rng = np.random.default_rng(2024)
14    epsilon = 1.0    # privacy budget
15    sens = 1.0    # sensitivity of a COUNT query
16
17    # True dengue case counts by region (confidential)
18    true_counts = {
19        "New York": 1_402,
20        "London": 387,
21        "Berlin": 201,
22        "Sydney": 94,
23        "Toronto": 47,
24    }
25
26    print(f"epsilon = {epsilon} (stronger privacy -> larger noise)"
27        )
28    print(f"{'Region':<14} {'True':>8} {'Published':>12} {'Error'
29        ':>8}")
30    print("-" * 46)
31
32    for region, count in true_counts.items():
33        published = laplace_mechanism(count, sens, epsilon, rng)
34        error = abs(published - count)
35        print(f"{'region':<14} {'count':>8} {'published':>12.1f} {'error'
36            ':>8.1f}")
37
38    # Compare: tighter budget -> more noise
39    for eps in [0.1, 0.5, 1.0, 2.0, 5.0]:
40        noised = laplace_mechanism(1_402, 1.0, eps, rng)
41        print(f"    epsilon={eps:<4}: published New York count = {
42            noised:.1f}")

```

Listing 11.4: Differential privacy with the Laplace mechanism: publishing district-level case counts

Real-world deployments.

- **Apple:** uses local differential privacy (LDP) for keyboard usage statistics on iOS—noise is added on the device before any data leaves the phone.
- **Google:** RAPPOR (Randomised Aggregatable Privacy-Preserving Ordinal Response) for Chrome usage telemetry.
- **US Census Bureau:** the 2020 US Census used differential privacy (the TopDown algorithm) to prevent re-identification of individuals in block-level population tables.

11.6.2 Federated Learning

(McMahan et al. 2017) introduced *federated learning* (FL) as a method for training a shared model on data held across multiple devices or institutions without centralising the raw data. The core algorithm, *FedAvg*, works as follows:

1. A **central server** initialises a global model w_0 and distributes it to all participating nodes (hospitals, mobile devices, banks).
2. Each node trains the model locally for E epochs on its own data, producing a locally updated model $w_i^{(t)}$.
3. Each node sends only its *model weight update* $\Delta w_i = w_i^{(t)} - w_0^{(t)}$ —not its raw data—to the server.
4. The server aggregates the updates using weighted averaging:

$$w_0^{(t+1)} = \sum_{i=1}^K \frac{n_i}{N} \Delta w_i$$

where n_i is node i 's dataset size and $N = \sum n_i$.

5. Repeat for T global rounds.

The raw data never leaves the originating node. This enables model training across organisational boundaries where data sharing is legally prohibited.

Production Lesson | Federated Learning Limitations in Production

Despite its privacy advantages, federated learning introduces engineering challenges that must be addressed before production deployment:

1. **Statistical heterogeneity (non-IID data):** each node's data reflects

local conditions (a rural hospital’s patient population differs from an urban hospital’s). Naive FedAvg converges slowly or to a poor global model when data distributions are highly non-IID. Techniques such as FedProx (adding a proximal term to the local loss) and SCAFFOLD (correction of client drift) address this.

2. **Communication cost:** transmitting full model weight vectors (sometimes hundreds of millions of parameters) after every round is expensive. Gradient compression, quantisation, and sparsification reduce communication by 10–100× at some accuracy cost.
3. **Gradient inversion attacks:** recent research has shown that model weight updates can leak information about the training data. Combining federated learning with differential privacy (DP-SGD: adding noise to gradients before uploading) provides formal privacy guarantees even against a malicious server.
4. **Partial participation:** in cross-device federated learning (e.g., mobile phones), only a fraction of nodes are available at each round (device is off, battery is low). The training algorithm must be robust to arbitrary dropouts.

11.7 The Dataflow Model and Apache Beam

Chapter 8 described the Lambda architecture as the practitioner’s solution to the batch–stream unification problem: run a batch pipeline for accuracy and a stream pipeline for latency, then merge their outputs in a serving layer. The Lambda architecture’s fundamental flaw is its *dual code path*: identical business logic must be maintained in two separate codebases that gradually drift.

(Akidau et al. 2015) proposed a more principled solution: a single programming model that is expressive enough to describe both batch and streaming workloads, treating batch as a special case of streaming.

11.7.1 The Four Questions

Any data pipeline can be fully characterised by four questions:

Definition | The Dataflow Model's Four Ws (Akidau et al., VLDB 2015)

1. **What** results are computed? The *transformation* applied to the data: sum, count, average, join, custom function. This is equivalent to the SELECT clause in SQL.
2. **Where** in event time are results computed? The *windowing strategy* partitions the unbounded event stream into finite chunks:
 - **Tumbling window:** fixed, non-overlapping windows (e.g., count transactions every 5 minutes).
 - **Sliding window:** fixed-size, overlapping windows (e.g., 1-hour window updated every 5 minutes).
 - **Session window:** variable-size window defined by activity—a window closes when there has been no event for a specified gap duration.
3. **When** in processing time are results materialised? The *trigger* controls when the system emits a result for a window:
 - **Watermark trigger:** emit when the watermark (the system's estimate of the latest event time it has seen minus an allowance for late arrivals) passes the window's end time.
 - **Periodic trigger:** emit every n seconds regardless of the watermark.
 - **Count trigger:** emit after n events.
 - **Composite:** e.g., emit early every 10 seconds AND emit a final result when the watermark passes.
4. **How** do refinements relate to previous results? The *accumulation mode* determines what happens when a window fires multiple times (early trigger, then final):
 - **Discarding:** each firing emits only the new data arrived since the last firing.
 - **Accumulating:** each firing emits the full running total since the window opened (overwrites previous output).
 - **Accumulating and retracting:** each firing emits both the new value AND a retraction of the previous value (for downstream

systems that need to subtract the old value).

Batch as a special case of streaming: in this model, a batch job is a streaming job with a single tumbling window covering the entire dataset, a watermark trigger set at the end of the input, and discarding accumulation. The same code expresses both.

11.7.2 Apache Beam: One Pipeline, Multiple Runners

Apache Beam (2016) is the open-source implementation of the Dataflow Model. A Beam pipeline is written in Python, Java, or Go using the Beam SDK. The same pipeline code then runs on multiple *runners*:

- **Apache Spark** runner: for batch processing on HDFS or S3.
- **Apache Flink** runner: for low-latency streaming.
- **Google Cloud Dataflow**: fully managed, auto-scaling stream and batch processing on GCP.
- **Direct runner**: for local testing without a cluster.

```
1 import apache_beam as beam
2 from apache_beam.options.pipeline_options import
   PipelineOptions
3 from apache_beam.transforms.window import FixedWindows
4 from apache_beam.transforms.trigger import (AfterWatermark,
5     AfterProcessingTime, AccumulationMode)
6 import json
7
8 # --- Configuration: change runner to switch between Spark /
   Flink / local ---
9 options = PipelineOptions([
10     "--runner=FlinkRunner",
11     "--flink_master=flink-master:6123",
12     # Use SparkRunner for batch:
13     # "--runner=SparkRunner", "--spark_master=spark://...",
14 ])
15
16 def parse_event(raw: bytes) -> dict:
17     """Deserialise a Kafka event bytes to dict."""
18     return json.loads(raw.decode("utf-8"))
19
```

```

20 def extract_key_amount(record: dict):
21     """Emit (region, amount_usd) pairs."""
22     return (record.get("region", "Unknown"),
23            float(record.get("amount_usd", 0.0)))
24
25 class SumValues(beam.CombineFn):
26     def create_accumulator(self): return 0.0
27     def add_input(self, acc, element): return acc + element
28     def merge_accumulators(self, accs): return sum(accs)
29     def extract_output(self, acc): return acc
30
31 def format_output(kv) -> str:
32     region, total = kv
33     return f"{region}: USD {total:,.0f}"
34
35 with beam.Pipeline(options=options) as p:
36     result = (
37         p
38         # WHAT: read from Kafka
39         | "ReadKafka" >> beam.io.ReadFromKafka(
40             consumer_config={
41                 "bootstrap.servers": "kafka:9092",
42                 "group.id": "beam-fraud-pipeline",
43             },
44             topics=["payment-events"],
45         )
46         | "ParseJSON" >> beam.Map(parse_event)
47         | "ExtractPairs" >> beam.Map(extract_key_amount)
48
49         # WHERE: 5-minute tumbling event-time windows
50         | "Window" >> beam.WindowInto(
51             FixedWindows(5 * 60),          # 5-minute windows
52             trigger=AfterWatermark(
53                 early=AfterProcessingTime(30), # emit every
54                 30s early
55             ),
56             accumulation_mode=AccumulationMode.ACCUMULATING,
57             allowed_lateness=2 * 60,        # accept events up to
58             2 min late
59         )
60
61         # WHAT: aggregate amount per region per window
62         | "SumByRegion" >> beam.CombinePerKey(SumValues())
63         | "Format" >> beam.Map(format_output)

```

```
62
63     # Output: write aggregates to the Lakehouse (or console
64     | "Write" >> beam.io.WriteToText("hdfs:///lakehouse/
65     fraud/agg")
    )
```

Listing 11.5: Apache Beam: fraud detection pipeline with tumbling windows (same code runs on Spark or Flink)

11.8 Research Spotlight

Research Spotlight | Armbrust et al. (2020) · Armbrust, Ghodsi, Xin & Zaharia (2021) · Akid

Work 1: Armbrust, M., et al. (2020). “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.” *Proceedings of the VLDB Endowment*, 13(12), 3411–3424. (Databricks / Apache Foundation; Rank A* venue.)

Delta Lake addresses a fundamental limitation of cloud object stores (S3, Azure ADLS, GCS): they provide no ACID semantics. A concurrent write from two Spark executors to the same S3 prefix produces undefined results; a partial write due to a job failure leaves the table in a corrupt state; there is no mechanism for rolling back a bad commit.

The paper’s solution is the *Delta Log*: an ordered sequence of atomic JSON commits stored inside the table directory in S3. Each commit records added and removed Parquet files, schema changes, and metadata. Readers reconstruct the table state by reading the log; they always see a consistent snapshot. Writers implement optimistic concurrency control by racing to write the next log entry number—only one succeeds. Time travel is a consequence of the log’s durability: any past version is recoverable by reading the log up to that version’s commit.

Three benchmarks from the paper:

1. Write throughput equal to raw Parquet on S3 (the log adds <2% overhead for writes; checkpoint files amortise read cost).
2. Z-ordering reduces the number of files scanned by queries with two-column filters by 95% compared to Hive partitioning on a single

column.

3. MERGE INTO (upsert) is 10× faster than the equivalent read–filter–rewrite pattern on raw Parquet.

Impact: Delta Lake is now the default table format for Databricks Lakehouse Platform, used by over 7,000 organisations. It has been adopted as a core component of AWS Lake Formation and Azure Synapse Analytics. Apache Iceberg (Netflix) and Apache Hudi (Uber) adopt the same transaction-log principle with different trade-offs.

Work 2: Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). “Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics.” *11th Annual Conference on Innovative Data Systems Research (CIDR 2021)*. (Databricks; invited vision paper.)

The Lakehouse paper formalises the observation that Delta Lake’s ACID semantics make it possible to use a data lake as a data warehouse—without a separate copy of the data in a proprietary warehouse system. Zaharia et al. benchmark the Lakehouse architecture (Delta Lake + Spark SQL on S3) against the leading cloud data warehouses (Redshift, Snowflake, BigQuery) on the TPC-DS benchmark:

- At the 3 TB scale factor, Databricks SQL on Delta Lake achieves comparable performance to Redshift RA3 at 3–5× lower cost.
- The “no ETL copy” property means analysts query data that is minutes old, not 12 hours old as in the traditional lake-to-warehouse pipeline.

The paper formally defines the Lakehouse using five properties (open storage, metadata layer, SQL performance, ML support, governance) and identifies DuckDB as an in-process analytical engine that further democratises Lakehouse query access without a Spark cluster.

Impact: Snowflake, Google, and Microsoft have all published blog posts and white papers in response to the Lakehouse paper. Snowflake’s Iceberg integration, Google’s BigLake, and AWS Lake Formation are all responses to the competitive pressure created by this paper.

Work 3: Akidau, T., et al. (2015). “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing.” *Proceedings of the VLDB Endowment*, 8(12), 1792–1803. (Google; VLDB Best Paper Award.)

Google’s internal production streaming system (MillWheel, 2013) revealed two fundamental inadequacies of the Lambda architecture: maintaining dual code paths introduced synchronisation bugs; and the existing streaming model (Storm, early Spark Streaming) provided no principled way to handle out-of-order events—events that arrive at the processing system after the window covering their event time has already been emitted.

Akidau et al.’s Dataflow Model answers four questions for any data pipeline (What, Where, When, How) and introduces the *watermark* as a first-class primitive: the watermark is the system’s estimate of the maximum event time for which all earlier events have been received. Triggers allow windows to emit early results (for latency) while waiting for late arrivals (for completeness). The accumulation mode controls how late-arriving events update previously emitted results.

Impact: The Dataflow Model was open-sourced as Apache Beam in 2016. Apache Flink adopted the Dataflow Model’s event-time windowing semantics directly (the `EventTimeSessionWindows` and `EventTimeTrigger` APIs are direct implementations of the Four Ws). Google Cloud Dataflow is the managed production deployment. Spark Structured Streaming’s watermark API (Chapter 8) is an implementation of the same model.

Work 4: Sculley, D., et al. (2015). “Hidden Technical Debt in Machine Learning Systems.” *Advances in Neural Information Processing Systems (NeurIPS 2015)*. (>5,000 citations; Google.)

Written by Google engineers managing internal ML production systems, this paper is one of the most practically influential papers of the 2010s. Its central argument is that the visible ML code—the model training loop—is only a small fraction of the actual cost of a production ML system. Nine categories of hidden technical debt (entanglement, un-

declared consumers, data dependency debt, configuration debt, fixed thresholds, monitoring gaps, training-serving skew, pipeline jungle, world changes) account for far more engineering effort than the ML code itself.

The paper's impact was to crystallise the *MLOps* discipline: toolmakers (Databricks MLflow, Tecton, Weights & Biases, Evidently AI) built their entire product around eliminating specific categories of the debt described in this paper. Stanford's CS329S (ML Systems) assigns the paper as the first required reading and maps its debt taxonomy to every tool in the MLOps stack.

11.9 Case Study: Enterprise Lakehouse Migration in Production

Case Study | From Lambda Architecture to Lakehouse — Pinterest, Comcast, and the Pr

Context: Lambda Pain at Scale. By 2018–2020, organisations running the Lambda architecture at petabyte scale were experiencing the same compounding pain:

- **Pinterest (300+ PB stored, 10+ billion events/day):** for every business metric (engagement rate, recommendation click-through, ad impression count), two separate codebases existed—a Spark batch job for the weekly reports and a Flink streaming job for the real-time dashboard. Engineers estimated they spent 40% of their time keeping the two code paths in sync. When business logic changed (new definition of an “engagement event”), both codebases had to be updated simultaneously—and failures in synchronisation produced inconsistent numbers that took days to diagnose.
- **Comcast (100 TB of new data per day):** data was ingested into Amazon S3 (the data lake) via Spark batch jobs. Every night, a separate ETL pipeline extracted the previous day's data from S3 and loaded it into Amazon Redshift (the data warehouse). Business analysts queried Redshift; ML teams queried S3. The copy introduced a 12-

hour data freshness lag for analysts, and the two stores inevitably diverged as some records were cleaned in the ETL pipeline and others were not.

Migration Architecture. Both organisations migrated to a Lakehouse using Delta Lake on Amazon S3 + Databricks (Spark) as the unified query engine. The migration strategy was *table-by-table*, not a big-bang cutover:

1. Convert the highest-value, most-queried table first (e.g., the daily active users table). Enable schema enforcement immediately. Run the Delta version and the legacy version in parallel for two weeks, comparing query results.
2. Migrate the Flink streaming pipeline to write to Delta using Spark Structured Streaming with the Delta Lake sink. The streaming table and the batch table are now the same Delta table—the same Parquet files, the same ACID log.
3. Redirect BI tools (Tableau, Superset) from Redshift to Delta Lake via Spark SQL. Validate dashboard numbers against the Redshift baseline.
4. Decommission the Redshift cluster and the nightly ETL pipeline.

Measured Outcomes.

Metric	Before (Lambda)	After (Lakehouse)
ETL pipeline count per metric	3–5 jobs	1 job
BI data freshness	12 hours	<15 minutes
Storage cost per TB-month	\$23 lake + \$230 DWH	\$23 (lake only)
ML training data access	File copy from S3	Direct Delta read
Schema violations detected	After landing	Blocked at ingest
GDPR record deletion	Rewrite all Parquet files	Single delta log entry
Analyst query latency (P95)	8 minutes (Hive)	45 seconds (Delta)
Code paths per pipeline	2 (batch + stream)	1

Five Engineering Lessons.

1. **Enable schema enforcement on day one.** Teams that enabled schema enforcement in the first week of migration discovered and fixed 90% of their data quality bugs within that week. Teams that deferred schema enforcement “until the migration was stable” spent months debugging corrupted tables that had accumulated malformed records during the deferral period.
2. **Time travel replaced manual snapshot exports.** Compliance teams had previously maintained a weekly manual export of the Redshift table to a separate S3 folder as an “audit snapshot.” After migration, `VERSION AS OF` or `TIMESTAMP AS OF` queries served this function automatically—eliminating 6 TB/month of redundant storage.
3. **Z-ordering beats blanket partitioning.** The legacy Hive tables were partitioned by date—reasonable for time-range queries but inadequate for the common multi-column filter `WHERE user_id = X AND date BETWEEN Y AND Z` (which still scanned the entire date partition). Z-ordering on `(user_id, date)` reduced scanned files by 78–85% for these queries.
4. **Migration, not replacement.** The table-by-table migration strategy eliminated downtime and allowed the team to validate each table independently. A big-bang migration that switches all tables simultaneously produces a situation where a bug in one table poisons all downstream consumers and is impossible to roll back cleanly.
5. **The bottleneck shifted from storage to governance.** After the technical migration, the dominant remaining challenge was *organizational*: who owns which Delta table? Which teams can write to it? Who is responsible when a table’s data quality degrades? These questions—the subject of the Data Mesh paradigm—proved harder to solve than the technical migration itself.

Chapter Summary

This chapter described the fourth era of big data technology and the architecture that defines it.

The Lakehouse combines cheap open-format object storage with an open table format (Delta Lake, Apache Iceberg, or Apache Hudi) that provides ACID transactions via a transaction log, time travel via log durability, and schema enforcement at the storage layer. Delta Lake’s optimistic concurrency control achieves serialisable writes on eventually-consistent S3 without a dedicated lock manager. The Lakehouse eliminates the lake-warehouse split: BI and ML workloads read the same ACID-consistent data, and the Lambda architecture’s 12-hour freshness lag disappears.

The Data Mesh addresses the organisational bottleneck created when a central platform team owns all data pipelines: it decentralises ownership to domain teams (domain ownership), standardises the interface of each dataset (data as a product), provides infrastructure abstractions that non-engineers can use (self-serve platform), and enforces global compliance policies automatically (federated governance).

MLOps makes production ML reliable by addressing the hidden technical debt identified by Sculley et al.: feature stores eliminate training-serving skew; MLflow tracks experiment configurations and enables result reproduction; model registries provide versioned, staged deployment; and monitoring detects data drift before it degrades business outcomes.

Edge analytics extends distributed processing to three tiers: lightweight on-device inference (TensorFlow Lite, ONNX), edge gateway aggregation (Greengrass, Azure IoT Edge), and cloud Lakehouse storage and model retraining. Cloud-native managed services (AWS EMR, Google Dataproc) decouple storage (S3, GCS) from compute, reducing cost by 60–80% compared to always-on HDFS clusters.

The Dataflow Model (Akidau et al., 2015) eliminates the Lambda architecture’s dual code path by providing a single programming model with four dimensions: What (transformation), Where (windowing), When (triggering), and How (accumulation). Apache Beam implements this model and runs on Spark, Flink, and Google Cloud Dataflow—one pipeline code, multiple backends.

Exercises

Short Questions

SQ11.1. Describe the four eras of big data technology. For each era, state the primary engineering bottleneck that the previous era's tools could not address and the key innovation that resolved it.

SQ11.2. What is the Delta Log in Delta Lake? How does it achieve atomicity, isolation, and durability on Amazon S3, which provides no native locking or atomic rename operations?

SQ11.3. A Spark job writes to a Delta table at the same time as a second Spark job. Only one write succeeds; the other throws a `ConcurrentModificationException`. Explain the optimistic concurrency control mechanism that produced this outcome.

SQ11.4. What is time travel in Delta Lake? Give two specific production use cases where the ability to query a past version of a Delta table provides direct business value.

SQ11.5. Describe the difference between Delta Lake's **schema enforcement** and **schema evolution**. What problem does each solve, and what is the consequence of not having schema enforcement in a data lake?

SQ11.6. What is Z-ordering? Explain why it outperforms single-column Hive partitioning for queries that filter on two columns simultaneously.

SQ11.7. State the four principles of the Data Mesh paradigm. For each, identify the specific bottleneck of centralised data architecture that it addresses.

SQ11.8. What is training-serving skew? Give a specific example from a fraud detection context, describe how it produces silent prediction errors, and explain how a feature store prevents it.

SQ11.9. Define (ϵ, δ) -differential privacy formally. If $\epsilon = 0.1$ vs. $\epsilon = 5.0$: which provides stronger privacy, which adds more noise, and which is more appropriate for a published aggregate statistic vs. an internal model training signal?

SQ11.10. Describe the Dataflow Model's four Ws. Explain how a batch MapReduce job is a special case of the streaming model under this framework.

SQ11.11. What is a watermark in the Dataflow Model? Why is a watermark necessary for correctly processing out-of-order streaming events?

SQ11.12. What is the “no ETL copy” advantage of the Lakehouse? Describe the specific data freshness and cost problems it eliminates.

Analytical Problems

AP11.1. Delta Lake Concurrency Analysis. A payment transaction table is stored as a Delta table on S3. Three concurrent Spark jobs attempt to write to the table simultaneously:

- Job A: append 10 million new transactions (started first).
 - Job B: UPDATE 500 transactions where `region='Northeast'` (started 2 seconds after Job A).
 - Job C: DELETE all transactions where `amount_usd < 0.50` (started 3 seconds after Job A).
- (a) Which job writes the Delta Log entry numbered 00042.json if the current latest version is 41? Which jobs conflict?
- (b) Under Delta Lake’s OCC: if Job A succeeds, does Job B automatically fail, or does it check whether its operation conflicts with Job A’s specific changes? Explain the conflict detection logic.
- (c) If Job C fails with a `ConcurrentModificationException`, what exactly does it retry? Does it re-read the data from scratch or simply retry the log append?
- (d) After all three operations complete, how does a time-travel query `VERSION AS OF 41` behave? What data does it return?

AP11.2. MLOps Debt Taxonomy. A hospital data science team built a sepsis prediction model in 2022. It is deployed in production at a large academic medical centre and makes predictions every 15 minutes for each ICU patient. In 2025, the team notices the model’s clinical alerts are being ignored by nurses, who say “the model keeps false-alarming.” An investigation reveals:

- The model was trained on 2019–2021 ICU data. In 2023, the hospital switched from manual vital-sign charting to an automated monitoring

system that records vitals every 1 minute instead of every 4 hours. The model was not retrained.

- The real-time feature computation at serving time uses a slightly different rolling-window logic than the training pipeline (off-by-one in window boundary calculation).
- No monitoring was configured to detect feature distribution drift.

For each issue: (a) identify the Sculley et al. technical debt category; (b) describe the specific MLOps tool or practice that would have prevented it; (c) estimate the severity of the clinical impact if the model's false positive rate has risen from 12% to 45%.

AP11.3. Dataflow Model Design. Design a Beam pipeline for a real-time ride-sharing driver location analytics system:

- Source: GPS ping events arrive at Kafka at 10 events/second per active driver (estimated 50,000 active drivers during peak hour $\Rightarrow$ 500,000 events/second).
- Events have a 5–30 second event-time lag (mobile network latency).
- Required output: count of active drivers per city zone per 5-minute window, emitted every 30 seconds with an early result, and a final result when the watermark passes the window end.

Answer the Four Ws:

- What:** specify the transformation. What is the key, what is aggregated, and what is the output schema?
- Where:** specify the windowing strategy. Why is a tumbling window preferable to a sliding window for this use case?
- When:** specify the trigger. Write the trigger configuration in pseudocode (AfterWatermark / AfterProcessingTime / composite). What allowed lateness should be set given the 5–30s event-time lag?
- How:** specify the accumulation mode. If the 30-second early trigger emits "Downtown: 12,400 drivers" and the final watermark trigger emits "Downtown: 12,800 drivers", what does the downstream system receive under *Accumulating* mode vs. *Accumulating and Retracting* mode?

AP11.4. Differential Privacy Budget Allocation. A public health agency

plans to publish monthly region-level disease statistics (dengue case counts, influenza hospitalisations, COVID-19 test positivity rate) for 50 regions. Each statistic is a COUNT or AVERAGE query over the patient records database. The total monthly privacy budget is $\epsilon_{\text{total}} = 2.0$.

- (a) The agency wants to publish 5 statistics per region per month (dengue count, influenza count, COVID count, COVID positivity rate, and RSV count). How should the budget be allocated across these 5 queries using *sequential composition* (each query spends from the budget independently)?
- (b) A statistician proposes using *parallel composition* instead: since the region-level counts are disjoint subsets of the total population, the budget does not need to be divided across regions. Explain why this is correct and what it implies for the per-region noise level.
- (c) With $\epsilon = 0.4$ per query (from part (a)), and a COUNT query sensitivity of $\Delta f = 1$, compute the standard deviation of the Laplace noise. Is this noise level acceptable for a region with 1,400 dengue cases? For a region with 14 cases? Discuss the small-count problem.
- (d) Propose a solution for the small-count problem that maintains differential privacy while still publishing useful statistics for low-incidence regions.

Design Problems

DP11.1. National Mobile Payment Lakehouse (capstone design). Design a complete Lakehouse-based data analytics platform for a nationwide mobile payment company (M-Pesa / PayPal scale: 50 million transactions/day, 5 years of historical data, 30 million registered users).

- (a) **Ingestion layer:** specify the Kafka topic structure (topic, partition count, key, value schema), the Spark Structured Streaming sink configuration for writing to Delta Lake, and the trigger interval.
- (b) **Storage design:** design the Delta table schema for the raw transaction table. Specify partitioning columns and Z-ordering columns. Justify each choice with reference to the query patterns (time-range queries for compliance, user-level queries for fraud, region-level aggregations

- for dashboards).
- (c) **Analytical layers:** describe the curated table hierarchy (bronze → silver → gold) using Delta Lake. What transformations happen at each layer? What SLA (freshness, accuracy) applies to each layer?
 - (d) **MLOps:** design the fraud detection MLOps pipeline. Specify: feature store features (at least 5), MLflow experiment tracking configuration, model registry stages, serving endpoint latency target, and drift monitoring metric.
 - (e) **Governance:** specify the Data Mesh domain ownership structure for three domains (Payments, Compliance, Marketing). Which Delta tables does each domain own? What global governance policies are enforced automatically?

DP11.2. National Health Analytics Lakehouse. Design a Lakehouse for a national health analytics platform (e.g., NHS England or US Health and Human Services), integrating data from 300 hospitals (HL7 FHIR), pharmacy chains, and community health workers (mobile app events via Kafka).

- (a) **Ingestion:** specify the Kafka Connect configuration for ingesting FHIR R4 Patient and Observation resources from 300 hospitals. How do you handle hospitals with different FHIR versions (R3 vs R4)?
- (b) **Privacy:** specify the de-identification pipeline that must run before any data enters the Lakehouse gold layer. Which HIPAA safe harbour identifiers are removed, and which are generalised (e.g., date-of-birth → age-band)?
- (c) **Federated learning:** the health authority wants to train a sepsis prediction model without ever centralising individual patient records from private hospitals. Design the federated learning architecture: number of rounds, local epochs per round, FedAvg aggregation, and validation strategy.
- (d) **Real-time alerting:** specify the Dataflow pipeline for detecting an influenza outbreak: window type and size, trigger, threshold (what counts as an alert), and output destination.

Programming Exercises

PE11.1. Delta Lake ACID Operations. Using PySpark and the delta-spark library locally, implement and verify Delta Lake’s ACID properties:

- (a) Create a Delta table for hospital admissions: (patient_id STRING, region STRING, diagnosis_code STRING, admit_date DATE, los_days INT). Write 10,000 synthetic rows.
- (b) Append 2,000 new records (Batch 2).
- (c) Execute an UPDATE to add 2 to los_days for all patients where region = 'Northeast'.
- (d) Execute a DELETE for all patients where los_days < 1 (invalid records).
- (e) Query VERSION AS OF 1 (after Batch 2 but before the UPDATE) and count the Northeast patients. Verify the count matches what you expect.
- (f) Run DESCRIBE HISTORY and display all four transaction entries. For each, identify the operation type.
- (g) Run OPTIMIZE with Z-ordering on (region, admit_date) and observe the reduction in the number of Parquet files.

PE11.2. MLflow Experiment Tracking and Model Registry. Build a complete MLflow experiment for payment fraud classification:

- (a) Generate 100,000 synthetic transaction records with (amount_usd, txn_per_hour, is_new_receiver, hour_of_day, is_fraud).
- (b) Train three models (Logistic Regression, Random Forest, Gradient Boosted Trees) and log each as a separate MLflow run with hyperparameters, AUROC, F1, and training duration.
- (c) Use the MLflow Tracking UI to compare the three runs. Identify the best model by AUROC.
- (d) Register the best model in the MLflow Model Registry with the name payment-fraud-v2. Transition it to the Staging stage.
- (e) Load the staged model using mlflow.spark.load_model() and run predictions on a 10,000-row test set. Verify the AUROC matches the training-time AUROC.

PE11.3. Apache Beam: Windowed Transaction Aggregation. Implement

an Apache Beam pipeline that aggregates payment transaction events:

- (a) Generate a synthetic stream of 1 million transaction events with (`txn_id`, `region`, `amount_usd`, `event_time`). Introduce random event-time lag of 0–30 seconds to simulate mobile network delay.
- (b) Write the pipeline using the Direct Runner (local execution). Apply 5-minute tumbling event-time windows and sum `amount_usd` per region per window.
- (c) Configure a composite trigger: emit an early result every 10 seconds of processing time AND emit a final result when the watermark passes the window end. Use accumulating mode.
- (d) Set `allowed_lateness` to 60 seconds. Verify that events arriving within 60 seconds after the window end are included in the final result.
- (e) Compare the output of the 5-minute tumbling window against a 10-minute sliding window with a 5-minute slide. Which produces more output records and why?

Further Reading

- **Armbrust, Das, et al. 2020** — Open access on the VLDB website. Read Sections 2 (motivation), 3 (transaction log design), and 5 (performance evaluation). Section 4 (MERGE INTO) is worth reading for the upsert use case.
- **Armbrust, Ghodsi, et al. 2021** — Four pages; reads quickly. Section 3 (Lakehouse Architecture) and Section 4 (Performance Evaluation) are most relevant to the course content. The final paragraph on DuckDB is prescient: by 2024, DuckDB had become a primary tool for in-process analytical queries on Delta tables.
- **Akidau et al. 2015** — The most technically dense paper in the course. Read Section 1 (introduction) and Section 3 (The Dataflow Model) carefully; Sections 4–6 describe the Google-internal execution system (Mill-Wheel) which is less relevant to open-source practitioners. The appendix examples (event-time windowing with watermarks) are highly recommended.

- **Sculley et al. 2015** — Five pages. Every ML practitioner should read this. Map each of the nine debt categories to a tool in the MLOps stack described in this chapter.
- **Dehghani 2022** — The definitive Data Mesh book. Chapters 1–3 (the problem) and 7–9 (the data product) are most relevant; Chapters 10–14 cover implementation which is beyond this course’s scope.
- **Designing Data-Intensive Applications (Kleppmann, 2017)**, Chapter 11 (*Stream Processing*). The clearest explanation of event-time vs. processing time and watermarks for a practitioner audience. Pairs perfectly with the Dataflow Model paper.
- **Delta Lake documentation at delta.io**: the Quickstart guide and the “How Delta Lake Works” deep-dive are the best practical complement to the VLDB paper. The Databricks Community Edition (free) allows running all code examples in this chapter without a local Spark installation.

References

- Acharya, S. and S. Chellappan (2015). *Big Data and Analytics*. Wiley.
- Akidau, T., R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle (2015). “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing”. In: *Proceedings of the VLDB Endowment*. Vol. 8. 12, pp. 1792–1803. DOI: [10.14778/2824032.2824076](https://doi.org/10.14778/2824032.2824076).
- Armbrust, M., T. Das, L. Sun, B. Yavuz, S. Zhu, M. Murthy, J. Torres, H. van Hovell, A. Ionescu, A. Łuszczak, M. Łysakowski, M. Switakowski, X. Li, T. Ueshin, M. Mokhtar, P. Boncz, A. Ghodsi, S. Paranjpye, P. Senster, R. Xin, and M. Zaharia (2020). “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores”. In: *Proceedings of the VLDB Endowment*. Vol. 13. 12, pp. 3411–3424. DOI: [10.14778/3415478.3415560](https://doi.org/10.14778/3415478.3415560).
- Armbrust, M., A. Ghodsi, R. Xin, and M. Zaharia (2021). “Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics”. In: *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR 2021)*.
- Beyer, M. A. and D. Laney (2012). “The Importance of “Big Data”: A Definition”. In: *Gartner Research*. Gartner Report G00235055.
- Bostock, M., V. Ogievetsky, and J. Heer (2011). “D³: Data-Driven Documents”. In: *IEEE Transactions on Visualization and Computer Graphics* 17.12, pp. 2301–2309. DOI: [10.1109/TVCG.2011.185](https://doi.org/10.1109/TVCG.2011.185).
- Brewer, E. A. (2000). “Towards Robust Distributed Systems”. In: *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC 2000)*. Keynote address. Portland, OR, USA: ACM, p. 7. DOI: [10.1145/343477.343502](https://doi.org/10.1145/343477.343502).
- Chang, F., J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber (2006). “Bigtable: A Distributed Storage System for Structured Data”. In: *Proceedings of the 7th Symposium on Operating System Design and Implementation (OSDI 2006)*. USENIX Association, pp. 205–218.

- Dean, J. and S. Ghemawat (2004). “MapReduce: Simplified Data Processing on Large Clusters”. In: *Proceedings of the 6th Symposium on Operating System Design and Implementation (OSDI 2004)*. USENIX Association, pp. 137–150.
- Dehghani, Z. (2022). *Data Mesh: Delivering Data-Driven Value at Scale*. O’Reilly Media.
- Ghemawat, S., H. Gobioff, and S.-T. Leung (2003). “The Google File System”. In: *Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP 2003)*. ACM, pp. 29–43. doi: [10.1145/945445.945450](https://doi.org/10.1145/945445.945450).
- Ghods, A., M. Zaharia, B. Hindman, A. Konwinski, S. Shenker, and I. Stoica (2011). “Dominant Resource Fairness: Fair Allocation of Multiple Resource Types”. In: *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2011)*. USENIX Association, pp. 323–336.
- Gilbert, S. and N. A. Lynch (2002). “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services”. In: *ACM SIGACT News* 33.2, pp. 51–59. doi: [10.1145/564585.564601](https://doi.org/10.1145/564585.564601).
- Halevy, A., A. Rajaraman, and J. Ordille (2006). “Data Integration: The Teenage Years”. In: *Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB 2006)*. Seoul, South Korea: VLDB Endowment, pp. 9–16.
- Jumper, J., R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis (2021). “Highly Accurate Protein Structure Prediction with AlphaFold”. In: *Nature* 596, pp. 583–589. doi: [10.1038/s41586-021-03819-2](https://doi.org/10.1038/s41586-021-03819-2).
- Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media.
- Kreps, J., N. Narkhede, and J. Rao (2011). “Kafka: A Distributed Messaging System for Log Aggregation”. In: *Proceedings of the NetDB Workshop at VLDB 2011*.

- Kwak, H., C. Lee, H. Park, and S. Moon (2010). "What is Twitter, a Social Network or a News Media?" In: *Proceedings of the 19th International Conference on World Wide Web (WWW 2010)*. ACM, pp. 591–600. doi: [10.1145/1772690.1772751](https://doi.org/10.1145/1772690.1772751).
- Laney, D. (Feb. 2001). *3D Data Management: Controlling Data Volume, Velocity and Variety*. Research Note 949. META Group.
- Lazer, D., R. Kennedy, G. King, and A. Vespignani (2014). "The Parable of Google Flu: Traps in Big Data Analysis". In: *Science* 343.6176, pp. 1203–1205. doi: [10.1126/science.1248506](https://doi.org/10.1126/science.1248506).
- Manyika, J., M. Chui, B. Brown, J. Bughin, R. Dobbs, C. Roxburgh, and A. H. Byers (May 2011). *Big Data: The Next Frontier for Innovation, Competition, and Productivity*. Tech. rep. McKinsey Global Institute.
- McMahan, H. B., E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas (2017). "Communication-Efficient Learning of Deep Networks from Decentralized Data". In: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)* 54, pp. 1273–1282.
- Obermeyer, Z. and E. J. Emanuel (2016). "Predicting the Future — Big Data, Machine Learning, and Clinical Medicine". In: *New England Journal of Medicine* 375.13, pp. 1216–1219. doi: [10.1056/NEJMp1606181](https://doi.org/10.1056/NEJMp1606181).
- Reinsel, D., J. Gantz, and J. Rydning (Nov. 2018). *The Digitization of the World: From Edge to Core*. Tech. rep. IDC White Paper, Doc. US44413318. International Data Corporation (IDC).
- Sculley, D., G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison (2015). "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems (NeurIPS 2015)*. Vol. 28, pp. 2503–2511.
- Sethi, R., M. Traverso, D. Sundstrom, D. Phillips, W. Xie, Y. Sun, N. Yigitbasi, H. Jin, E. Hwang, N. Shingte, and C. Berner (2019). "Presto: SQL on Everything". In: *Proceedings of the 35th IEEE International Conference on Data Engineering (ICDE 2019)*. IEEE, pp. 1802–1813. doi: [10.1109/ICDE.2019.00196](https://doi.org/10.1109/ICDE.2019.00196).
- Shneiderman, B. (1996). "The Eyes Have It: A Task by Data Type Taxonomy for Information Visualizations". In: *Proceedings of the IEEE Symposium on Visual Languages*. IEEE, pp. 336–343. doi: [10.1109/VL.1996.545307](https://doi.org/10.1109/VL.1996.545307).
- Taylor, S. J. and B. Letham (2018). "Forecasting at Scale". In: *The American Statistician* 72.1, pp. 37–45. doi: [10.1080/00031305.2017.1380080](https://doi.org/10.1080/00031305.2017.1380080).

- Thusoo, A., J. S. Sarma, N. Jain, Z. Shao, P. Chakka, S. Anthony, H. Liu, P. Wyckoff, and R. Murthy (2009). "Hive – A Warehousing Solution Over a Map-Reduce Framework". In: *Proceedings of the VLDB Endowment (PVLDB)*. Vol. 2. 2, pp. 1626–1629. doi: [10.14778/1687553.1687609](https://doi.org/10.14778/1687553.1687609).
- Tufte, E. R. (1983). *The Visual Display of Quantitative Information*. Cheshire, CT: Graphics Press.
- Vavilapalli, V. K., A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, B. Saha, C. Curino, O. O'Malley, S. Radia, B. Reed, and E. Baldeschwieler (2013). "Apache Hadoop YARN: Yet Another Resource Negotiator". In: *Proceedings of the 4th Annual Symposium on Cloud Computing (SoCC 2013)*. ACM, 5:1–5:16. doi: [10.1145/2523616.2523633](https://doi.org/10.1145/2523616.2523633).
- White, T. (2015). *Hadoop: The Definitive Guide*. 4th. O'Reilly Media.
- Zaharia, M., M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica (2012). "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing". In: *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2012)*. Best Paper Award. USENIX Association, pp. 15–28.
- Zaharia, M., M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica (2010). "Spark: Cluster Computing with Working Sets". In: *Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 2010)*. USENIX Association.